HO CHI MINH CITY UNIVERSITY OF TRANSPORT (UTH)
FACULTY OF INFORMATION TECHNOLOGY

Capstone Project Document

Horse Racing Tournament Management System

Group Members	Ha Duy Hung - 074206002060 *(Team Leader)* Dinh Thuy Anh - 068306011497 Ho Nguyen Thai Chau - 054206003685 Le Thi My Hue - 052306012275 Bui Le Quang Huy - 231206000064 Nguyen Gia Huy - 2251120017 Doan Thi Thu May - 068306004742
Supervisor	Lecturer Nguyen Van Chien
Capstone Project code	Hourse Racing Tournament

- Ho Chi Minh City, July 2026 -

Contents

Acknowledgement

We would like to express our sincere gratitude to the faculty of Ho Chi Minh City University of Transport (UTH), and especially to our supervisor, Nguyen Van Chien, for their dedicated guidance and support throughout the development of this Capstone Project.

We would also like to thank every member of our team for their effort and close collaboration, which made it possible to complete the "Horse Racing Tournament Management System" project on schedule.

Definition and Acronyms

Term / Abbreviation	Definition
MVP	Minimum Viable Product
SRS	Software Requirement Specification
SDD	Software Design Description
ERD	Entity Relationship Diagram
UC	Use Case
BR	Business Rule
CR	Common Requirement
MSG	Application Message
UI	User Interface
API	Application Programming Interface
DB	Database

I. Project Introduction

1.1. Overview

1.1.1 Project Information

- **Project name:** Horse Racing Tournament Management System
- **Project code:**
- **Group name:** Group 3
- **Software Type:** Web application

1.1.2 Project Team

Full Name	Role	Email	Mobile
Nguyen Van Chien	Supervisor	chiennv@ut.edu.vn	
Ha Duy Hung	Leader	haduydung0912@gmail.com	
Dinh Thuy Anh	Member	anhdt1497@ut.edu.vn	
Ho Nguyen Thai Chau	Member	chauhnt3685@ut.edu.vn	
Le Thi My Hue	Member	lemyhue1803@gmail.com	
Nguyen Gia Huy	Member	nhuy22174@gmail.com	
Doan Thi Thu May	Member	maydtt6742@ut.edu.vn	
Bui Le Quang Huy	Member	huyblq0064@ut.edu.vn	

Table 1.1: Project Team

1.2. Product Background

In the context of Vietnam's growing interest in sports entertainment and digital transformation, horse racing organizations are increasingly facing the need to manage tournaments, participants, horses, jockeys, and race registration processes more efficiently. However, many racing-related activities may still depend on manual management methods such as paper forms, scattered Excel files, direct communication, or disconnected systems. This can lead to difficulties in managing tournament information, horse and jockey profiles, owner registrations, approval status, and communication between horse owners, jockeys, referees, and administrators.

Recognizing these limitations, a student team has proposed the development of a Horse Racing Management System. This system aims to digitize the main workflow of horse racing event management, from tournament creation and participant registration to approval, race organization, and result tracking. The target users include horse owners who register horses for tournaments, jockeys who participate in races, administrators who manage tournaments and registration requests, referees who monitor race results, and viewers or related users who need to follow tournament information in a transparent and convenient way.

1.3. Existing Systems

1.3.1 Prism Horse

Link: `https://prism.horse`

Description: A horse racing management platform supporting race events, horse information, and participant management.

Actors:

- Horse Owners
- Jockeys
- Administrators
- Referees

Features:

- Horse and jockey management.
- Tournament organization.
- Race registration.
- Participant management.

Pros:

- Designed specifically for horse racing activities.
- Supports race and participant management.

Cons:

- Limited owner–jockey collaboration workflow.
- No invitation management mechanism.
- Limited registration status tracking.
- Lack of integrated communication features.

1.3.2 RaceDB

Link: `https://www.racedb.com`

Description: A race event management system that supports competition registration, participant management, race scheduling, and result tracking. The platform is primarily designed for cycling and sporting competitions.

Actors:

- Participants
- Event Organizers
- Administrators
- Race Officials

Features:

- Online participant registration.
- Race scheduling and event management.
- Result recording and ranking.
- Participant and competition tracking.

Pros:

- Supports online registration and race management.
- Provides race scheduling and result tracking.
- Offers a centralized system for competition administration.

Cons:

- Not specifically designed for horse racing.
- Does not support horse owner, horse, and jockey relationships.
- Lacks invitation and approval workflows between horse owners and jockeys.
- Does not provide horse profile management and ownership information.
- Limited support for horse racing officials and tournament-specific workflows.

1.4. Business Opportunity

The horse racing industry is gradually embracing digital transformation as racing organizations and clubs seek more efficient ways to manage tournaments, participants, and race operations. However, many horse racing activities still rely on manual registration processes, paper documents, and fragmented communication between horse owners, jockeys, and organizers. This creates an opportunity to develop a specialized management system that can streamline tournament registration, improve participant coordination, and enhance transparency throughout the racing process.

The proposed Horse Racing Management System aims to optimize race organization by supporting online tournament registration, horse and jockey management, invitation workflows, and registration approval processes. The system can reduce administrative workload, minimize registration errors, and improve communication among stakeholders.

The system will compete in the sports event management and competition management software market while specifically targeting the horse racing domain, where existing solutions often lack comprehensive support for horse owner–jockey collaboration, race registration workflows, and tournament management requirements. By focusing on the unique characteristics of horse racing activities, the system has the potential to provide a more specialized and efficient solution for racing organizations and participants.

1.5. Software Product Vision

For horse racing organizations seeking to digitalize tournament operations, the Horse Racing Management System is a web-based platform that enables horse owners to register horses, invite jockeys, and participate in racing tournaments through an integrated online process. Jockeys can manage invitations and confirm participation, while administrators and referees can efficiently monitor registrations, approve requests, and oversee race activities.

Unlike traditional management methods that rely on paper documents, manual registration, and fragmented communication, the Horse Racing Management System reduces administrative workload, improves the accuracy and transparency of tournament management, and enhances collaboration among horse owners, jockeys, referees, and administrators. By digitalizing registration workflows, participant information, and competition management, the system also establishes a foundation for future services such as performance analytics, race statistics, online event management, and data-driven decision making in horse racing organizations.

1.6. Project Scope & Limitations

1.6.1 Major Features

In order to establish a comprehensive management solution for horse racing tournaments, the system defines specific functional scopes tailored to five primary user roles: Administrator, Horse Owner, Jockey, Referee, and Spectator.

Major Features for Admin

- **FE-AD-01: User Account Management:** Register, view, update, and manage accounts for Referees, Jockeys, Horse Owners, and Spectators.
- **FE-AD-02: Tournament & Schedule Management:** Define new tournaments, set up races, assign tracks, and schedule tournament timelines.
- **FE-AD-03: System Configuration:** Configure parameters such as registration fees, base points, and reward distributions.

Major Features for Horse Owner

- **FE-OW-01: Horse Registry:** Register and maintain detailed profiles for participating horses.
- **FE-OW-02: Tournament Registration:** Register horses to upcoming tournaments and track the review/approval status from the organization board.
- **FE-OW-03: Jockey Invitation:** Draft and send invitations to qualified Jockeys for tournament partnerships.

Major Features for Jockey

- **FE-JK-01: Invitation Handling:** View, accept, or decline race partnership invitations received from Horse Owners.
- **FE-JK-02: Race Schedule Tracking:** View personal racing schedules and details of assigned tournaments.
- **FE-JK-03: Performance Overview:** Track personal race history, rankings, and career statistics.

Major Features for Referee

- **FE-RF-01: Race Assignments:** View the list of races assigned by the Administrator.
- **FE-RF-02: Participant Inspection:** Log checking-in processes, including verifying registration details and recording pre-race weight checks for jockeys and horses.
- **FE-RF-03: Violation Reporting:** Report and document any rule infractions or warnings during the race.
- **FE-RF-04: Result Recording:** Input and confirm race completion times and final placements through a two-step validation flow.

Major Features for Spectator

- **FE-SP-01: Schedule & Result Information:** Browse active tournaments, view upcoming schedules, and inspect historical race results.
- **FE-SP-02: Leaderboards:** View dynamic performance leaderboards for horses, jockeys, and top predicting spectators.
- **FE-SP-03: Rank Prediction:** Predict the top-ranking horses before a race starts to earn virtual reward points.
- **FE-SP-04: Personal Profile & History:** Manage profile details and review prediction history and accumulated rewards.

1.6.2 Limitations & Exclusions

To ensure project feasibility within the designated timeframe and resources, the following constraints and exclusions are defined:

- **LI-01: No Real-Money Betting:** The prediction module operates purely on virtual reward points. No integration with real currency transactions, financial betting systems, or gambling licenses is supported.
- **LI-02: No Automatic IoT/Hardware Integration:** The system does not interface with physical hardware such as GPS trackers on horses, automatic weight scales, or high-speed photofinish cameras at the finish line. All inspection metrics and race results are logged manually by Referees.
- **LI-03: Offline & Simulated Payment Processing:** Standard fees (e.g., tournament registrations) are simulated or handled offline. No production payment gateways (e.g., credit card processors, electronic wallets) are integrated.
- **LI-04: Lack of Live Simulation:** The application does not provide real-time 3D simulation or video streaming of active races. The race status changes are updated statefully through Referee inputs.

II. Software Project Management Plan

This chapter describes project planning, scheduling, risk management, communication and configuration management.

2.1. Overview

The Software Project Management Plan (SPMP) outlines the managerial, technical, and resource allocation strategies for the development of the Horse Racing Tournament Management System. This project aims to digitize and automate the processes of managing horse racing tournaments, including horse owner registration, jockey invitation workflows, race scheduling, referee assignment, result recording, and spectator interactions.

The primary objective of this plan is to ensure that the project is delivered on time, within scope, and meets all defined quality criteria. To achieve these goals, the team employs an Agile-Scrum methodology, which promotes iterative development, continuous feedback, and rapid response to changes. This document details the project scope, estimation, quality objectives, risk management, and configuration management processes that govern the lifecycle of this project.

2.2. Scope & Estimation

To ensure the project is delivered successfully within the constraints of time and resources, the project scope has been broken down into a detailed Work Breakdown Structure (WBS). The WBS divides the Horse Racing Tournament Management System into 7 distinct modules, covering administrative tasks, core infrastructure, and functional components for various user roles.

Each work item is assessed for complexity (Simple, Medium, or Complex) based on the database interactions, business logic density, and interface requirements:

- **Simple**: Basic CRUD operations, simple UI layouts, or minor configurations requiring 5 to 6 man-days.
- **Medium**: Standard functional features with moderate business logic (e.g., forms validation, standard data list with filters) requiring 7 to 8 man-days.
- **Complex**: Critical components involving advanced logic, multiple integrations, or strict constraints (e.g., conflict checking algorithms, two-step referee verification, real-time leaderboard calculations) requiring 9 to 15 man-days.

The table below outlines the estimated effort in man-days for each WBS item based on these parameters:

No.	WBS Item	Complexity	Est. Effort (man-days)
1	**PM01. Project Management & Administration**		**20**
1.1	Project Setup & Git Workflow Config	Simple	5
1.2	Code Review, Integration & Conflict Support	Complex	15
2	**DB01. Database & Infrastructure**		**12**
2.1	Database Schema Setup & ERD Design	Medium	7
2.2	API Core Config & DB Connection	Simple	5
3	**FE01. Authentication & User Management**		**14**
3.1	Login, Registration & Authorization UI/API	Medium	8
3.2	User Management Panel (Pagination/Search)	Medium	6
4	**FE02. Horse & Jockey Management**		**23**
4.1	Horse Owner Profile Update & Registration	Medium	8
4.2	Jockey Profile Update & Invitation UI/API	Complex	15
5	**FE03. Tournament & Race Scheduling**		**23**
5.1	Tournament Management & Race Creation	Medium	8
5.2	Race Scheduling & Conflict Validation	Complex	15
6	**FE04. Race Execution & Results**		**24**
6.1	Referee Two-step Result Submission	Complex	12
6.2	Real-time Leaderboard & Prize Config	Complex	12
7	**FE05. Spectator Portal**		**14**
7.1	Schedules & Live Results UI/API	Simple	6
7.2	Place Prediction Form & Timer Lock	Complex	8

Table 2.1: Work Breakdown Structure (WBS) and Effort Estimation

2.3. Project Objectives

To ensure the Horse Racing Tournament Management System meets the quality standards required for deployment and user satisfaction, the team has established specific quality objectives. These objectives focus on functional correctness, performance, security, data integrity, and usability, matching the real development and testing guidelines of the project.

The table below outlines the core quality objectives and target criteria for the project:

No.	Quality Objective	Target Metric / Criteria	Measurement Method
1	Functional Suitability	100% of defined SRS requirements pass manual E2E test cases	Execution of Manual E2E Test Cases (Phase 1, 2, 3, 4)
2	Data Integrity & Encoding	100% successful storage and display of Vietnamese characters (Unicode) in DB and UI	Database inspection (SQL Server) & UI display validation
3	Security & Authentication	Secure password hashing (bcrypt) and JWT authorization for protected role-based routes	Manual code review of app security handlers and route guards
4	Performance Efficiency	Response time $<$ 2s for all frontend page transitions and backend API calls	Network monitoring via Browser Developer Tools
5	Usability & Aesthetics	Responsive dark-mode glassmorphic interface with consistent date/time formats	Manual peer testing & UI checklist validation

Table 2.2: Project Quality Objectives

2.3.1 Defect Tolerance by Testing Phase

Defect density and resolution rates are tracked continuously across various testing levels: Document Review, Integration Testing (Phase 1-2), Integration Testing (Phase 3), and System Testing/UAT. Defects are classified into three severity levels:

- **Critical**: System crash, database connection failure, or core functions completely blocked (e.g., authentication failure, scheduling conflicts, failed result recalculation).
- **Major**: Features malfunctioning or displaying incorrect data without completely blocking the flow (e.g., jockey showing ID instead of real name, date formats displaying raw ISO string).
- **Minor**: UI layout alignment bugs, minor text formatting glitches, or spelling errors that do not affect functional operations.

The table below details the maximum allowed defects and the target resolution process for each testing phase:

Testing Phase	Max Critical	Max Major	Max Minor	Resolution Process
Document Review	0	Max 2	Max 5	Resolved before coding phase
Unit Testing	0	0	Max 3	Resolved before PR merge
Integration Test	0	Max 1	Max 8	Resolved before System Test
System Test	0	Max 2	Max 12	Resolved before UAT phase
Acceptance Test (UAT)	0	0	Max 3	Resolved before final release

Table 2.3: Maximum Allowed Defect Rates by Testing Phase

2.4. Risk Management

To ensure the smooth execution of the Horse Racing Tournament Management System development, the team has identified potential technical, operational, and managerial risks. Managing these risks involves analyzing their probability of occurrence, potential impact, and establishing proactive mitigation and contingency plans.

The table below presents the key risks identified during the project lifecycle and the corresponding response strategies:

No.	Risk Description	Probability	Impact	Mitigation / Response Plan
1	**Git Merge Conflicts**: Code integration conflicts due to multiple developers working on adjacent modules (e.g., dashboard panels).	High	Major	Split large files (like `dashboard/page.js`) into smaller components. Enforce strict branching guidelines: pull changes from `dev-GiaHuy` before coding, and create pull requests for review.
2	**Vietnamese Encoding Errors**: Vietnamese names and text showing as `?` in UI or database due to non-Unicode configurations in SQL Server.	Medium	Moderate	Change string definitions to `Unicode` and text to `UnicodeText` in SQLAlchemy models (`database_models.py`) to map to `NVARCHAR` columns in SQL Server.
3	**Race Scheduling Conflicts**: Double-booking of referees, jockeys, or horses in overlapping races or tournaments.	Medium	Major	Implement robust backend validations (`Conflict Validation`) for schedule updates and referee assignments before persisting to the database.
4	**React Hooks Rule Violations**: Frontend crashes and rendering bugs caused by calling Hooks inside loops or conditional statements (e.g., in Admin panels).	Medium	Major	Perform mandatory code reviews and utilize ESLint static analysis to identify Hook violations prior to code integration.
5	**Inconsistent Local Databases**: Out-of-sync local database schemas among team members causing API integration failures.	High	Major	Provide a centralized database setup and seeding script (`db_setup.py`). Enforce running the script immediately after git pulls.

Table 2.4: Project Risk Analysis and Mitigation Plans

2.5. Management Approach

To effectively manage the development of the Horse Racing Tournament Management System, the team employs the Agile-Scrum methodology. This approach allows the team to deliver features iteratively, adapt to changes quickly, and ensure constant communication among members.

2.5.1 Agile-Scrum Implementation

The development cycle is organized into weekly sprints. Each sprint begins with a planning meeting and ends with a review and retrospective.

- **Sprint Planning**: Held at the start of each week to define the sprint goal and allocate tasks from the product backlog on Jira.
- **Weekly Syncs**: Held twice a week (on Wednesdays and Saturdays) to track progress, identify blockers, and align integration efforts. Daily updates and progress reports are shared via Zalo.
- **Sprint Review & Retrospective**: Conducted at the end of the sprint to review completed tasks, demo features, and reflect on improvements for the next sprint.

2.5.2 Coding Standards and Bug Prevention

To maintain code quality and prevent bugs early in the lifecycle, the team enforces the following rules and standards:

- **Coding Style Guidelines**: The backend follows PEP 8 guidelines for Python (FastAPI), while the frontend adheres to ESLint standards for React/Next.js. React Hooks rules are strictly monitored (e.g., avoiding calling hooks inside loops or conditions).
- **Database Consistency**: Any database schema modifications must be updated in the SQLAlchemy models. Developers are required to run the automated database setup and seeding script (`db_setup.py`) locally after pulling the latest code to prevent database mismatches.
- **Git Workflow Rules**: Direct pushes to the main branch (`main`) or integration branch (`dev-GiaHuy`) are prohibited. All work must be completed on feature branches (`feature/*`) and integrated via Pull Requests (PRs). The Team Leader reviews and approves all PRs before merging.

2.6. Deliverables Schedule

The project deliverables are scheduled and tracked weekly, aligning with the Agile sprints and the development phases of the Horse Racing Tournament Management System. The following schedule details the deliverables for both source code components (API and UI) and project documentation from Week 1 to Week 9.

2.6.1 Deliverables Timeline (Week 1 - Week 9)

The timeline below structures the actual development history, pull requests, and documentation milestones into weekly progress updates from the beginning of the project through the final LaTeX compilation:

Week	Phase / Focus	Key Deliverables (Source Code & Documentation)	Status
Week 1	Scope & Context	System Scope definition, Actors identification, Context Diagram, and initial database setup (`schema.sql`).	Completed
Week 2	Requirements	Overall Use Case Diagram for MVP, Use Case Descriptions, and initial project layout setup.	Completed
Week 3	Phase 1 (Core Auth)	Next.js and FastAPI project skeleton, user registration, login flow, and dashboard layout restructuring.	Completed
Week 4	Phase 1 (Admin Flow)	User Management API, lock/unlock accounts, dynamically calling referees list, and fixing React Hooks rendering violations. Merged PR #4 and #6 (16/06/2026).	Completed
Week 5	Phase 2 (Owner & Jockey)	Jockey invitation workflow (Accept/Reject), horse profiles CRUD with age validation (2-10 years), and registration status updates. Merged (19/06/2026).	Completed
Week 6	Phase 3 (Referee & Spectator)	Two-step result verification (`RESULTS_ENTERED` → `COMPLETED`), `RaceInspections` form, spectator place prediction form with dynamic filters, and auto-reward points logic. Merged (21/06/2026).	Completed
Week 7	Phase 4 (Admin & Database)	Timezone `Asia/Ho_Chi_Minh` integration, ±2h referee conflict check, validation rules, database schema update (missing columns for users, spectators), and fixing local connections (127.0.0.1). Merged (24/06/2026).	Completed
Week 8	LaTeX	LaTeX compilation environment setup, project structure creation, and initial drafts of Chapter 1 and 2 sections.	Completed
Week 9	LaTeX Compilation	Finalizing Chapter 2 (SPMP) sections, Chapter 3 (SRS) requirements, Chapter 4 (SDD) system design, and compiling the official PDF document.	Completed

Table 2.5: Key Deliverables Schedule (Week 1 - 9)

2.7. Responsibility Assignment Matrix (DRSI)

To clarify roles and responsibilities across the team, the project utilizes a DRSI matrix (Do, Review, Support, Informed). The abbreviations used in the matrix are defined as follows:

- **D (Do)**: The member responsible for developing and implementing the deliverable.

- **R (Review)**: The member responsible for reviewing and approving the quality of the work.
- **S (Support)**: The member providing support during development or integration.
- **I (Informed)**: Members kept updated on the status of the deliverable.
- **- (Omitted)**: Member has no direct responsibility for the deliverable.

The table below outlines the DRSI matrix for the key project deliverables:

Deliverable / Module	DH	TA	GH	TC	MH	BH	TM
Project Setup & Configuration	D	I	I	I	I	I	I
Database & Infrastructure Setup	R	I	D	I	I	S	I
Authentication & User Management	R	I	D	I	D	I	I
Horse Owner Registration	R	D	I	I	I	I	I
Jockey Profile & Invitations	R	S	I	D	I	I	I
Race Scheduling & Referee Assign	R	I	S	I	D	I	I
Race Result & Ranking Calculation	R	I	I	I	I	D	I
Spectator Prediction & Rewards	R	I	I	I	I	S	D
LaTeX Report & Presentation	D	D	D	D	D	D	D

Table 2.6: DRSI Responsibility Assignment Matrix

2.7.1 Document Management

To ensure consistent document formatting and prevent conflicts during collaborative writing, the team follows a structured LaTeX document management workflow on GitHub:

- **Three-Tier Architecture**: The document structure is divided into three tiers:
  1. `main.tex`: The root file that only handles package imports and includes document wrappers.
  2. `documents/`: Wrapper files for each chapter (e.g., `project_management_plan.tex`).
  3. `sections/`: The folder containing the actual content files divided by section and assignee.
- **Conflict Avoidance**: Team members are strictly restricted to editing files under their assigned sections. Editing `main.tex` or files in the `documents/` folder is prohibited without prior team alignment.
- **Figure Convention**: Figures must be stored in the `figures/` directory and named according to the format: `<chapter>_<content>.<ext>` (e.g., `figures/srs_erd.png`). Generic names like `image1.png` are avoided.

2.7.2 Source Code Management

The project source code is hosted on GitHub and managed using a customized feature-branching workflow to guarantee code quality and repository stability:

- **Branching Strategy**:
  - `main`: Reserved for the final, production-ready release used for demos.
  - `dev-GiaHuy`: The main integration branch where all validated feature branches are merged.
  - `feature/*`: Temporary branches used by developers to work on assigned tasks (e.g., `feature/auth-user`, `feature/spectator-view`).
- **Development Protocol**:
  1. Pull the latest updates from the remote integration branch before coding: `git pull origin dev-GiaHuy`.
  2. Develop the feature on a separate `feature/*` branch.
  3. Push the feature branch to the remote repository.
  4. Open a Pull Request (PR) from the feature branch to `dev-GiaHuy`. Direct pushes to `dev-GiaHuy` or `main` are disabled.
- **Commit Message Convention**: Commit messages must clearly state the scope of the change. For example:
  - `git commit -m "Add login API"`
  - `git commit -m "Create horse management page"`
- **Review & Merge**: The Team Leader reviews all PRs, checks for build errors, resolves conflicts, and merges the code into the integration branch.

2.7.3 Tools and Infrastructure

The development and execution environment for the Horse Racing Tournament Management System utilizes the following tech stack, tools, and local infrastructure:

- **Programming Languages & Frameworks**:
  - **Backend**: Python 3 utilizing the FastAPI framework for clean architecture REST APIs.
  - **Frontend**: JavaScript/React.js utilizing Next.js (App Router) for the web application UI.
- **Database Management**:
  - Local Microsoft SQL Server Express instance (`localhost\SQLEXPRESS`) with database named `HorseRacing`.
  - Connection library: `pyodbc` (ODBC Driver 17 for SQL Server) and SQLAlchemy ORM.

- **Development Platforms**:
  - **Operating System**: Windows 10/11 workstation.
  - **Integrated Development Environment (IDE)**: VS Code equipped with LaTeX Workshop extension.
  - **Git Client**: Git for Windows / Git Bash.
  - **LaTeX Compiler**: MiKTeX or TeX Live for generating report PDFs locally.
- **Local Hosting Configuration**:
  - Backend server running locally on `http://127.0.0.1:8000` (API Client configured directly to IP address to prevent connections dropouts).
  - Frontend Next.js development server running locally on `http://localhost:3000`.

III. Software Requirement Specification

3.1. Product Overview

The Horse Racing Tournament Management System (HRTMS) is a comprehensive web-based application designed to digitize and optimize the operational workflows of horse racing tournament organizations, race scheduling, and spectator participation.

- **Operational Environment**:
  - **Platform**: Web-based application accessible via standard web browsers (Chrome, Edge, Firefox, Safari).
  - **Connectivity**: Requires a stable Internet connection to function (no offline mode).
  - **Infrastructure**: Powered by a FastAPI backend API and a Next.js frontend, utilizing Microsoft SQL Server for the database.
- **Anticipated Users**:
  - **Internal Users**: Administrator and Race Referees.
  - **External Users**: Horse Owners, Jockeys, and Spectators.
- **Constraints & Assumptions**:
  - **Language**: The system database and interface support UTF-8 encoding for Vietnamese characters, with localized date/time formatting.
  - **Compliance**: Complies with user account data privacy and local tournament scheduling rules in Vietnam.
  - **Scope Limitation**: Does not support actual financial betting transactions; spectator predictions use virtual reward points only. Timezone handling is restricted to `Asia/Ho_Chi_Minh`.
- **Dependencies**:
  - Relies on local Python environment libraries (`pyodbc`, `SQLAlchemy`, `FastAPI`).
  - Requires a configured Microsoft SQL Server database.

3.1.1 System Context Diagram

The system context diagram below defines the boundaries and exchanges between the Horse Racing Tournament Management System and its external entities:

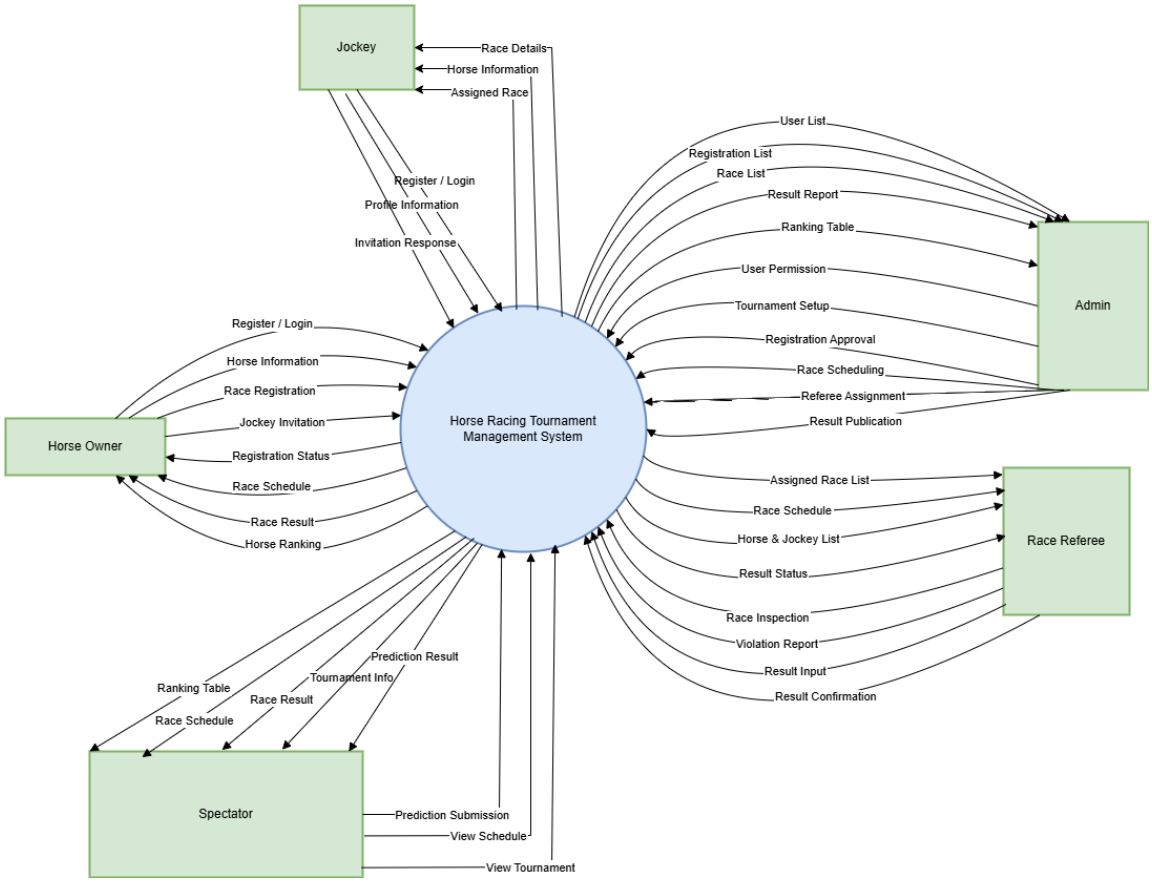


Figure 3.1: System Context Diagram

3.1.2 Interaction Data Flows

The data interactions between the system and each external actor are defined as follows:

- **Horse Owner**:
  - *Data Input to System*: Register / Login, Horse Information, Race Registration, and Jockey Invitation.
  - *Data Output from System*: Registration Status, Race Schedule, Race Result, and Horse Ranking.
- **Jockey**:
  - *Data Input to System*: Register / Login, Profile Information, and Invitation Response.
  - *Data Output from System*: Assigned Race, Race Details, and Horse Information.
- **Administrator (Admin)**:

  - *Data Input to System*: User Permission, Tournament Setup, Registration Approval, Race Scheduling, Referee Assignment, and Result Publication.
  - *Data Output from System*: User List, Registration List, Race List, Result Report, and Ranking Table.

- **Race Referee**:
  - *Data Input to System*: Race Inspection, Violation Report, Result Input, and Result Confirmation.
  - *Data Output from System*: Assigned Race List, Race Schedule, Horse & Jockey List, and Result Status.

- **Spectator**:
  - *Data Input to System*: View Tournament, Prediction Submission, and View Schedule.
  - *Data Output from System*: Tournament Info, Race Schedule, Race Result, Ranking Table, and Prediction Result.

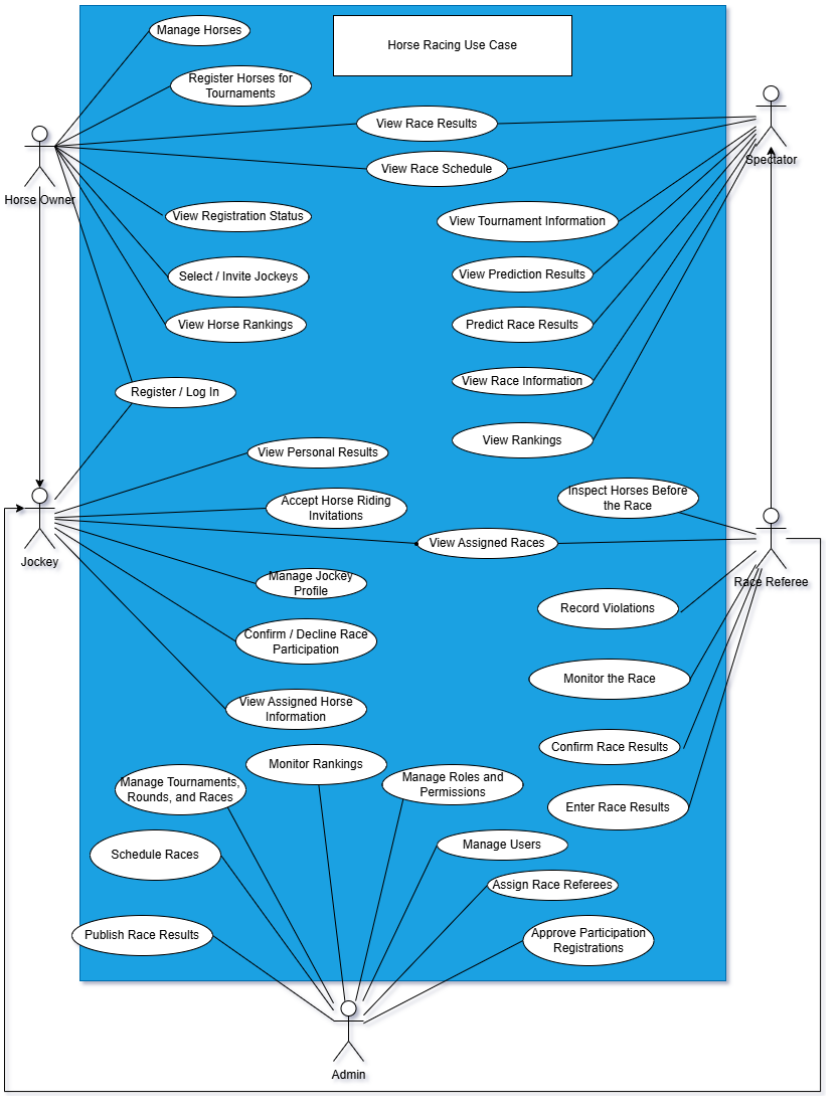
3.2. User Requirements

3.2.1 System Actors

Actor	**Type**	**Description / Responsibilities**
Admin	Human	Responsible for platform administration: user and role management, system configuration, approvals, and overall system monitoring.
Referee	Human	Manages race supervision and validation: verifies race results, resolves disputes, and updates official results (backend endpoints in ‘results.py‘ / ‘referees.py‘).
Jockey	Human	Represents the rider; manages jockey profile information and race assignments.
Horse Owner	Human	Registers and manages horse profiles, submits horses to races, and maintains horse-related records and health information.
Spectator	Human	Views race schedules, results, leaderboards, and (optionally) places bets or follows favorite horses/jockeys via the frontend panels.

3.2.2 Use Cases

Use Case Diagram



Use Case Description

ID	Use Case	Primary Actor	Brief Description
UC-01	Manage Horses	Horse Owner	Create, update and maintain horse profiles and related health records.
UC-02	Register Horses for Tournaments	Horse Owner	Submit horses to tournament participation and view registration status.
UC-03	Manage Jockey Profile	Jockey	Create and update jockey personal/profile information and availability.
UC-04	Confirm / Decline Race Participation	Jockey	Accept or decline invitations to participate in assigned races.
UC-05	View Race Schedule	Spectator	Browse upcoming races, schedules and basic race information.
UC-06	View Race Results	Spectator	View final results, rankings and statistics after a race is published.
UC-07	Predict / View Prediction Results	Spectator	Submit predictions (if supported) and view prediction outcomes.
UC-08	Inspect Horses / Monitor Race	Referee	Inspect horses before the race, monitor the race and record any violations.
UC-09	Enter / Confirm Race Results	Referee	Enter raw race results and confirm official results for publication.
UC-10	Manage Races and Tournaments	Admin	Create and schedule races/tournaments, assign referees and publish results.
UC-11	Manage Users and Roles	Admin	Manage platform users, roles and permissions (admin panel).

3.3. Functional Requirements

3.3.1 System Functional Overview

Admin Screen Flow

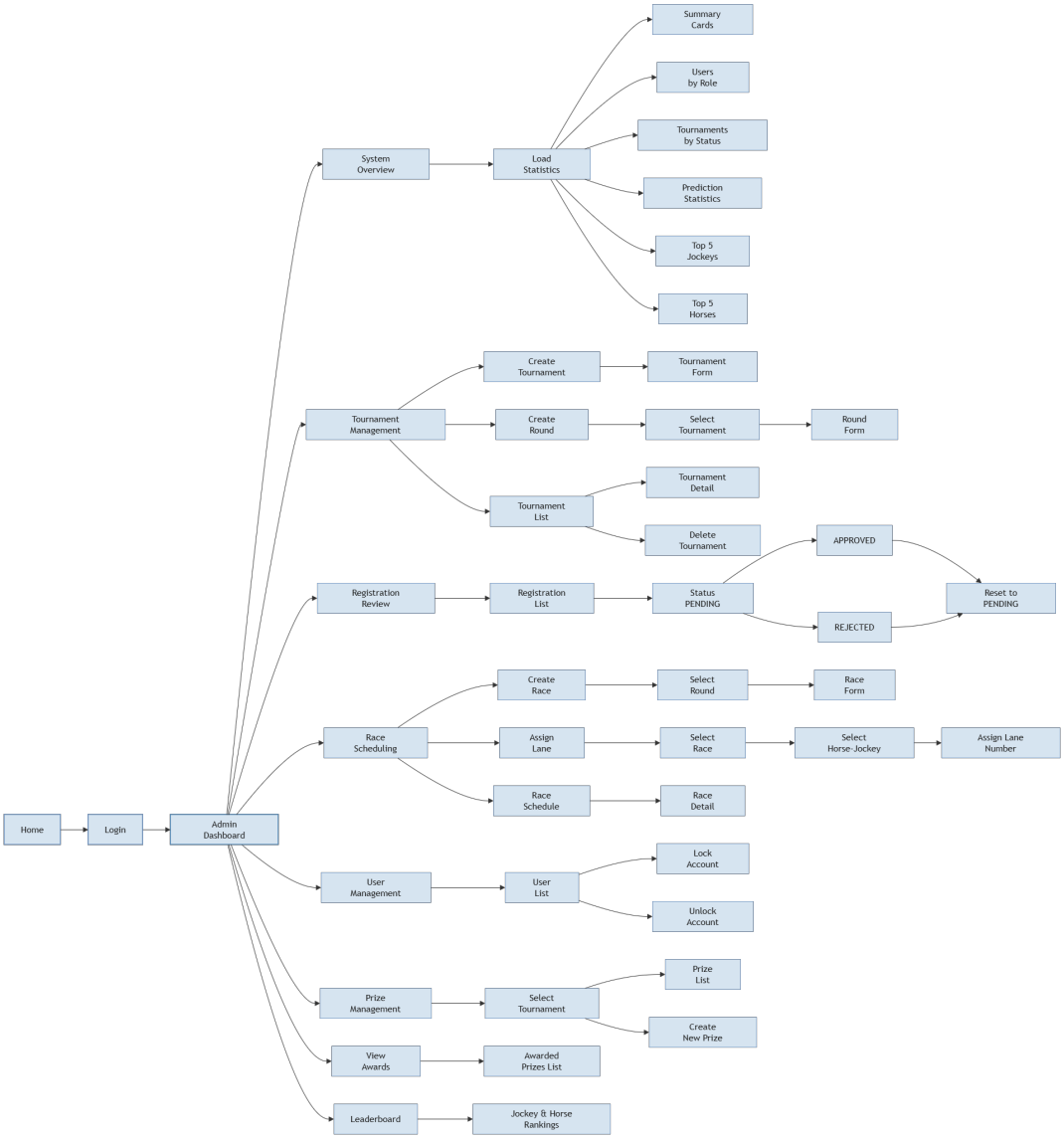


Figure 3.2: Admin screen flow

Admin Screen description

#	Feature	Screen	Description
1	Navigation	Home	Initial landing page that leads to the login screen for an admin user.
2	Authentication	Login	Secure admin sign-in screen with credentials and access validation.
3	Dashboard	Admin Dashboard	Main admin hub that provides links to system overview, management panels, and key actions.
4	Overview	System Overview	Displays the high-level system summary and provides access to statistics modules.
5	Statistics	Load Statistics	Shows dashboard widgets such as summary cards, user counts by role, tournament status, prediction metrics, top jockeys, and top horses.
6	Tournament Management	Tournament List	Lists tournaments with controls to view details, approve or reject, and delete tournaments.
7	Tournament Management	Tournament Form	Screen for creating a new tournament with required details and rules.
8	Tournament Management	Round Form	Screen for creating a new tournament round after selecting a tournament.
9	Registration Review	Registration List	Displays pending horse registration requests and allows the admin to review status.
10	Registration Review	Registration Status	Shows the current approval state of a registration, including options to approve, reject, or reset to pending.
11	Race Scheduling	Create Race	Screen for creating a new race based on selected tournament rounds.
12	Race Scheduling	Assign Lane	Allows the admin to assign a lane number to a selected horse-jockey pair for a specific race.
13	Race Scheduling	Race Schedule	Displays the full race schedule and navigation to race detail pages.
14	User Management	User List	Lists platform users with options to lock or unlock accounts and manage roles.
15	Prize Management	Prize List	Shows existing prize entries for a selected tournament and management actions.
16	Prize Management	Create New Prize	Screen to add a new prize associated with a selected tournament.
17	Rankings	Leaderboard	Displays jockey and horse rankings and performance standings.

Table 3.1: Admin screen descriptions

Horse Owner Screen Flow

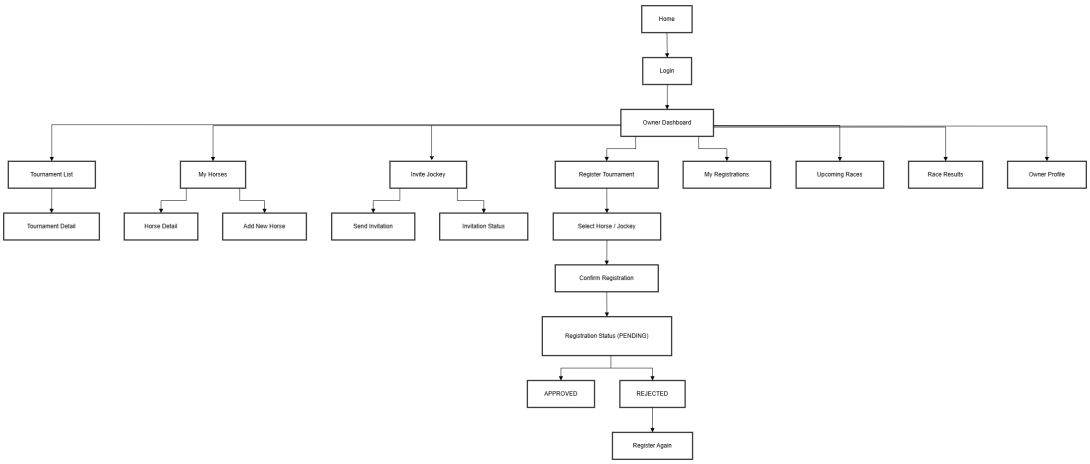


Figure 3.3: Horse Owner screen flow

Horse Owner Screen description

#	Feature	Screen	Description
1	General	Home	Landing page for all users; navigate to Login and Owner access.
2	General	Login	Authenticate the owner and enter the Owner Dashboard.
3	Dashboard	Owner Dashboard	Main hub for Owner with navigation to horse management, invitations, tournament registration, schedules, results, and profile.
4	Horse Management	My Horses	View and manage owned horses, including add, edit, and delete operations.
5	Horse Management	Horse Detail	Edit horse details: name, age (2-10), breed, gender, and save changes.
6	Invitation	Invite Jockey	Send invitations to jockeys for a selected horse and tournament with a custom message.
7	Invitation	Invitation Status	Track invitations with jockey, horse, tournament, and current acceptance status.
8	Registration	Register Tournament	Register accepted horse/jockey pairs for tournaments and create registration requests.
9	Registration	My Registrations	Display tournament registration history with approval status: APPROVED, PENDING, REJECTED.
10	Schedule	Upcoming Races	Show upcoming races for the owner's horses with date, time, and location.
11	Results	Race Results	Show historical race results for the owner's horses with rank, points, notes, and violations.
12	Profile	Owner Profile	View and update owner details: full name, phone, company, avatar.

Table 3.2: Horse Owner screen descriptions

Jockey Screen Flow

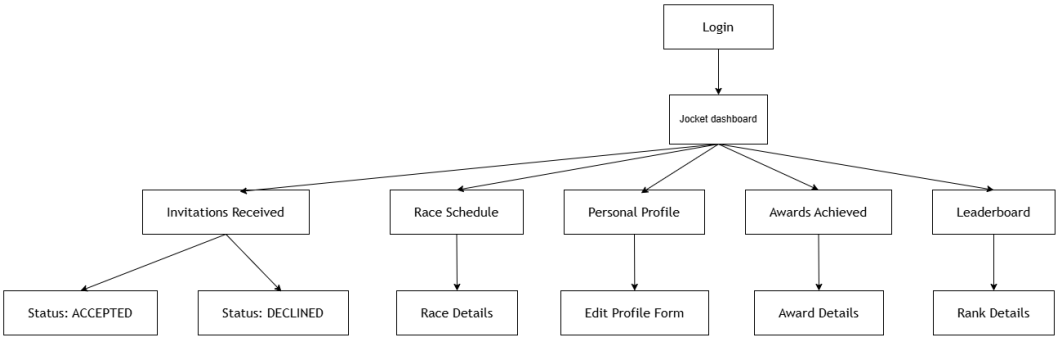


Figure 3.4: Jockey screen flow

Jockey Screen description

#	Feature	Screen	Description
1	General	Home	Landing page for all users; navigate to Login.
2	General	Login	Authenticate the jockey and enter the Jockey Dashboard.
3	Dashboard	Jockey Dashboard	Main hub for jockey with navigation to invitations, race schedule, profile, awards, and leaderboard.
4	Invitations	Invitations Received	View invitations received from horse owners with current status.
5	Invitations	Status: ACCEPTED	Show accepted invitations and allow jockey to proceed to race registration.
6	Invitations	Status: DECLINED	Show declined invitations and provide feedback details.
7	Schedule	Race Schedule	Display upcoming races and enable navigation to race details.
8	Schedule	Race Details	Show detailed information for a selected race.
9	Profile	Personal Profile	View jockey profile information and access the edit profile form.
10	Profile	Edit Profile Form	Update personal details such as name, contact, weight, height, and experience.
11	Awards	Awards Achieved	Display awards and achievements earned by the jockey.
12	Awards	Award Details	Show detailed information for a selected award.
13	Leaderboard	Leaderboard	Display jockey rankings with filters by tournament and category.
14	Leaderboard	Rank Details	Show detailed rank information for selected jockey standings.

Table 3.3: Jockey screen descriptions

Race Scheduling Management Screen Flow

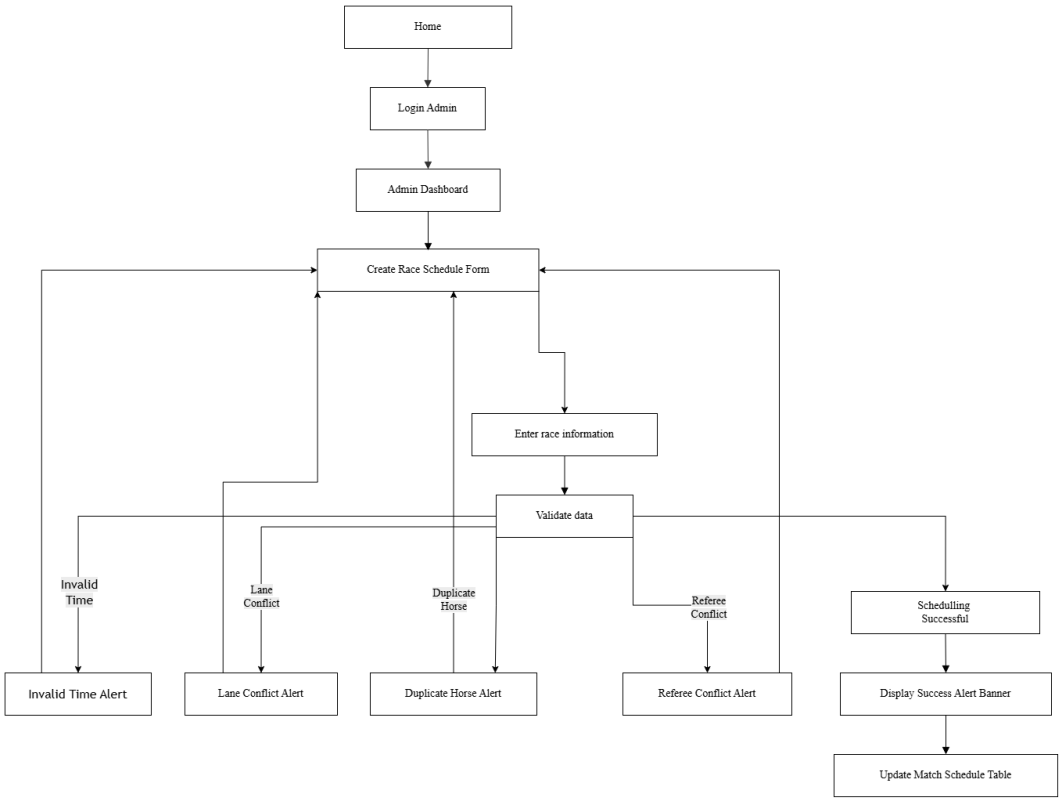


Figure 3.5: Race scheduling management screen flow

Screen description for Race Scheduling

#	Feature	Screen	Description
1	General	Home	The main landing page of the system, acting as the entry point before authentication.
2	Authentication	Login Admin	The dedicated login interface restricted to authorized Administrator accounts.
3	General	Admin Dashboard	The landing page post-login; displays the ongoing tournament list to select a specific workspace.
4	Configuration	Create Race Schedule Form	Main input interface to set up tournament stage, race name, time, track conditions, referee assignment, and lane allocation.
5	Configuration	Enter race information	Step where detailed race attributes are inputted before the system initiates the validation process.
6	Validation	Validate data	Automated backend process triggered to verify the validity of data and check for any constraints or scheduling conflicts.
7	Validation	Invalid Time Alert	Validation error state triggered if the selected race time falls outside the boundaries of the tournament start and end dates.
8	Validation	Lane Conflict Alert	Validation error state triggered if multiple horses are mistakenly assigned to the exact same lane number.
9	Validation	Duplicate Horse Alert	Validation error state triggered if a single horse is selected more than once within the same race setup.
10	Validation	Referee Conflict Alert	Validation error state triggered if the assigned referee has another active commitment within a 2-hour window.
11	Validation	Scheduling Successful	System state confirming that the schedule data passed all validations and has been successfully recorded.
12	Validation	Display Success Alert Banner	A success banner prompted at the top of the form displaying: “Race created successfully!”.
13	Feedback	Update Match Schedule Table	Persistent historical data table below the form that appends and displays the newly scheduled race details instantly.

Table 3.4: Race Scheduling screen descriptions

Spectator Screen Flow

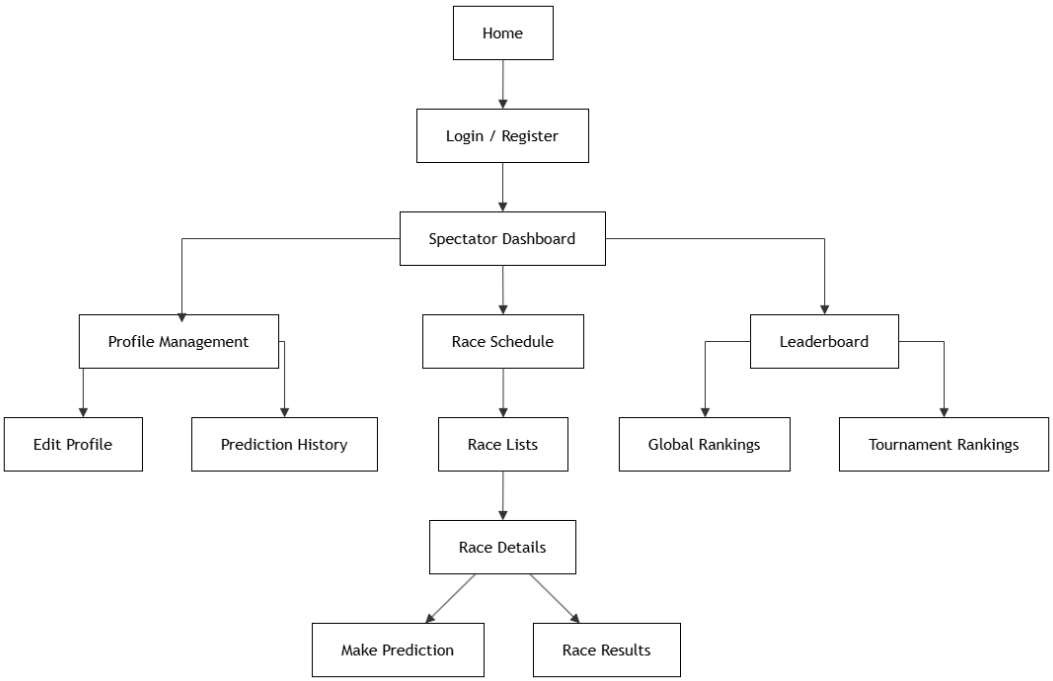


Figure 3.6: Spectator screen flow

Spectator Screen description

#	Feature	Screen	Description
1	General	Home	Landing page for all users; navigate to Login and Registration access.
2	General	Login / Register	Authenticate spectators or register a new account to access the system.
3	Dashboard	Spectator Dashboard	Main hub for Spectator with navigation shortcuts to profile management, race schedules, and leaderboards.
4	Profile	Profile Management	Manage personal account information, providing options to edit profile details and view prediction history.
5	Profile	Edit Profile	Edit and update spectator profile details (full name, phone number, avatar, etc.).
6	Profile	Prediction History	Review the history of placed race predictions along with their corresponding earned points status.
7	Schedule	Race Schedule	View an overview schedule of upcoming tournaments and race rounds.
8	Schedule	Race Lists	Display the detailed list of specific races under the selected schedule.
9	Schedule	Race Details	View comprehensive details of a specific race (participants, time, status) with options to make predictions or view results.
10	Schedule	Make Prediction	Place predictions on results or rankings for upcoming/unstarted races.
11	Schedule	Race Results	View the final outcomes, standings, and technical statistics of a completed race.
12	Leaderboard	Leaderboard	Main section to view rankings, performance, and scores.
13	Leaderboard	Global Rankings	Display the global leaderboard based on total accumulated user points.
14	Leaderboard	Tournament Rankings	Display detailed standings categorized by specific tournaments.

Table 3.5: Spectator screen descriptions

Referee Screen Flow

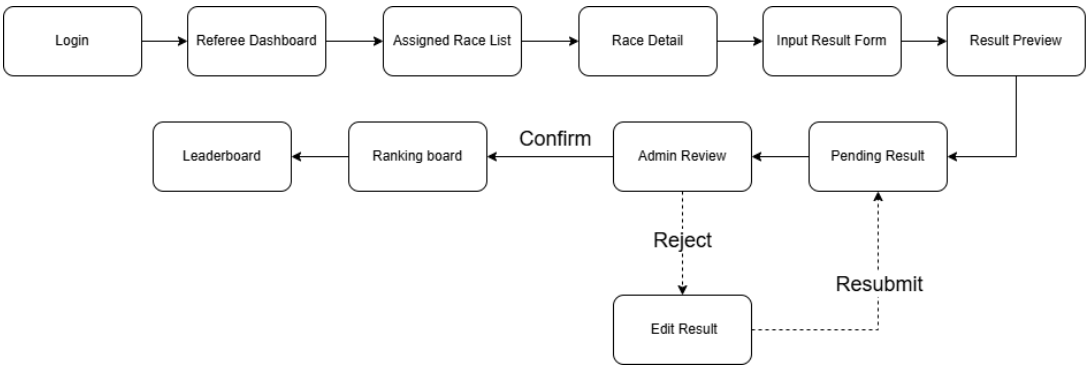


Figure 3.7: Referee screen flow

Referee Screen Description

#	Feature	Screen	Description
1	General	Login	Authenticate the referee and enter the Referee Dashboard.
2	Dashboard	Referee Dashboard	Main hub for the referee with navigation to assigned races, results management, and profile.
3	Race Management	Assigned Race List	Display all races assigned to the referee, including tournament, date, venue, and current status.
4	Race Management	Race Detail	Show detailed information of the selected race, including participating horses, jockeys, and schedule.
5	Result Entry	Input Result Form	Allow the referee to enter finishing positions, times, and relevant notes for each horse in the race.
6	Result Entry	Result Preview	Display a summary of the entered results for the referee to review before submission.
7	Result Submission	Pending Result	Hold the submitted result in a pending state awaiting approval from the administrator.
8	Result Review	Admin Review	Administrator reviews the submitted result and either confirms or rejects it.
9	Result Correction	Edit Result	Allow the referee to correct and update the result after it has been rejected by the administrator.
10	Ranking	Ranking Board	Display the official race ranking after the result has been confirmed by the administrator.
11	Ranking	Leaderboard	Show the overall tournament leaderboard updated with the latest confirmed race results.

Table 3.6: Referee screen descriptions

3.3.2 Screen Authorization

This section defines the access control matrix for the Horse Racing Tournament Management System (HRTMS). It specifies which web modules and screen views are accessible to each user role, along with the typical actions (Create, Read, Update, Delete, or Special operations) they are authorized to perform.

#	Web Module / Screen	Typical Actions (C-R-U-D / Special)	Admin	Owner	Jockey	Referee	Spectator
1	Authentication	C-R-U (Login, Register, Logout)	✓	✓	✓	✓	✓
2	Dashboard Overview	R (Role-specific overview & stats)	✓	✓	✓	✓	✓
3	Personal Profile	R-U (Update weight, experience, contact)		✓	✓		✓
4	User Management	C-R-U / Lock (Account administration)	✓				
5	Horse Management	C-R-U-D (Manage horse profile & health)	R	✓	R	R	R
6	Jockey Invitation	C-R-U (Invite Jockeys to race pairs)	R	✓	✓		
7	Tournament Management	C-R-U-D (Set up tournament details)	✓	R	R	R	R
8	Tournament Registration	C-R-U (Submit horse-jockey registrations)	✓	✓	R		
9	Race Scheduling	C-R-U-D (Race setup & Referee assign)	✓	R	R	R	R
10	Pre-Race Inspection	C-R-U (Track, weather & health check)	R			✓	
11	Race Results & Violations	C-R-U / Confirm (Confirm results & rank)	R	R	R	✓	R
12	Prediction Management	C-R-U (Predict race outcomes with points)					✓
13	Prize & Award Management	C-R-U-D / R (Manage cash prizes & awards)	✓		R		
14	Leaderboard & Rankings	R (View horse, jockey & prediction ranks)	R	R	R	R	R

Table 3.7: Screen Authorization Matrix

3.3.3 Non-Function Requirement

External Interfaces

To ensure effective communication between users and external systems, the following interfaces shall be supported.

User Interface

- The system shall provide a user-friendly and intuitive interface for Horse Owners, Jockeys, Referees, and Administrators.
- The graphical interface shall follow modern web design standards and support responsive layouts.

- Users shall be able to access the system through desktop computers, tablets, and mobile devices.
- Navigation menus and functions shall remain consistent throughout the system.

Authentication Interface

- The system shall support secure authentication using username and password.
- Role-based access control shall restrict system functions according to user roles.
- User sessions shall be managed securely to prevent unauthorized access.

Database Interface

- The application shall communicate with the database system to store and retrieve information.
- Data synchronization between the application server and database server shall be maintained.
- The database shall support CRUD operations for horses, tournaments, registrations, invitations, and users.

Quality Attributes

Usability

User-Friendly Interface

- The system shall provide a simple and intuitive interface.
- Functions such as tournament registration and jockey invitation shall require minimal user actions.
- The interface shall be easy to understand for both novice and experienced users.

Training and Familiarization Time

- Horse Owners and Jockeys shall require less than 30 minutes to learn the basic functions.
- Administrators shall require approximately 1–2 hours to understand management operations.

Usability Metrics

- Tournament registration shall be completed within 3 minutes.
- Error messages shall clearly guide users to correct invalid input.
- User feedback shall be collected to improve usability.

Reliability

Availability

- The system shall maintain at least 99% availability.
- Maintenance shall be scheduled during low-traffic periods.

Data Integrity

- Registration and tournament information shall be stored accurately.
- Duplicate registrations shall be prevented.
- Data consistency shall be maintained across all modules.

Error Handling

- The system shall display meaningful error messages.
- Critical system errors shall be logged for administrators.
- The system shall recover from failures with minimal data loss.

Performance

Response Time

- Normal user requests shall respond within 2 seconds.
- Under heavy load conditions, response time shall not exceed 5 seconds.

Concurrent Users

- The system shall support at least 500 concurrent users.
- Multiple users shall access the system simultaneously without significant performance degradation.

Database Performance

- Search operations shall complete within 3 seconds.
- Registration transactions shall be processed efficiently.

Security

- Passwords shall be encrypted before storage.
- Unauthorized access attempts shall be logged.
- Session timeout mechanisms shall automatically log out inactive users.
- Input validation shall prevent invalid data submission.

Maintainability

- The system shall follow a modular architecture.
- Source code shall comply with coding standards.
- New features shall be added with minimal impact on existing modules.
- API endpoints and database structures shall be documented.

Portability

- The system shall support Google Chrome, Microsoft Edge, and Mozilla Firefox.
- The application shall operate on both Windows and Linux servers.
- Users shall access the system without additional software installation.

3.3.4 Entity Relationship Diagram

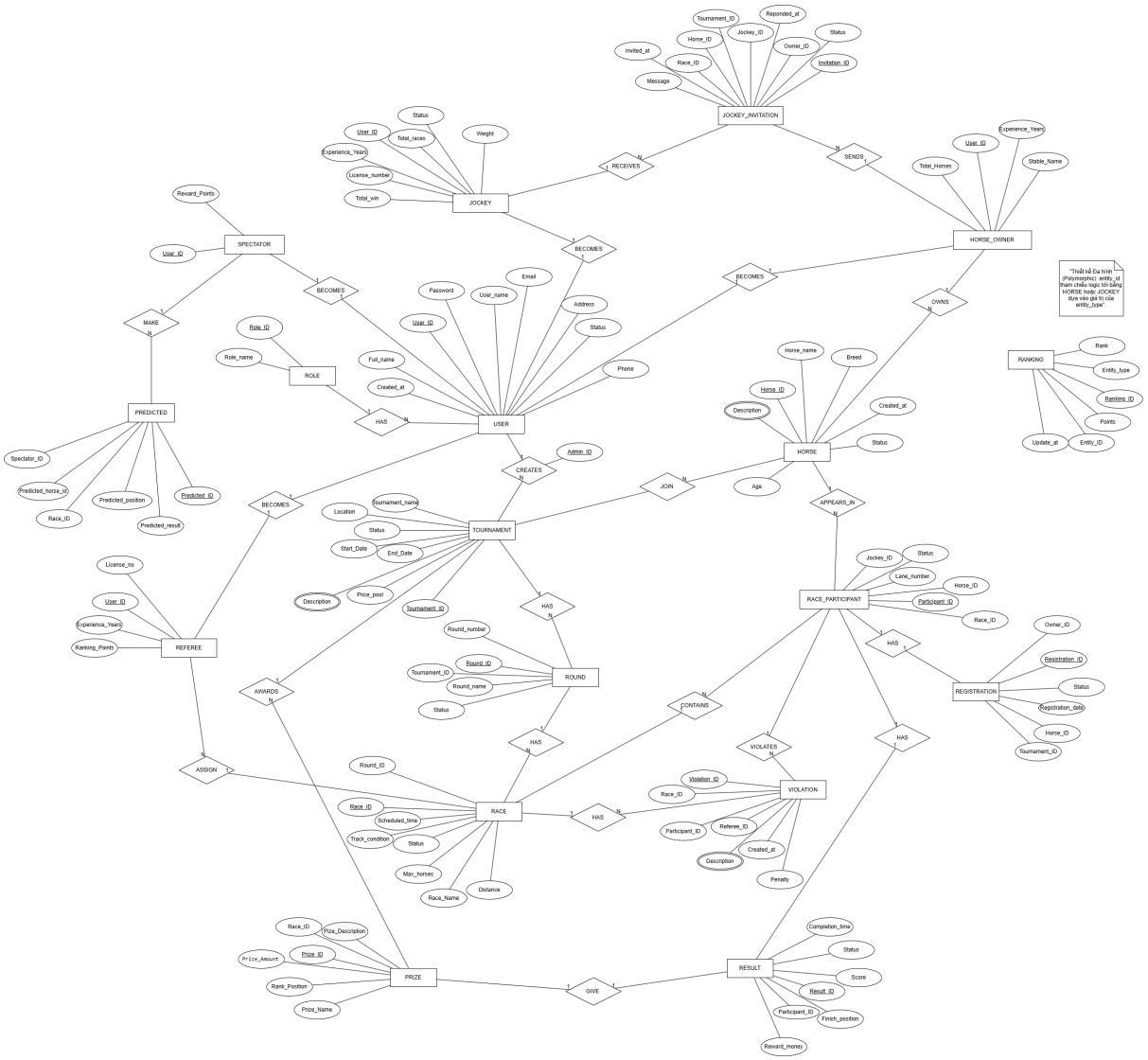


Figure 3.8: Entity Relationship Diagram

3.3.5 Web Application

Horse Owner Login

Role: Horse Owner

- **Function trigger:** The owner enters account credentials to log in.
- **Function description:**
  - **Purpose:**
    1. Verify the owner's identity.
    2. Grant access to owner functionalities.
  - **Data processing:**

1. The owner enters username and password.
2. The system validates credentials.
3. The system identifies the OWNER role.
4. The system creates a login session.

- **Function details:**
  - Successful login redirects to Owner Dashboard.
  - Invalid credentials display an error message.
  - Users without accounts may register.
  - **Bussiness rule:** BR-13

Screen layout:

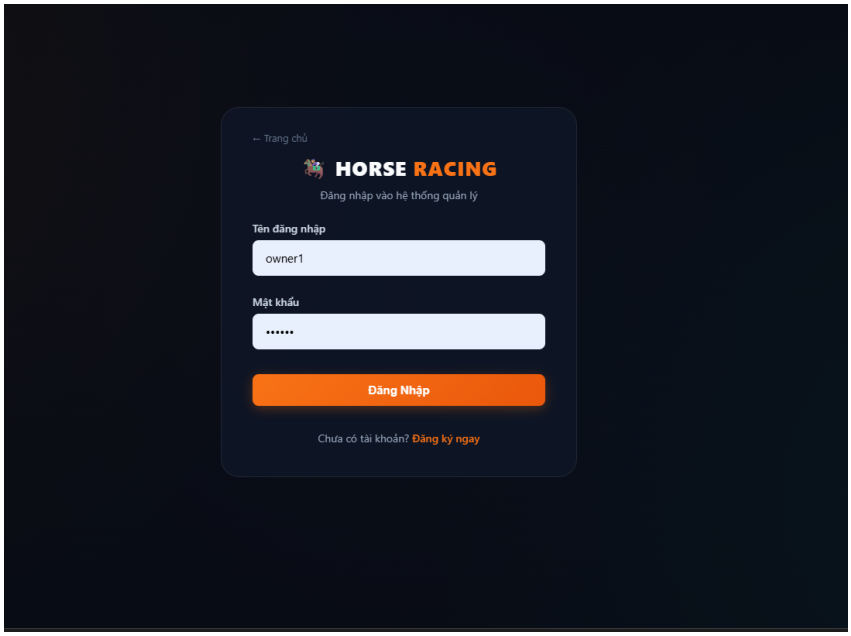


Figure 3.9: Horse Owner Login

Manage Horses

Role: Horse Owner

- **Function trigger:** Select "Quan ly Ngua".
- **Function description:**
  - **Purpose:**
    1. Register new horses.
    2. Manage owned horses.
  - **Data processing:**
    1. Enter horse name.
    2. Enter age.

3. Select breed.
4. Select gender.
5. Save horse information.

- **Function details:**
  - Add new horses.
  - Edit horse information.
  - Delete horse information.
  - Display horse list.
  - **Bussiness rule:** BR-14, BR-15, BR-23

Screen layout:

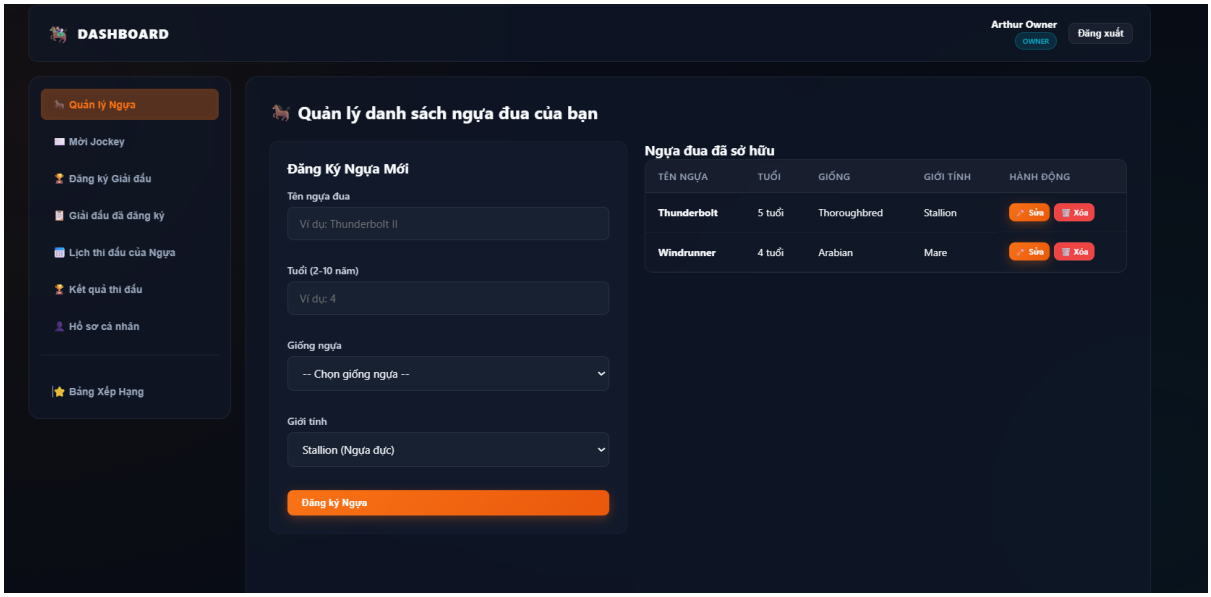


Figure 3.10: Manage Horses

Invite Jockey

Role: Horse Owner

- **Function trigger:** Select “Moi Jockey”.
- **Function description:**
  - **Purpose:**
    1. Invite jockeys.
    2. Create horse-jockey partnerships.
  - **Data processing:**
    1. Select jockey.
    2. Select horse.

3. Select tournament.
4. Enter invitation message.
5. Send invitation.

- **Function details:**
  - Invitation status is displayed.
  - Pending invitations can be monitored.
  - Accepted invitations may be used for registration.
  - **Bussiness rule:** BR-16, BR-17, BR-23

Screen layout:

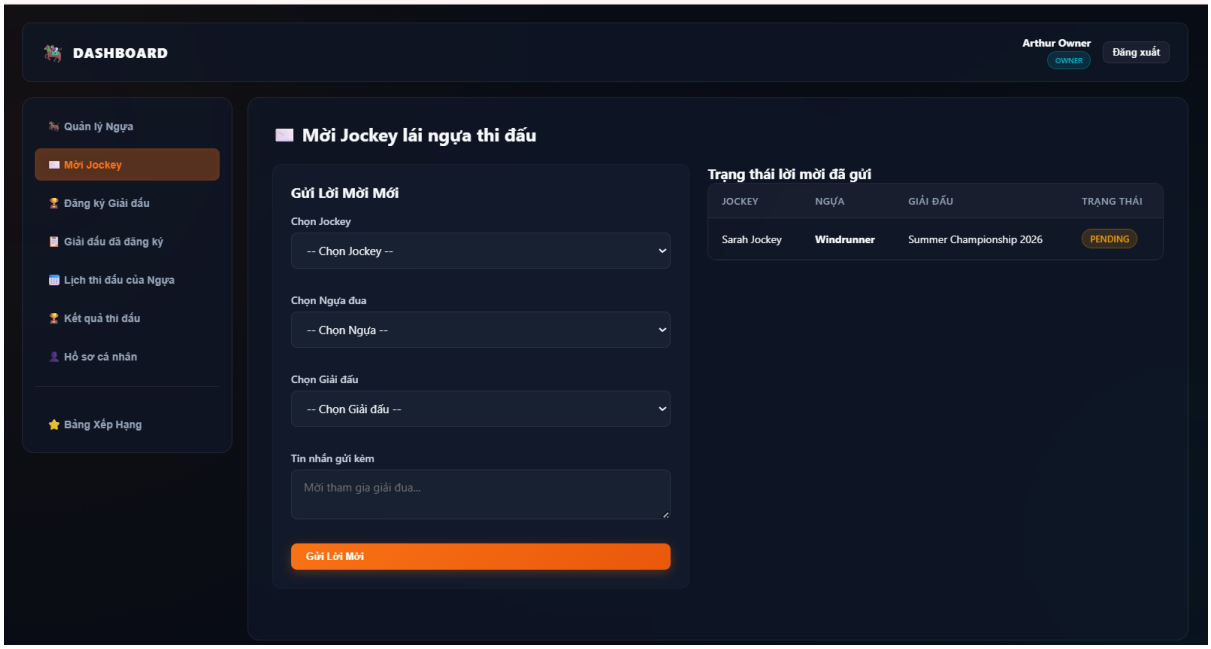


Figure 3.11: Invite Jockey

Register Tournament

Role: Horse Owner

- **Function trigger:** Select “Dang ky Giai dau”.
- **Function description:**
  - **Purpose:**
    1. Register horse-jockey pairs.
    2. Participate in tournaments.
  - **Data processing:**
    1. Retrieve available tournaments.
    2. Check accepted jockey invitations.

3. Select horse-jockey pair.
4. Submit registration request.

- **Function details:**
  - Only accepted invitations may register.
  - Registration requests are sent to administrators.
  - Invalid pairs cannot participate.
  - **Bussiness rule:** BR-18, BR-19, BR-23

Screen layout:

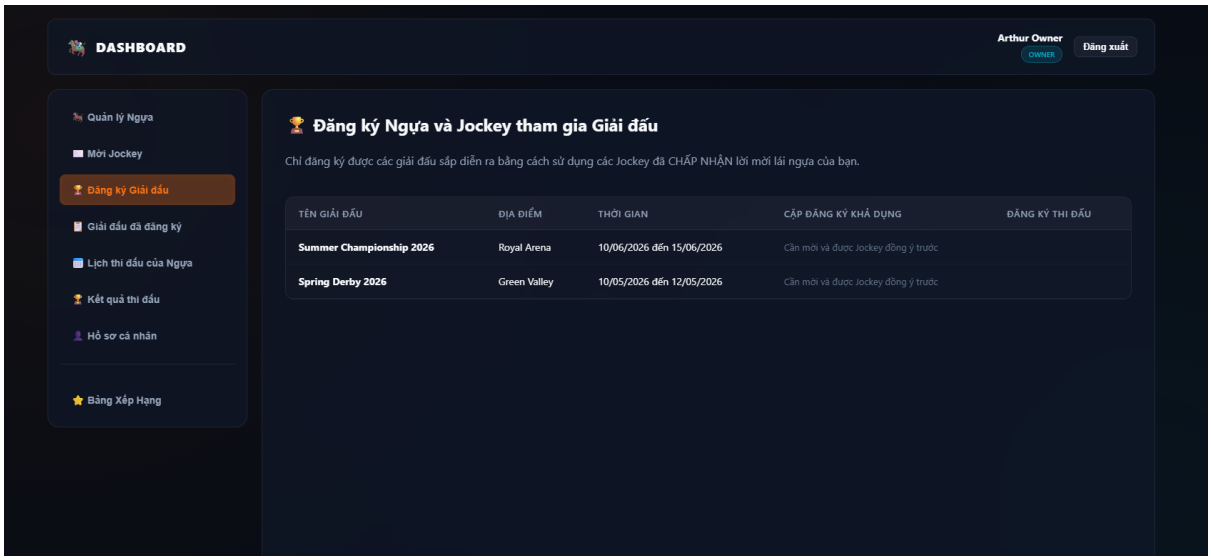


Figure 3.12: Tournament Registration

Registered Tournaments

Role: Horse Owner

- **Function trigger:** Select “Giai dau da dang ky”.
- **Function description:**
  - **Purpose:**
    1. Monitor registration status.
    2. View approval results.
  - **Data processing:**
    1. Retrieve registration records.
    2. Retrieve approval status.
    3. Display registration history.
- **Function details:**

- Display horse information.
- Display jockey information.
- Display approval status.
- **Bussiness rule:** BR-20

Screen layout:

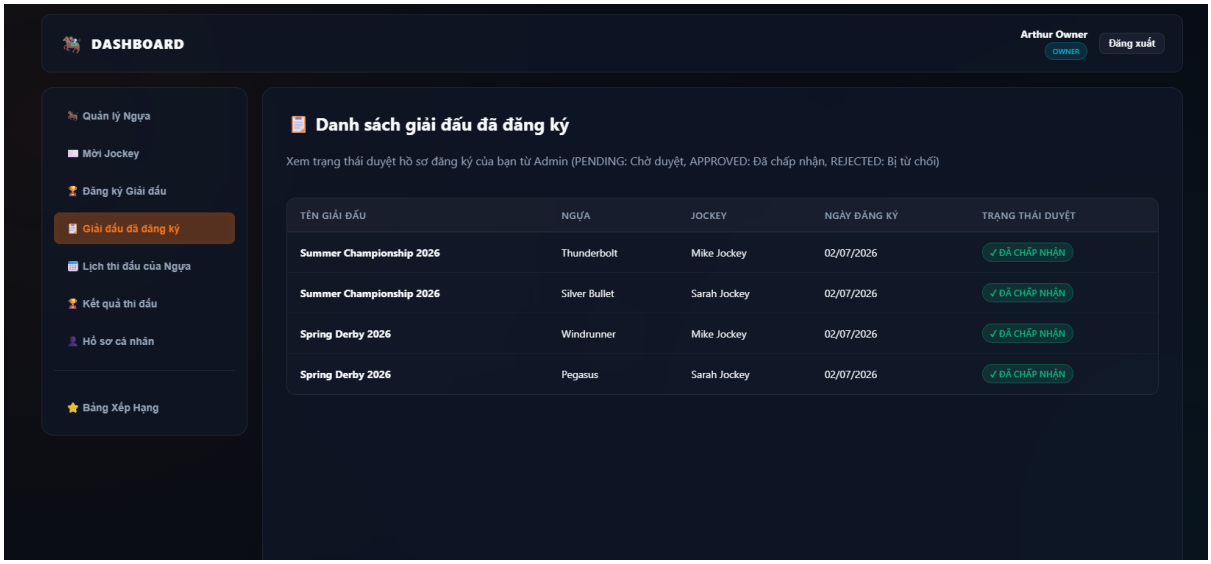


Figure 3.13: Registered Tournaments

Horse Schedule

Role: Horse Owner

- **Function trigger:** Select “Lich thi dau cua Ngua”.
- **Function description:**
  - **Purpose:**
    1. View upcoming races.
    2. Prepare for competitions.
  - **Data processing:**
    1. Retrieve race schedules.
    2. Retrieve tournament information.
    3. Display race information.
- **Function details:**
  - Display race date.
  - Display location.
  - Display tournament information.

- **Bussiness rule:** BR-21

Screen layout:

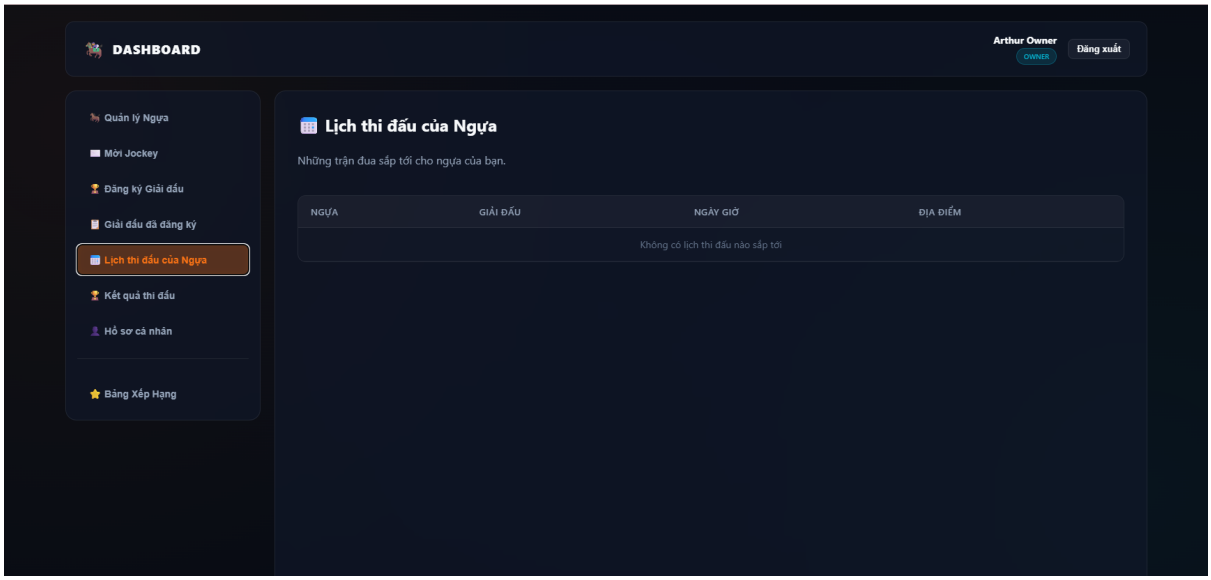


Figure 3.14: Horse Schedule

Race Results

Role: Horse Owner

- **Function trigger:** Select “Ket qua thi dau”.
- **Function description:**
  - **Purpose:**
    1. Review race results.
    2. Evaluate horse performance.
  - **Data processing:**
    1. Retrieve completed races.
    2. Retrieve rankings.
    3. Retrieve scores.
    4. Retrieve violations.
- **Function details:**
  - Display race rankings.
  - Display accumulated points.
  - Display remarks.
  - Display violations.
  - **Bussiness rule:** BR-21

Screen layout:

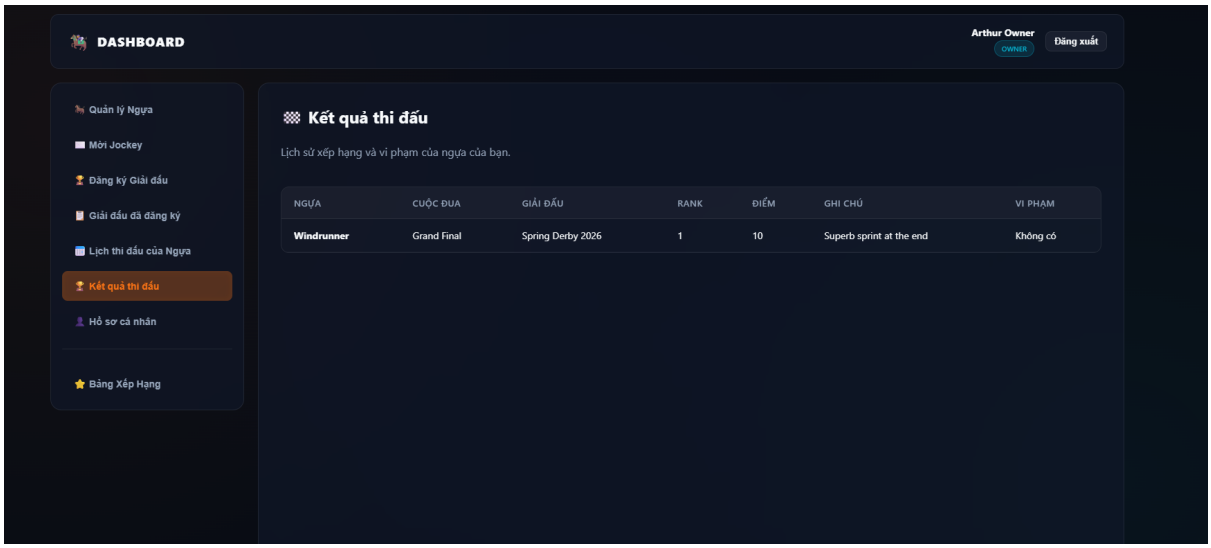


Figure 3.15: Race Results

Owner Profile

Role: Horse Owner

- **Function trigger:** Select "Ho so ca nhan".
- **Function description:**
  - **Purpose:**
    1. Manage personal information.
    2. Update profile information.
  - **Data processing:**
    1. Update personal information.
    2. Update contact information.
    3. Update company information.
    4. Save profile data.
- **Function details:**
  - Edit owner profile.
  - Edit biography.
  - Update social information.
  - **Bussiness rule:** BR-22, BR-23

Screen layout:

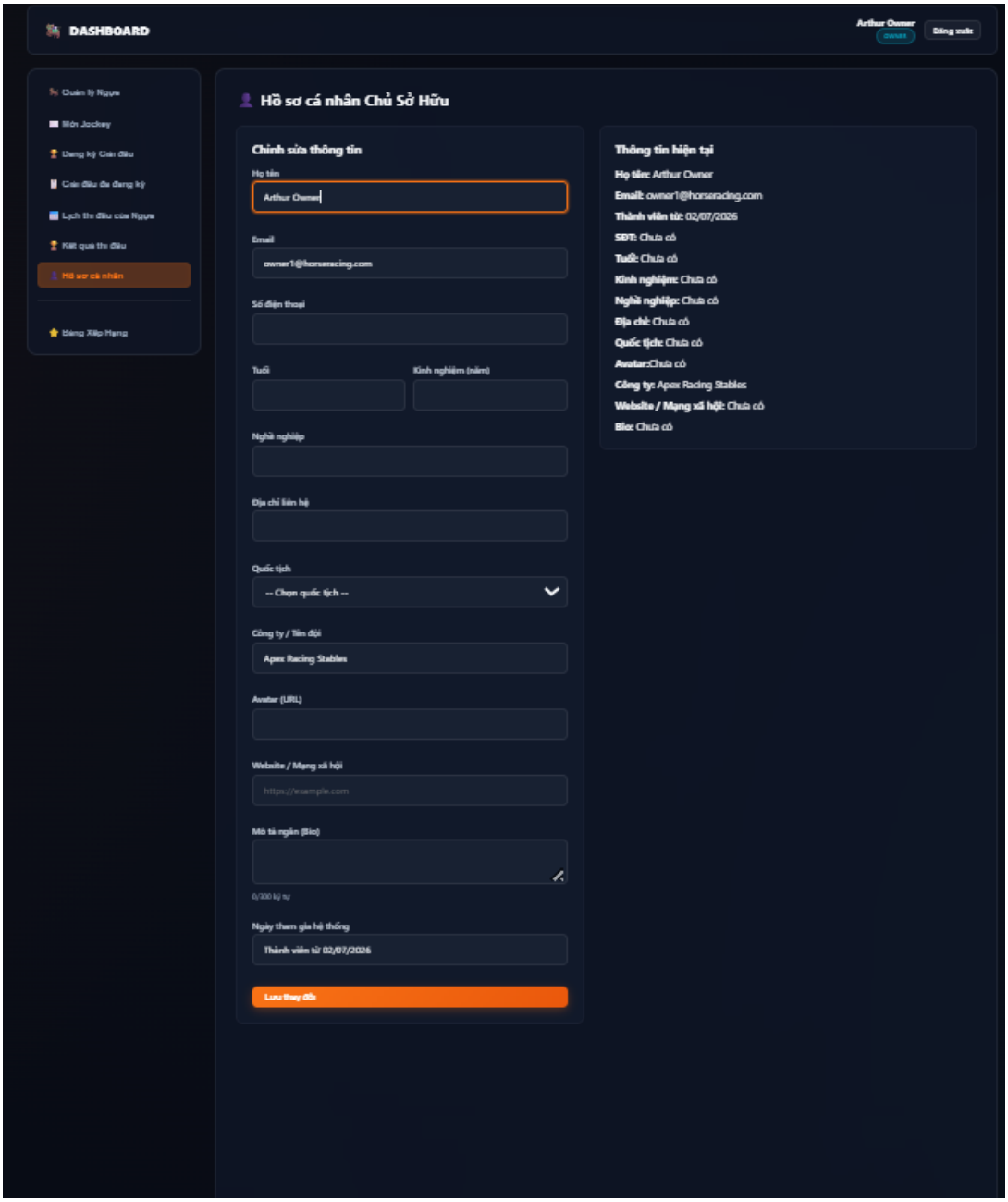


Figure 3.16: Owner Profile

Ranking

Role: Horse Owner

- **Function trigger:** Select “Bang xep hang”.
- **Function description:**
  - **Purpose:**
    1. View official rankings.
    2. Compare competition performance.
  - **Data processing:**

1. Retrieve horse rankings.
2. Retrieve jockey rankings.
3. Filter tournaments.

- **Function details:**
  - Display ranking positions.
  - Display accumulated scores.
  - Display tournament filters.
  - **Bussiness rule:** BR-24

Screen layout:

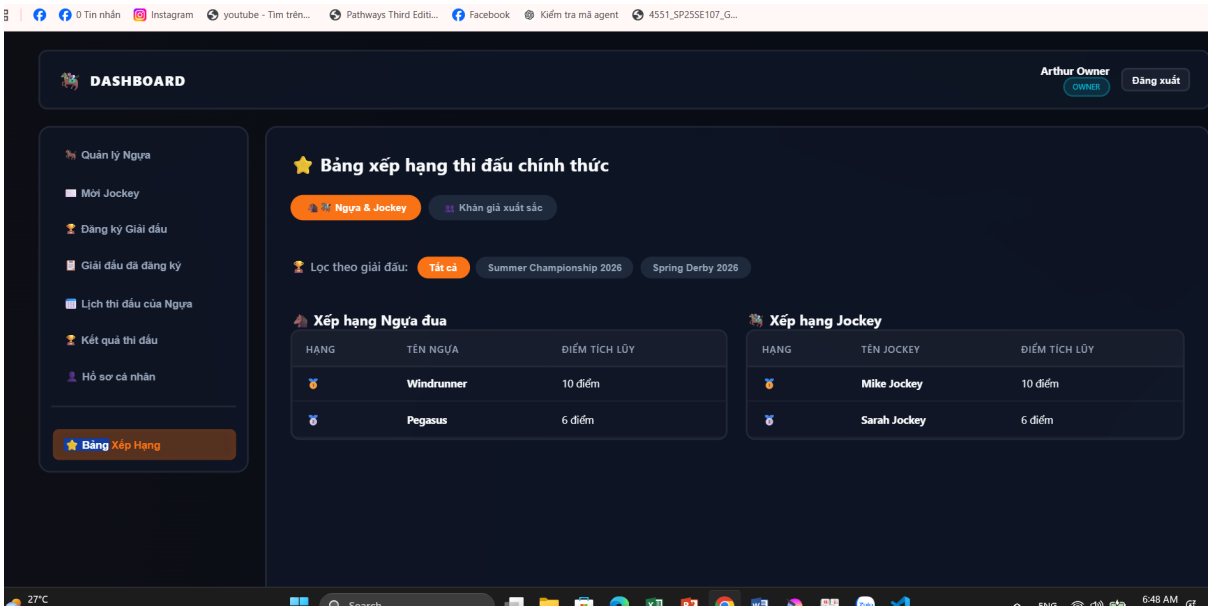


Figure 3.17: Ranking

Jockey Login

Role: Jockey

- **Function trigger:** The jockey enters account credentials to log in.
- **Function description:**
  - **Purpose:** Allow a jockey to authenticate and access the Jockey Dashboard where they can view invitations, schedules, results and manage their profile.
  - **Pre-conditions:** User has a registered jockey account. Internet connection available.
  - **Post-conditions:** Successful login redirects to the Jockey Dashboard. Failed login shows an error message.
- **Function details:**

- Enter username/email and password.
- Support for "Forgot password" flow.
- Role-based redirect to jockey views after authentication.

Screen layout:

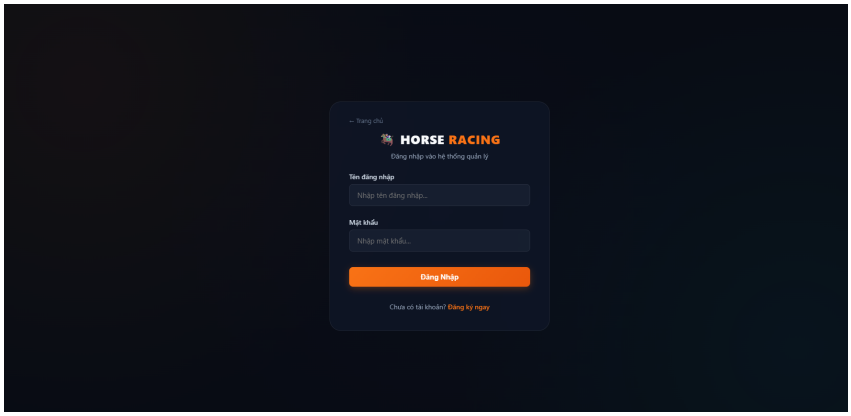


Figure 3.18: Jockey Login

Invitations Inbox

Role: Jockey

- **Function trigger:** Select "Invitations" or open dashboard inbox.
- **Function description:**
  - **Purpose:** Show invitations sent by horse owners with horse, tournament and message details so the jockey can accept or reject.
  - **Pre-conditions:** Owner has sent an invitation to this jockey.
  - **Post-conditions:** Accepting an invitation registers the jockey/horse pair for registration flows; rejecting notifies the owner.
- **Function details:**
  - List invitations with columns: Owner, Horse, Tournament, Message, Status, Actions.
  - Actions: Accept, Reject, View details.
  - Status badges: PENDING, ACCEPTED, REJECTED.

Screen layout:

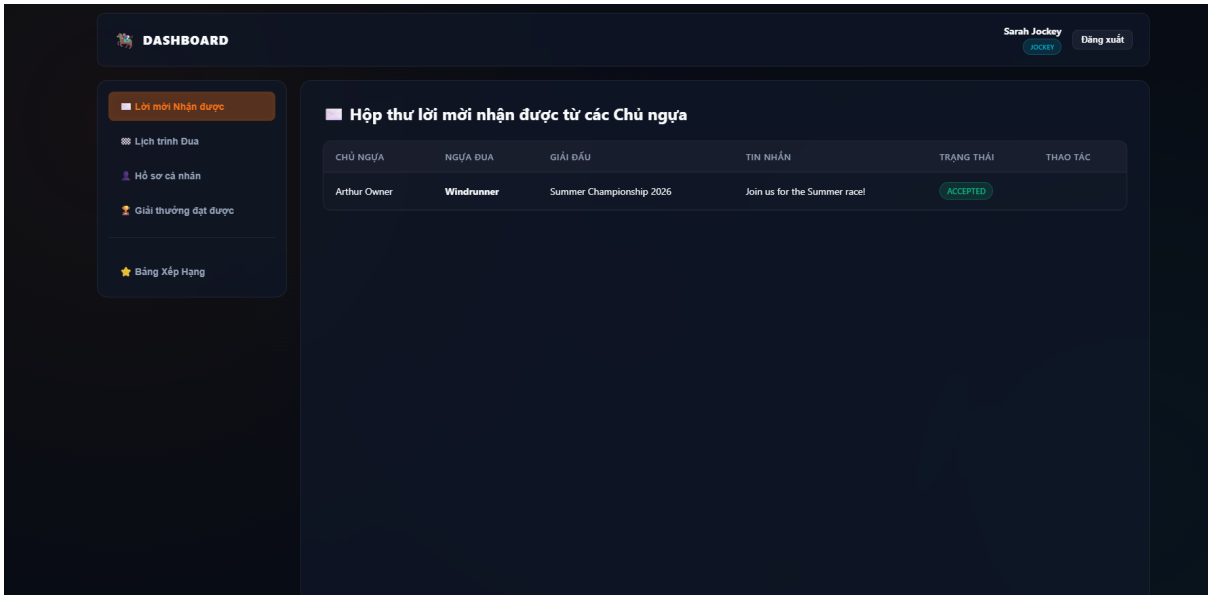


Figure 3.19: Jockey Invitations Inbox

Race Schedule

Role: Jockey

- **Function trigger:** Select "Schedule" or view upcoming races on dashboard.
- **Function description:**
  - **Purpose:** Display upcoming races the jockey is registered for, with date, time, distance, track and assigned horse.
  - **Pre-conditions:** Jockey has accepted invitations and registrations are processed.
  - **Post-conditions:** Jockey can prepare for races and view race details.
- **Function details:**
  - Show list grouped by tournament and date.
  - Provide quick links to race details and results (if available).

Screen layout:

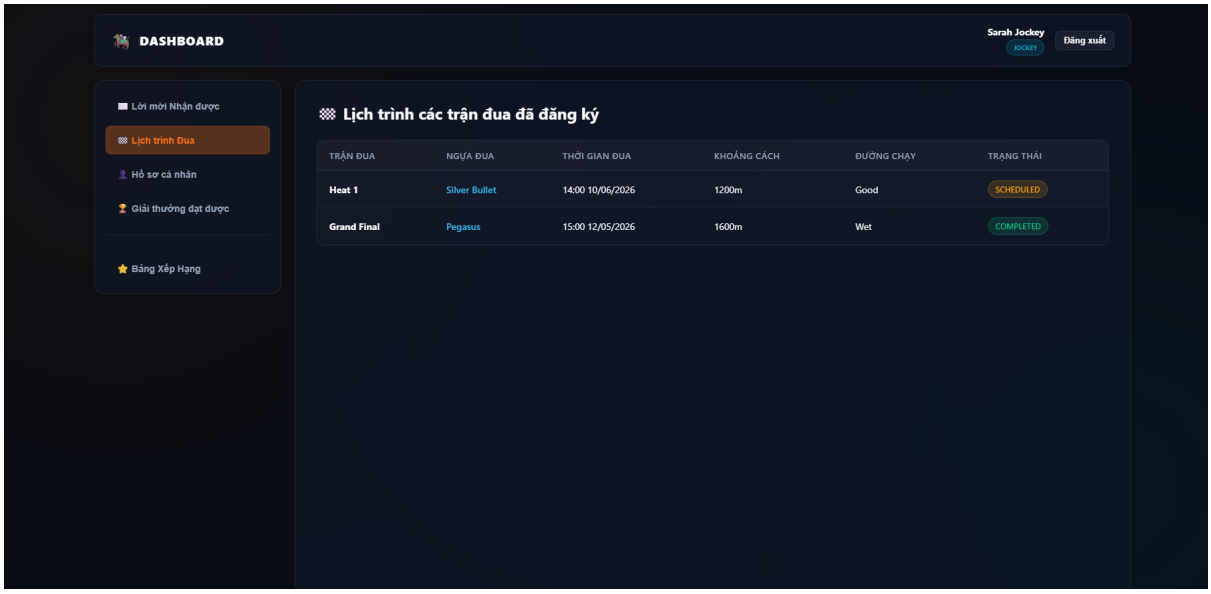


Figure 3.20: Jockey Upcoming Races

Jockey Profile

Role: Jockey

- **Function trigger:** Select "Profile" from the dashboard.
- **Function description:**
  - **Purpose:** View and update personal information: full name, email, phone, gender, weight, height, experience years and bio.
  - **Pre-conditions:** Jockey is authenticated.
  - **Post-conditions:** Changes are saved and shown across the application.
- **Function details:**
  - Editable fields with validation for required fields.
  - Avatar upload support.
  - Save changes with confirmation message.

Screen layout:

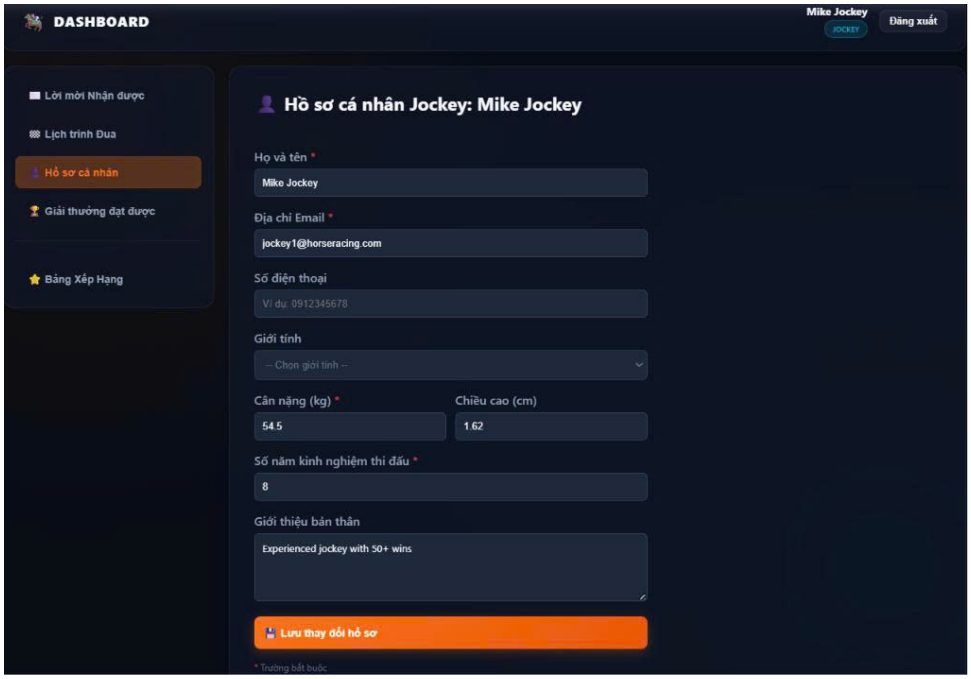


Figure 3.21: Jockey Personal Profile

Rewards & Achievements

Role: Jockey

- **Function trigger:** Select "Rewards" or "Achievements" from the dashboard.
- **Function description:**
  - **Purpose:** Show accumulated points, awards and historical results for the jockey across tournaments.
  - **Pre-conditions:** Race results have been recorded in the system.
  - **Post-conditions:** Jockey can filter awards by tournament and view details.
- **Function details:**
  - Summary cards for total points, best ranking and number of participations.
  - Table of awards per tournament with dates and points.

Screen layout:

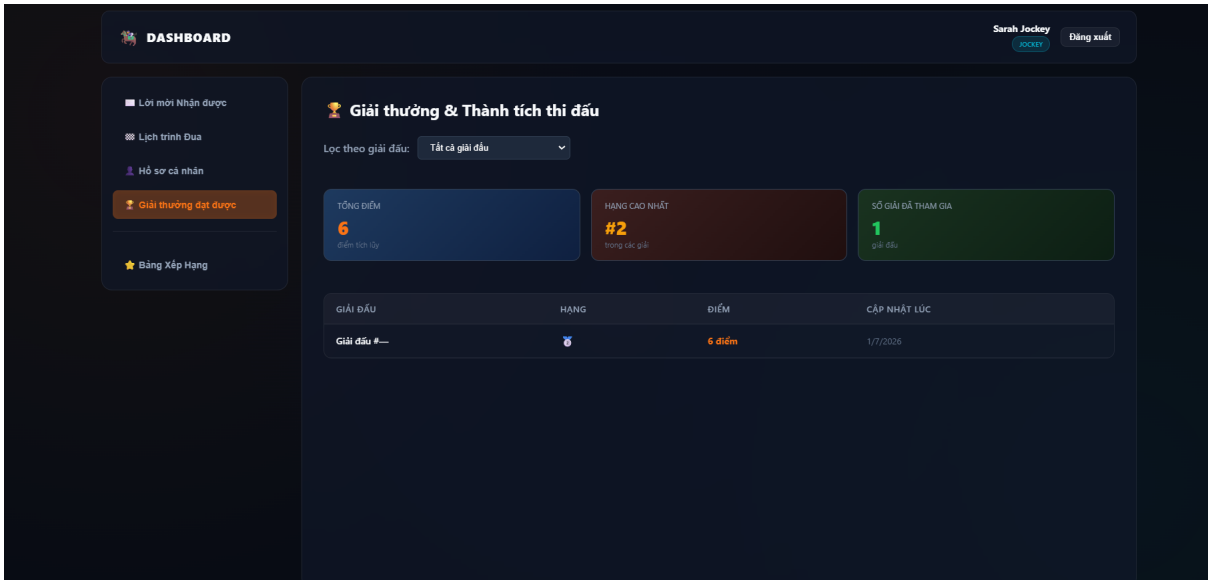


Figure 3.22: Jockey Rewards and Achievements

Leaderboard

Role: Jockey

- **Function trigger:** Select "Leaderboard" from the navigation.
- **Function description:**
  - **Purpose:** Display rankings for jockeys (and horses) with filtering by tournament.
  - **Pre-conditions:** Results and scores have been computed.
  - **Post-conditions:** User can view overall and tournament-specific leaderboards.
- **Function details:**
  - Two panels: Horse ranking and Jockey ranking.
  - Filter dropdown for tournaments and tabs for category selection.

Screen layout:

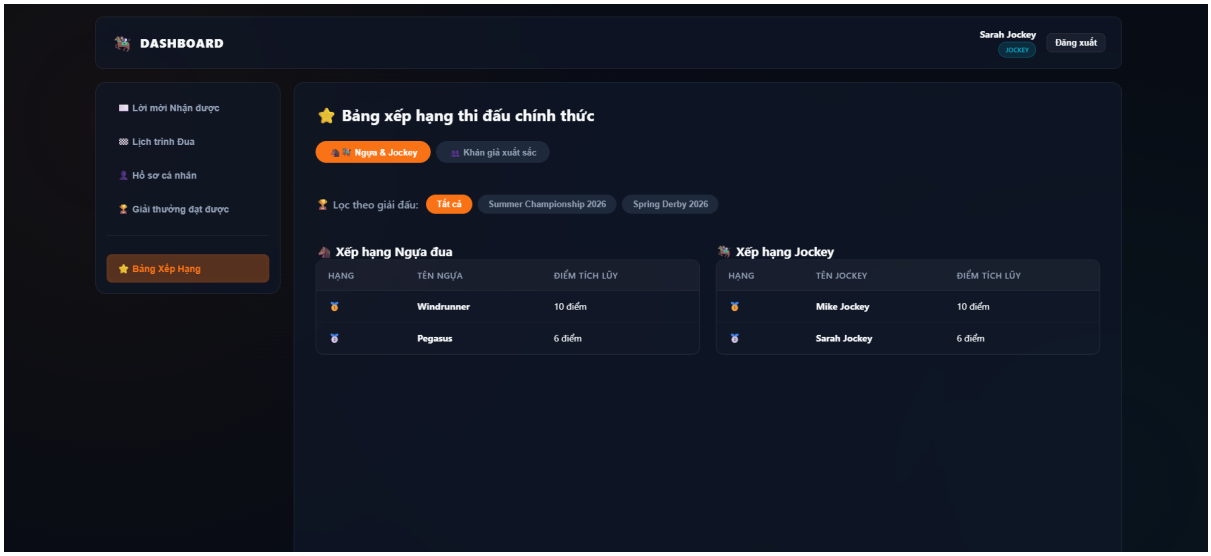


Figure 3.23: Leaderboard (Jockey and Horse Rankings)

Spectator Login

Role: Spectator

- **Function trigger:** The spectator enters account credentials to log in.
- **Function description:**
  - **Purpose:**
    1. Verify the spectator's identity.
    2. Grant access to spectator functionalities.
  - **Data processing:**
    1. The spectator enters username and password.
    2. The system validates credentials.
    3. The system identifies the SPECTATOR role.
    4. The system creates a login session.
- **Business rules:** BR-47

Screen layout:

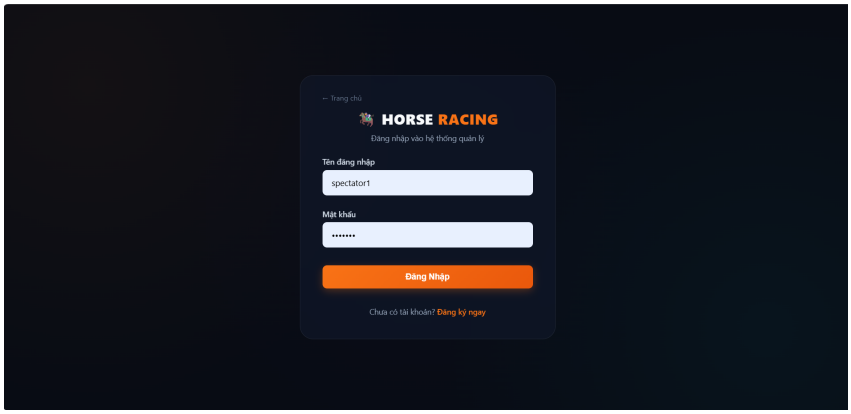


Figure 3.24: Spectator Login

Prediction Management

Role: Spectator

- **Function trigger:** The spectator navigates to the "Du doan Tran dua" (Predict Race) menu.
- **Function description:**
  - **Purpose:**
    1. Allow the spectator to submit a new race ranking prediction to earn reward points.
    2. Allow the spectator to view, edit, or delete their existing prediction history.
  - **Data processing:**
    1. **For creating a new prediction:**
       * The spectator selects a race, selects a horse, and chooses the predicted finishing rank from the dropdown menus.
       * The spectator clicks the "Gui du doan" button.
       * The system validates the prediction data and saves it to the database.
    2. **For managing prediction history:**
       * The system retrieves and displays the spectator's accumulated reward points and prediction history list (including Race, Horse, Predicted Rank, Result, and Actions).
       * If the race status is pending ("Cho ket qua"), the system allows the spectator to click "Sua" (Edit) or "Xoa" (Delete) to update their prediction.
       * If the race has ended, the actions are disabled ("Da khoa"), and the system updates the prediction status ("DUNG" / "SAI") along with the spectator's total reward points.
- **Business rules:** BR-48, BR-49, BR-50, BR-54, BR-55

Screen layout:

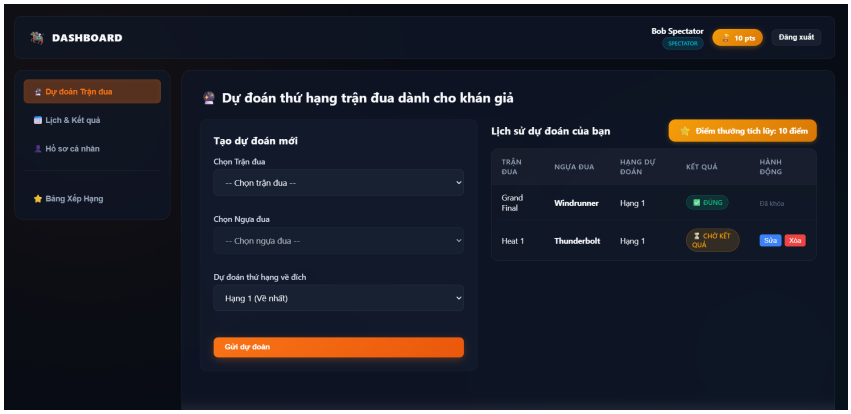


Figure 3.25: Spectator Prediction

View Schedule and Results

Role: Spectator

- **Function trigger:** The spectator navigates to the "Lich & Ket qua" (Schedule & Results) menu.
- **Function description:**
  - **Purpose:**
    1. Allow the spectator to view the list of upcoming race schedules.
    2. Allow the spectator to view the detailed results and rankings of completed races.
  - **Data processing:**
    1. **For upcoming race schedules ("Lich thi dau sap dien ra"):**
       * The system retrieves and displays scheduled races with status "SCHEDULED".
       * Information includes: Race name (e.g., Heat 1), referee name, date, time, distance, track condition, and the list of participating horses ("Danh sach tham gia") with their respective jockeys.
    2. **For completed race results ("Ket qua cac tran da ket thuc"):**
       * The system retrieves and displays finished races with status "HOAN THANH".
       * For each completed race (e.g., Grand Final), the spectator can expand to view the detailed ranking table including: Rank ("Hang"), Horse ("Ngua dua"), Jockey, Score ("Diem"), and Notes ("Ghi chu").
- **Business rules:** BR-51, BR-53, BR-56

Screen layout:

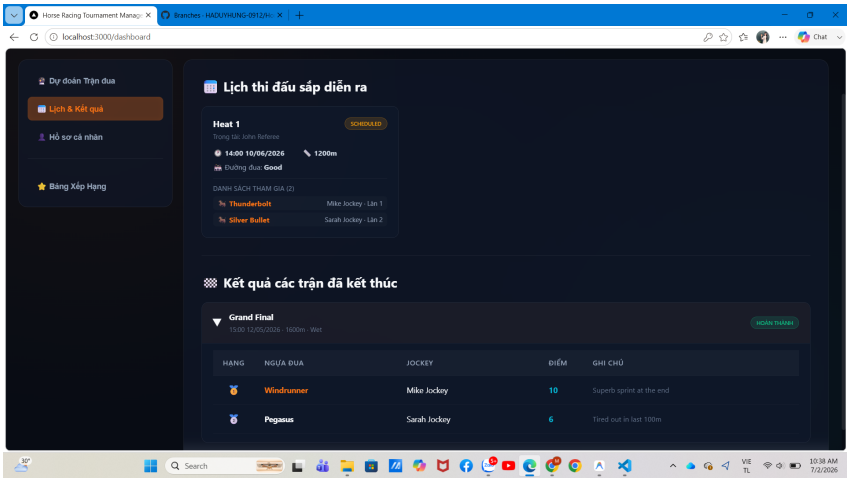


Figure 3.26: View Schedule and Results

Personal Profile

Role: Spectator

- **Function trigger:** The spectator navigates to the "Ho so ca nhan" (Personal Profile) menu.
- **Function description:**
  - **Purpose:**
    1. Allow the spectator to view and update their personal profile information.
    2. Display the spectator's prediction statistics, performance overview, and recent prediction history.
  - **Data processing:**
    1. **For updating profile information:**
       * The spectator can view and edit the following fields: Full Name ("Ho va Ten"), Email, Phone Number ("So dien thoai"), Avatar URL ("URL Anh dai dien"), Favorite Horse Breed ("Giong ngua yeu thich"), and Favorite Jockey ("Nai ngua yeu thich").
       * The spectator clicks the "Cap nhat thong tin" button.
       * The system validates the entered data and updates the profile details in the database.
    2. **For viewing statistics and history:**
       * The system retrieves and displays statistical insights ("Thong ke & Thanh tich") including: Current Rank ("Thu hang hien tai"), Total Predictions ("Tong so tran du doan"), Accuracy Rate ("Ty le chinh xac"), and Reward Points ("Diem thuong").
       * The system retrieves and lists the recent prediction history ("Lich su du doan gan day") with columns: Race Name ("Ten tran dua"), Selected Horse ("Ngua da chon"), Time ("Thoi gian"), Status ("Trang thai"), and Reward Points ("Diem thuong").

- **Business rules:** BR-52, BR-56

Screen layout:

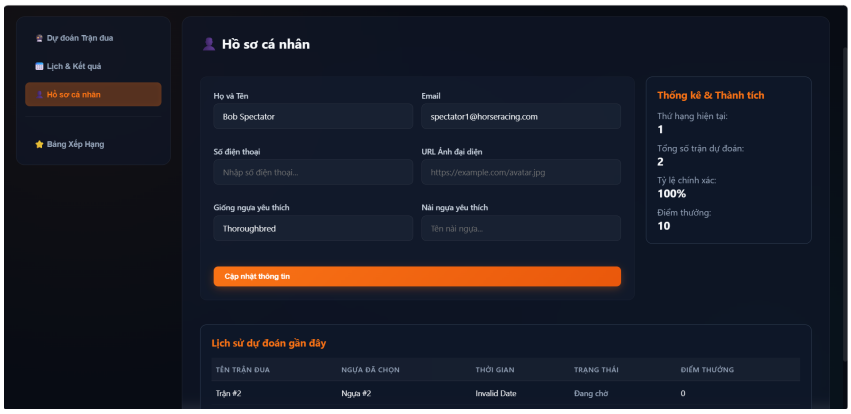


Figure 3.27: View Personal Profile

Leaderboard

Role: Spectator

- **Function trigger:** The spectator navigates to the "Bang Xep Hang" (Leaderboard) menu.
- **Function description:**
  - **Purpose:**
    1. Allow the spectator to view the official competition rankings of horses and jockeys, filtered by tournament.
    2. Allow the spectator to view the top 10 spectators with the highest prediction accuracy scores.
  - **Data processing:**
    1. **For Horse & Jockey Leaderboard ("Ngua & Jockey" tab):**
       * The spectator can filter rankings by tournament ("Loc theo giai dau") including options: All ("Tat ca"), Summer Championship 2026, or Spring Derby 2026.
       * The system retrieves and displays the Horse Ranking table ("Xep hang Ngua dua") including columns: Rank ("Hang"), Horse Name ("Ten ngua"), and Accumulated Points ("Diem tich luy").
       * The system retrieves and displays the Jockey Ranking table ("Xep hang Jockey") including columns: Rank ("Hang"), Jockey Name ("Ten jockey"), and Accumulated Points ("Diem tich luy").
    2. **For Top Spectators Leaderboard ("Khan gia xuat sac" tab):**
       * The system retrieves and displays the top 10 spectators with the highest prediction scores ("Top 10 Khan gia co diem du doan cao nhat").
       * The leaderboard table includes columns: Rank ("Hang"), Spectator Name ("Khan gia"), Favorite Horse Breed ("Giong ngua yeu thich"), and Accumulated Points ("Diem tich luy").

- **Business rules:** BR-53

Screen layout:

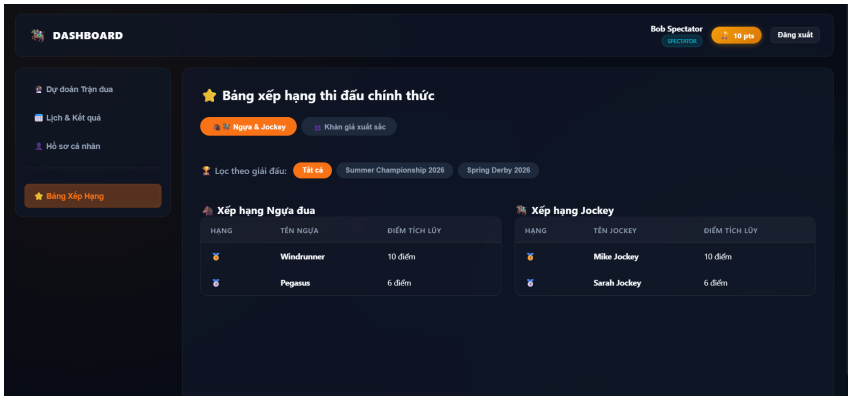


Figure 3.28: Official Competition Leaderboard for Horses and Jockeys

Screen layout:

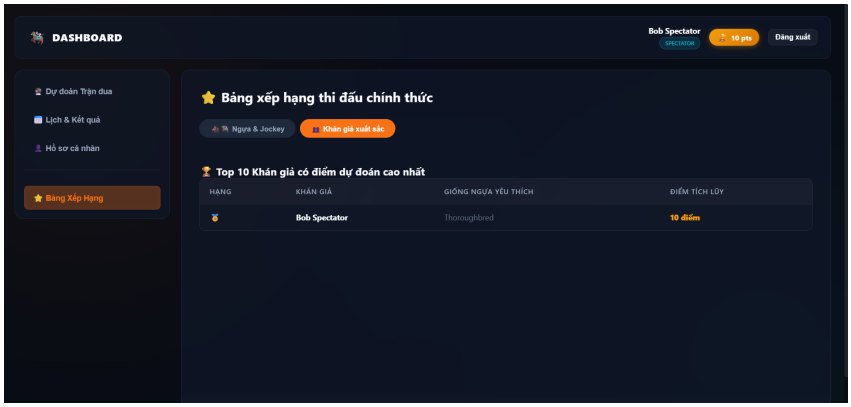


Figure 3.29: View Top 10 Spectators Prediction Leaderboard

Referee Login

Role: Race Referee

- **Function trigger:** The referee enters account credentials to log in.
- **Function description:**
  - **Purpose:**
    1. Verify the referee's identity.
    2. Grant access to referee functionalities.
  - **Data processing:**
    1. The referee enters username and password.
    2. The system validates credentials.
    3. The system identifies the REFEREE role.

4. The system creates a login session.

- **Function details:**
  - Successful login redirects to Referee Dashboard.
  - Invalid credentials display an error message.
  - Users without accounts may register.
- **Business rules:** BR-25

Screen layout:

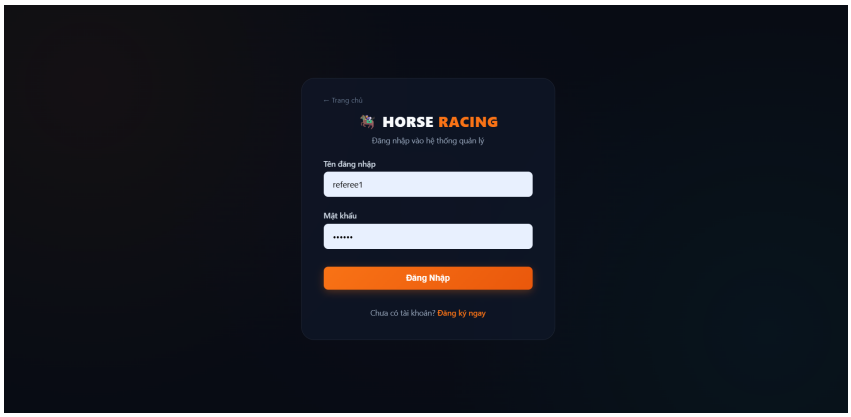


Figure 3.30: Referee Login

Assigned Race List

Role: Race Referee

- **Function trigger:** Select "Tran dua phan cong".
- **Function description:**
  - **Purpose:**
    1. View all races assigned to the referee for supervision.
    2. Navigate to race inspection and result entry actions.
  - **Data processing:**
    1. Retrieve races assigned to the current referee.
    2. Retrieve race details including time, distance, track condition, and number of participants.
    3. Retrieve current race status.
    4. Display available action buttons based on race status.
- **Function details:**
  - Display race name, scheduled time, distance, and track condition.
  - Display the number of participating horses.
  - Display current race status (e.g., SCHEDULED, COMPLETED).

- Provide “Kiem tra duong dua” and “Nhap ket qua” buttons for scheduled races.
- Provide “Sua ket qua” button for completed races.

- **Business rules:** BR-26, BR-36

Screen layout:

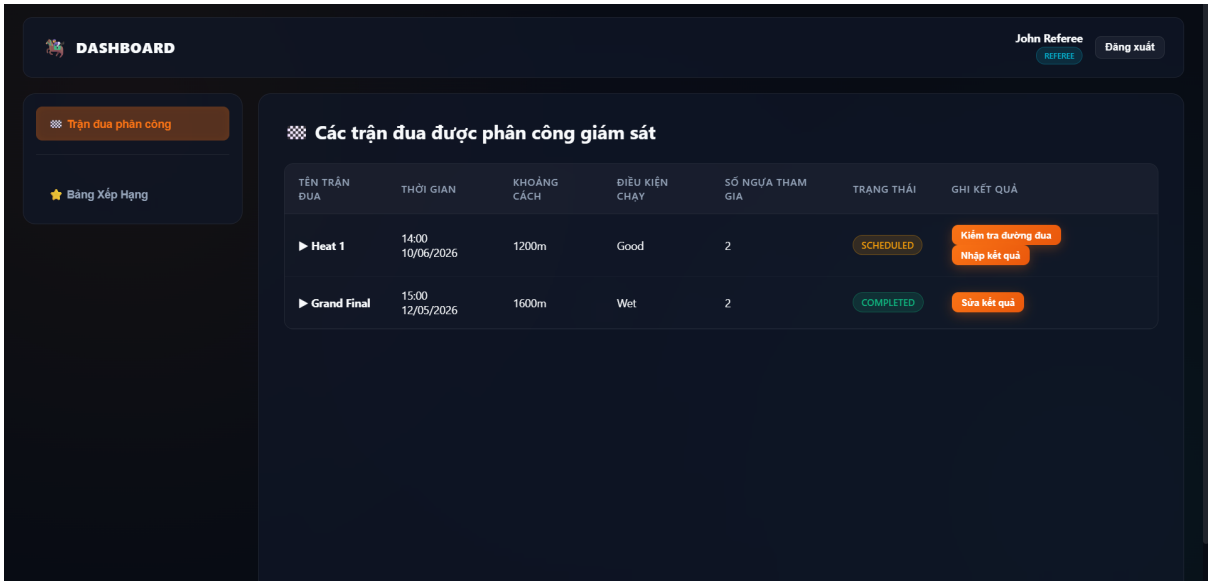


Figure 3.31: Assigned Race List

Race Inspection

Role: Race Referee

- **Function trigger:** Select “Kiem tra duong dua” on a scheduled race.
- **Function description:**
  - **Purpose:**
    1. Record pre-race track and environmental conditions.
    2. Assess horse health and fitness before the race.
  - **Data processing:**
    1. Enter current weather conditions.
    2. Enter track surface condition.
    3. Enter horse health assessment notes.
    4. Submit the inspection report to the system.
- **Function details:**
  - Input field for weather (e.g., Sunny, Rainy).
  - Input field for track condition (e.g., Good, Wet, Muddy).

- Text area for horse health assessment.
- "Gui bao cao kiem tra" button to submit the inspection report.
- "Huy" button to cancel without saving.

Screen layout:

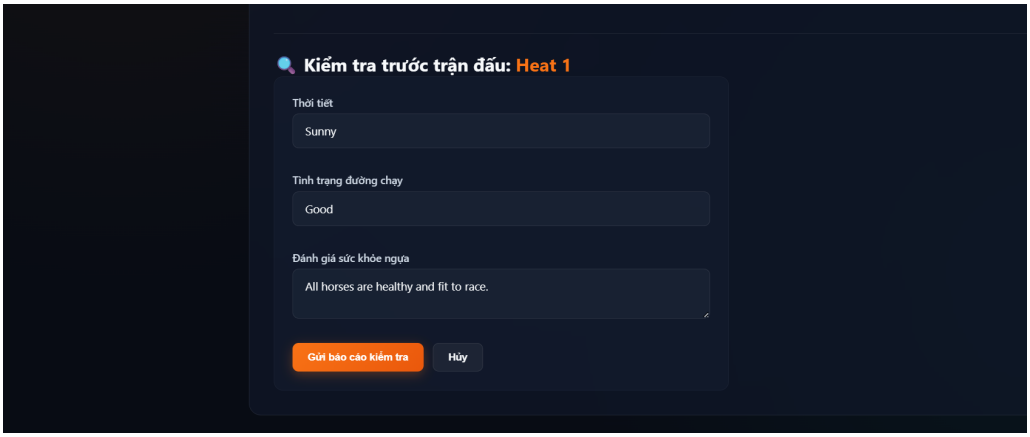


Figure 3.32: Race Inspection

Input Result Form

Role: Race Referee

- **Function trigger:** Select "Nhap ket qua" on a scheduled race.
- **Function description:**
  - **Purpose:**
    1. Record finishing positions and scores for each participating horse.
    2. Add race notes or remarks for each horse.
  - **Data processing:**
    1. Retrieve the list of participating horses for the selected race.
    2. Enter finishing position (Hang ve dich) for each horse.
    3. Enter bonus/penalty points (Diem cong/tru) for each horse.
    4. Enter optional race notes (Ghi chu) for each horse.
    5. Save the result to the system.
- **Function details:**
  - Display each participating horse by name.
  - Input field for finishing position per horse.
  - Input field for bonus/penalty score adjustment per horse.
  - Optional text field for race notes per horse.
  - "Luu ket qua cuoc dua" button to save results.
  - "Huy" button to cancel without saving.

- **Business rules:** BR-28, BR-29, BR-34

Screen layout:

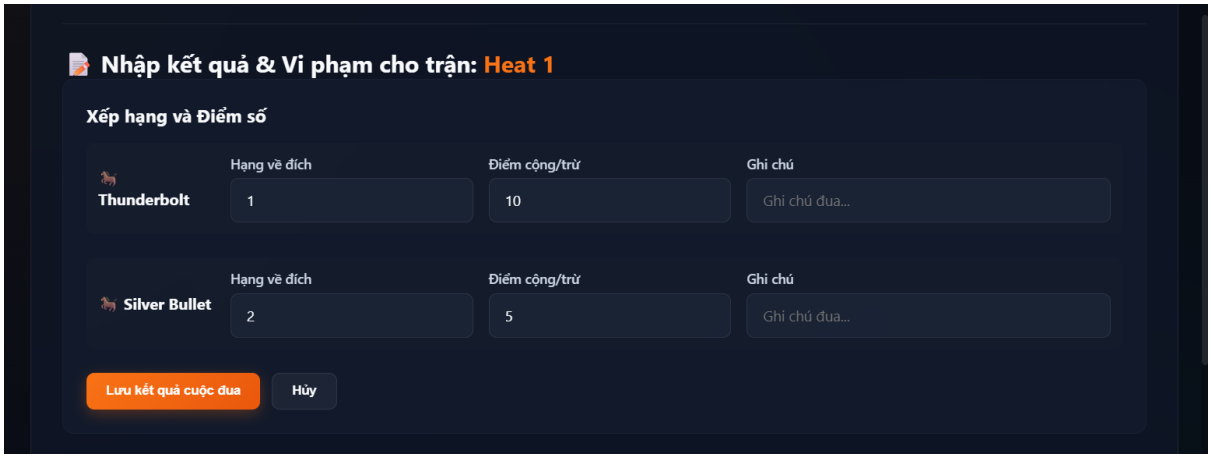


Figure 3.33: Input Result Form

Violation Report

Role: Race Referee

- **Function trigger:** Access violation reporting within the result entry workflow.
- **Function description:**
  - **Purpose:**
    1. Document rule violations observed during the race.
    2. Apply appropriate penalties to the offending horse.
  - **Data processing:**
    1. Select the offending horse from the race participant list.
    2. Enter a description of the violation.
    3. Select the penalty type (e.g., Canh cao – Warning, Phat tien – Fine, Cam thi dau – Disqualification).
    4. Enter the fine amount if applicable.
    5. Submit the violation report.
- **Function details:**
  - Dropdown to select the violating horse.
  - Text area for violation description.
  - Dropdown for penalty type selection.
  - Numeric input for fine amount.
  - "Bao cao vi pham" button to submit the report.

Screen layout:

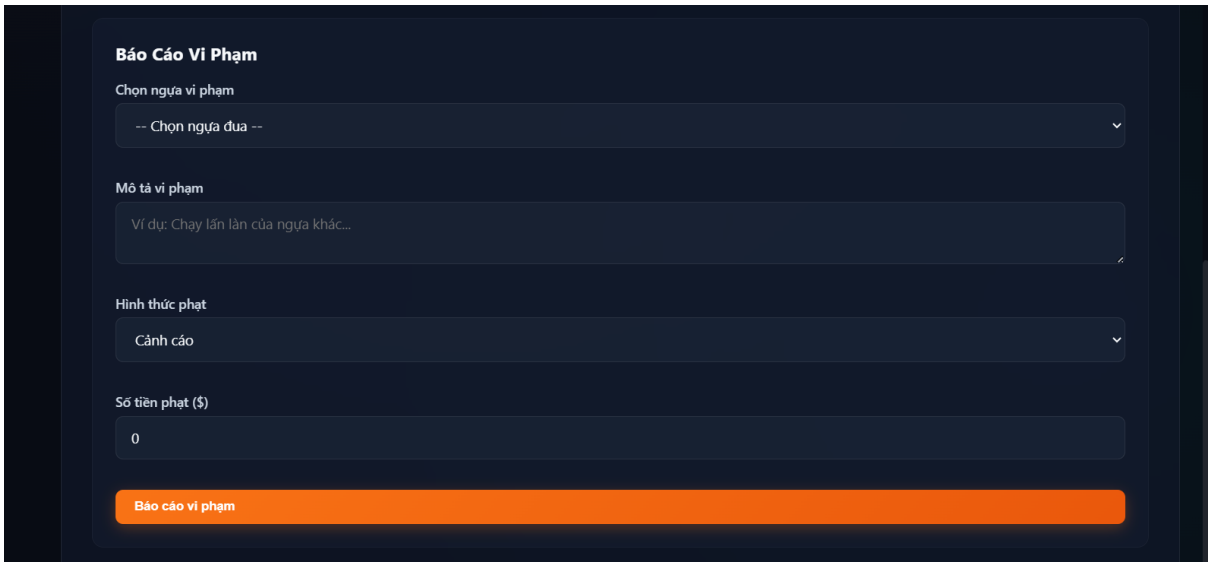


Figure 3.34: Violation Report

Result Confirmed

Role: Race Referee

- **Function trigger:** Administrator approves the submitted result; referee views the updated race list.
- **Function description:**
  - **Purpose:**
    1. Display confirmed race results after administrator approval.
    2. Allow the referee to resubmit corrected results if needed.
  - **Data processing:**
    1. Retrieve updated race status from the system.
    2. Display the race with status RESULTS_ENTERED or COMPLETED.
    3. Provide result confirmation and editing options based on status.
- **Function details:**
  - Display race list with updated statuses after administrator review.
  - "Xac nhan ket qua chinh thuc" button available for races with RESULTS_ENTERED status.
  - "Sua ket qua" button available for COMPLETED races requiring correction.
  - "Nhap ket qua" button to re-enter results if necessary.
- **Business rules:** BR-29, BR-34, BR-35

Screen layout:

Figure 3.35: Result Confirmed

Ranking Board

Role: Race Referee

- **Function trigger:** Select "Bang Xep Hang".
- **Function description:**
  - **Purpose:**
    1. View official tournament standings for horses, jockeys, and spectators.
    2. Filter rankings by tournament.
  - **Data processing:**
    1. Retrieve horse and jockey accumulated scores from confirmed race results.
    2. Retrieve spectator prediction scores.
    3. Filter rankings by selected tournament.
    4. Display ranked lists for each category.
- **Function details:**
  - Toggle tabs between "Ngua & Jockey" and "Khan gia xuat sac" views.
  - Filter rankings by tournament (e.g., All, Summer Championship 2026, Spring Derby 2026).
  - Display horse ranking table with position, horse name, and accumulated points.
  - Display jockey ranking table with position, jockey name, and accumulated points.
- **Business rules:** BR-35

Screen layout:

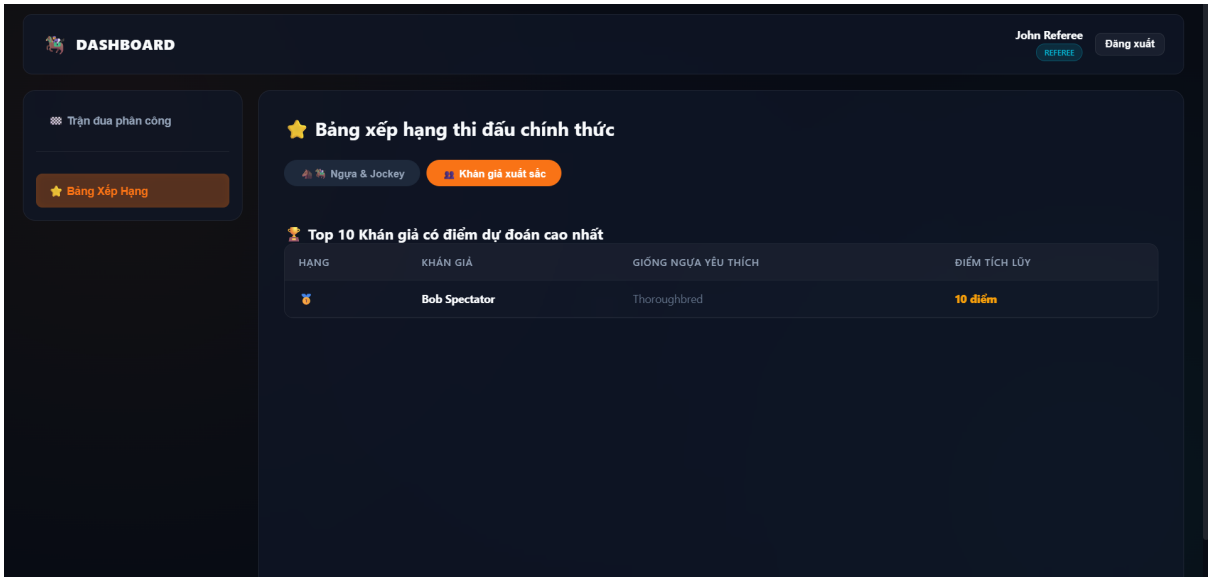


Figure 3.36: Ranking Board

Leaderboard

Role: Race Referee

- **Function trigger:** Select the "Khan gia xuat sac" tab within the Ranking Board.
- **Function description:**
  - **Purpose:**
    1. View the top spectators ranked by their prediction accuracy.
    2. Recognize outstanding spectator engagement.
  - **Data processing:**
    1. Retrieve spectator prediction scores from the system.
    2. Sort spectators by accumulated prediction points in descending order.
    3. Display the top 10 spectators.
- **Function details:**
  - Display ranking position, spectator name, preferred horse breed, and accumulated points.
  - Highlight top-ranked spectator with a medal icon.
  - Read-only view accessible to the referee for reference.
- **Business rules:** BR-35

Screen layout:

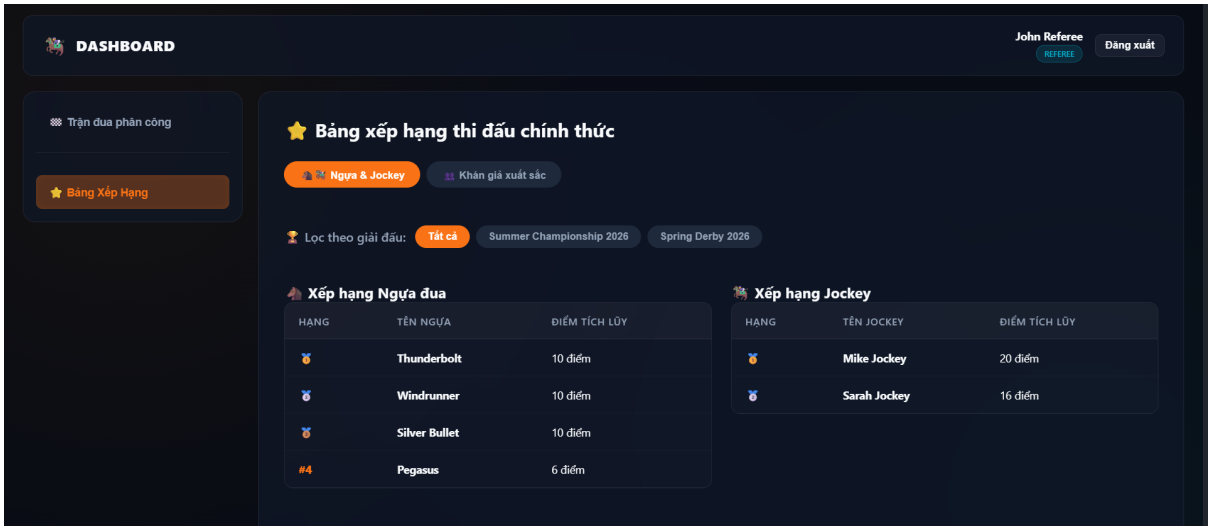


Figure 3.37: Leaderboard

Admin Login

Role: Admin

- **Function trigger:** The admin enters account credentials and selects the login button.
- **Function description:**
  - **Purpose:**
    1. Verify the administrator identity.
    2. Provide access to administrative functions.
  - **Data processing:**
    1. The admin enters username and password.
    2. The system validates the credentials.
    3. The system identifies the ADMIN role.
    4. The system creates an admin session.
- **Function details:**
  - Successful login redirects to the Admin Dashboard.
  - Invalid credentials show an error message.
  - Admin can access system management screens after login.
- **Business rules:** BR-01, BR-10

Screen layout:

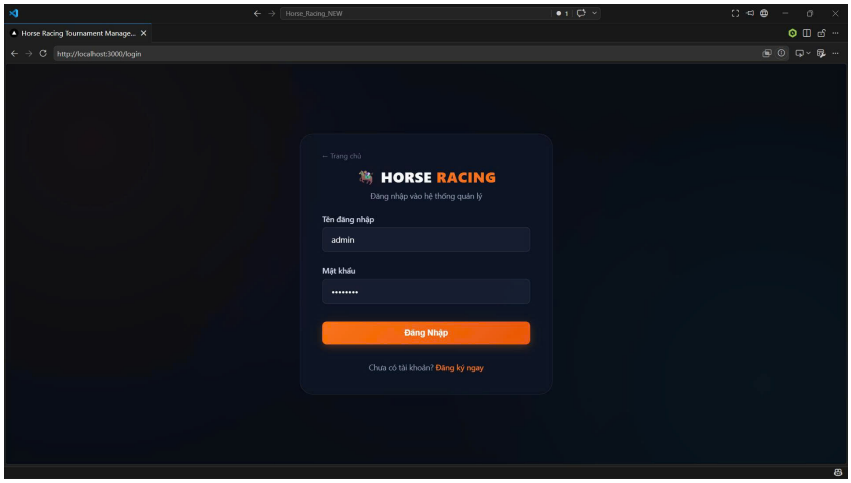


Figure 3.38: Admin Login

System Overview

Role: Admin

- **Function trigger:** Select "Tong quan he thong" from the admin menu.
- **Function description:**
  - **Purpose:**
    1. Provide a summary of system status.
    2. Display key metrics and role distributions.
  - **Data processing:**
    1. Retrieve user counts by role.
    2. Retrieve tournament and race statistics.
    3. Retrieve award and registration metrics.
- **Function details:**
  - Display total active users, tournaments, races, horses, and awards.
  - Show distribution of users by role.
  - Show tournament status and prediction statistics.
- **Business rules:** BR-11, BR-12

Screen layout:

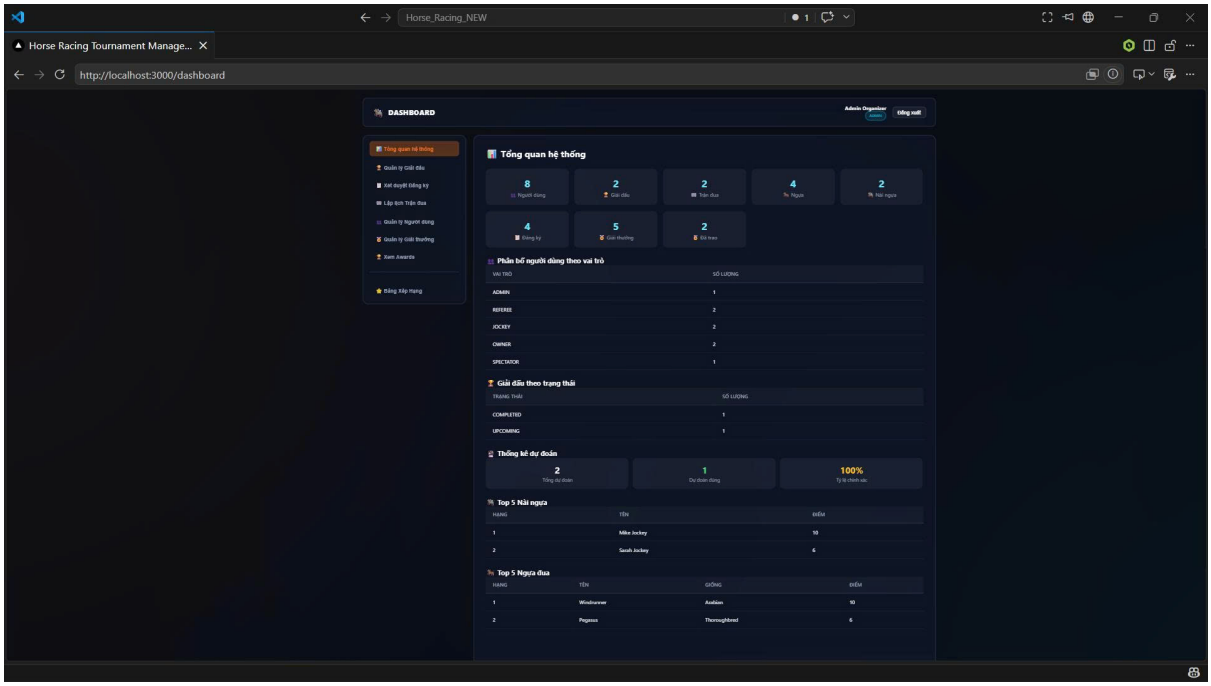


Figure 3.39: System Overview

Manage Tournaments

Role: Admin

- **Function trigger:** Select "Quan ly Giai dau".
- **Function description:**
  - **Purpose:**
    1. Create and manage tournament information.
    2. Add new rounds to tournaments.
  - **Data processing:**
    1. Enter tournament name, description, dates, and location.
    2. Save tournament details.
    3. Add competition rounds with sequence order.
- **Function details:**
  - Create new tournaments.
  - Edit or delete tournaments.
  - Manage the number and order of rounds.
  - Display current tournament list and status.
- **Business rules:** BR-02, BR-03, BR-10, BR-11

Screen layout:

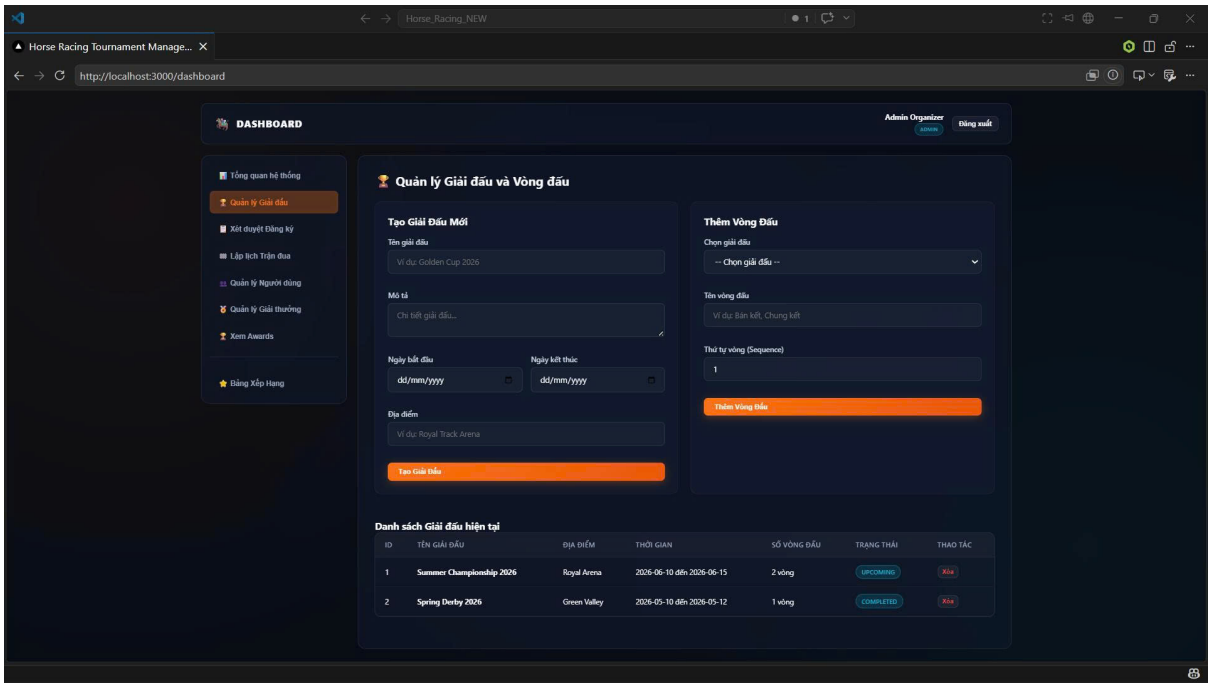


Figure 3.40: Manage Tournaments

Schedule Races

Role: Admin

- **Function trigger:** Select "Lap lich Tran dua".
- **Function description:**
  - **Purpose:**
    1. Schedule race heats and assign race details.
    2. Organize race lanes and pairs.
  - **Data processing:**
    1. Choose the tournament round.
    2. Enter race name, date, distance, and conditions.
    3. Select approved horse-jockey pairs.
    4. Assign lane numbers for each pair.
- **Function details:**
  - Create and manage races.
  - Assign horses and jockeys to lanes.
  - Display current race schedule and status.
- **Business rules:** BR-04 ,BR-05, BR-06, BR-10, BR-11

Screen layout:

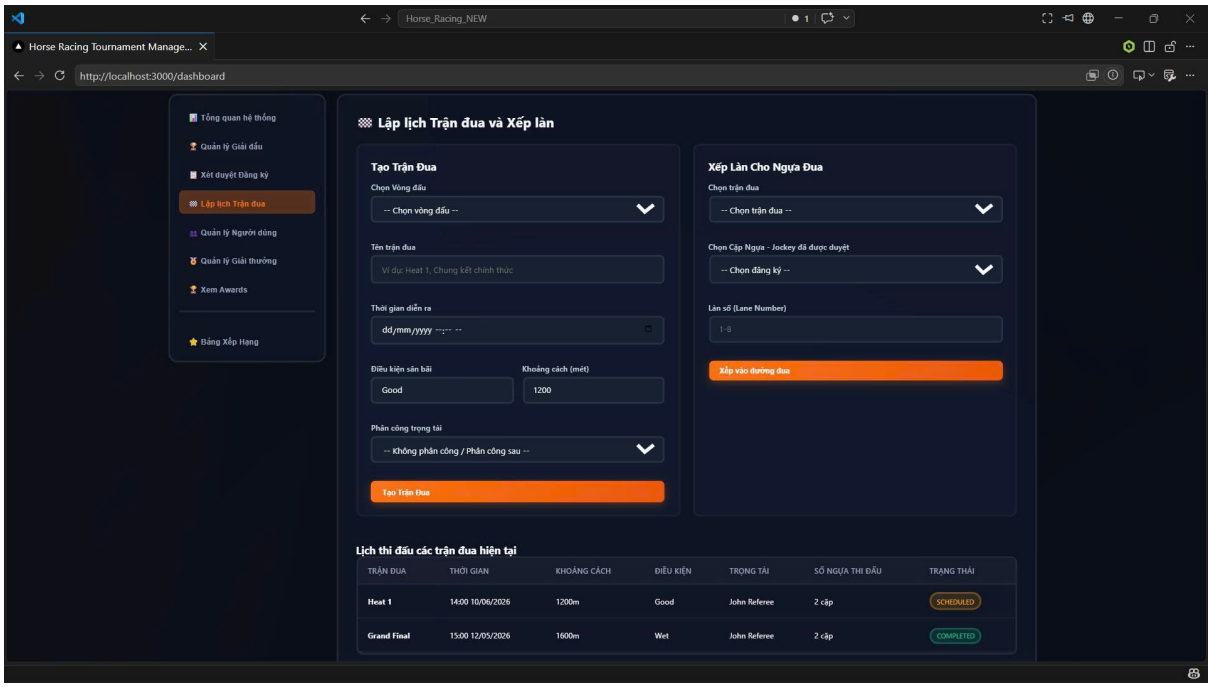


Figure 3.41: Schedule Races

Manage Rewards

Role: Admin

- **Function trigger:** Select "Quan ly Giai thuong".
- **Function description:**
  - **Purpose:**
    1. Define prizes for tournament positions.
    2. Manage award distributions.
  - **Data processing:**
    1. Select the tournament.
    2. Enter prize rank, title, value, and description.
    3. Save reward information.
- **Function details:**
  - Add new awards for tournaments.
  - Edit or delete existing awards.
  - Display award list and delivery status.
- **Business rules:** BR-08, BR-10, BR-11

Screen layout:

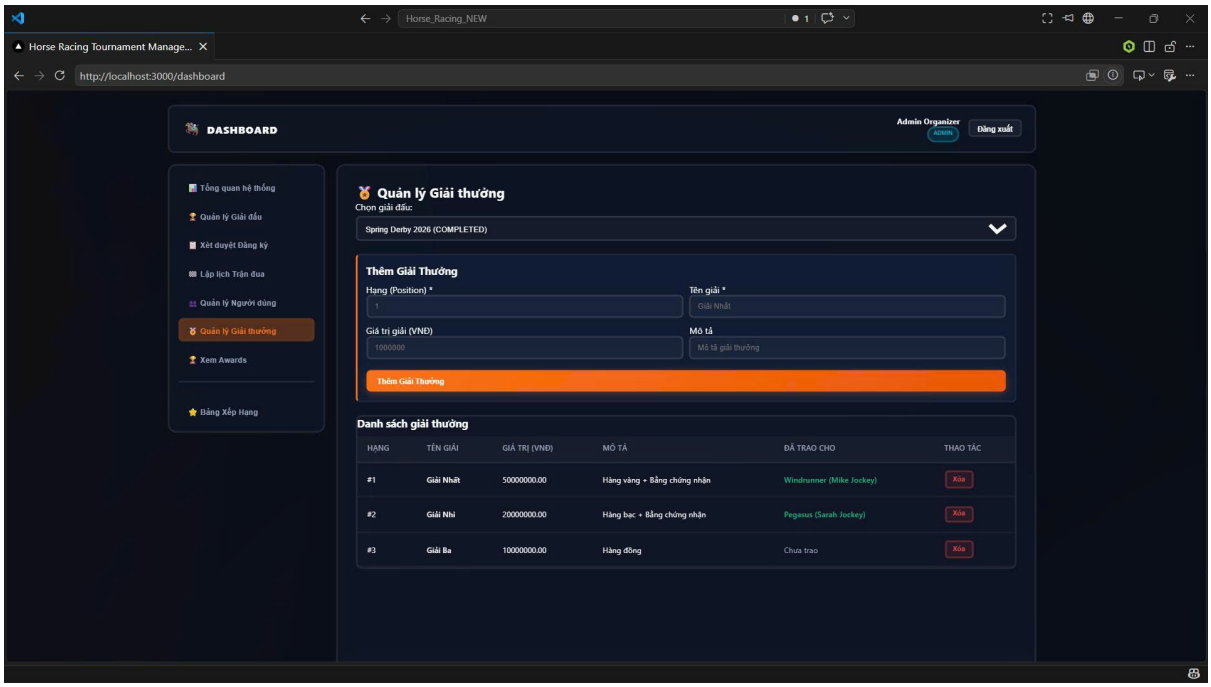


Figure 3.42: Manage Rewards

User Management

Role: Admin

- **Function trigger:** Select "Quan ly Nguoi dung".
- **Function description:**
  - **Purpose:**
    1. Manage system users and roles.
    2. Control access and account status.
  - **Data processing:**
    1. Retrieve user accounts.
    2. Display role and status information.
    3. Perform account lock or unlock actions.
- **Function details:**
  - Display user list with roles and statuses.
  - Lock or unlock user accounts.
  - Filter users by role.
- **Business rules:** BR-09, BR-10, BR-11

Screen layout:

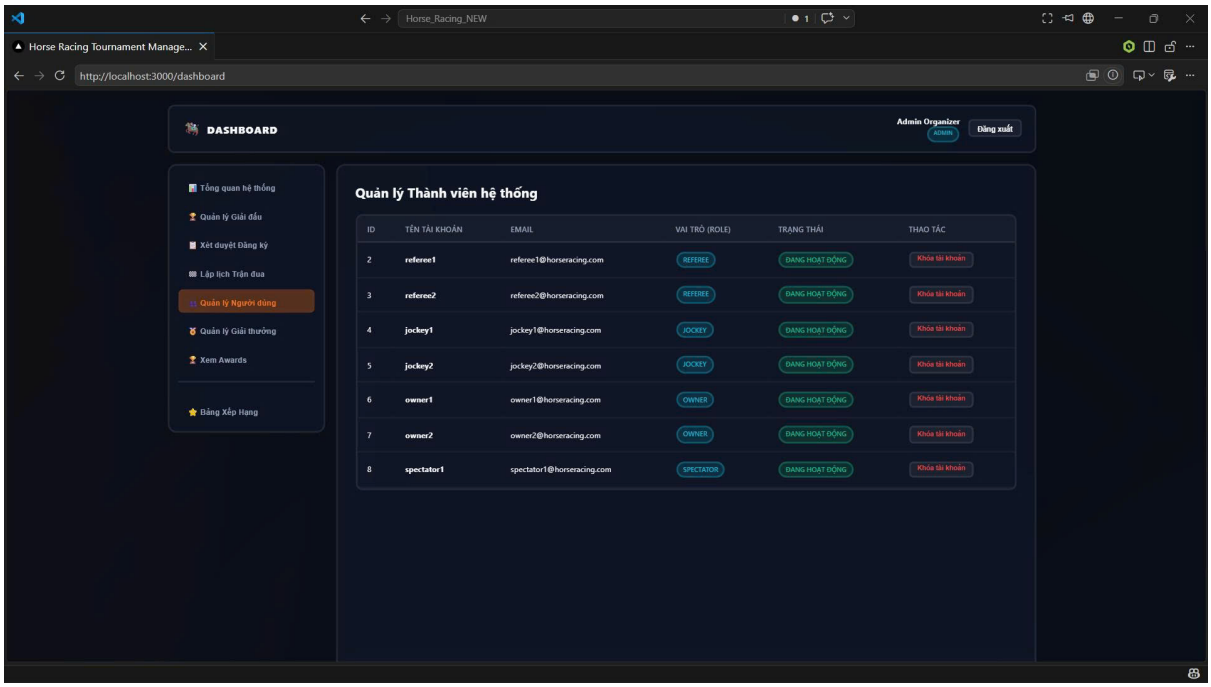


Figure 3.43: User Management

Confirm Registrations

Role: Admin

- **Function trigger:** Select "Xet duyet Dang ky".
- **Function description:**
  - **Purpose:**
    1. Review and confirm registration requests.
    2. Approve or reject tournament entries.
  - **Data processing:**
    1. Retrieve pending registrations.
    2. Display horse, jockey, and tournament details.
    3. Update registration approval status.
- **Function details:**
  - Show pending registration entries.
  - Approve or reject registrations.
  - Display current approval status.
- **Business rules:** BR-10, BR-11

Screen layout:

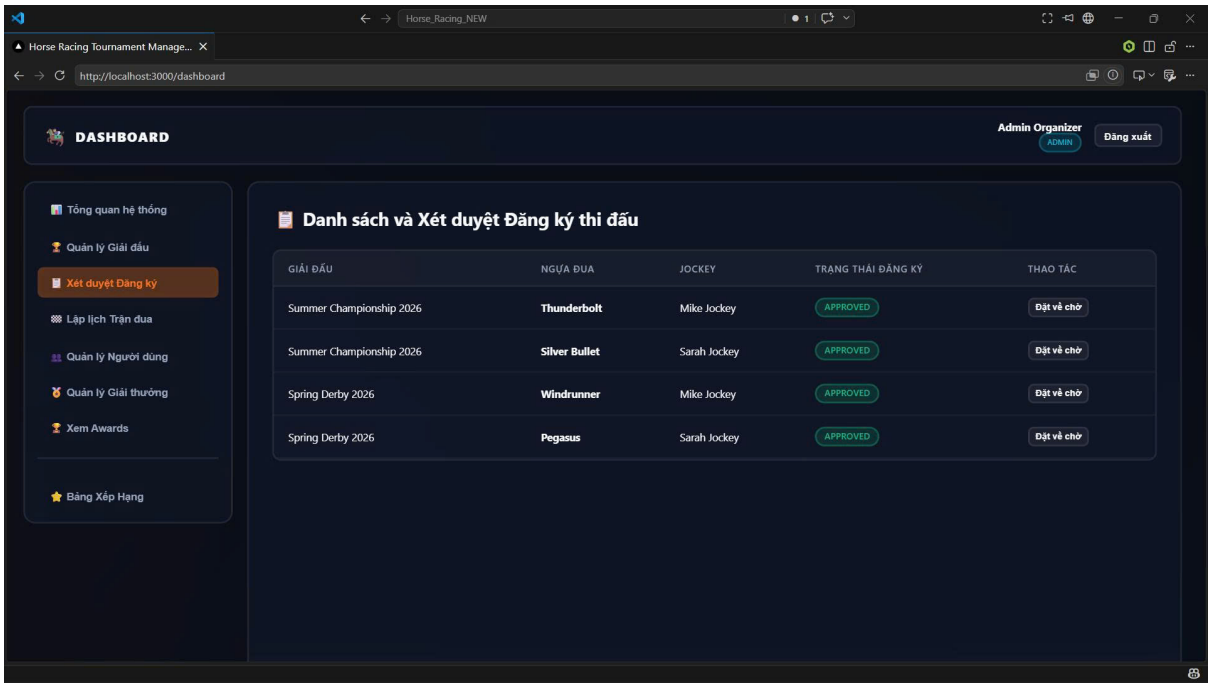


Figure 3.44: Confirm Registrations

Leaderboard

Role: Admin

- **Function trigger:** Select "Xem Awards" or "Bang Xep Hang".
- **Function description:**
  - **Purpose:**
    1. Display the official leaderboard.
    2. Review award winners and rankings.
  - **Data processing:**
    1. Retrieve ranking data for horses and jockeys.
    2. Retrieve award recipient details.
- **Function details:**
  - Display award and ranking summaries.
  - Show top positions by horse and jockey performance.
- **Business rules:** BR-12, BR-11

Screen layout:

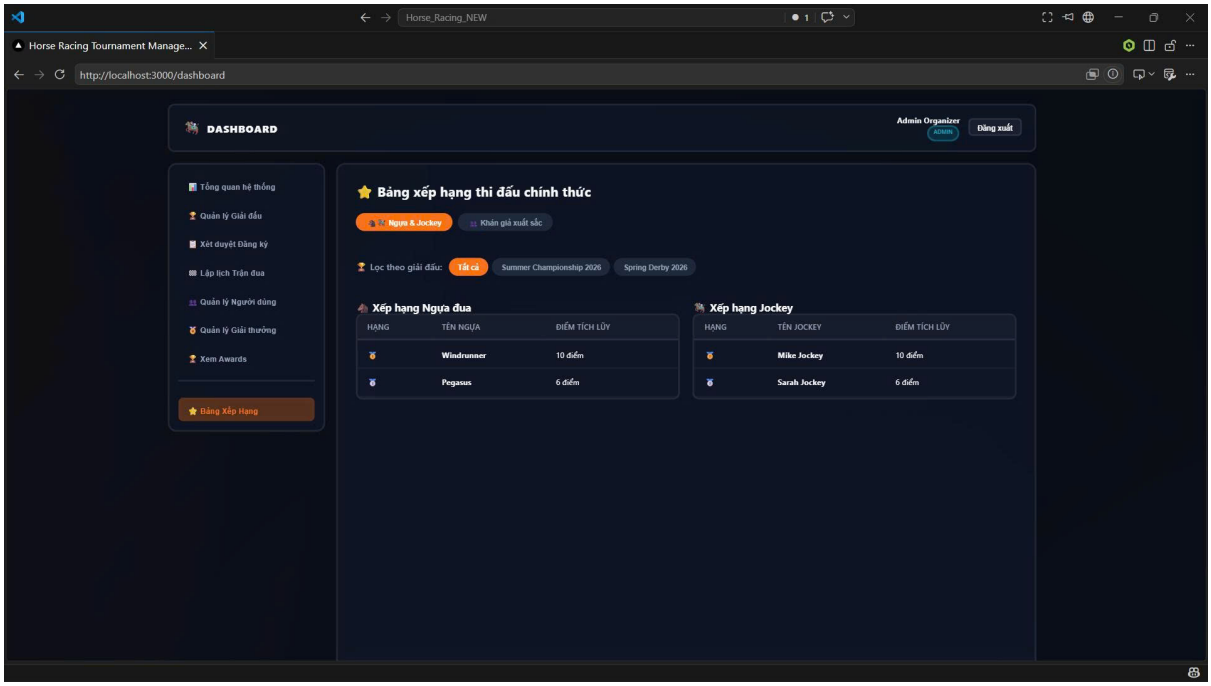


Figure 3.45: Leaderboard

3.4. Requirement Appendix

3.4.1 Business Rules

ID	Rule Definition
BR-01	Only an authenticated Administrator with the ADMIN role may access the admin management screens and perform administrative operations.
BR-02	The Administrator must create or update a tournament only when all required information, including name, dates, location, and round structure, has been provided.
BR-03	A tournament cannot be deleted while it still contains active races, registered participants, or reward definitions that depend on it.
BR-04	Every race must be associated with an existing tournament round, and its scheduled date and time must remain within the valid round timeframe.
BR-05	The Administrator must assign a unique lane number to each horse-jockey pair within the same race heat.
BR-06	The system must prevent the Administrator from creating duplicate lane assignments or scheduling a horse into overlapping races.
BR-07	A referee may be assigned to a race only when the referee is available and has no conflicting assignment within the allowed time window.
BR-08	The Administrator may define rewards only for an existing tournament, and prize ranks must be unique within that tournament.
BR-09	User account management by the Administrator must validate role assignment and allow create, update, lock, or delete actions only for valid account records.
BR-10	The system must reject invalid administrative input and display a clear error message when tournament, race, reward, or user data is incomplete or inconsistent.
BR-11	A successful create, update, or delete action by the Administrator must immediately refresh the relevant list or dashboard so the latest system state is visible.
BR-12	The Administrator must be able to review current tournament status, race schedules, rewards, and user records from the admin overview and management screens.

BR-13	Only an authenticated Horse Owner with the OWNER role may access the owner dashboard and perform owner-related operations.
BR-14	The Horse Owner may register a horse only when all required horse information, including name, age, breed, and gender, has been provided.
BR-15	A horse record may be updated or deleted only by its owner and only when the horse is not currently involved in an active or locked registration.
BR-16	The Horse Owner may send a jockey invitation only when both the selected jockey and the selected horse are valid and belong to an eligible tournament context.
BR-17	A jockey invitation must contain valid invitation details and may be accepted or rejected by the recipient before it can be used for tournament registration.
BR-18	Tournament registration is allowed only when the Horse Owner has an accepted jockey invitation and the selected horse-jockey pair is valid for the chosen tournament.
BR-19	The system must prevent duplicate tournament registrations for the same horse-jockey pair in the same tournament.
BR-20	The Horse Owner may view the current registration status and approval result of each submitted registration request from the registered tournaments screen.
BR-21	The Horse Owner may view upcoming race schedules and completed race results for owned horses, including rankings, scores, remarks, and violations.
BR-22	The Horse Owner may update personal profile information, including contact and organization details, only after valid input is provided and saved successfully.
BR-23	The system must display clear success or error messages after each owner action, such as horse registration, invitation sending, registration submission, or profile update.
BR-24	The Horse Owner must be able to view rankings and performance summaries for horses and jockeys through the ranking page.

BR-25	Only an authenticated Race Referee with the REFEREE role may access the referee dashboard and manage assigned race supervision tasks.
BR-26	The Race Referee may view only the races assigned to them and must be able to inspect race information such as time, distance, track condition, and participant count.
BR-27	A race inspection report may be submitted only when the referee provides valid weather, track, and horse health information for that race.
BR-28	The referee may enter race results only for races that are currently scheduled or available for result submission, and each result must be tied to a valid race record.
BR-29	Result entry must include a finishing position and valid score adjustment for each participating horse before the record can be saved.
BR-30	The referee may submit a violation report only when the violating horse, violation description, penalty type, and applicable fine amount (if required) are provided.
BR-31	The system must prevent duplicate or conflicting result submissions for the same race after the result has already been accepted or confirmed.
BR-32	When a result is corrected or updated, the system must preserve the revision history and display the latest confirmed state to the referee and administrator.
BR-33	The referee must receive a confirmation message when an inspection report, race result, or violation report is successfully submitted.
BR-34	The system must reject invalid referee input and display clear error messages when required inspection or result data is missing or inconsistent.
BR-35	The referee may review the status of race results after administrator approval and may resubmit corrected results when necessary.
BR-36	The referee dashboard must display the latest assigned race status, result state, and available actions for each assigned race.

BR-37	Only an authenticated Jockey with the JOCKEY role may access the jockey dashboard and manage assigned race-related activities.
BR-38	The Jockey may accept or decline an invitation only when the invitation is valid, active, and still pending.
BR-39	A jockey profile may be updated only when the provided information is complete and valid, including licensing and contact details.
BR-40	The system must prevent a jockey from accepting duplicate invitations for the same horse-tournament context.
BR-41	A jockey registration request must be linked to a valid accepted invitation before it can be considered for tournament participation.
BR-42	The system must display the current invitation status and registration outcome to the jockey in the dashboard.
BR-43	The jockey can view upcoming races and assigned schedules only for the tournaments or races in which they are officially registered.
BR-44	The system must reject invalid jockey input and show a clear message when required profile or invitation data is missing.
BR-45	A successful profile update or invitation response must immediately refresh the relevant jockey views.
BR-46	The jockey must be able to review race participation history and current status through the jockey dashboard.

BR-47	Only an authenticated Spectator with the SPECTATOR role may access spectator features such as predictions, schedules, and results.
BR-48	The Spectator may submit a prediction only when a valid race and horse selection have been chosen and the prediction rank is provided.
BR-49	The system must prevent the Spectator from submitting duplicate predictions for the same race and selected horse.
BR-50	A prediction may be edited or deleted only while the associated race is still pending and not locked for result confirmation.
BR-51	The Spectator may view upcoming race schedules and completed race results for all available public races.
BR-52	The system must display the Spectator's reward point balance and prediction history after each prediction action.
BR-53	The Spectator may view ranking tables and race outcome details for completed races from the public results screens.
BR-54	The system must reject invalid spectator input and display a clear error message when required prediction information is missing or inconsistent.
BR-55	A successful prediction submission, update, or deletion must immediately refresh the spectator dashboard and history list.
BR-56	The spectator must be able to review race status, result outcomes, and reward information from the spectator portal.

3.4.2 Common Requirements

This appendix summarizes the common requirements that apply across all modules of the Horse Racing Tournament Management System.

Req. ID	Requirement Name	Description
CR-01	Data Validation	All mandatory fields marked as required must be validated on both the client side and the server side. The system shall prevent submission if any required information is missing.
CR-02	Data Format Validation	The system shall validate input formats before processing data. • Email must follow the standard email format. • Phone numbers must contain exactly 10 digits. • Dates must use the DD/MM/YYYY format.

Req. ID	Requirement Name	Description
CR-03	Authentication	Users shall authenticate before accessing protected system resources. Unauthorized users shall be denied access.
CR-04	Authorization	The system shall enforce role-based access control for Administrator, Horse Owner, Jockey, Referee, and Spectator.
CR-05	Audit Logging	Important user activities, including race scheduling, result submission, and approval actions, shall be recorded with timestamps and user information.
CR-06	Notification	The system shall notify users when important events occur, such as tournament registration approval, race assignment, or result approval.
CR-07	Performance	The system should respond to user requests within 3 seconds under normal operating conditions and support multiple concurrent users.
CR-08	Security	Passwords shall be encrypted before storage. All communication shall use HTTPS, and inactive sessions shall expire automatically.
CR-09	Availability	The system shall remain available during tournament operations except for scheduled maintenance periods.
CR-10	Data Integrity	The system shall maintain consistent and accurate data across all modules and prevent duplicate or inconsistent records.

3.5. Application Messages

This section lists standard messages used across the Horse Racing Tournament Management System. Messages are grouped by feature and include a short type and the text shown to users.

3.5.1 Message Type Definitions

- **Inline** — Displayed directly below a field or section (neutral information).
- **Inline (Red)** — Displayed below a field when validation fails (error highlight).
- **Toast (Success)** — Floating notification (green) shown after a successful action.
- **Toast (Error)** — Floating notification (red) shown when an action fails.
- **Toast (Warning)** — Floating notification (yellow) shown as a caution before an action.

#	Code	Message Type	Context	Content
1	MSG01	Inline	Search result empty	No search results found.
2	MSG02	Inline (Red)	Form validation	The `{field_name}` field is required.
3	MSG03	Toast (Success)	Data update	Update `{entity_name}` successfully.
4	MSG04	Toast (Success)	Data creation	Add new `{entity_name}` successfully.
5	MSG05	Toast (Error)	Data deletion	Delete `{entity_name}` failed. Please try again.
6	MSG06	Toast (Success)	Email action	A confirmation email has been sent to `{email_address}`.
7	MSG07	Inline (Red)	Login – wrong credentials	Invalid username or password. Please try again.
8	MSG08	Toast (Error)	Login – account locked	Your account has been locked. Please contact the administrator.
9	MSG09	Toast (Warning)	Session expired	Your session has expired. Please log in again.
10	MSG10	Toast (Error)	Unauthorized access	You do not have permission to access this page.
11	MSG11	Toast (Success)	Password reset	Password reset instructions sent to your email.
12	MSG12	Inline (Red)	Registration – duplicate	This horse has already been registered for the selected race.
13	MSG13	Toast (Success)	Registration – submitted	Your registration has been submitted successfully.
14	MSG14	Toast (Warning)	Registration – deadline	The registration deadline for this race has passed.
15	MSG15	Toast (Error)	Registration – capacity full	This race has reached its maximum number of participants.
16	MSG16	Toast (Success)	Invitation sent	Your invitation has been sent successfully. Status: Pending.
17	MSG17	Toast (Error)	Invitation – duplicate	You have already sent an invitation to this jockey for the selected race.
18	MSG18	Toast (Error)	Invitation – jockey unavailable	This jockey is not available for the selected race date.

#	Code	Message Type	Context	Content
19	MSG19	Toast (Error)	Invitation – deadline passed	The invitation deadline has passed for this race.
20	MSG20	Toast (Success)	Invitation accepted (Jockey)	You have accepted the invitation. The race has been added to your upcoming races.
21	MSG21	Toast (Success)	Invitation declined (Jockey)	You have declined the invitation.
22	MSG22	Toast (Success)	Invitation accepted (Owner)	The jockey has accepted your invitation.
23	MSG23	Toast (Warning)	Invitation declined (Owner)	The jockey has declined your invitation.
24	MSG24	Inline	Invitation status	This invitation is still pending. The jockey has not responded yet.
25	MSG25	Toast (Success)	Race created	The race has been created and published successfully.
26	MSG26	Toast (Error)	Race scheduling – conflict	A race is already scheduled at the selected date and track.
27	MSG27	Toast (Warning)	Race cancelled	This race has been cancelled. All registered participants have been notified.
28	MSG28	Toast (Success)	Race updated	Race information has been updated successfully.
29	MSG29	Toast (Success)	Result submitted	Race results have been recorded successfully.
30	MSG30	Toast (Error)	Result – already exists	Results for this race have already been submitted.
31	MSG31	Toast (Warning)	Result – incomplete	Please enter the result for all participating horses before submitting.
32	MSG32	Toast (Success)	Ranking updated	Rankings have been updated based on the latest race results.
33	MSG33	Toast (Error)	Server error	An unexpected error occurred. Please refresh the page or contact support.
34	MSG34	Inline (Red)	Not found	The requested resource was not found.
35	MSG35	Toast (Warning)	Unsaved changes	You have unsaved changes. Are you sure you want to leave this page?

#	Code	Message Type	Context	Content
36	MSG36	Toast (Success)	Horse added	Horse `{horse_name}` has been added successfully.
37	MSG37	Toast (Success)	Horse updated	Horse information has been updated successfully.
38	MSG38	Toast (Error)	Horse – not found	The selected horse could not be found. Please refresh and try again.
39	MSG39	Toast (Warning)	Horse – health not verified	This horse has not passed the health check. It cannot be registered for a race.
40	MSG40	Inline (Red)	Horse – duplicate name	A horse with this name already exists in your account.
41	MSG41	Toast (Error)	Jockey – license expired	This jockey's license has expired. Please update the license before sending an invitation.
42	MSG42	Toast (Warning)	Jockey – no upcoming races	You have no upcoming races at this time.
43	MSG43	Toast (Success)	Profile updated	Your profile has been updated successfully.
44	MSG44	Inline (Red)	Profile – invalid phone	Please enter a valid phone number.
45	MSG45	Inline (Red)	Profile – invalid email format	Please enter a valid email address.
46	MSG46	Toast (Success)	Tournament created	The tournament has been created and is now open for registration.
47	MSG47	Toast (Success)	Tournament updated	Tournament information has been updated successfully.
48	MSG48	Toast (Error)	Tournament – cannot delete	This tournament cannot be deleted because it has active registrations.
49	MSG49	Toast (Warning)	Tournament – closing soon	Registration for this tournament closes in `{X}` day(s).
50	MSG50	Toast (Success)	Tournament closed	Registration has been closed for this tournament.
51	MSG51	Toast (Success)	Registration approved	The registration has been approved successfully.
52	MSG52	Toast (Success)	Registration rejected	The registration has been rejected. The owner has been notified.

#	Code	Message Type	Context	Content
53	MSG53	Toast (Warning)	Registration – pending review	This registration is awaiting admin approval.
54	MSG54	Toast (Success)	Approval notified (Owner)	Your registration has been approved. You may now proceed.
55	MSG55	Toast (Error)	Rejection notified (Owner)	Your registration has been rejected. Please contact the administrator for more details.
56	MSG56	Toast (Success)	Result confirmed	The race result has been confirmed by the referee.
57	MSG57	Toast (Error)	Result – conflict detected	A ranking conflict has been detected. Please review and resubmit the results.
58	MSG58	Toast (Warning)	Protest received	A protest has been filed for this race. Results are temporarily on hold.
59	MSG59	Toast (Success)	Protest resolved	The protest has been reviewed and resolved. Final results have been published.
60	MSG60	Inline	No notifications	You have no new notifications.
61	MSG61	Toast (Success)	Notifications cleared	All notifications have been marked as read.
62	MSG62	Inline	Notification – new invitation	You have a new race invitation. Please review the details.
63	MSG63	Inline	Notification – race reminder	Reminder: your race `{race_name}` is scheduled in `{X}` day(s).
64	MSG64	Inline (Red)	Upload – file too large	The uploaded file exceeds the maximum allowed size of 5MB.
65	MSG65	Inline (Red)	Upload – invalid format	Invalid file format. Only JPG, PNG, and PDF files are accepted.
66	MSG66	Toast (Success)	Upload – success	File uploaded successfully.
67	MSG67	Toast (Error)	Upload – failed	File upload failed. Please check your connection and try again.
68	MSG68	Toast (Warning)	Weight – out of range	The jockey's weight does not meet the requirements for this race category.
69	MSG69	Toast (Success)	Weight check passed	Weight check passed. The jockey is eligible for this race.

#	Code	Message Type	Context	Content
70	MSG70	Toast (Error)	Network error	Unable to connect to the server. Please check your internet connection.

IV. Software Design Description

This chapter presents the software architecture, database design and detailed design.

4.1. Overall Design

4.1.1 System Architecture

The Horse Racing Tournament Management System is designed following a modern 3-tier software architecture. This architectural pattern guarantees a strict separation of concerns, scalability, and ease of maintenance by dividing the application into three distinct layers: the Presentation Layer, the Application Server Layer, and the Data Persistence Layer.

Figure 4.1 illustrates the high-level system architecture and how data flows across these layers:

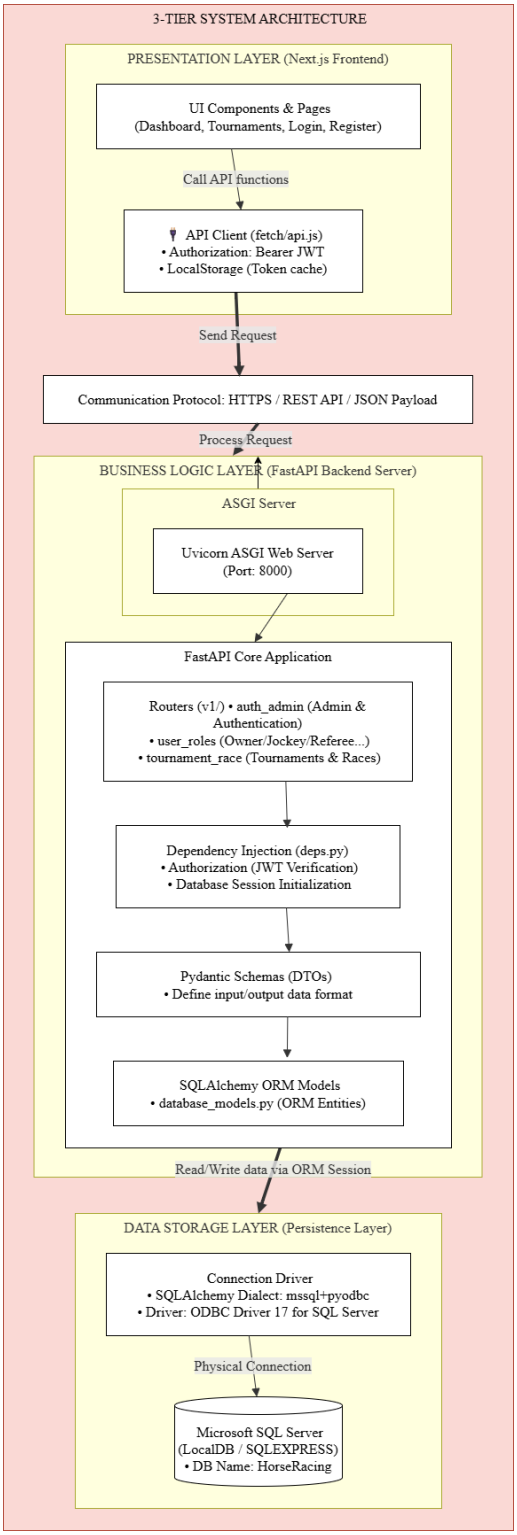


Figure 4.1: Overall System Architecture

- **Presentation Layer (Frontend):** Implemented using Next.js (React Framework). It runs directly inside the client's web browser, rendering interactive and responsive user interfaces (such as registration forms, tournament dashboards, scoreboards, and leaderboards). It communicates with the backend exclusively via HTTP requests carrying JSON payloads, passing JSON Web Tokens (JWT) for authentication.

- **Application Server Layer (Backend):** Built on FastAPI (Python). It is hosted on a high-performance ASGI server (Uvicorn). This layer exposes secure REST API endpoints, handles business logic (such as scheduling races, validating horse registrations, allocating referees, recording race violations, and computing points), performs user authentication, and coordinates transactions with the database.
- **Data Persistence Layer (Database):** Managed by Microsoft SQL Server (MSSQL). It stores the persistent relational data including user credentials, role definitions, horse profiles, registration records, tournament schedules, race results, and award distributions. It connects to the backend through SQLAlchemy ORM utilizing the `pyodbc` driver.

4.1.2 Backend Architecture

The backend application utilizes the FastAPI framework to provide a modular, performant, and secure API server. Figure 4.2 depicts the internal architectural layers of the backend system:

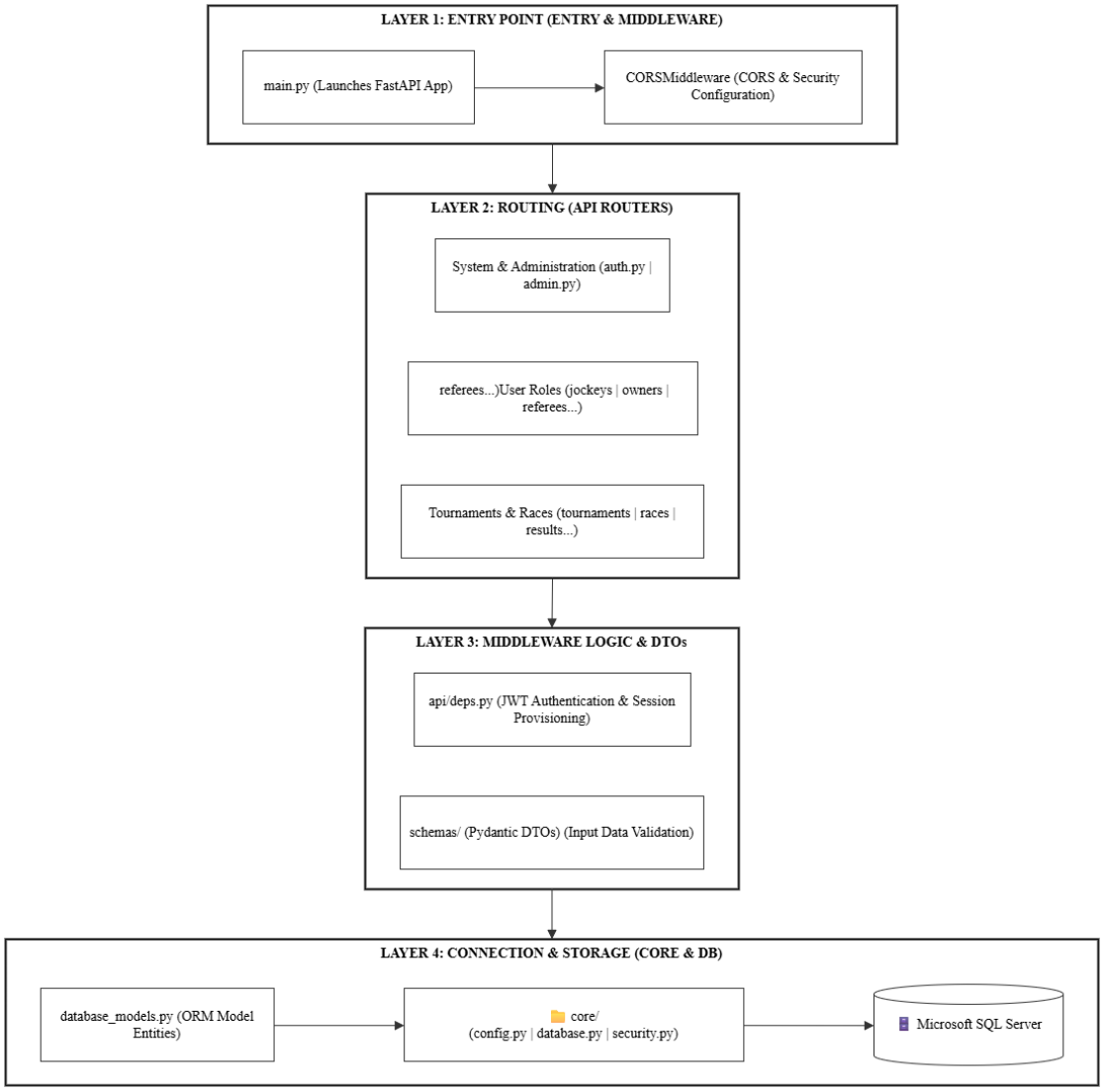


Figure 4.2: FastAPI Backend Core Architecture

The backend processing flow is structured into the following operational stages:

1. **Entry Point & Security Middleware:** The application entry is defined in `main.py`. All incoming requests are filtered through `CORSMiddleware` to validate security origins and ensure safe cross-origin communication.

2. **API Routing:** Requests are routed to specific endpoints under `/api/v1/` managed by modular routers (such as `auth`, `admin`, `user_roles`, and `tournament_race`).

3. **Dependency Injection & DTO Validation:** Request payloads are validated against Pydantic Schemas (Data Transfer Objects). Meanwhile, the dependency injection system in `deps.py` validates JWT authorization and provisions active SQL database sessions.

4. **Core Logic & ORM Mapping:** Validated requests are processed through business logic and mapped to relational database entities using SQLAlchemy ORM declarative models (`database_models.py`).

5. **Database Persistence:** SQLAlchemy executes the transactions against Microsoft SQL Server using the SQL Server connection string defined in `config.py` and database session helper in `database.py`.

4.1.3 Web Application Architecture

The user interface is structured as a Single Page Application (SPA) using Next.js. Figure 4.3 details the architectural layers and components of the Web Application:

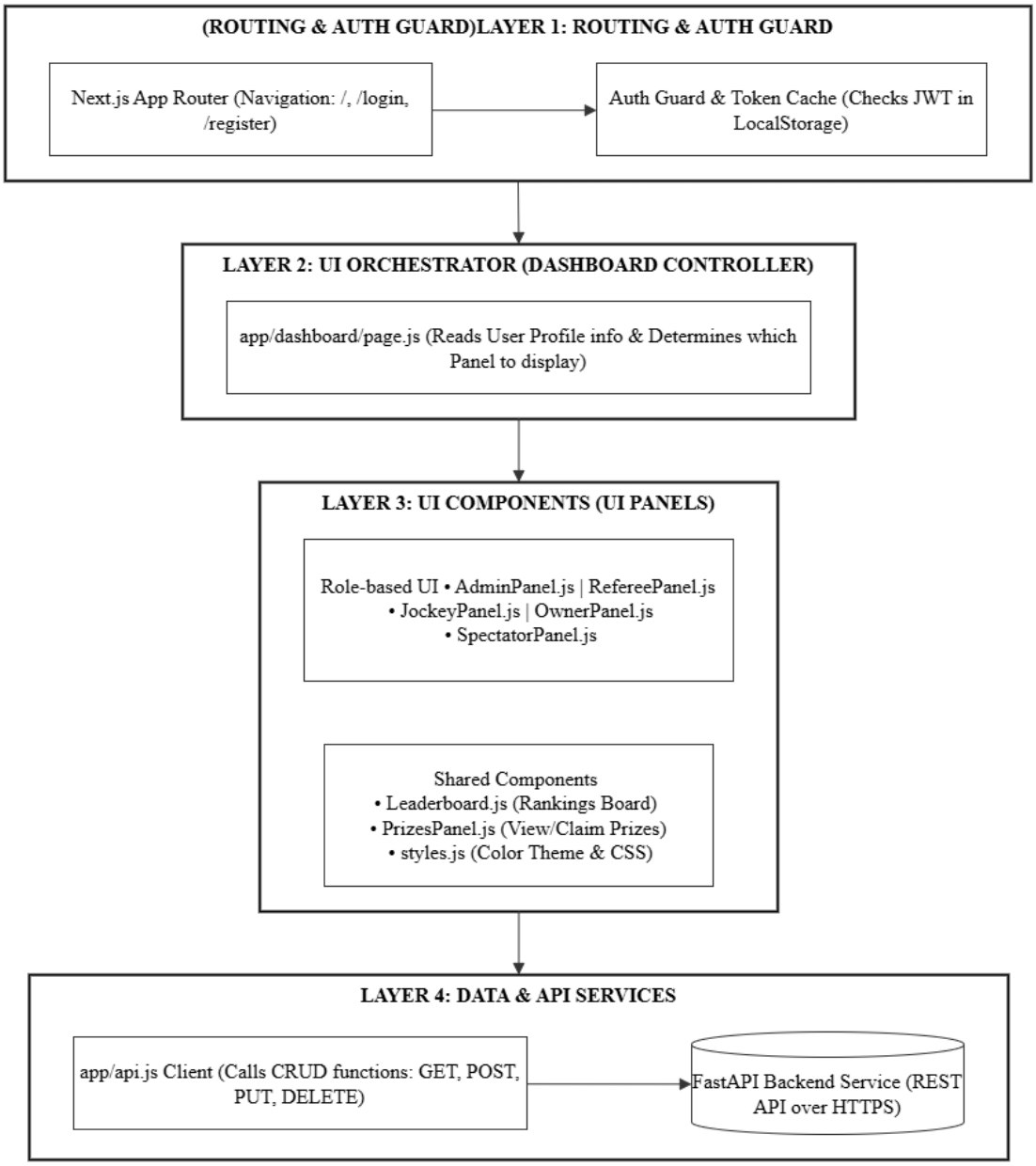


Figure 4.3: Next.js Web Application Frontend Architecture

The frontend architectural workflow is organized as follows:

1. **Routing & Authentication Guard:** Next.js App Router controls client-side navigation. The `Auth Guard` inspects the browser's local storage for a valid JWT token. Unauthenticated users are redirected to the Login or Register routes, while authenticated users are allowed access to the dashboard.

2. **Dashboard Controller:** Located in `dashboard/page.js`, it fetches the authenticated user's profile details. It dynamically checks the user's role (Admin, Jockey, Horse Owner, Referee, Spectator) to determine which UI component to mount.

3. **Role-based UI Panels:** Specialized interfaces are mounted dynamically based on role configurations (such as `AdminPanel`, `JockeyPanel`, `OwnerPanel`, `RefereePanel`, or `SpectatorPanel`).

4. **Shared Components & Styles:** Common interface sections like `Leaderboard.js` and `PrizesPanel.js` are imported by multiple roles, applying uniform CSS styling definitions.

5. **API Client Layer:** The centralized HTTP client utility in `api.js` communicates with the FastAPI Backend server via async `fetch` operations, managing request headers and auth tokens.

4.1.4 Backend Package Diagram

The backend code structure is organized into modular packages to isolate concerns and enforce clear boundaries between different components of the FastAPI application. Figure 4.4 displays the layout of the backend directory:

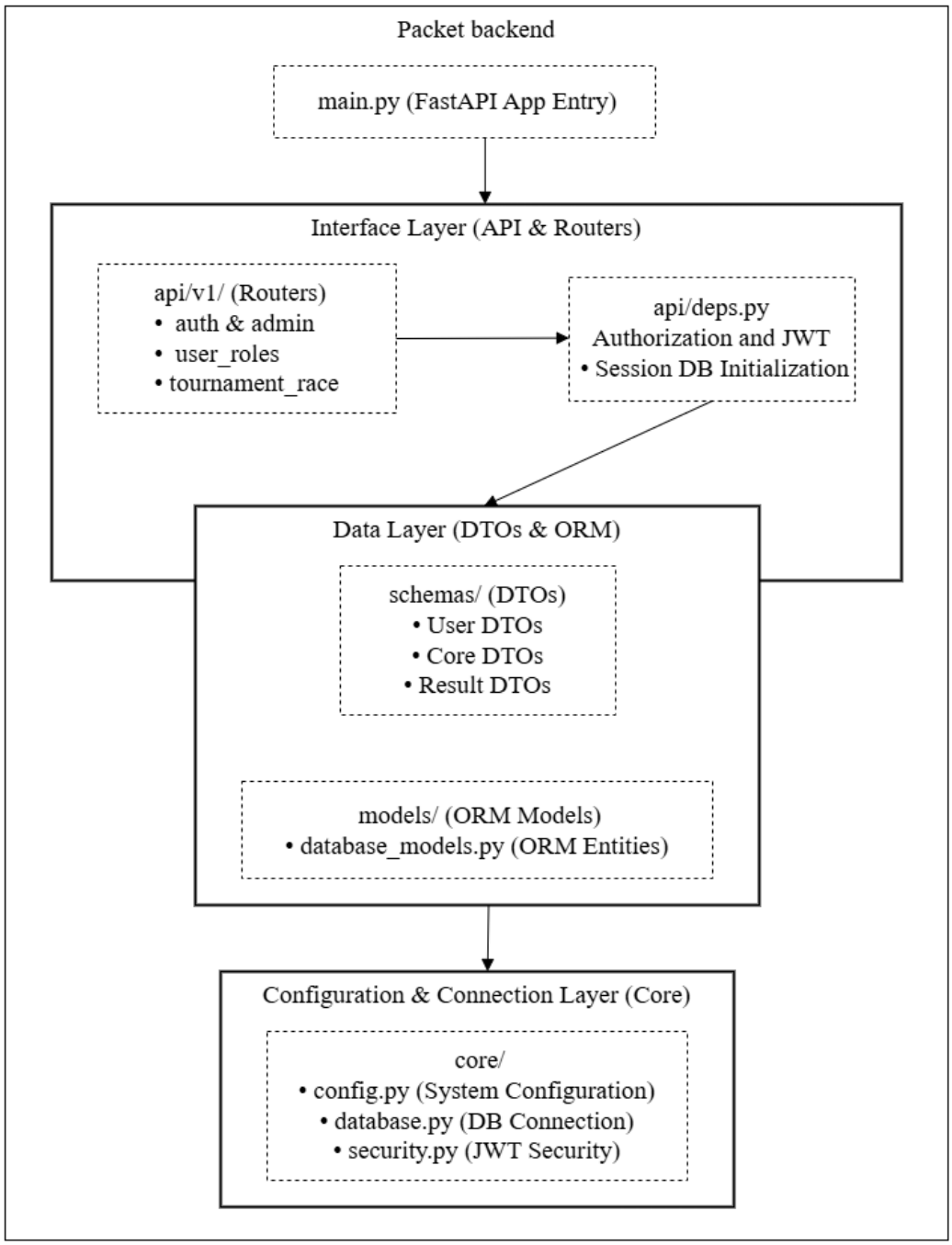


Figure 4.4: Backend Package Structure

The packages and modules are structured as follows:

- **`api/`:** Houses the API routers. The subpackage `v1/` contains separate router modules for each logical subdomain (such as authentication, user roles, and tournament/race management). The file `deps.py` implements dependency injection providers.
- **`schemas/`:** Defines Pydantic data serialization models (DTOs) representing the structured format of requests and responses for all major subdomains (user, horse, race, prize, result, tournament).

- **`models/`:** Contains SQLAlchemy declarative models (`database_models.py`) mapped to the physical database tables.
- **`core/`:** Holds core system modules, including system configurations (`config.py`), JWT security utilities (`security.py`), and database connection managers (`database.py`).

4.1.5 Web Application Package Diagram

The frontend codebase is organized using the Next.js App Router structure. Figure 4.5 displays the frontend package structure:

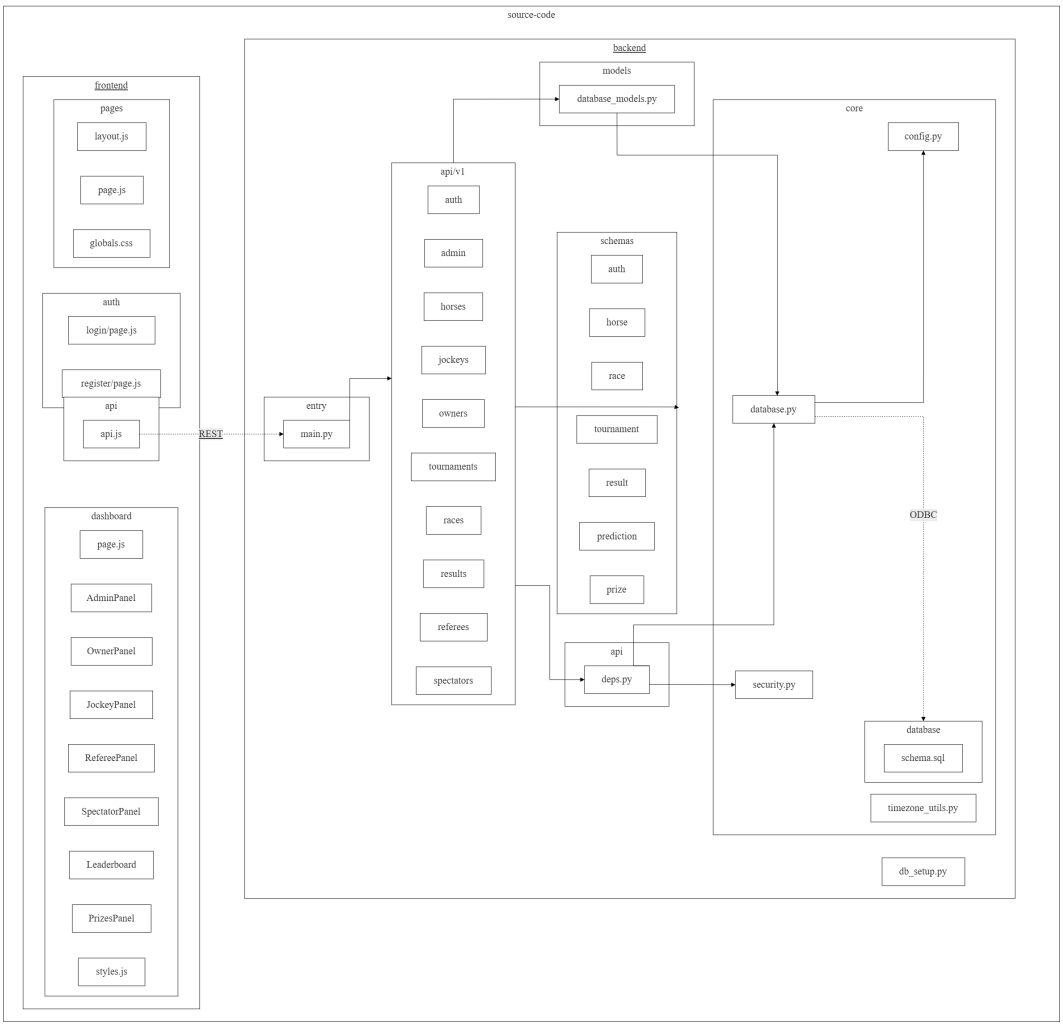


Figure 4.5: Web Application Package Structure

The major frontend modules are organized inside the following directories:

- **`app/`:** Root of the application router. The files `layout.js` and `page.js` render the root layout and landing page. The file `api.js` contains utility methods for interacting with the backend API.
- **`login/` and `register/`:** Route directories managing authentication pages.
- **`dashboard/`:** Route directory for the dashboard. It contains the controller `page.js` and a subfolder `components/` that groups the role-specific interface panels (Admin, Referee, Jockey, Owner, Spectator).

4.1.6 Database Design

This section presents the database design for the Horse Racing Tournament Management System. The persistent data layer is implemented using Microsoft SQL Server to store relational entity information.

Figure 4.6 displays the physical database design schema. The interactive database schema is also available online and can be accessed directly at: dbdiagram.io - Horse Racing Tournament Management System.

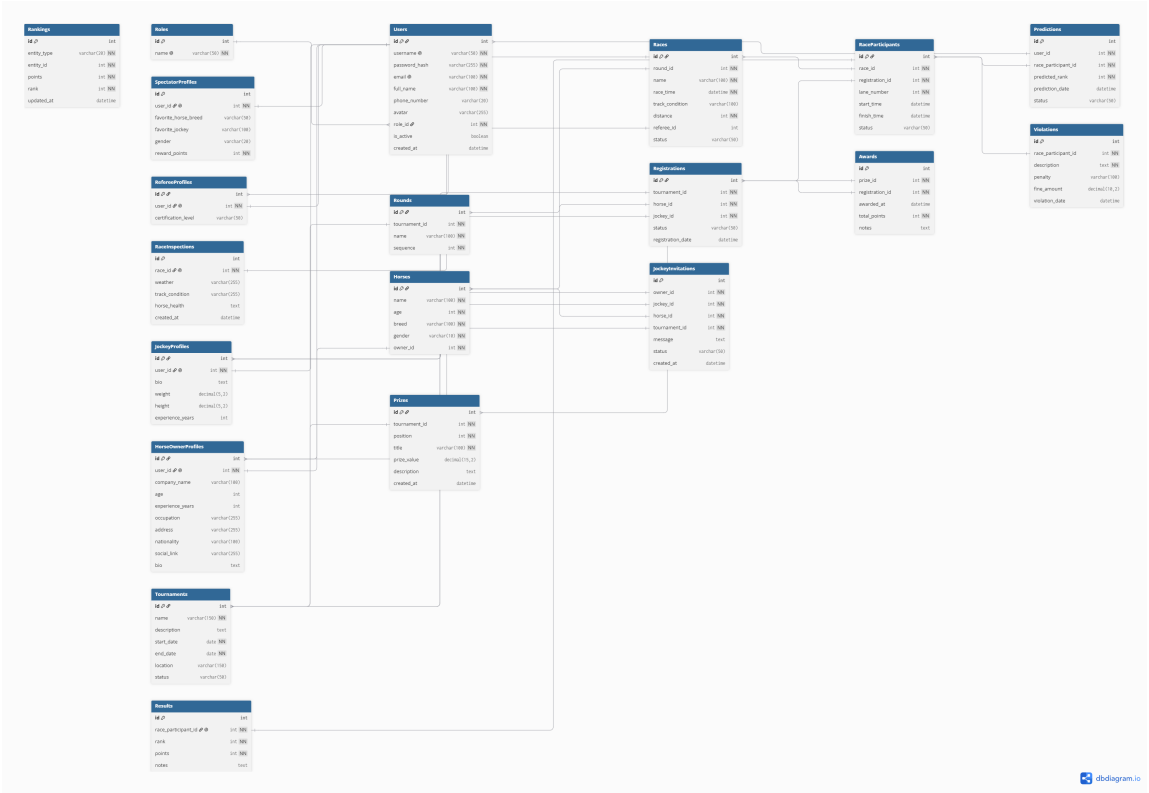


Figure 4.6: Physical Database Design Schema

The relational database schema is structured to capture all operations including user role-based access control, horse ownership details, tournament scheduling, race registration, and result calculation. Below is a detailed description of the physical table attributes:

- **Roles:** Stores authorization group definitions (such as Admin, Referee, Jockey, Horse Owner, Spectator).

- **Users:** Holds centralized user profiles, password hashes, and links back to role definitions.
- **Profile Tables (JockeyProfiles, HorseOwnerProfiles, RefereeProfiles, SpectatorProfiles):** Extend the user table with domain-specific properties using 1-to-1 relationships.
- **Horses:** Tracks horse attributes and connects each horse to its respective owner profile.
- **Tournaments, Rounds, and Races:** Create a hierarchical structure representing active events, stages of competition, and individual races.
- **Registrations:** Acts as a junction table recording which horse and jockey pairings are approved to compete in a specific tournament.
- **RaceParticipants:** Manages lane allocations, race results (ranks/points), and violations for each registered participant in a specific race.

4.2. Detailed Design

4.2.1 Admin System Overview

Class Diagram

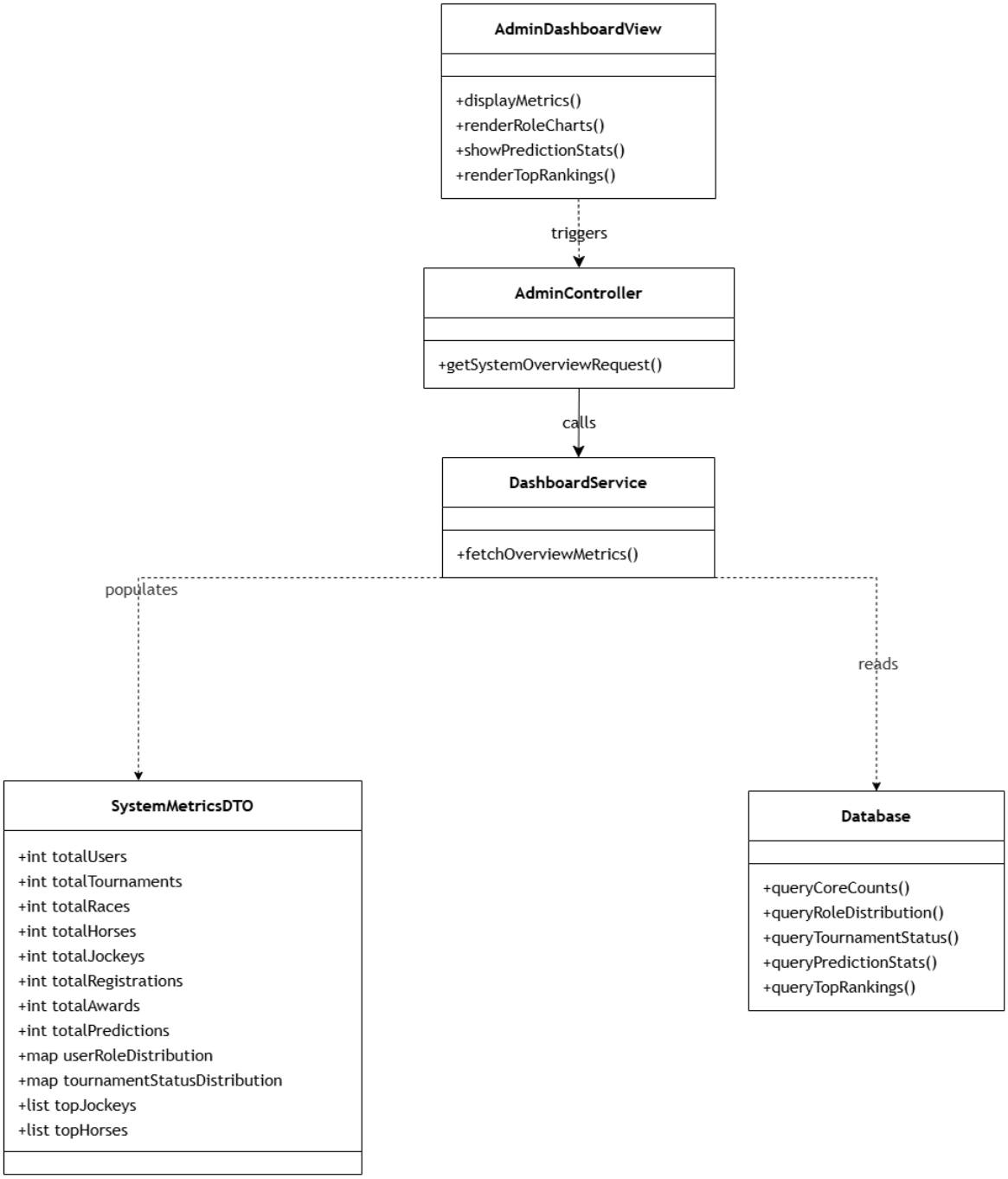


Figure 4.7: Admin System Overview Class Diagram

Sequence Diagram

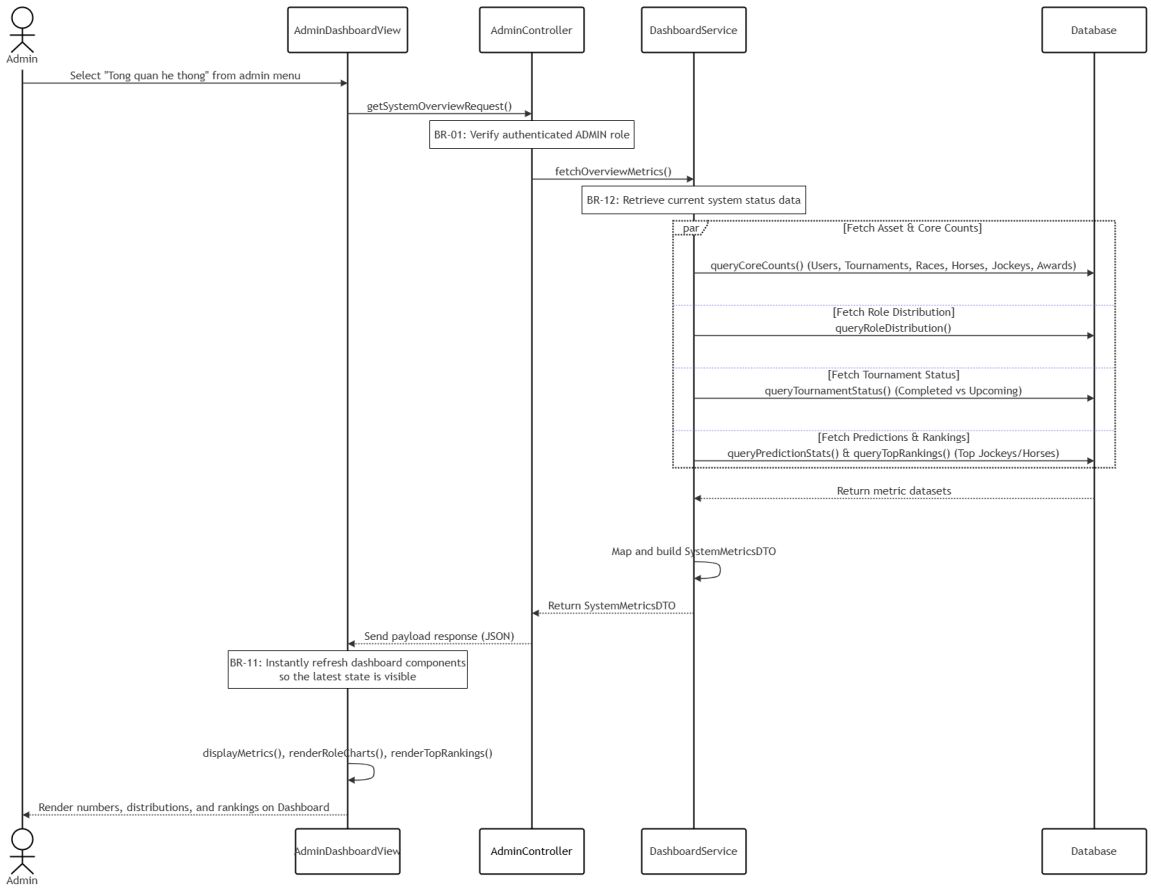


Figure 4.8: Admin System Overview Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
AdminDashboard Router	get_system_overview()	controller / route	Handles HTTP GET requests to retrieve centralized analytics, validating BR-01 authorization.
SystemMetricsOut	total_users, total_tournaments, total_races, total_horses, total_jockeys, total_registrations, total_awards, user_roles, tournament_status, top_jockeys, top_horses	DTO	Output data transfer object providing all aggregated dashboard numbers and top standings.
User	id, username, role, is_active	entity	Database entity representing application users to fetch aggregate accounts and role groupings.
Tournament	id, name, status, start_date, end_date	entity	Database entity representing tournaments used to map status metrics.
Race	id, round_id, status	entity	Database entity representing individual scheduled racing heats.
Registration	id, horse_id, jockey_id, status	entity	Database entity managing structural signups used to count processing assets.

Table 4.1: Admin System Overview class specification

4.2.2 Admin Tournament Management

Class Diagram

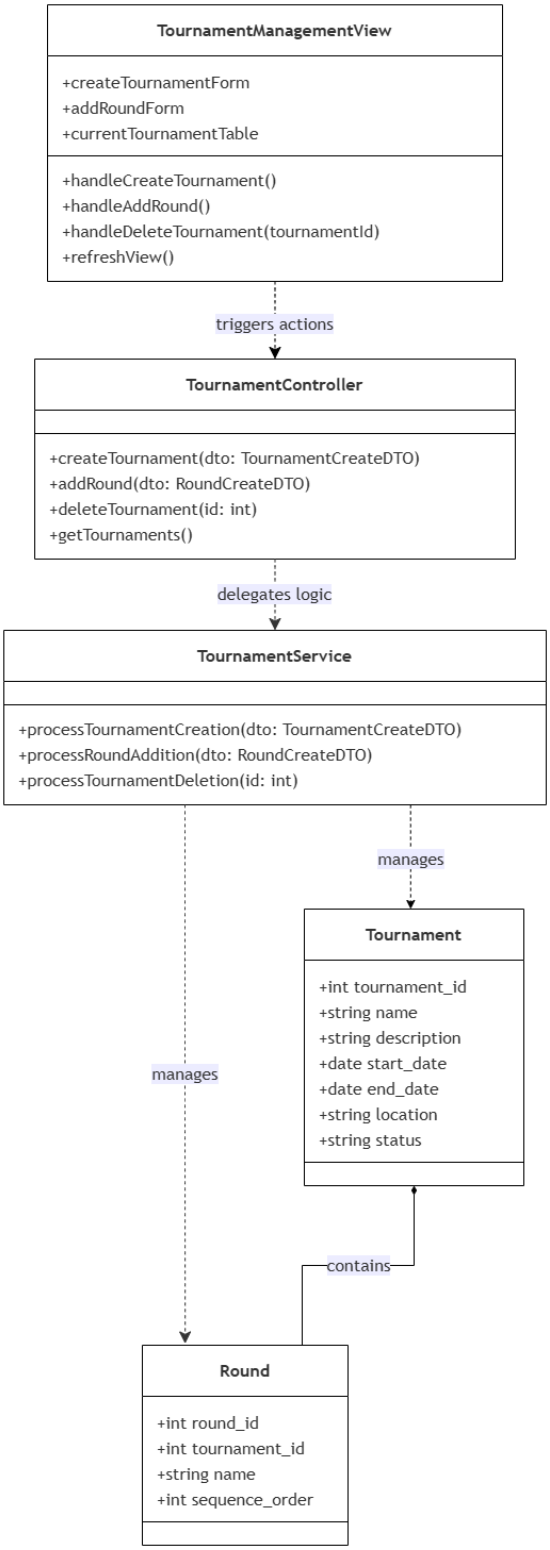


Figure 4.9: Class Diagram for Tournament Management

Sequence Diagram

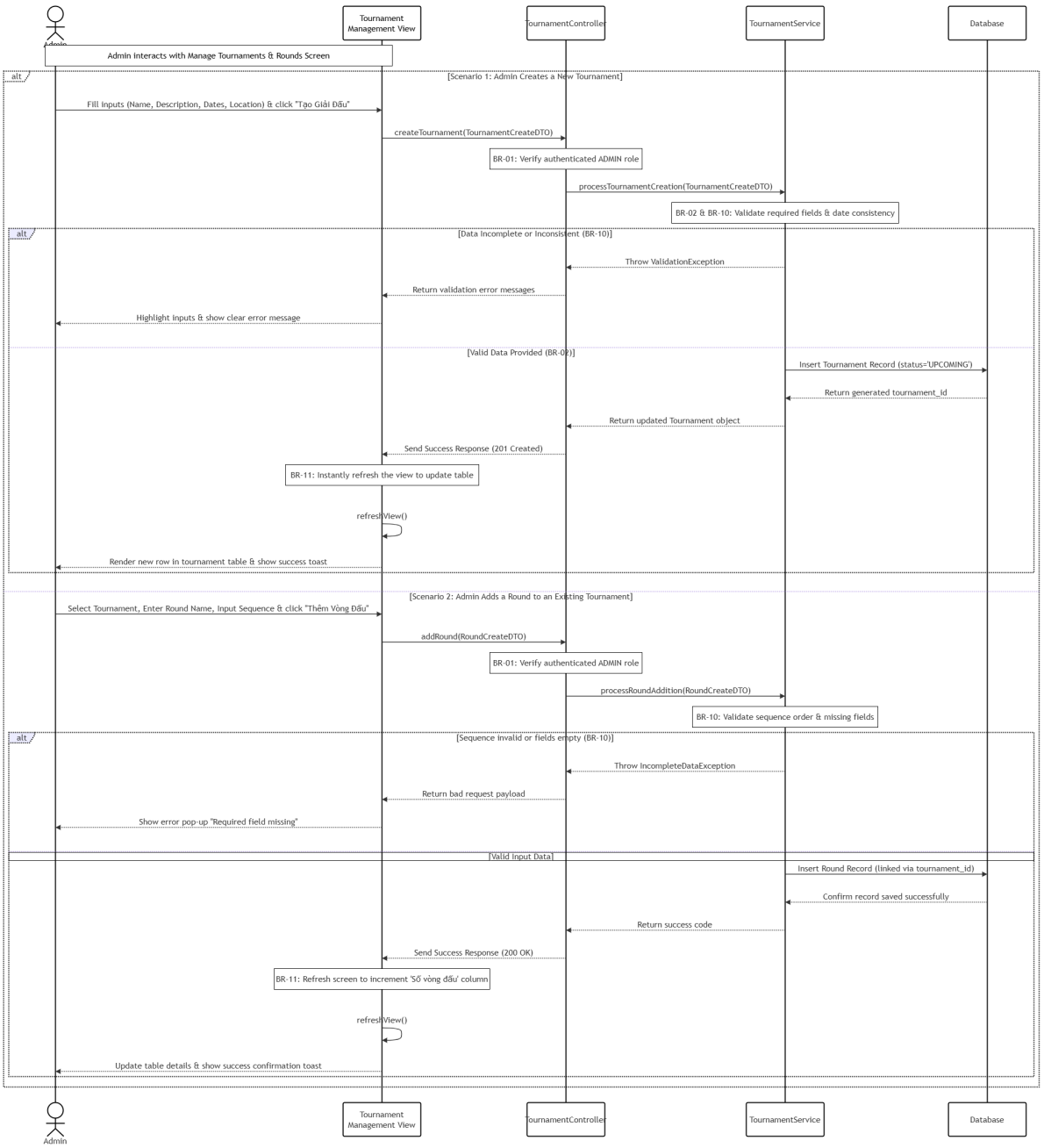


Figure 4.10: Admin Tournament Management Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
TournamentRouter	create_tournament(), add_round(), get_tournaments(), delete_tournament()	controller / route	Handles administrative operations for structuring tournaments and competition stages.
TournamentCreate	name, description, start_date, end_date, location	DTO	Input data transfer object containing essential metrics to provision a new event profile.
RoundCreate	tournament_id, name, sequence_order	DTO	Input data transfer object configuring ordered stages inside a specific tournament context.
TournamentOut	id, name, description, start_date, end_date, location, status, round_count	DTO	Output data transfer object supplying summary details of stored events.
Tournament	id, name, description, start_date, end_date, location, status	entity	Core domain entity storing structured racing tournament profile boundaries.
Round	id, tournament_id, name, sequence_order	entity	Database entity representing individual racing stages arranged sequentially.

Table 4.2: Admin Tournament Management class specification

4.2.3 Admin Registration Confirmation

Class Diagram

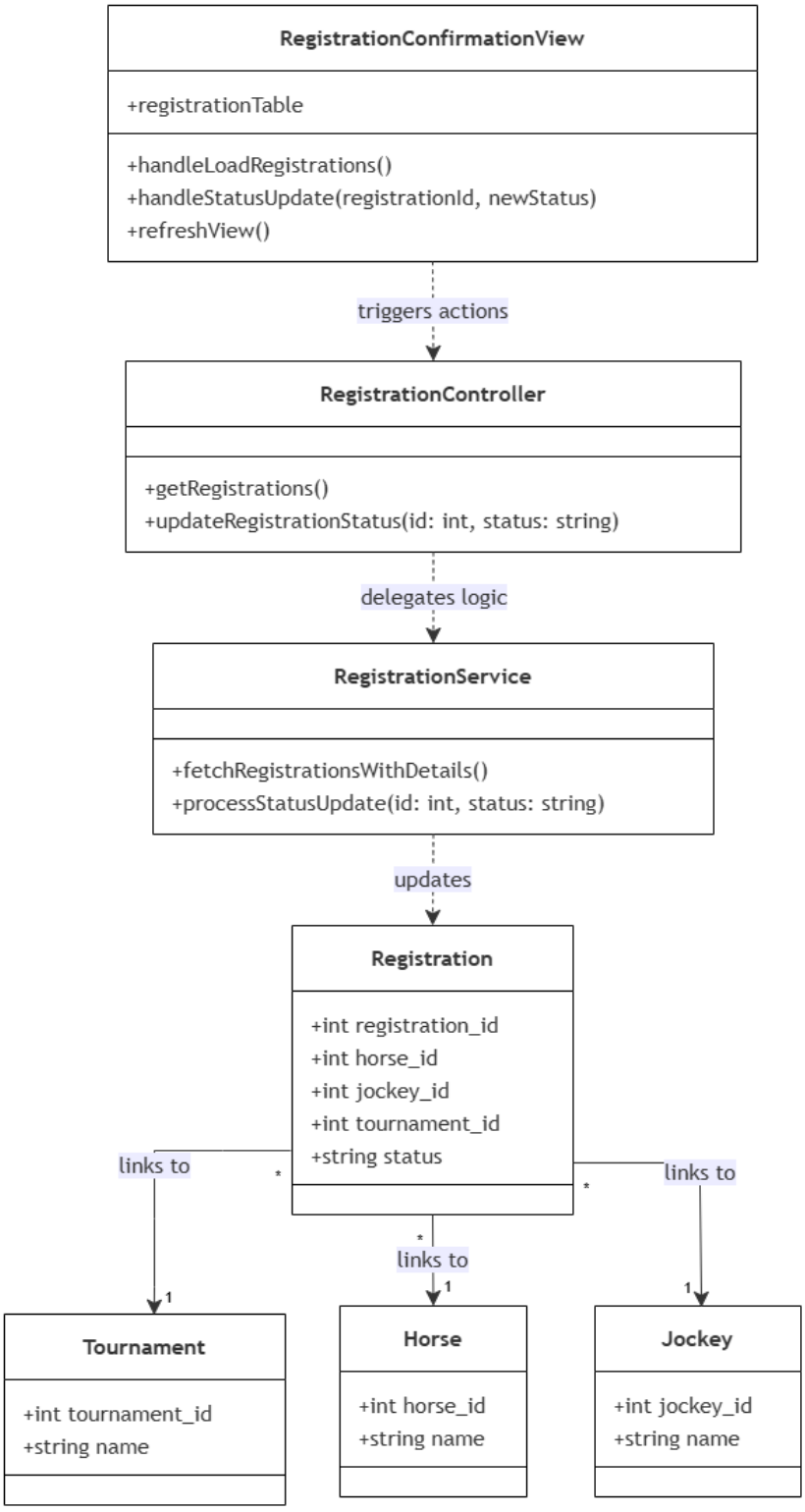


Figure 4.11: Class Diagram for Registration Confirmation

Sequence Diagram

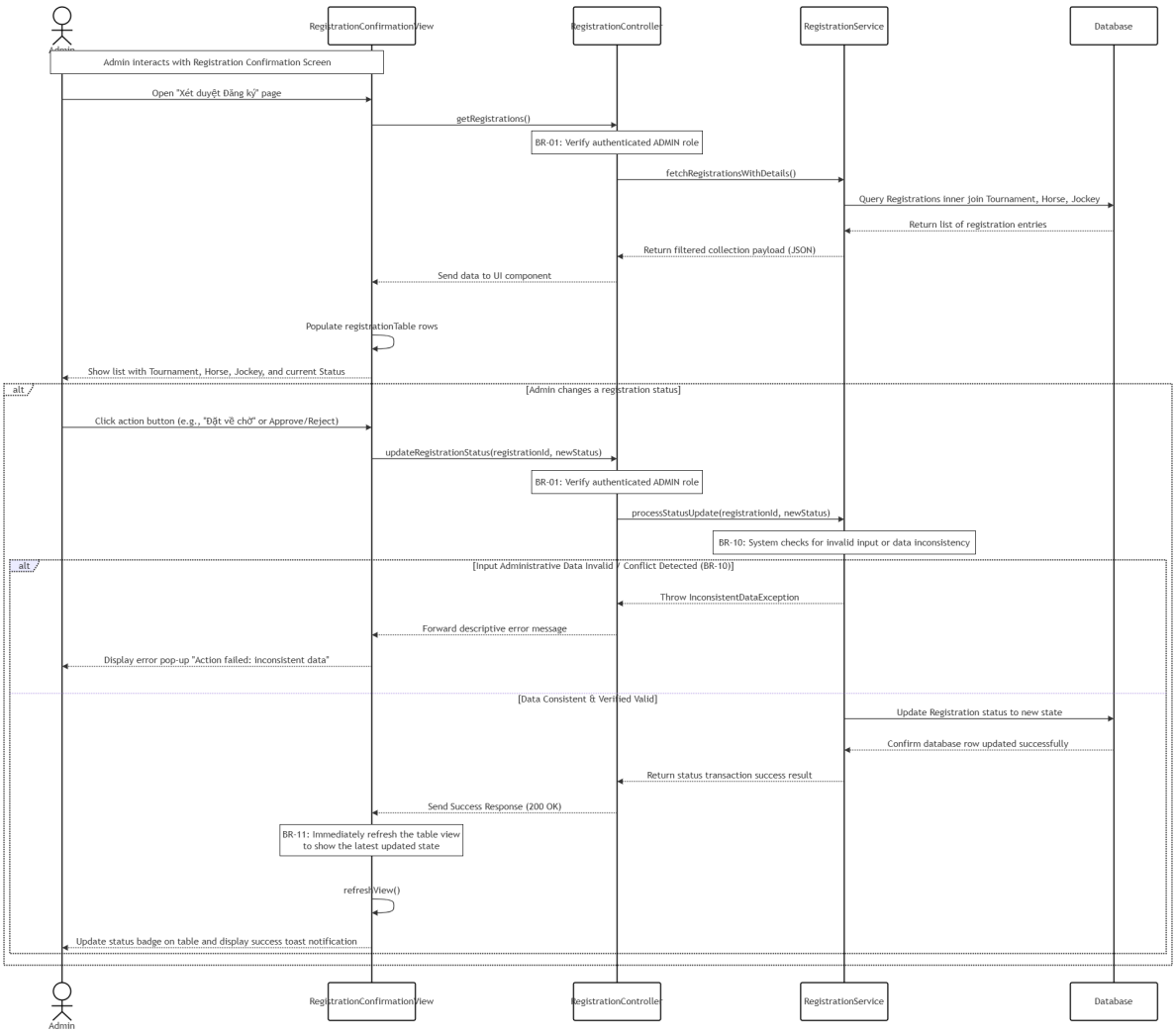


Figure 4.12: Admin Registration Confirmation Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
RegistrationRouter	get_registrations(), update_registration_status()	controller / route	Exposes endpoints for loading applications and updating approval statuses.
RegistrationStatus Update	status	DTO	Input data transfer object carrying the updated status command (e.g., PENDING, APPROVED, REJECTED).
RegistrationDetailOut	id, tournament_name, horse_name, jockey_name, status	DTO	Flattened output payload providing structural identities for administrative grid displays.
Registration	id, horse_id, jockey_id, tournament_id, status	entity	Operational data entity containing processing records of team entries.
Horse	id, name	entity	Database entity providing naming contexts for registered horse components.
Jockey	id, name	entity	Database entity referencing profile information for underlying jockey entries.

Table 4.3: Admin Registration Confirmation class specification

4.2.4 Admin Schedule Races

Class Diagram

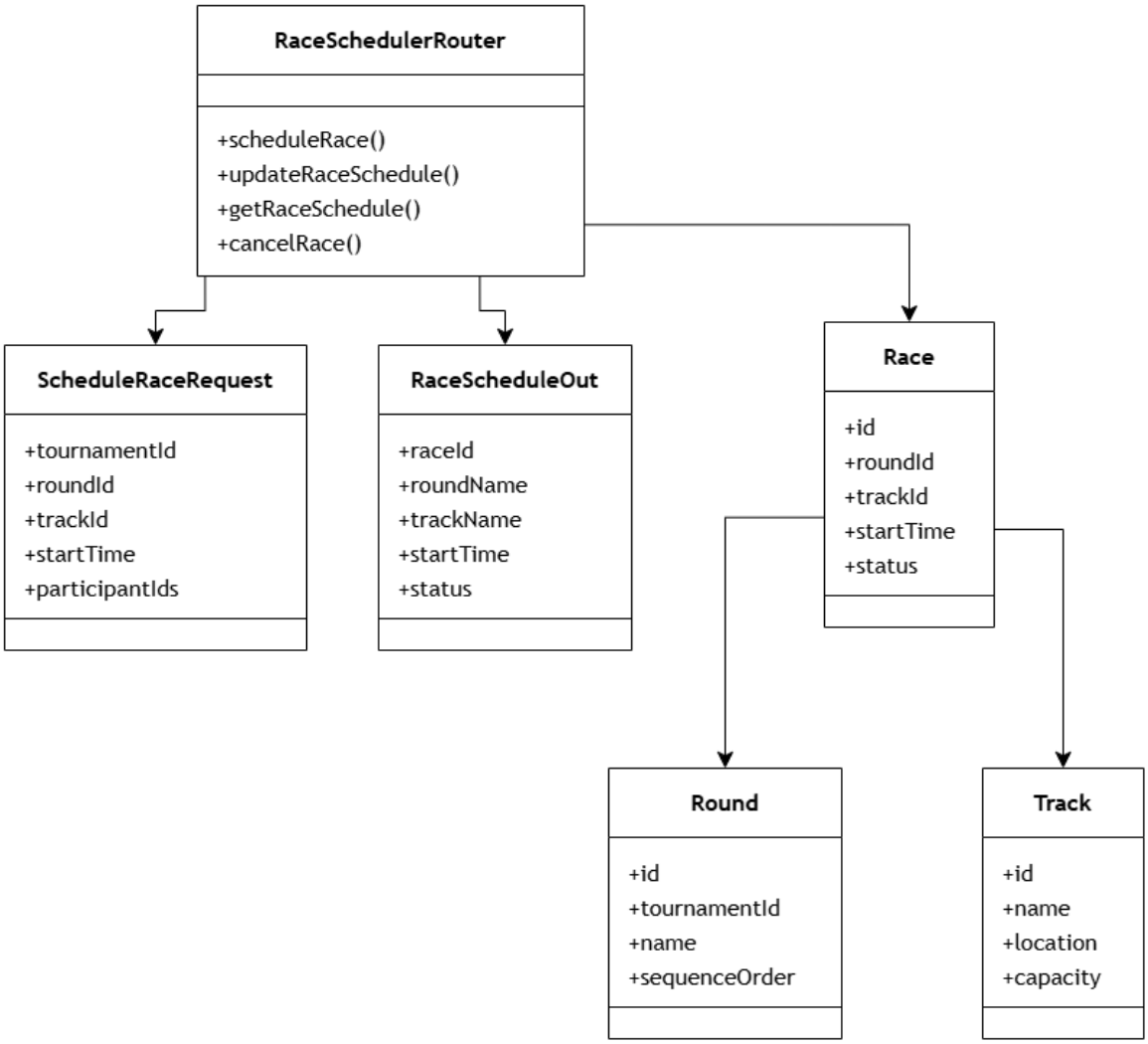


Figure 4.13: Class Diagram for Schedule Races

Sequence Diagram

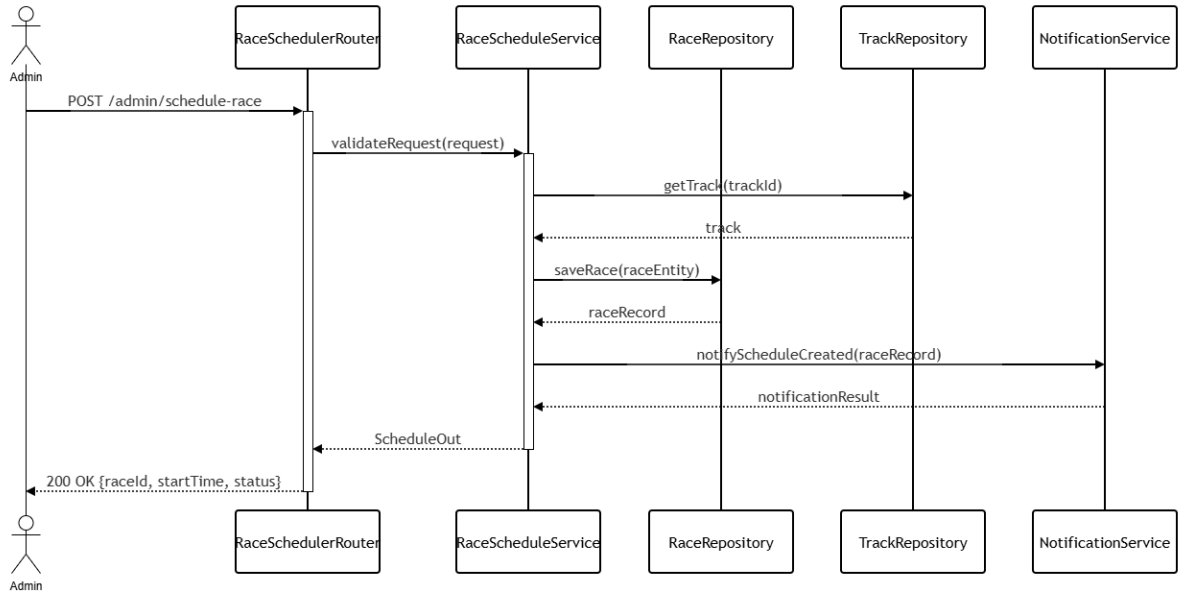


Figure 4.14: Admin Schedule Races Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
RaceSchedulerRouter	schedule_race(), update_race(), get_race_schedule(), cancel_race()	controller / route	Handles race scheduling operations and event lifecycle changes.
ScheduleRaceRequest	tournament_id, round_id, track_id, start_time, participant_ids	DTO	Input DTO for creating or modifying a race schedule.
RaceScheduleOut	race_id, round_name, track_name, start_time, status, lane_assignments	DTO	Output DTO delivering scheduled race details.
Race	id, round_id, track_id, start_time, status, lane_assignments	entity	Entity representing a scheduled race entry.
Round	id, tournament_id, name, sequence_order	entity	Entity representing a tournament round that contains races.
Track	id, name, location, capacity	entity	Entity representing a race track and its configuration.
Participant	id, horse_id, jockey_id, lane_number	entity	Entity representing a horse-jockey entry assigned to a race lane.

Table 4.4: Admin Schedule Races class specification

4.2.5 Admin Manage Users

Class Diagram

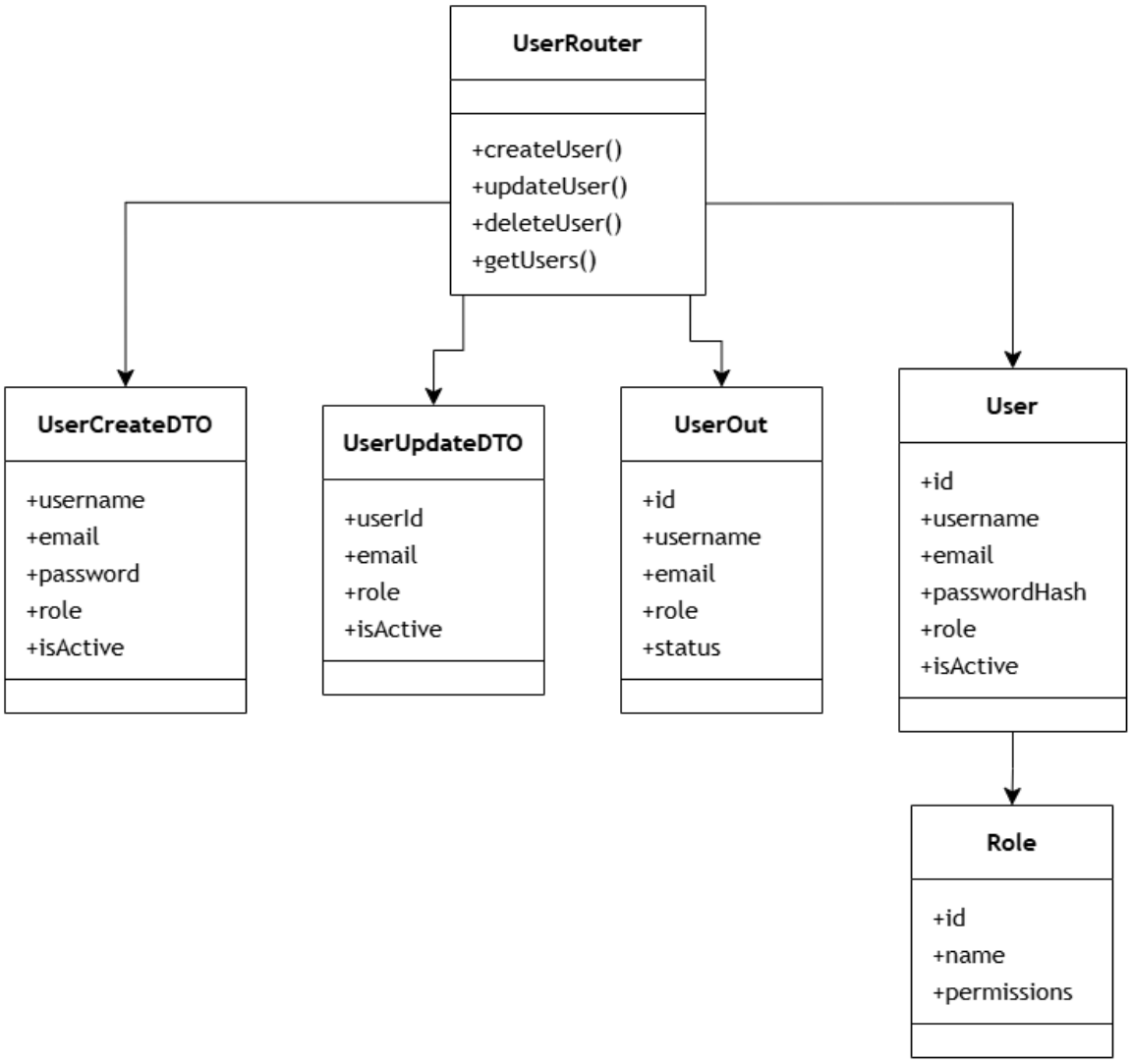


Figure 4.15: Class Diagram for Manage Users

Sequence Diagram

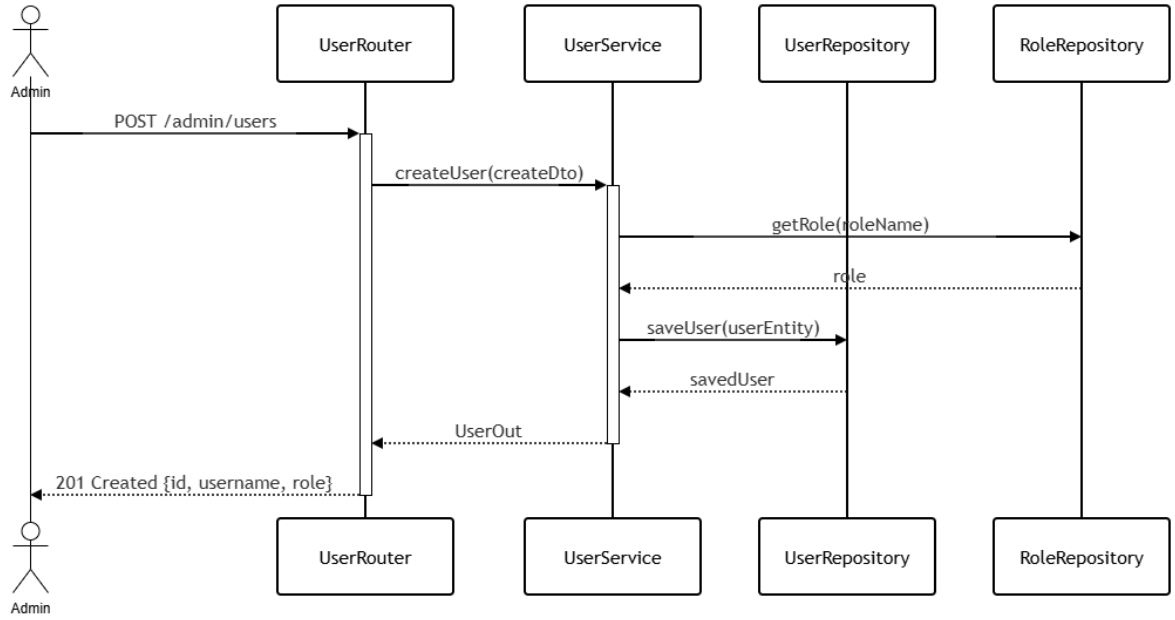


Figure 4.16: Admin Manage Users Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
UserRouter	create_user(), update_user(), delete_user(), get_users()	controller / route	Manages user administration requests and account lifecycle actions.
UserCreate	username, email, password, role, is_active	DTO	Input DTO used to create a new user account.
UserUpdate	user_id, email, role, is_active	DTO	Input DTO used to update user profile and status.
UserOut	id, username, email, role, status	DTO	Output DTO returning user details for display.
User	id, username, email, password_hash, role, is_active	entity	Entity storing user authentication and role data.
Role	id, name, permissions	entity	Entity representing role definitions and access permissions.

Table 4.5: Admin Manage Users class specification

4.2.6 Admin Manage Rewards

Class Diagram

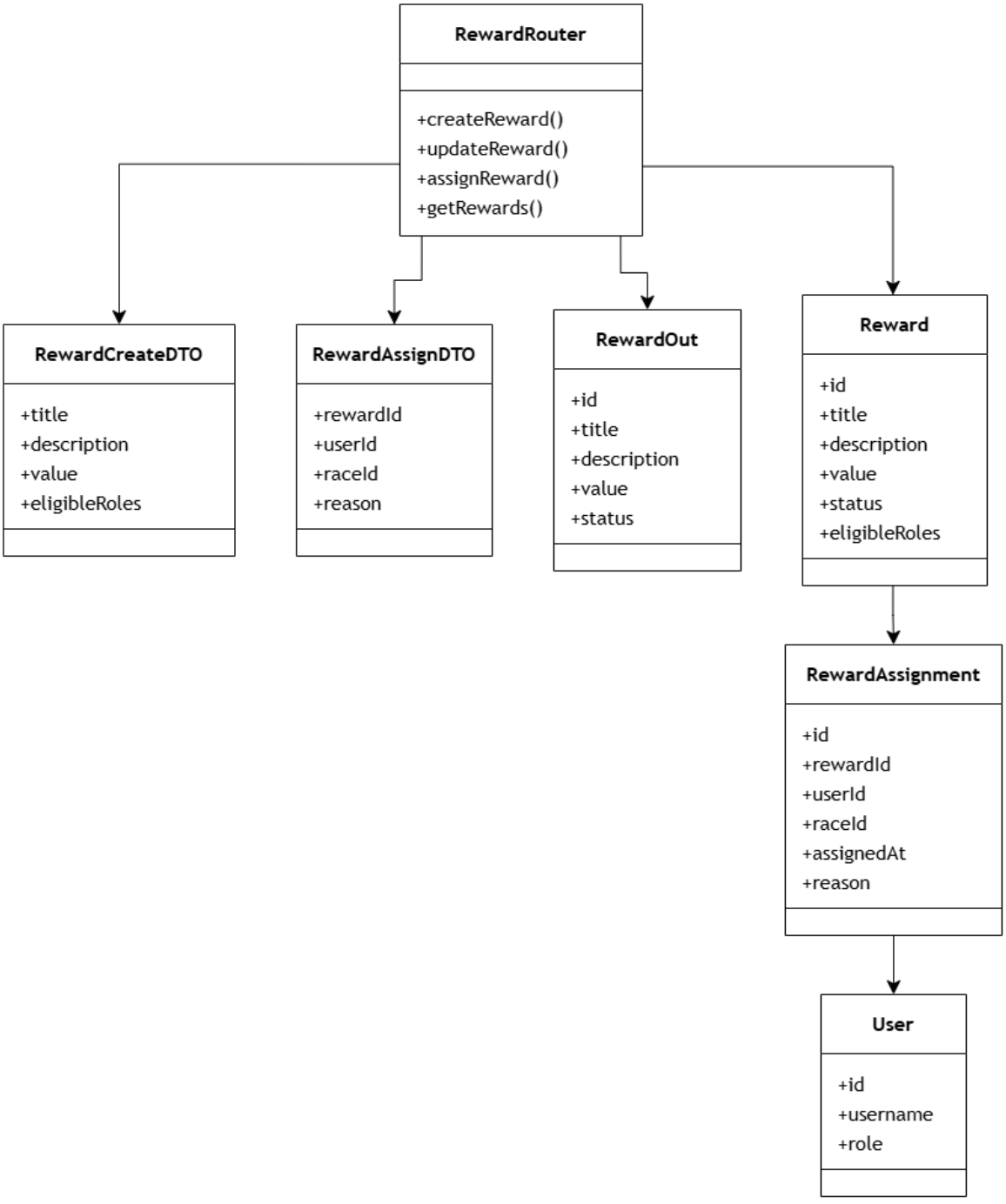


Figure 4.17: Class Diagram for Manage Rewards

Sequence Diagram

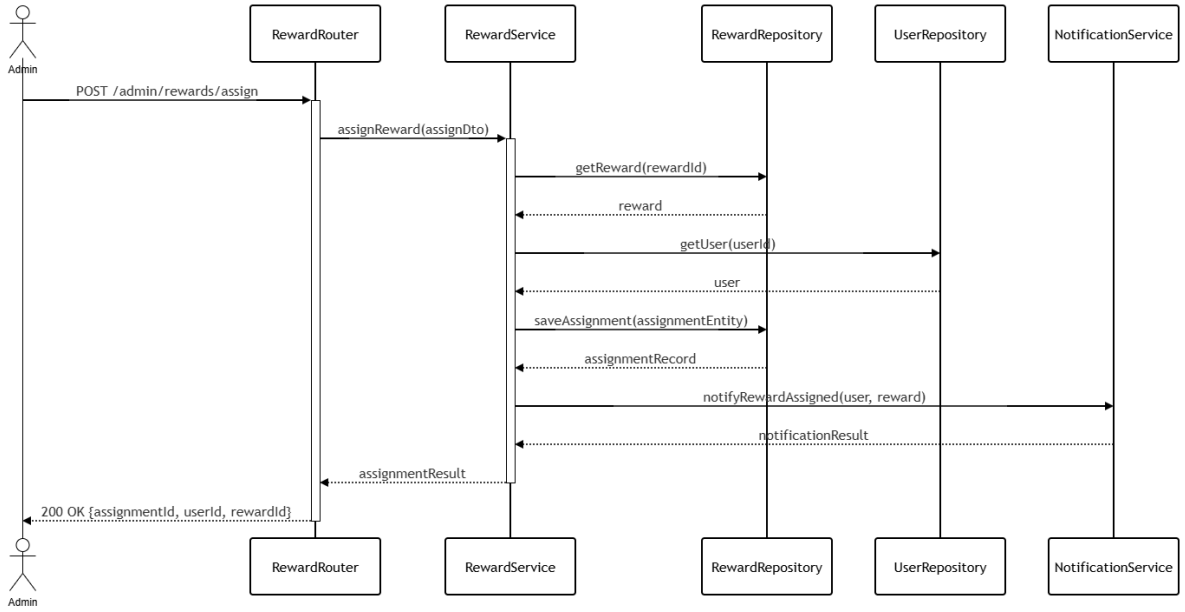


Figure 4.18: Admin Manage Rewards Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
RewardRouter	create_reward(), update_reward(), assign_reward(), get_rewards()	controller / route	Manages reward configuration, assignment, and retrieval endpoints.
RewardCreate	title, description, value, rank, tournament_id	DTO	Input DTO for defining a new reward within a tournament context.
RewardAssign	reward_id, user_id, race_id, reason	DTO	Input DTO for assigning a reward to a user or race result.
RewardOut	id, title, description, value, rank, status	DTO	Output DTO returning reward information and distribution status.
Reward	id, title, description, value, rank, status, tournament_id	entity	Entity representing a reward definition and its current state.
RewardAssignment	id, reward_id, user_id, race_id, assigned_at, reason	entity	Entity capturing reward assignment history and references.

Table 4.6: Admin Manage Rewards class specification

4.2.7 Login and Registration

Class Diagram

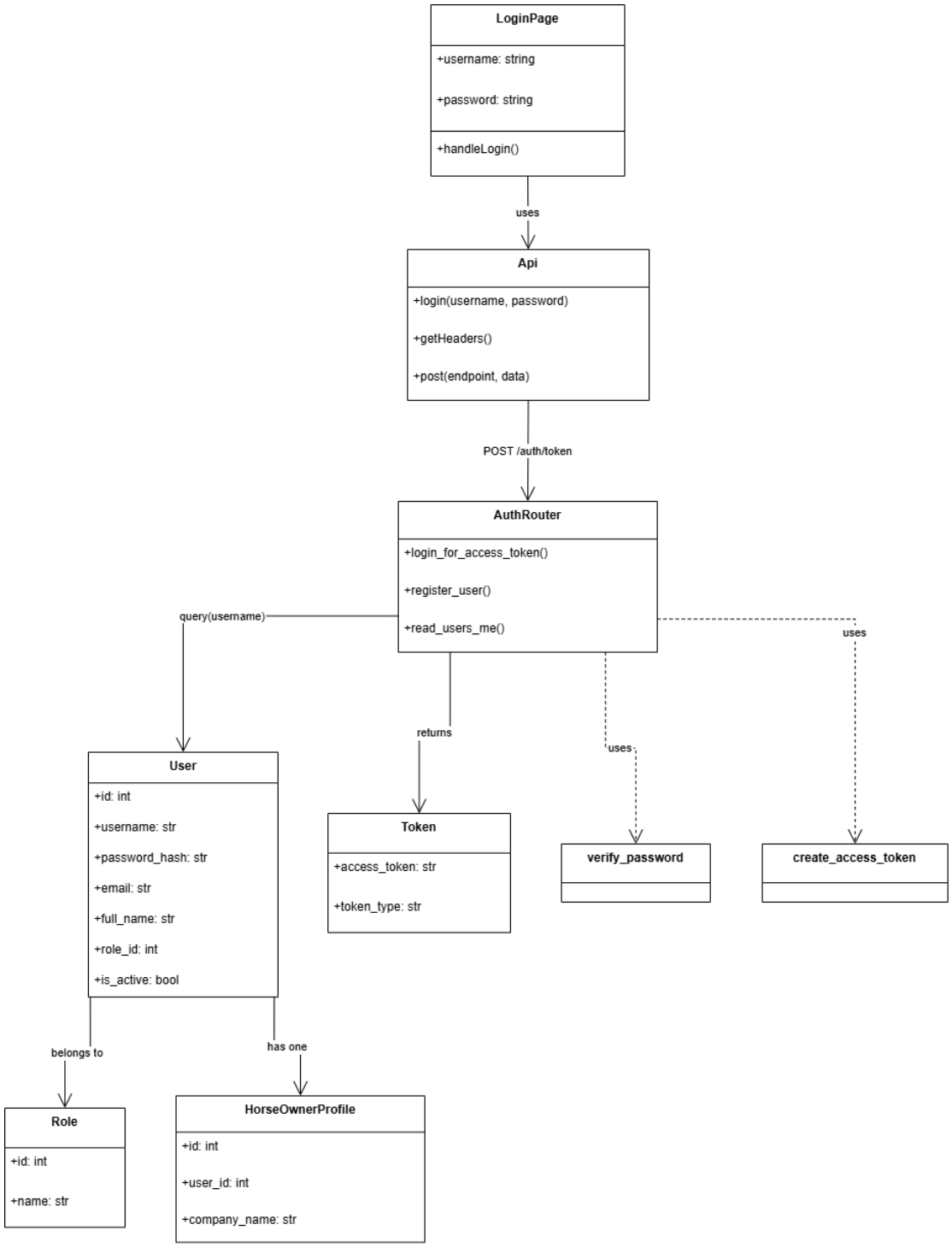


Figure 4.19: Login Class Diagram

Sequence Diagram

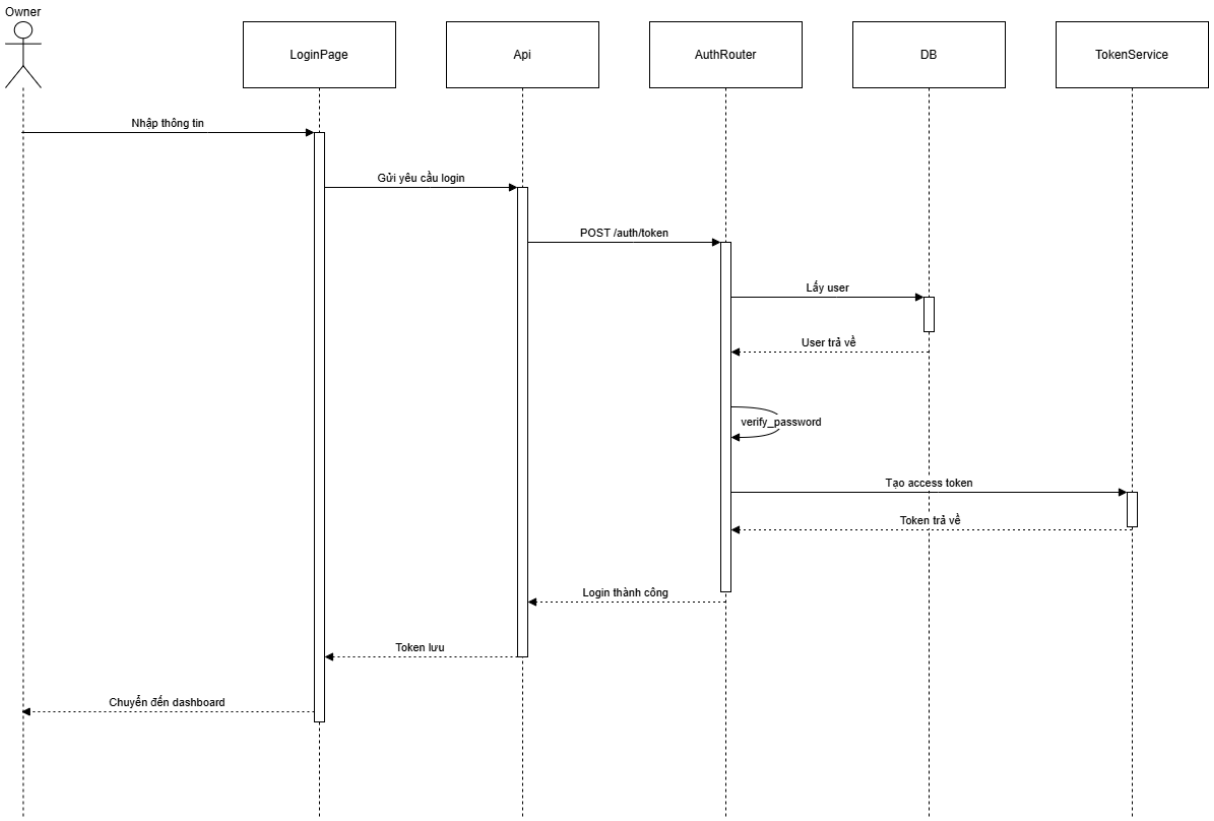


Figure 4.20: Login Sequence Diagram

Class Specification

Table 4.7: Login class specification

Class	Attributes / Methods	Type	Description
LoginPage	username, password, handleLogin()	state / action	UI component that collects login credentials and calls the API.
Api	login(), post(), getHeaders()	service / helper	Calls `POST /auth/token`, stores JWT, and prepares request headers.
AuthRouter	login_for_access_token(), read_users_me()	controller / route	Validates username/password and issues bearer tokens.

Class	Attributes / Methods	Type	Description
User	id, username, password_hash, email, full_name, role_id, is_active	entity	Database entity representing an application user.
Role	id, name	entity	Defines user roles, including `OWNER`.
Token	access_token, token_type	DTO	Response payload returned after successful login.
HorseOwner Profile	id, user_id, company_name	entity	Owner-specific profile linked to the User entity.

4.2.8 Tournament Registration

Tournament Registration Data Preparation Class Diagram

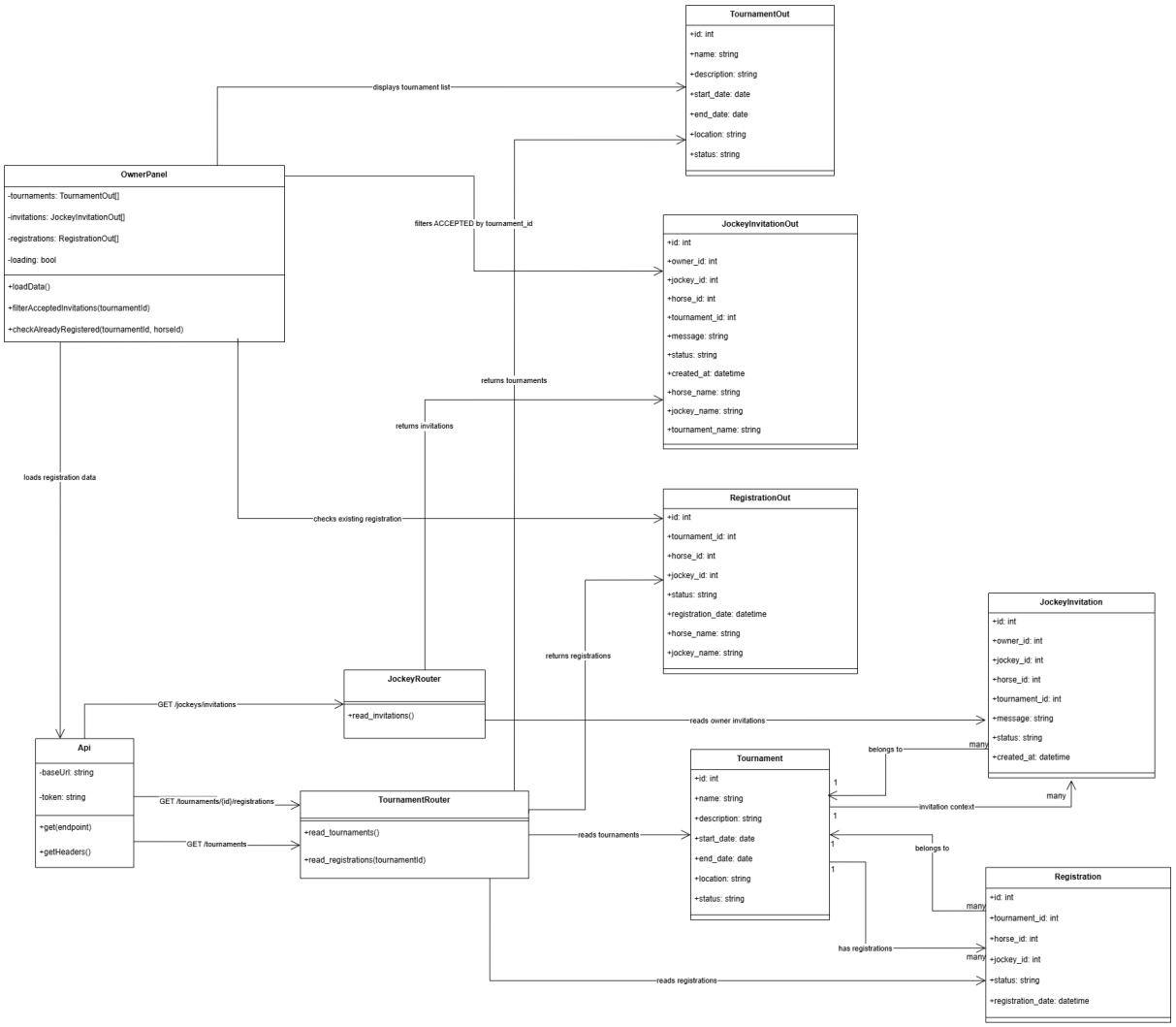


Figure 4.21: Tournament Registration Data Preparation Class Diagram

Load Accepted Registration Options Sequence Diagram

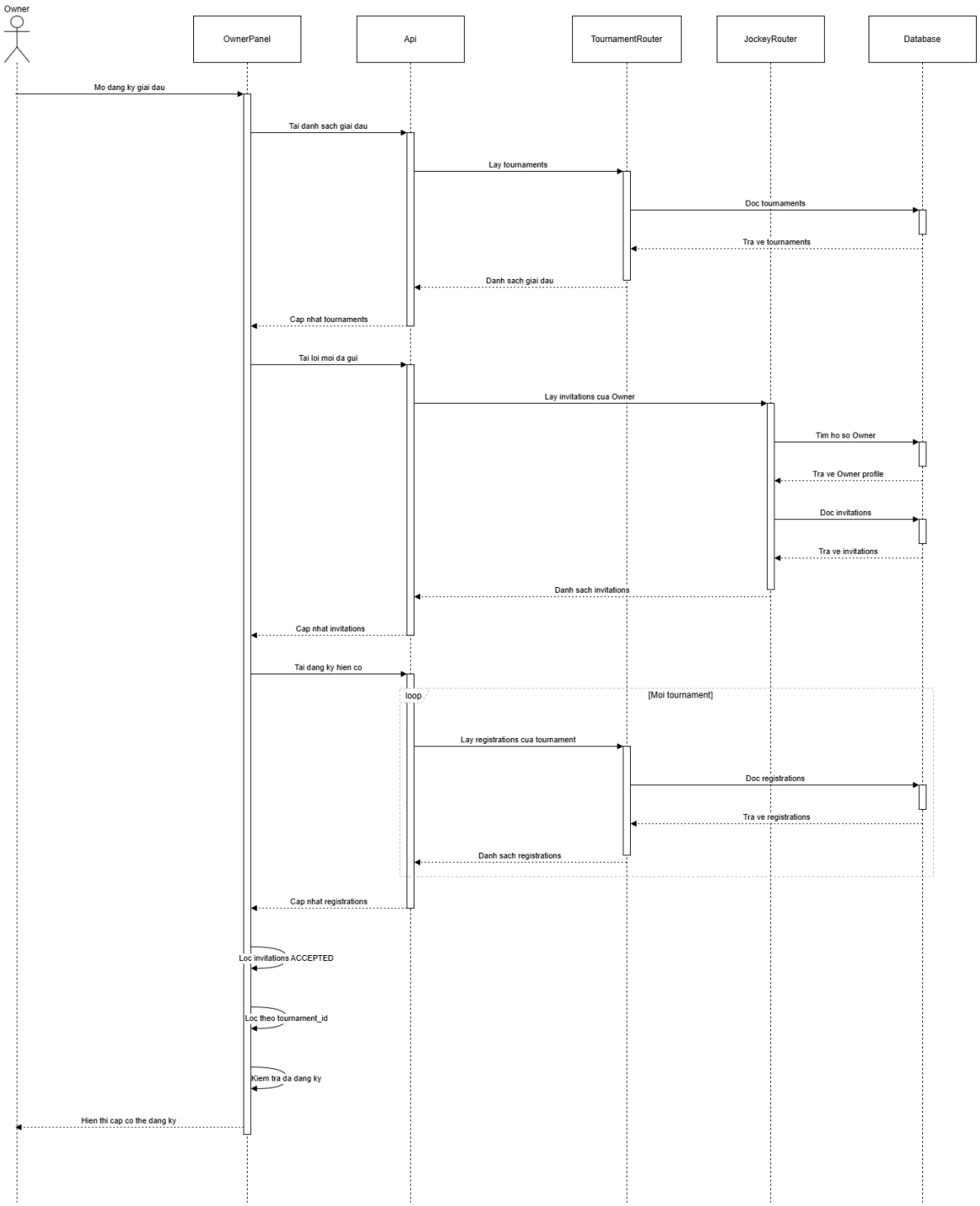


Figure 4.22: Load Accepted Registration Options Sequence Diagram

Class Specification

Table 4.8: Tournament registration data preparation class specification

Class	Attributes / Methods	Type	Description
OwnerPanel	tournaments, invitations, registrations, loading, loadData(), filterAcceptedInvitations(), checkAlreadyRegistered()	boundary / UI	Loads tournaments, owner invitations, and existing registrations, then filters accepted horse-jockey pairs available for tournament registration.
Api	baseUrl, token, get(), getHeaders()	service / helper	Sends data-loading requests to tournament and jockey endpoints with the owner authentication token.
Tournament Router	read_tournaments(), read_registrations()	controller / route	Provides tournament records and registration lists used by the frontend to prepare registration options.
JockeyRouter	read_invitations()	controller / route	Provides invitations sent by the owner, including their acceptance status and related horse-jockey information.
Tournament Out	id, name, description, start_date, end_date, location, status	DTO / response	Returns tournament data displayed in the registration screen.
JockeyInvitation Out	id, owner_id, jockey_id, horse_id, tournament_id, message, status, created_at, horse_name, jockey_name, tournament_name	DTO / response	Returns invitation data that the frontend filters by `ACCEPTED` status and matching tournament.
Registration Out	id, tournament_id, horse_id, jockey_id, status, registration_date, horse_name, jockey_name	DTO / response	Returns existing registrations so the frontend can prevent duplicate registration buttons for the same horse and tournament.
Tournament	id, name, description, start_date, end_date, location, status	entity	Stores tournament records that may be selected for registration.

Class	Attributes / Methods	Type	Description
Jockey Invitation	id, owner_id, jockey_id, horse_id, tournament_id, message, status, created_at	entity	Stores owner-jockey invitation records and their current acceptance status.
Registration	id, tournament_id, horse_id, jockey_id, status, registration_date	entity	Stores tournament registration records and approval status.

Tournament Registration Submission Class Diagram

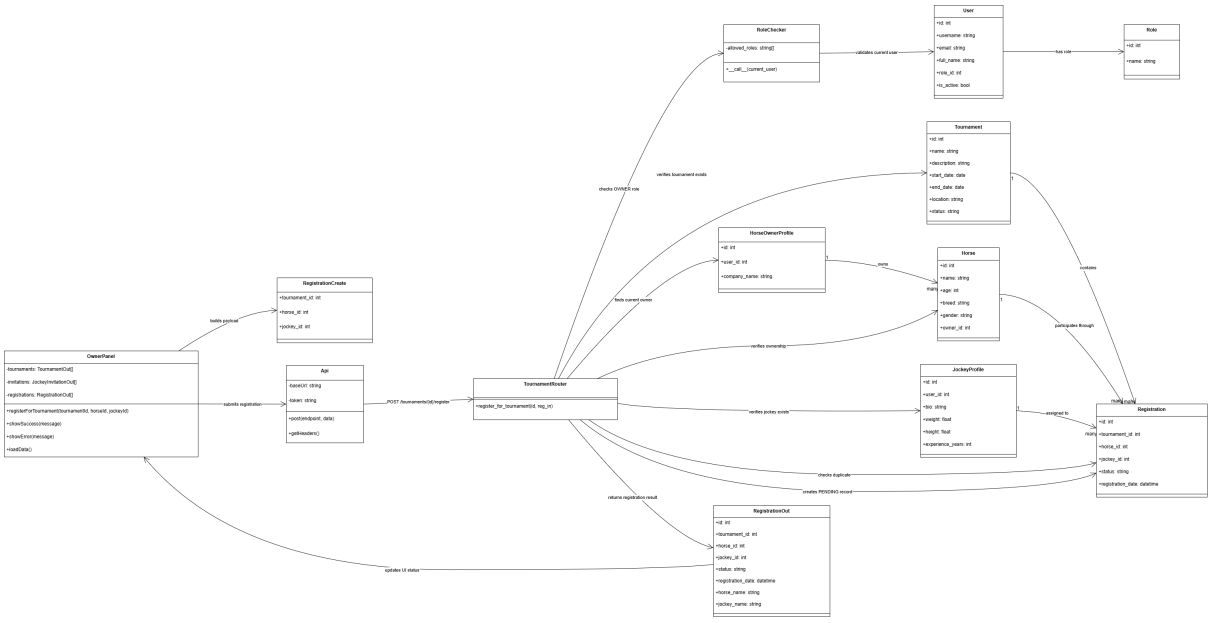


Figure 4.23: Tournament Registration Submission Class Diagram

Submit Tournament Registration Sequence Diagram

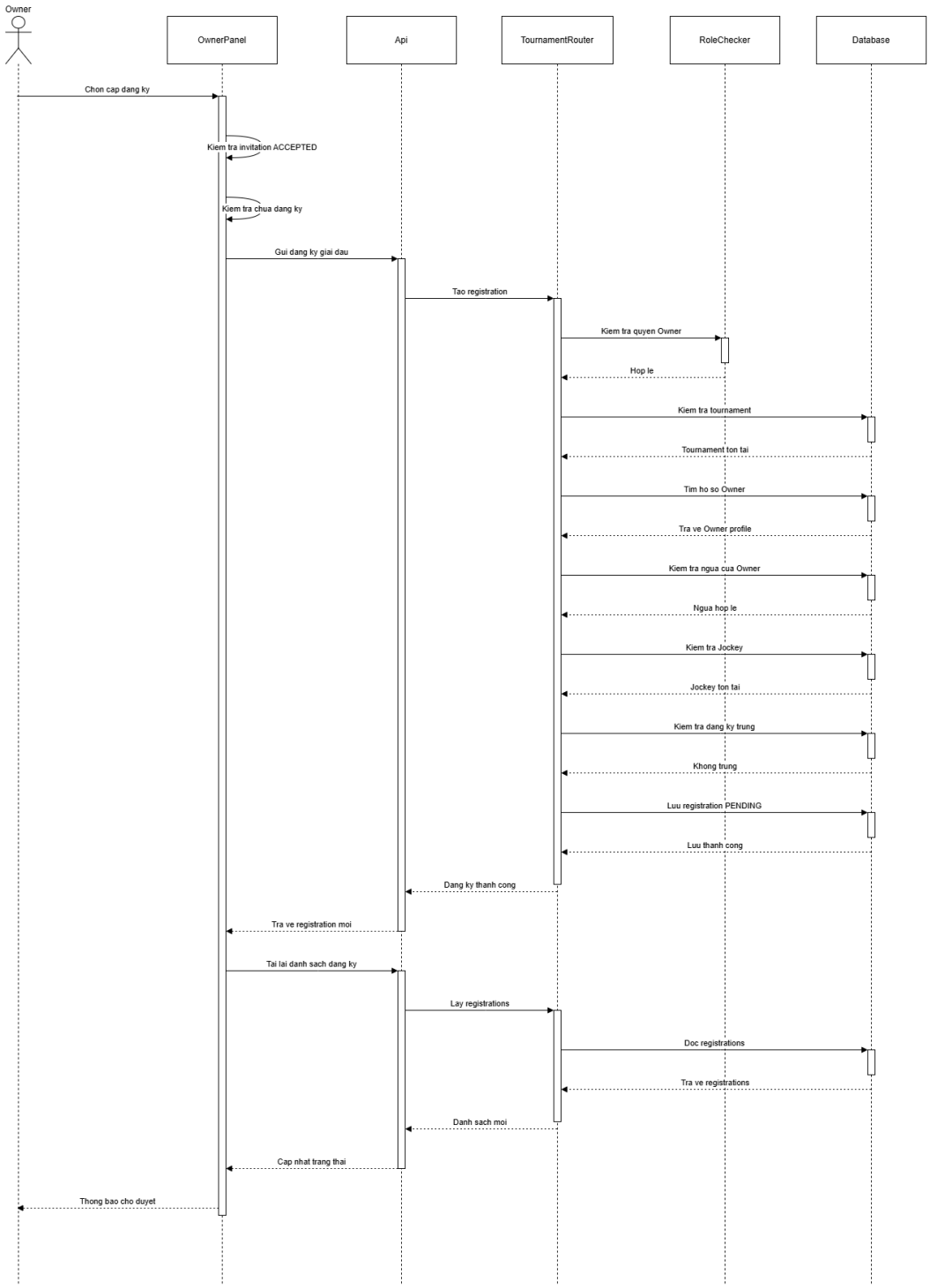


Figure 4.24: Submit Tournament Registration Sequence Diagram

Class Specification

Table 4.9: Tournament registration submission class specification

Class	Attributes / Methods	Type	Description
OwnerPanel	tournaments, invitations, registrations, registerForTournament(), showSuccess(), showError(), loadData()	boundary / UI	Submits an accepted horse-jockey pair to a selected tournament and refreshes registration status after submission.
Api	baseUrl, token, post(), getHeaders()	service / helper	Sends the tournament registration payload to `POST /tournaments/{id}/register`.
Tournament Router	register_for_tournament()	controller / route	Validates the tournament registration request and creates a pending registration when all rules are satisfied.
RoleChecker	allowed_roles, ___call___()	service / authorization	Ensures that only an authenticated `OWNER` user can submit tournament registrations.
Registration Create	tournament_id, horse_id, jockey_id	DTO / request	Carries the selected tournament, horse, and jockey identifiers in the registration request.
Registration Out	id, tournament_id, horse_id, jockey_id, status, registration_date, horse_name, jockey_name	DTO / response	Returns the created registration with status information for display on the owner dashboard.
User	id, username, email, full_name, role_id, is_active	entity	Represents the authenticated account used to resolve the current owner.
Role	id, name	entity	Defines user roles and supports owner-only registration authorization.
HorseOwner Profile	id, user_id, company_name	entity	Links the authenticated owner account to horses that may be registered.

Class	Attributes / Methods	Type	Description
Tournament	id, name, description, start_date, end_date, location, status	entity	Represents the tournament that receives the registration request.
Horse	id, name, age, breed, gender, owner_id	entity	Represents the selected horse and is validated to ensure ownership by the current owner.
JockeyProfile	id, user_id, bio, weight, height, experience_years	entity	Represents the selected jockey who will ride the horse in the tournament.
Registration	id, tournament_id, horse_id, jockey_id, status, registration_date	entity	Stores the submitted tournament registration with default status `PENDING`.
Jockey Invitation	id, owner_id, jockey_id, horse_id, tournament_id, status	entity	Represents the accepted invitation pair used by the frontend before submitting registration.

4.2.9 Horse Owner Authentication and Authorization

Class Diagram

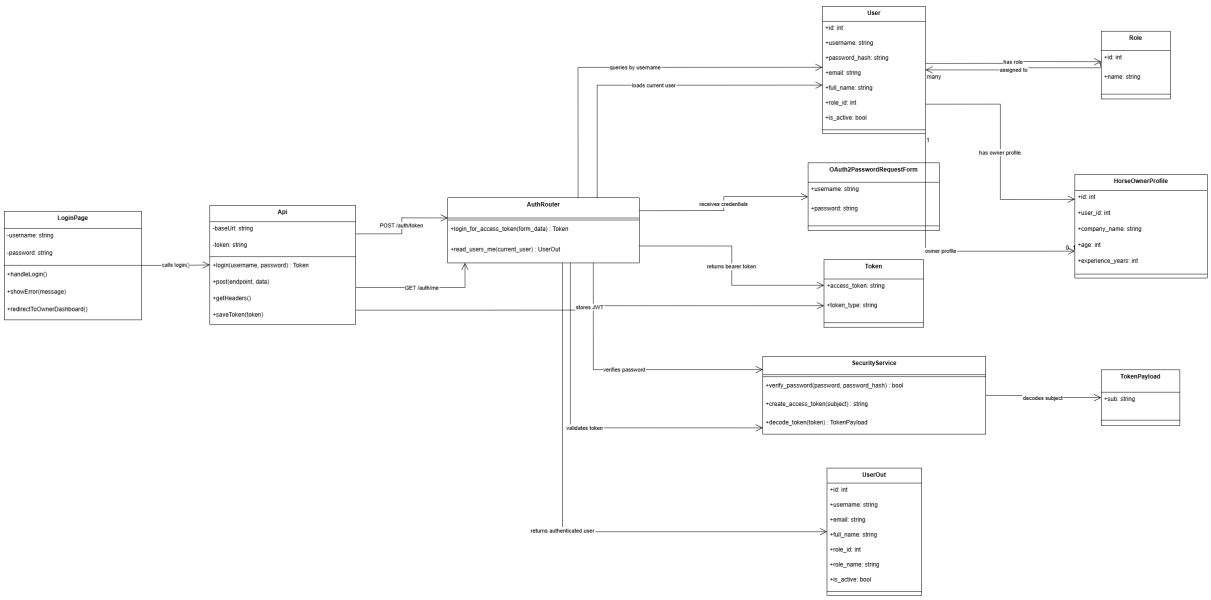


Figure 4.25: Horse Owner Authentication and Authorization Class Diagram

Sequence Diagram

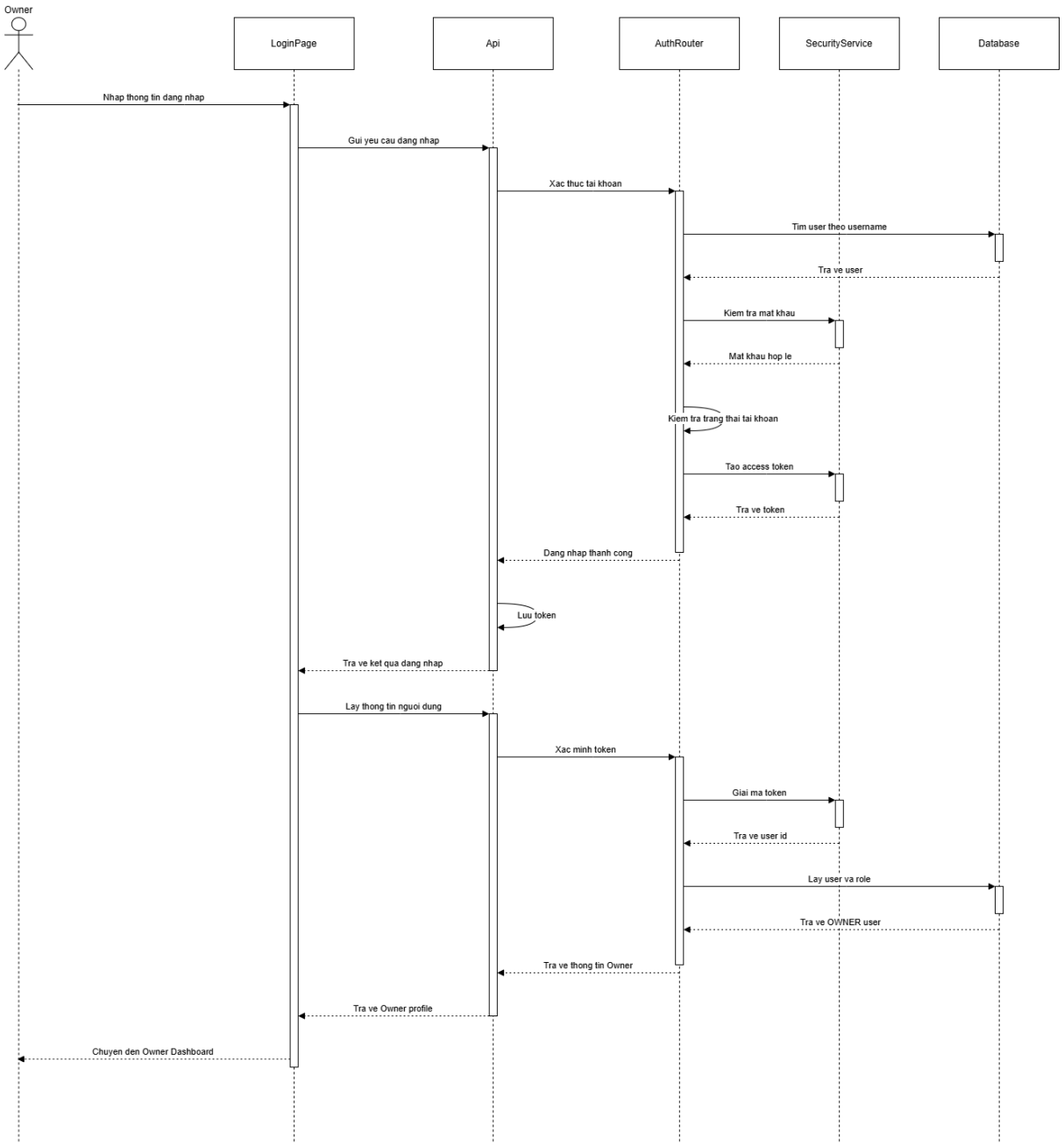


Figure 4.26: Horse Owner Authentication and Authorization Sequence Diagram

Class Specification

Table 4.10: Horse Owner authentication and authorization class specification

Class	Attributes / Methods	Type	Description
LoginPage	username, password, handleLogin(), showError(), redirectToOwnerDashboard()	boundary / UI	Collects owner credentials, sends the login request, displays authentication errors, and redirects a valid owner to the owner dashboard.
Api	baseUrl, token, login(), post(), getHeaders(), saveToken()	service / helper	Sends authentication requests to the backend, stores the JWT token, and prepares authorization headers for protected owner operations.
AuthRouter	login_for_access_token(), read_users_me()	controller / route	Processes login requests, validates submitted credentials, issues bearer tokens, and returns authenticated user information.
Security Service	verify_password(), create_access_token(), decode_token()	service	Verifies password hashes and creates or validates JWT tokens used in the owner authentication flow.
OAuth2PasswordRequestForm	username, password	DTO / request	Carries the submitted username and password from the login form to the authentication endpoint.
Token	access_token, token_type	DTO / response	Represents the bearer token returned to the frontend after successful authentication.
TokenPayload	sub	DTO / payload	Stores the user identifier decoded from the JWT token when resolving the current authenticated user.

Class	Attributes / Methods	Type	Description
User	id, username, password_hash, email, full_name, role_id, is_active	entity	Stores account and credential information used to authenticate the owner and verify account status.
Role	id, name	entity	Defines user roles and allows the system to confirm that the authenticated account has the `OWNER` role.
HorseOwner Profile	id, user_id, company_name, age, experience_years	entity	Stores owner-specific profile information linked to the authenticated user account.
UserOut	id, username, email, full_name, role_id, role_name, is_active	DTO / response	Returns authenticated user details to the frontend so the dashboard can render owner-specific screens.

4.2.10 Horse Owner Management Operations

Class Diagram

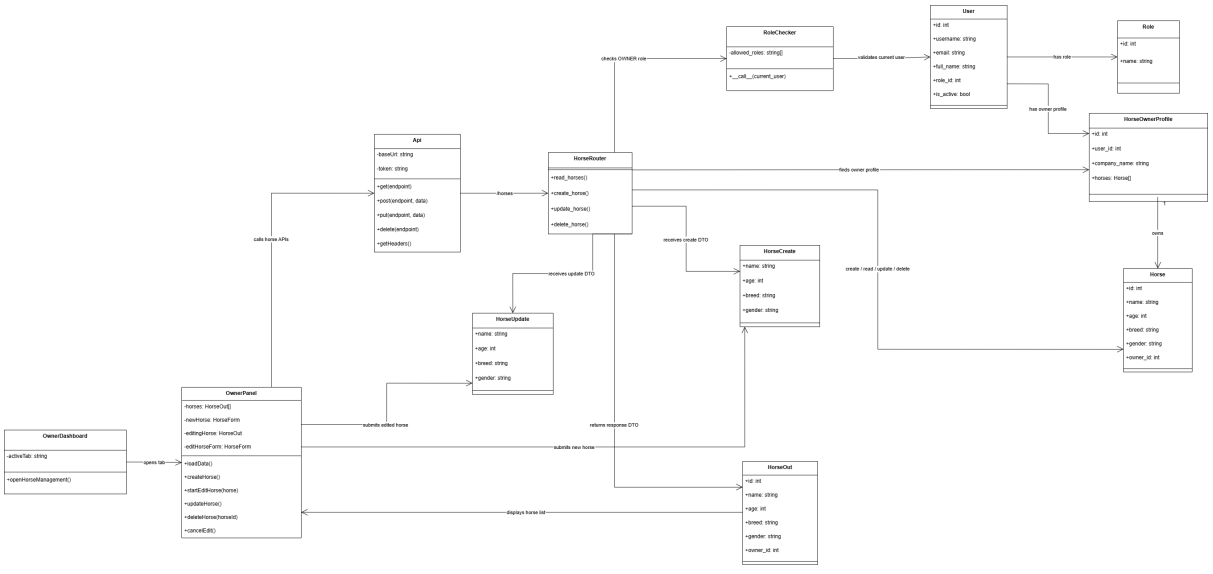


Figure 4.27: Horse Owner Management Operations Class Diagram

Sequence Diagram

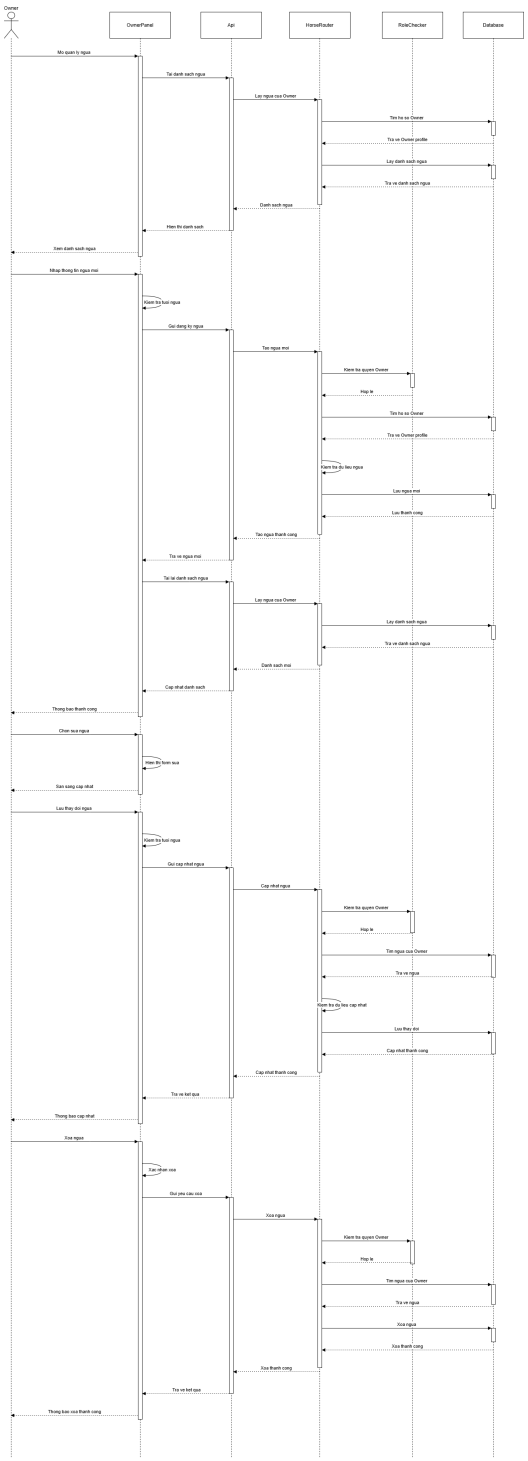


Figure 4.28: Horse Owner Management Operations Sequence Diagram

Class Specification

Table 4.11: Horse Owner management operations class specification

Class	Attributes / Methods	Type	Description
Owner Dashboard	activeTab, openHorseManagement()	boundary / UI	Provides the owner dashboard navigation and opens the horse management tab for owner operations.
OwnerPanel	horses, newHorse, editingHorse, editHorseForm, loadData(), createHorse(), startEditHorse(), updateHorse(), deleteHorse(), cancelEdit()	boundary / UI	Displays the list of owned horses and manages the create, update, and delete interactions from the frontend.
Api	baseUrl, token, get(), post(), put(), delete(), getHeaders()	service / helper	Sends horse management requests to the backend and attaches the owner authentication token to protected API calls.
HorseRouter	read_horses(), create_horse(), update_horse(), delete_horse()	controller / route	Handles HTTP requests for viewing, registering, editing, and deleting horses owned by the authenticated owner.
RoleChecker	allowed_roles, ___call___()	service / authorization	Verifies that the current authenticated user has the `OWNER` role before allowing protected horse operations.
HorseCreate	name, age, breed, gender	DTO / request	Carries the input data required to register a new horse; age must be between 2 and 10.
HorseUpdate	name, age, breed, gender	DTO / request	Carries optional horse fields submitted when an owner edits an existing horse record.
HorseOut	id, name, age, breed, gender, owner_id	DTO / response	Returns horse information to the frontend after list, create, or update operations.

Class	Attributes / Methods	Type	Description
User	id, username, email, full_name, role_id, is_active	entity	Represents the authenticated account used to identify the owner performing horse management operations.
Role	id, name	entity	Defines the user's role and supports authorization checks for owner-only operations.
HorseOwner Profile	id, user_id, company_name, horses	entity	Stores owner-specific information and links the authenticated owner to the horses they manage.
Horse	id, name, age, breed, gender, owner_id	entity	Database entity storing horse records owned and managed by a horse owner.

4.2.11 Horse Owner Send Jockey Invitation

Class Diagram

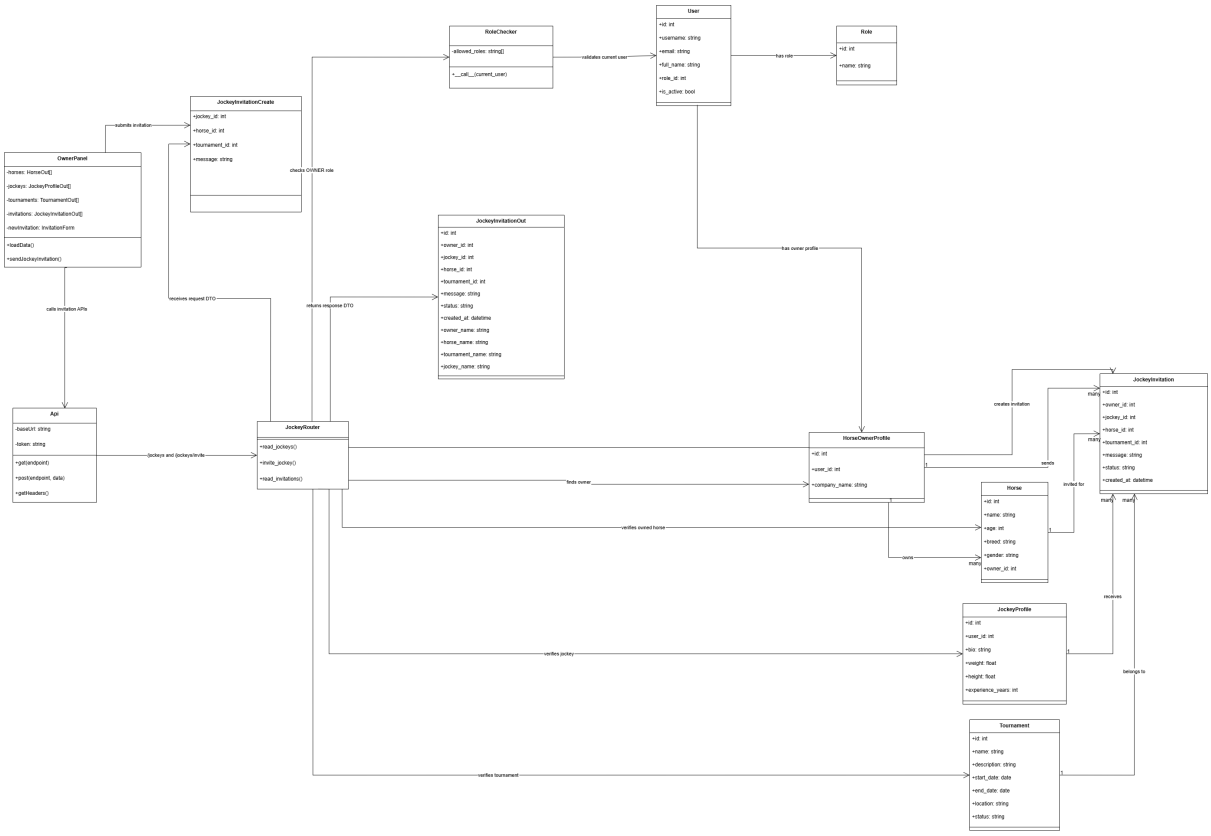


Figure 4.29: Send Jockey Invitation Class Diagram

Sequence Diagram

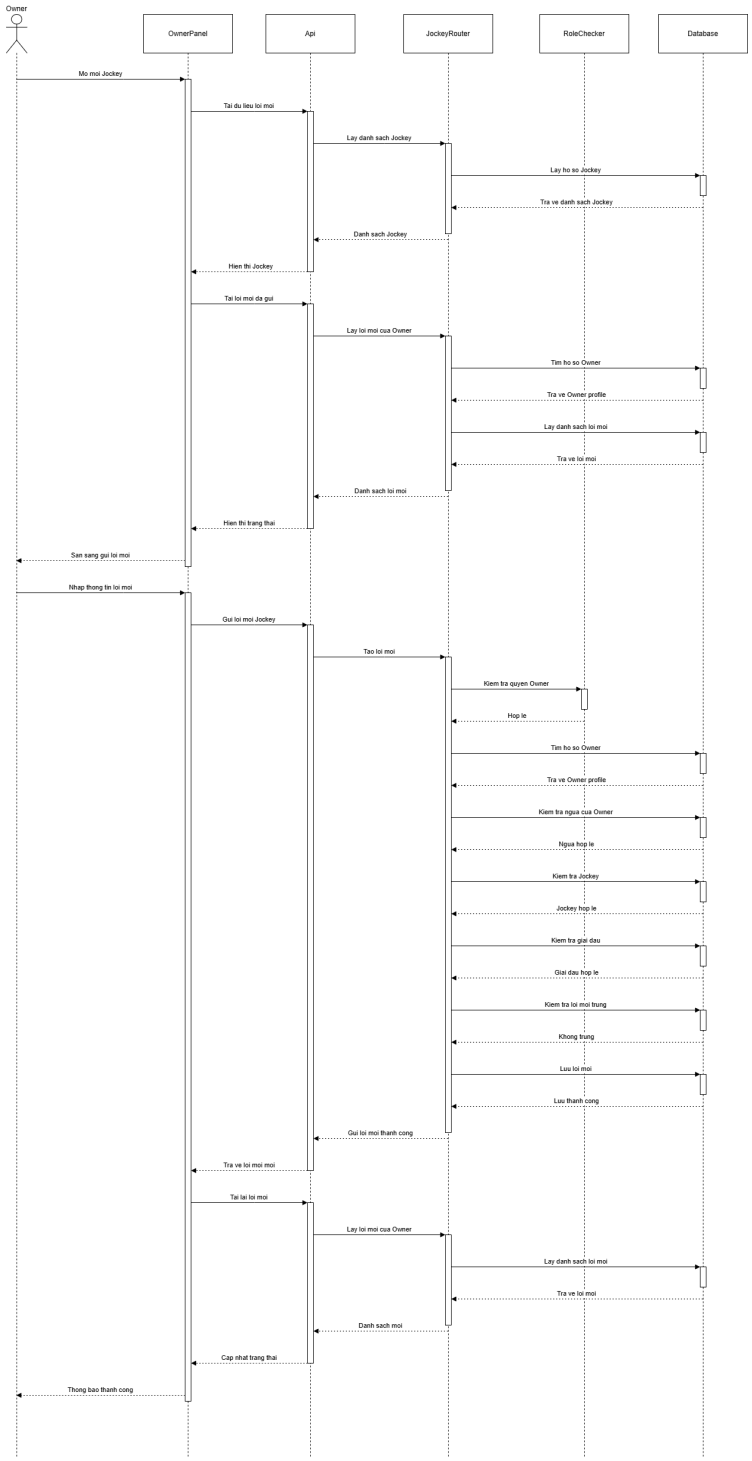


Figure 4.30: Send Jockey Invitation Sequence Diagram

Class Specification

Table 4.12: Send Jockey Invitation class specification

Class	Attributes / Methods	Type	Description
OwnerPanel	horses, jockeys, tournaments, invitations, newInvitation, loadData(), sendJockeyInvitation()	boundary / UI	Displays the invitation form, loads available horses, jockeys, tournaments, and shows the status of invitations already sent by the owner.
Api	baseUrl, token, get(), post(), getHeaders()	service / helper	Sends requests to retrieve jockeys and invitations, and submits new invitation data to the backend with authorization headers.
JockeyRouter	read_jockeys(), invite_jockey(), read_invitations()	controller / route	Handles HTTP requests for listing jockey profiles, creating owner invitations, and retrieving invitation status records.
RoleChecker	allowed_roles, ___call___()	service / authorization	Ensures that only an authenticated user with the `OWNER` role can send a jockey invitation.
JockeyInvitationCreate	jockey_id, horse_id, tournament_id, message	DTO / request	Carries the selected jockey, horse, tournament, and optional invitation message submitted by the owner.
JockeyInvitationOut	id, owner_id, jockey_id, horse_id, tournament_id, message, status, created_at, owner_name, horse_name, tournament_name, jockey_name	DTO / response	Returns the created or retrieved invitation information, including related display names and current invitation status.
User	id, username, email, full_name, role_id, is_active	entity	Represents the authenticated account used to identify the owner and resolve jockey display information.

Class	Attributes / Methods	Type	Description
Role	id, name	entity	Defines user roles and supports owner-only authorization for sending invitations.
HorseOwner Profile	id, user_id, company_name	entity	Stores owner-specific information and links the authenticated owner to the invitations they send.
Horse	id, name, age, breed, gender, owner_id	entity	Represents the horse selected by the owner and is validated to ensure it belongs to the current owner.
JockeyProfile	id, user_id, bio, weight, height, experience_years	entity	Represents the jockey profile selected as the invitation recipient.
Tournament	id, name, description, start_date, end_date, location, status	entity	Represents the tournament context in which the owner invites a jockey to partner with a horse.
Jockey Invitation	id, owner_id, jockey_id, horse_id, tournament_id, message, status, created_at	entity	Stores the invitation record and tracks whether it is `PENDING`, `ACCEPTED`, or `REJECTED`.

4.2.12 View Registered Tournaments

Class Diagram

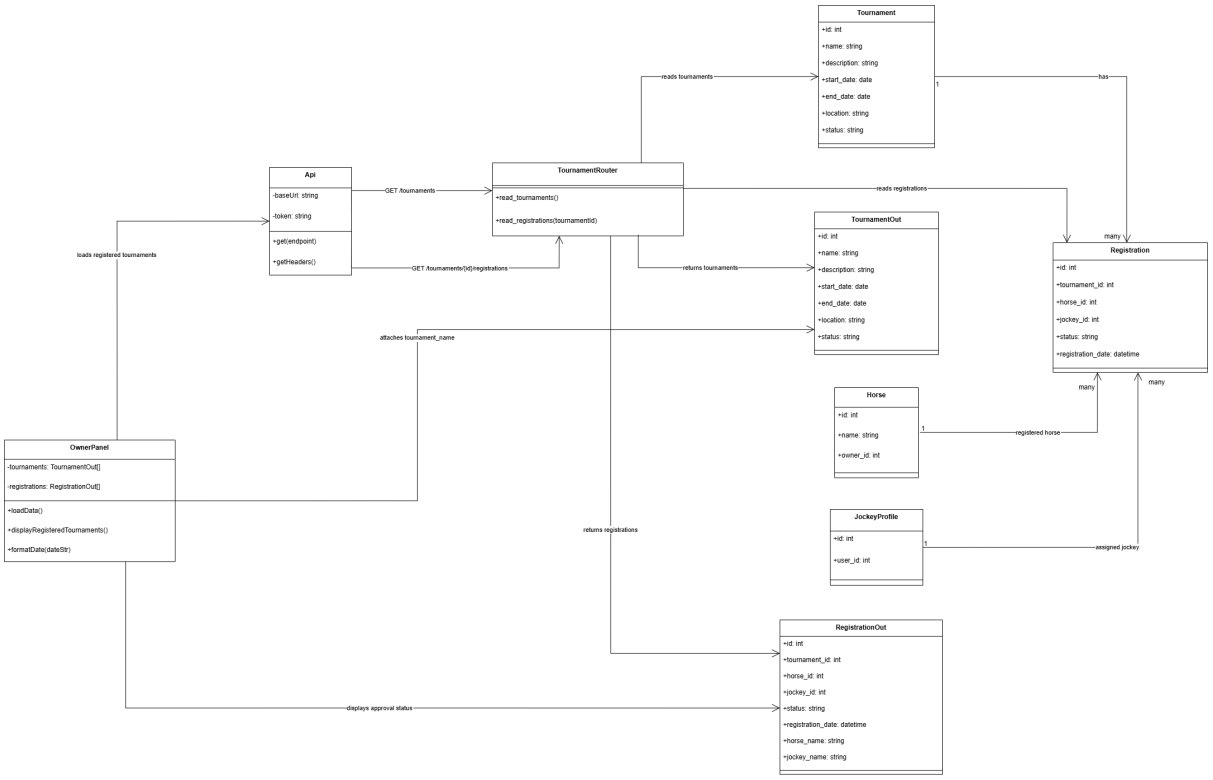


Figure 4.31: View Registered Tournaments Class Diagram

Sequence Diagram

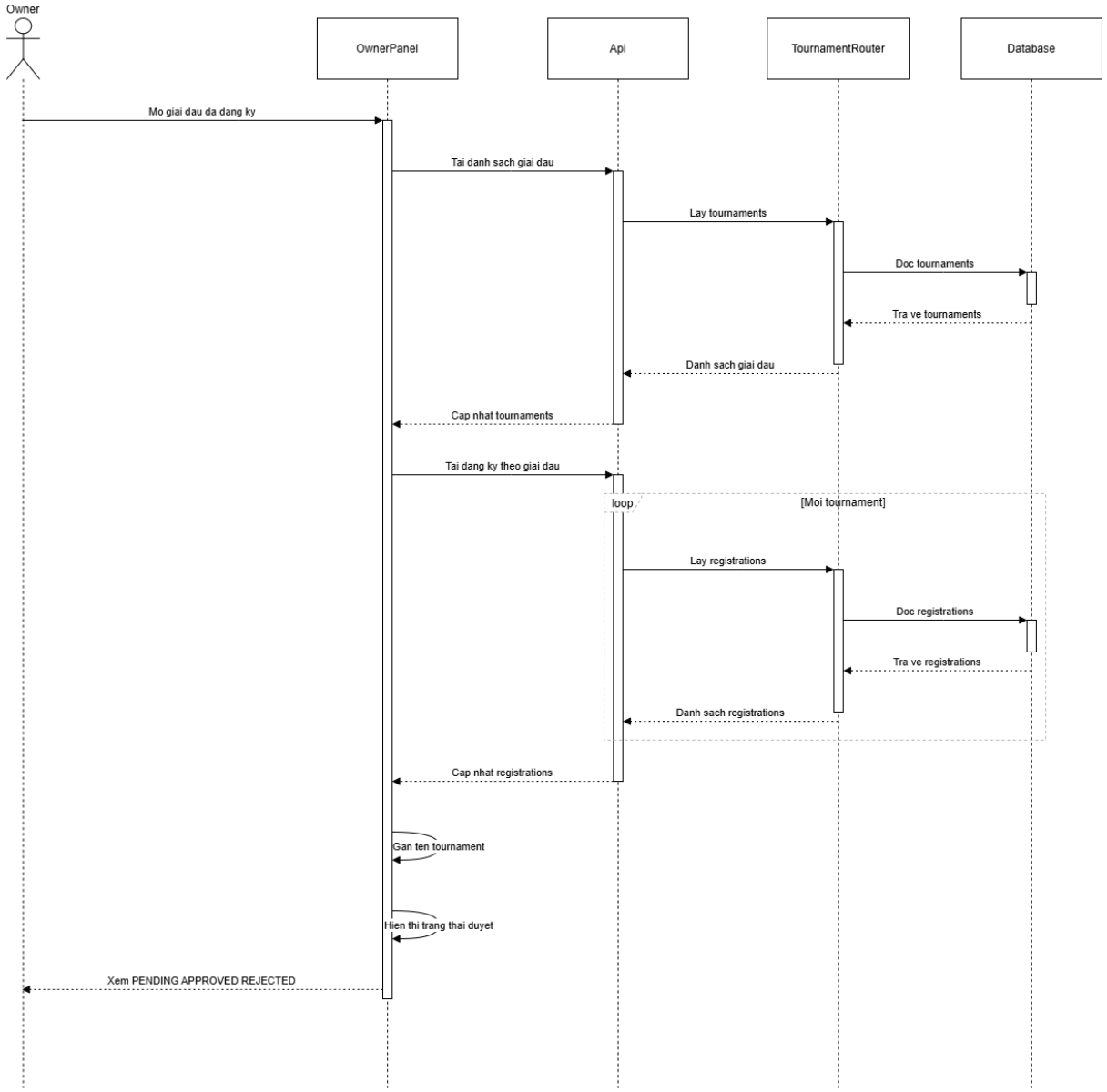


Figure 4.32: View Registered Tournaments Sequence Diagram

Class Specification

Table 4.13: View registered tournaments class specification

Class	Attributes / Methods	Type	Description
OwnerPanel	tournaments, registrations, loadData(), displayRegisteredTournaments(), formatDate()	boundary / UI	Displays the list of tournaments registered by the owner and shows the approval status for each registration.
Api	baseUrl, token, get(), getHeaders()	service / helper	Sends requests to retrieve tournaments and registration records with authorization headers.
Tournament Router	read_tournaments(), read_registrations()	controller / route	Provides tournament lists and registration records used to build the registered tournaments table.
Tournament Out	id, name, description, start_date, end_date, location, status	DTO / response	Returns tournament information that is attached to registration records for display.
Registration Out	id, tournament_id, horse_id, jockey_id, status, registration_date, horse_name, jockey_name	DTO / response	Returns registration history and current approval status such as `PENDING`, `APPROVED`, or `REJECTED`.
Tournament	id, name, description, start_date, end_date, location, status	entity	Stores tournament data referenced by each registration.
Registration	id, tournament_id, horse_id, jockey_id, status, registration_date	entity	Stores submitted tournament registrations and their approval status.
Horse	id, name, owner_id	entity	Represents the horse registered by the owner for a tournament.
JockeyProfile	id, user_id	entity	Represents the jockey assigned to a registered horse.

4.2.13 View Horse Schedules for Owner

Class Diagram

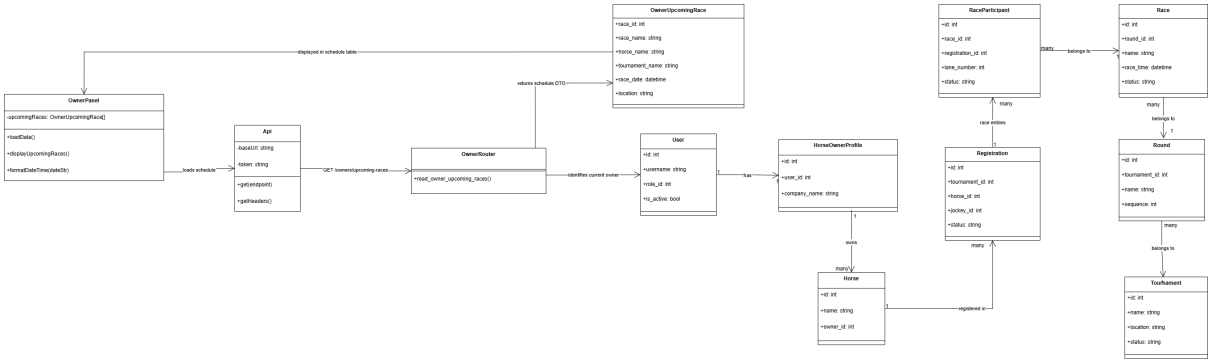


Figure 4.33: View Horse Schedules Class Diagram

Sequence Diagram

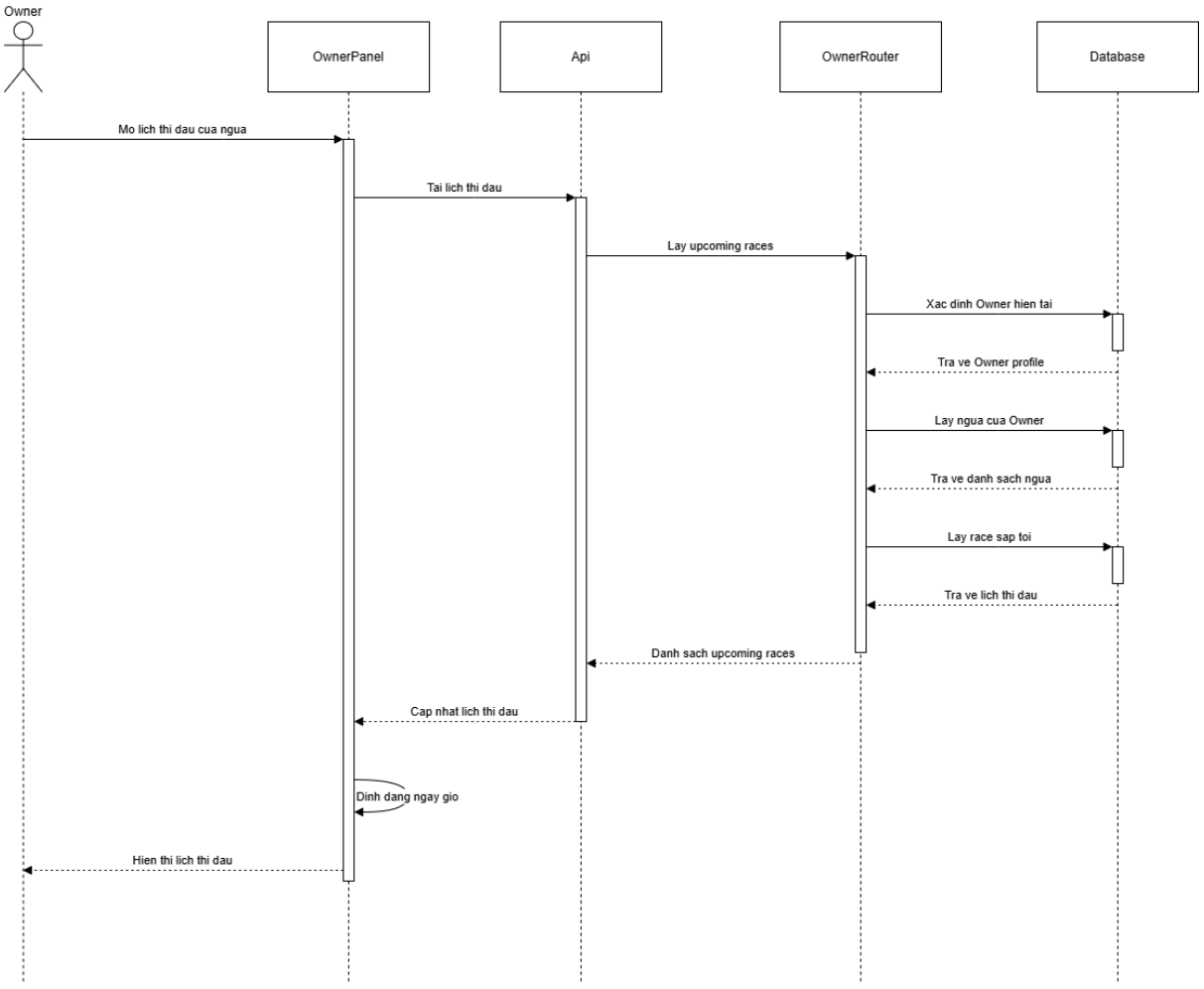


Figure 4.34: View Horse Schedules Sequence Diagram

Class Specification

Table 4.14: View horse schedules class specification

Class	Attributes / Methods	Type	Description
OwnerPanel	upcomingRaces, loadData(), displayUpcomingRaces(), formatDateTime()	boundary / UI	Displays upcoming race schedules for horses owned by the authenticated owner.
Api	baseUrl, token, get(), getHeaders()	service / helper	Sends the schedule request to `GET /owners/upcoming-races`.
OwnerRouter	read_owner_upcoming_races()	controller / route	Retrieves scheduled future races for horses that belong to the current owner.
OwnerUpcomingRace	race_id, race_name, horse_name, tournament_name, race_date, location	DTO / response	Returns formatted schedule information displayed in the owner schedule screen.
User	id, username, role_id, is_active	entity	Represents the authenticated owner account used to filter schedule data.
HorseOwnerProfile	id, user_id, company_name	entity	Links the authenticated user to owned horses.
Horse	id, name, owner_id	entity	Represents horses owned by the current owner.
Registration	id, tournament_id, horse_id, jockey_id, status	entity	Connects owned horses to tournament participation records.
RaceParticipant	id, race_id, registration_id, lane_number, status	entity	Represents a registered horse-jockey pair assigned to a race.
Race	id, round_id, name, race_time, status	entity	Stores race schedule data and is filtered to upcoming scheduled races.
Round	id, tournament_id, name, sequence	entity	Links a race to its tournament round.
Tournament	id, name, location, status	entity	Provides tournament name and location displayed in the schedule.

4.2.14 View Race Results for Owner

Class Diagram

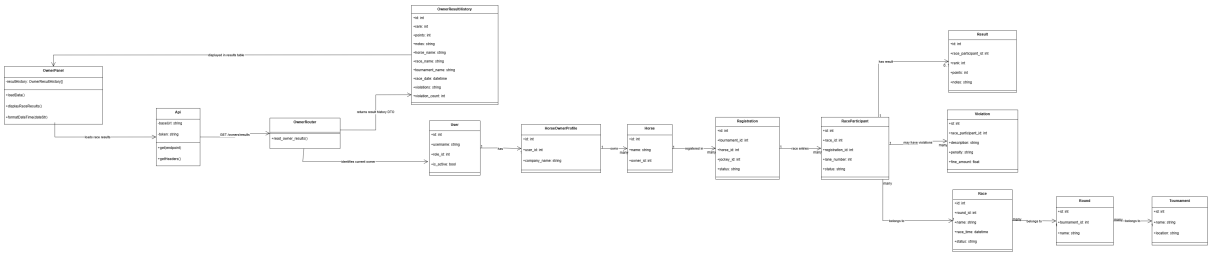


Figure 4.35: View Race Results Class Diagram

Sequence Diagram

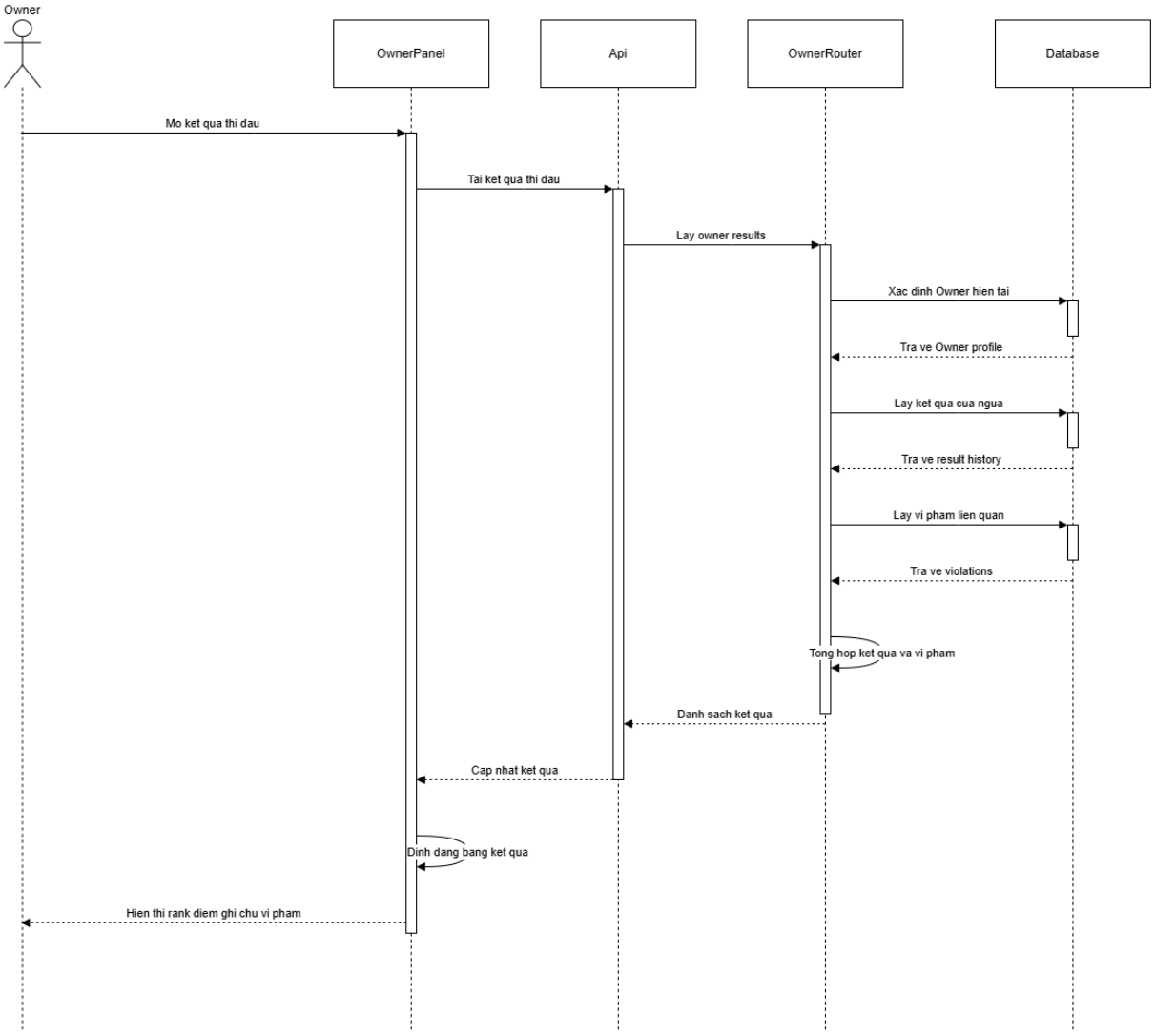


Figure 4.36: View Race Results Sequence Diagram

Class Specification

Table 4.15: View race results class specification

Class	Attributes / Methods	Type	Description
OwnerPanel	resultHistory, loadData(), displayRaceResults(), formatDateTime()	boundary / UI	Displays race result history for the owner's horses, including rank, points, notes, and violations.
Api	baseUrl, token, get(), getHeaders()	service / helper	Sends the result history request to `GET /owners/results`.
OwnerRouter	read_owner_results()	controller / route	Retrieves completed race results and related violations for horses owned by the current owner.
OwnerResult History	id, rank, points, notes, horse_name, race_name, tournament_name, race_date, violations, violation_count	DTO / response	Returns detailed result history displayed in the owner result table.
User	id, username, role_id, is_active	entity	Represents the authenticated owner account used to filter result data.
HorseOwner Profile	id, user_id, company_name	entity	Links the authenticated owner to horses whose results are displayed.
Horse	id, name, owner_id	entity	Represents the owner's horses that have participated in races.
Registration	id, tournament_id, horse_id, jockey_id, status	entity	Connects an owned horse to a tournament participation record.
Race Participant	id, race_id, registration_id, lane_number, status	entity	Represents a horse-jockey pair participating in a specific race.
Result	id, race_participant_id, rank, points, notes	entity	Stores official race result information for a race participant.
Violation	id, race_participant_id, description, penalty, fine_amount	entity	Stores rule violations associated with a race participant.

Class	Attributes / Methods	Type	Description
Race	id, round_id, name, race_time, status	entity	Provides race name and race date for the result history.
Round	id, tournament_id, name	entity	Links each race to its tournament context.
Tournament	id, name, location	entity	Provides tournament information displayed with race results.

4.2.15 Horse Owner Profile Management

Class Diagram

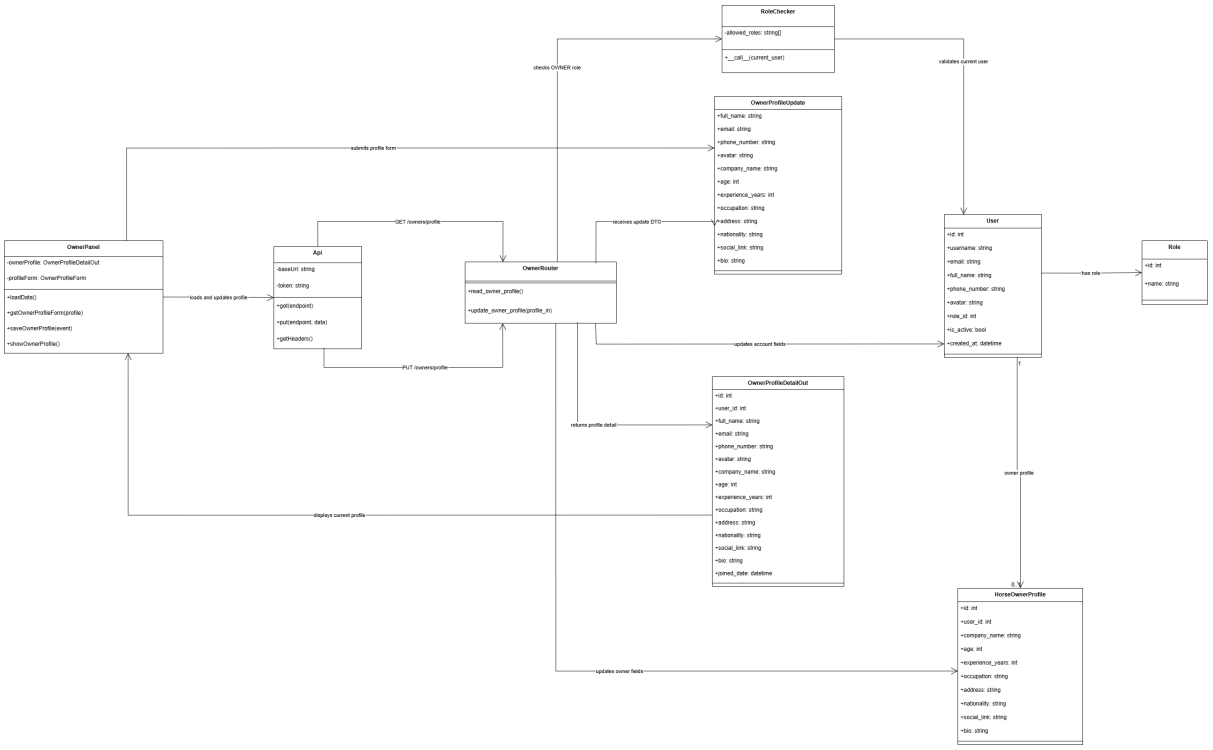


Figure 4.37: Horse Owner Profile Management Class Diagram

Sequence Diagram

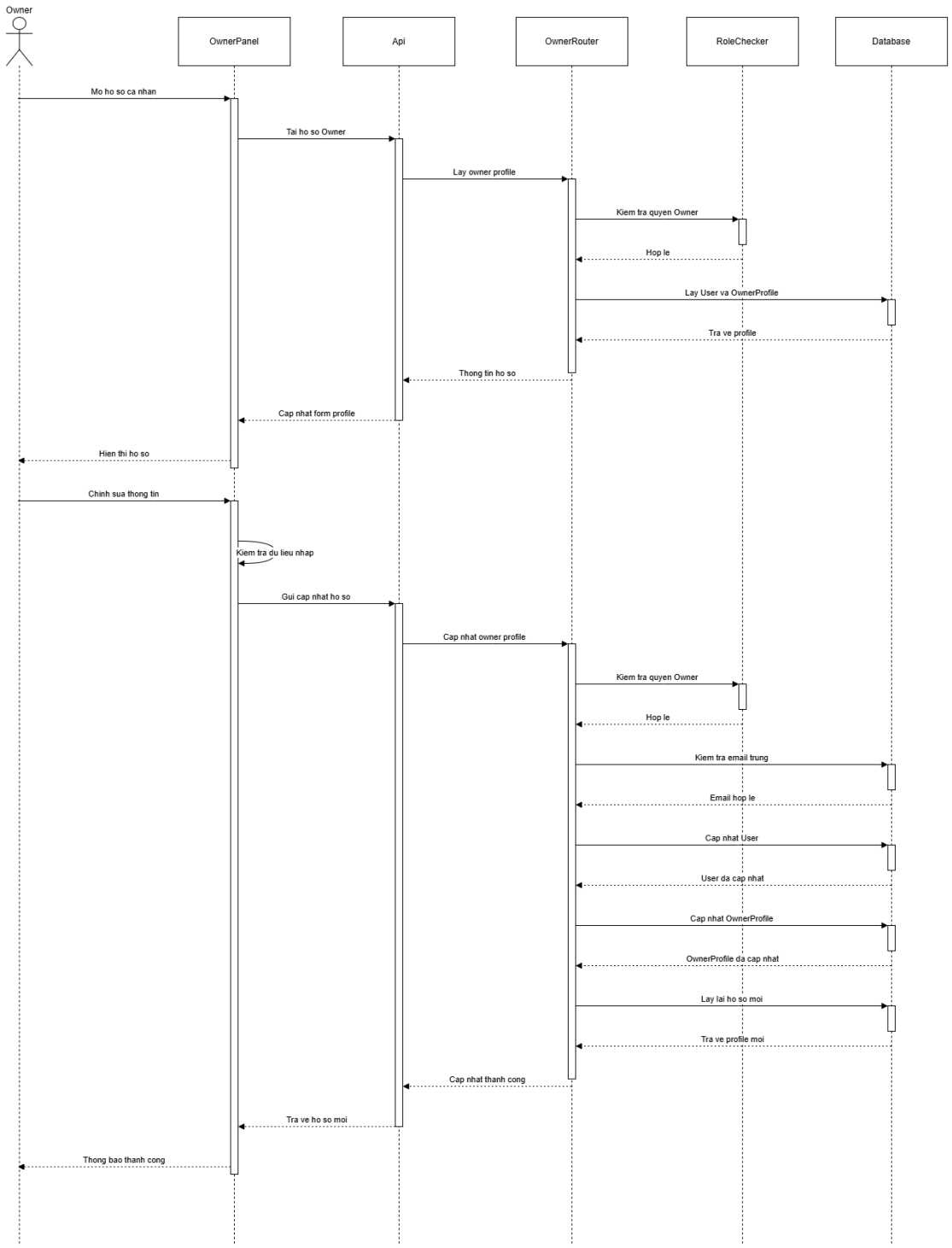


Figure 4.38: View and Update Horse Owner Profile Sequence Diagram

Class Specification

Table 4.16: Horse Owner profile management class specification

Class	Attributes / Methods	Type	Description
OwnerPanel	ownerProfile, profileForm, loadData(), getOwnerProfileForm(), saveOwnerProfile(), showOwnerProfile()	boundary / UI	Displays the current owner profile, fills the edit form, validates input, and submits profile updates.
Api	baseUrl, token, get(), put(), getHeaders()	service / helper	Sends profile retrieval and update requests to owner endpoints with authorization headers.
OwnerRouter	read_owner_profile(), update_owner_profile()	controller / route	Handles requests to view and update the authenticated horse owner's profile information.
RoleChecker	allowed_roles, ___call___()	service / authorization	Ensures that only an authenticated user with the `OWNER` role can view or update owner profile data.
OwnerProfile Update	full_name, email, phone_number, avatar, company_name, age, experience_years, occupation, address, nationality, social_link, bio	DTO / request	Carries editable user and owner profile fields submitted from the owner profile form.
OwnerProfile DetailOut	id, user_id, full_name, email, phone_number, avatar, company_name, age, experience_years, occupation, address, nationality, social_link, bio, joined_date	DTO / response	Returns complete owner profile details for display after loading or updating the profile.
User	id, username, email, full_name, phone_number, avatar, role_id, is_active, created_at	entity	Stores account-level profile fields such as full name, email, phone number, avatar, and join date.
Role	id, name	entity	Defines the user's role and supports owner-only profile access.

Class	Attributes / Methods	Type	Description
HorseOwner Profile	id, user_id, company_name, age, experience_years, occupation, address, nationality, social_link, bio	entity	Stores horse-owner-specific profile information linked to the user account.

4.2.16 Jockey Dashboard

Class Diagram

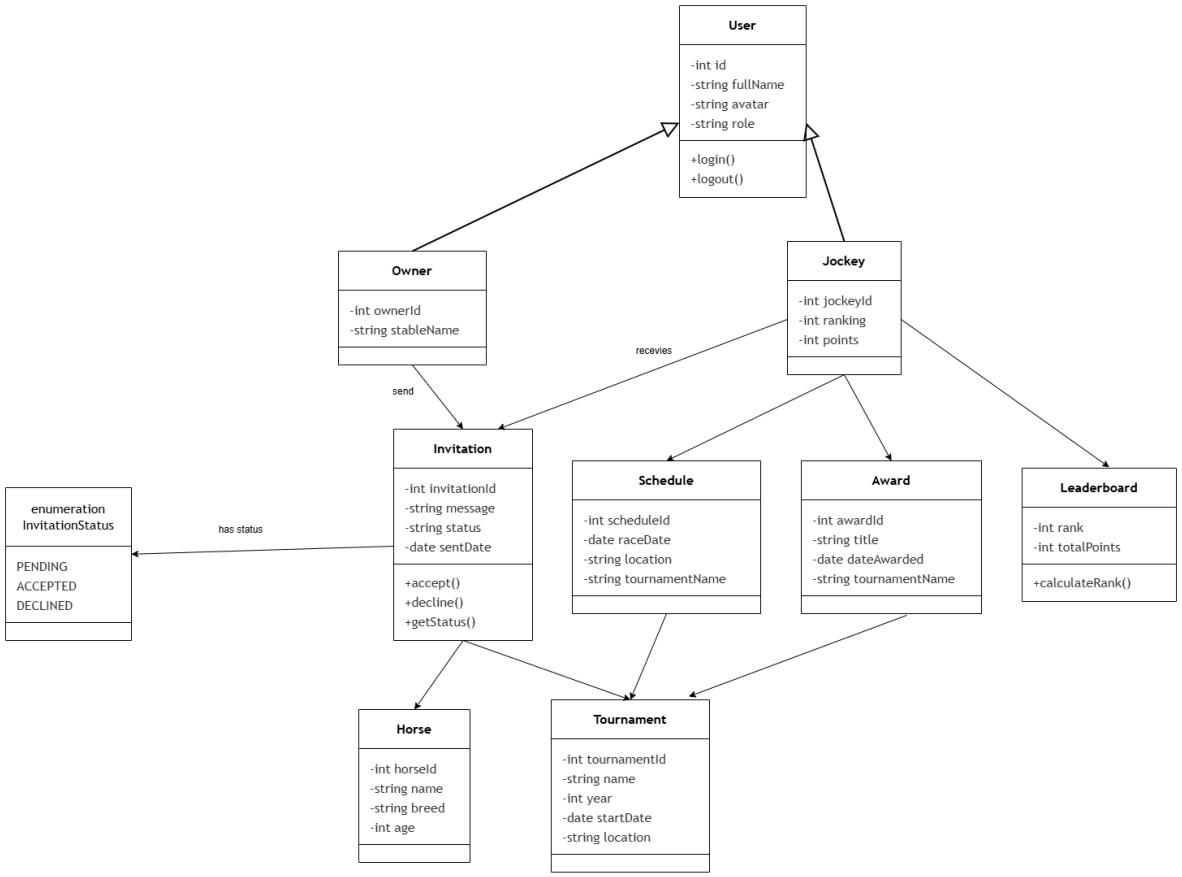


Figure 4.39: Jockey Dashboard Class Diagram

Sequence Diagram

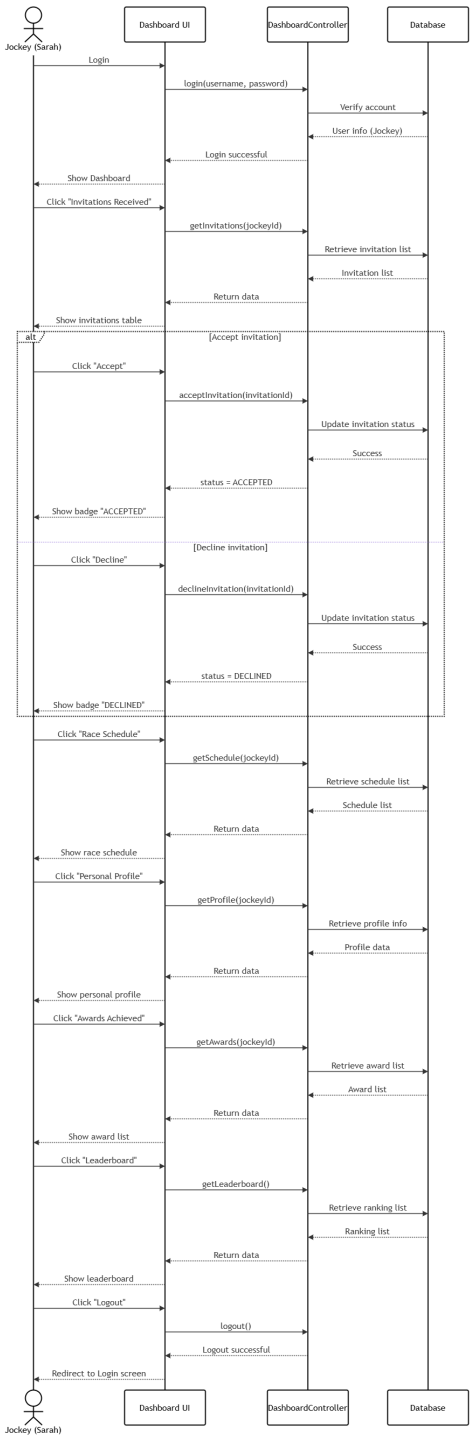


Figure 4.40: Jockey Dashboard Sequence Diagram

Class Specification

Table 4.17: Jockey Dashboard class specification

Class	Attributes / Methods	Type	Description
User	id, fullName, avatar, role, login(), logout()	entity	Represents the base account of a system user. Both `Owner` and `Jockey` inherit from this class.
Jockey	jockeyId, ranking, points	entity	Extends `User` with jockey-specific attributes; linked to invitations, schedules, awards, and the leaderboard.
Owner	ownerId, stableName	entity	Extends `User` and sends invitations to jockeys on behalf of a stable.
Invitation	invitationId, message, status, sentDate, accept(), decline(), getStatus()	entity	Records a race invitation sent by an owner to a jockey, with the current status tracked via `InvitationStatus`.
Invitation Status	PENDING, ACCEPTED, DECLINED	enumeration	Enumerates the possible lifecycle states of a jockey invitation.
Schedule	scheduleId, raceDate, location, tournamentName	entity	Stores race schedule details visible to the jockey after an invitation is accepted.
Award	awardId, title, dateAwarded, tournamentName	entity	Records awards and achievements earned by the jockey in tournaments.
Leaderboard	rank, totalPoints, calculateRank()	entity	Maintains the global jockey ranking and calculates each jockey's position based on accumulated points.
Horse	horseId, name, breed, age	entity	Represents the horse assigned to a jockey through an invitation for a specific tournament.

Class	Attributes / Methods	Type	Description
Tournament	tournamentId, name, year, startDate, location	entity	Represents the racing tournament context that links invitations, schedules, and awards together.
DashboardUI	login(), getInvitations(), acceptInvitation(), declineInvitation(), getSchedule(), getProfile(), getAwards(), getLeaderboard(), logout()	boundary / UI	The jockey-facing dashboard interface that handles login, displays invitations, schedule, profile, awards, and the leaderboard.
Dashboard Controller	login(username, password), getInvitations(jockeyId), acceptInvitation(invitationId), declineInvitation(invitationId), getSchedule(jockeyId), getProfile(jockeyId), getAwards(jockeyId), getLeaderboard()	controller	Processes all dashboard requests from the UI, coordinates data retrieval and status updates with the database.

4.2.17 Jockey Invitation Management

Class Diagram

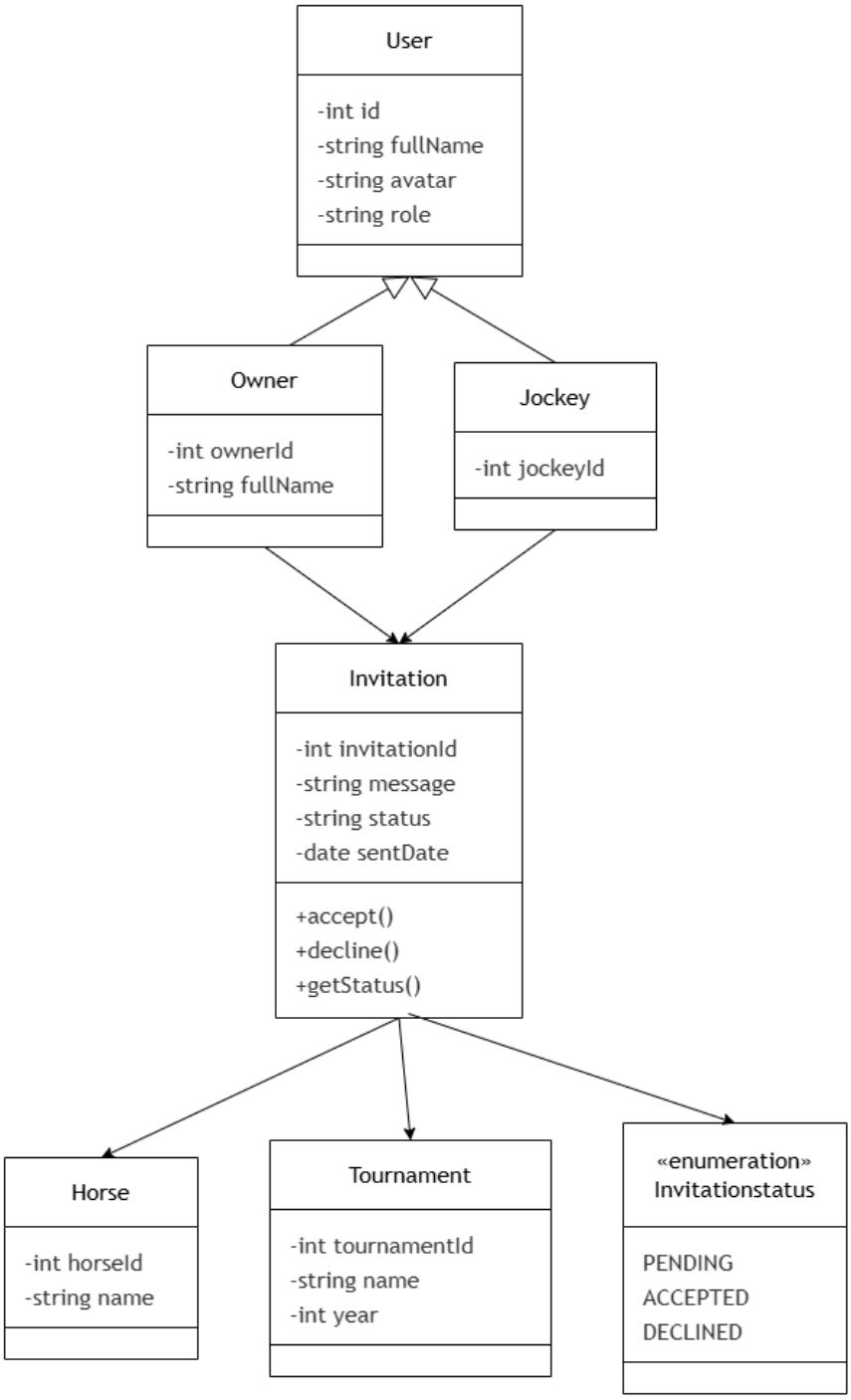


Figure 4.41: Jockey Invitation Management Class Diagram

Sequence Diagram

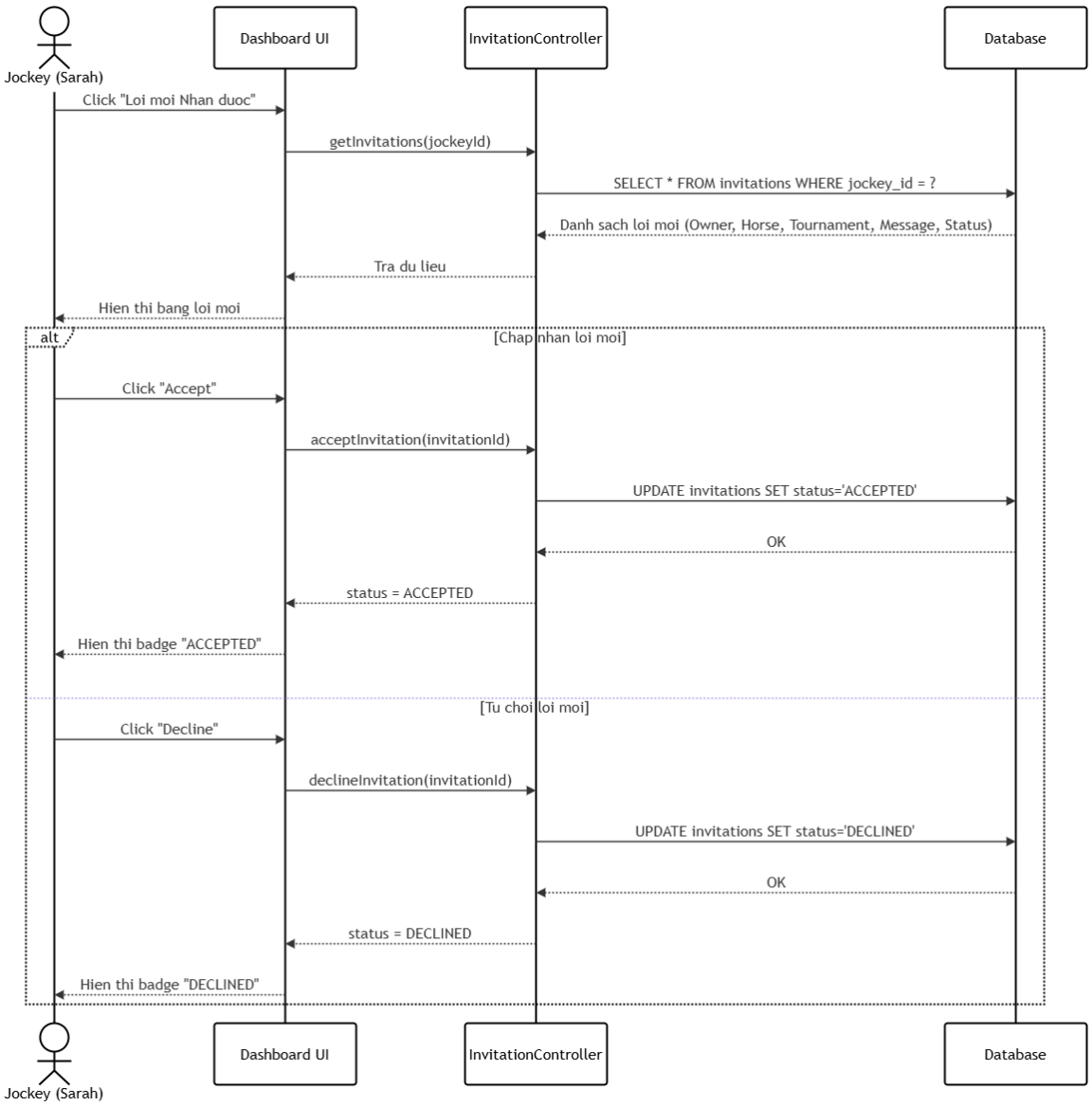


Figure 4.42: Jockey Invitation Management Sequence Diagram

Class Specification

Table 4.18: Jockey Invitation Management class specification

Class	Attributes / Methods	Type	Description
User	id, fullName, avatar, role	entity	Base account class inherited by both `Owner` and `Jockey`; provides shared identity attributes.
Owner	ownerId, fullName	entity	Extends `User`; represents the horse owner who sends race invitations to jockeys.
Jockey	jockeyId	entity	Extends `User`; represents the jockey who receives and responds to invitations.
Invitation	invitationId, message, status, sentDate, accept(), decline(), getStatus()	entity	Records the invitation sent from an owner to a jockey, supporting accept and decline actions with status tracking.
Invitation Status	PENDING, ACCEPTED, DECLINED	enumeration	Defines the three possible lifecycle states of a jockey invitation.
Horse	horseId, name	entity	Represents the horse associated with the invitation that the jockey is being asked to ride.
Tournament	tournamentId, name, year	entity	Provides the tournament context for the invitation, linking the jockey and horse to a specific racing event.
DashboardUI	getInvitations(jockeyId), acceptInvitation(invitationId), declineInvitation(invitationId)	boundary / UI	Displays the invitation list to the jockey and handles accept and decline user interactions.
Invitation Controller	getInvitations(jockeyId), acceptInvitation(invitationId), declineInvitation(invitationId)	controller	Processes invitation retrieval and status update requests, coordinates between the UI and the database.

4.2.18 Race Schedule

Class Diagram

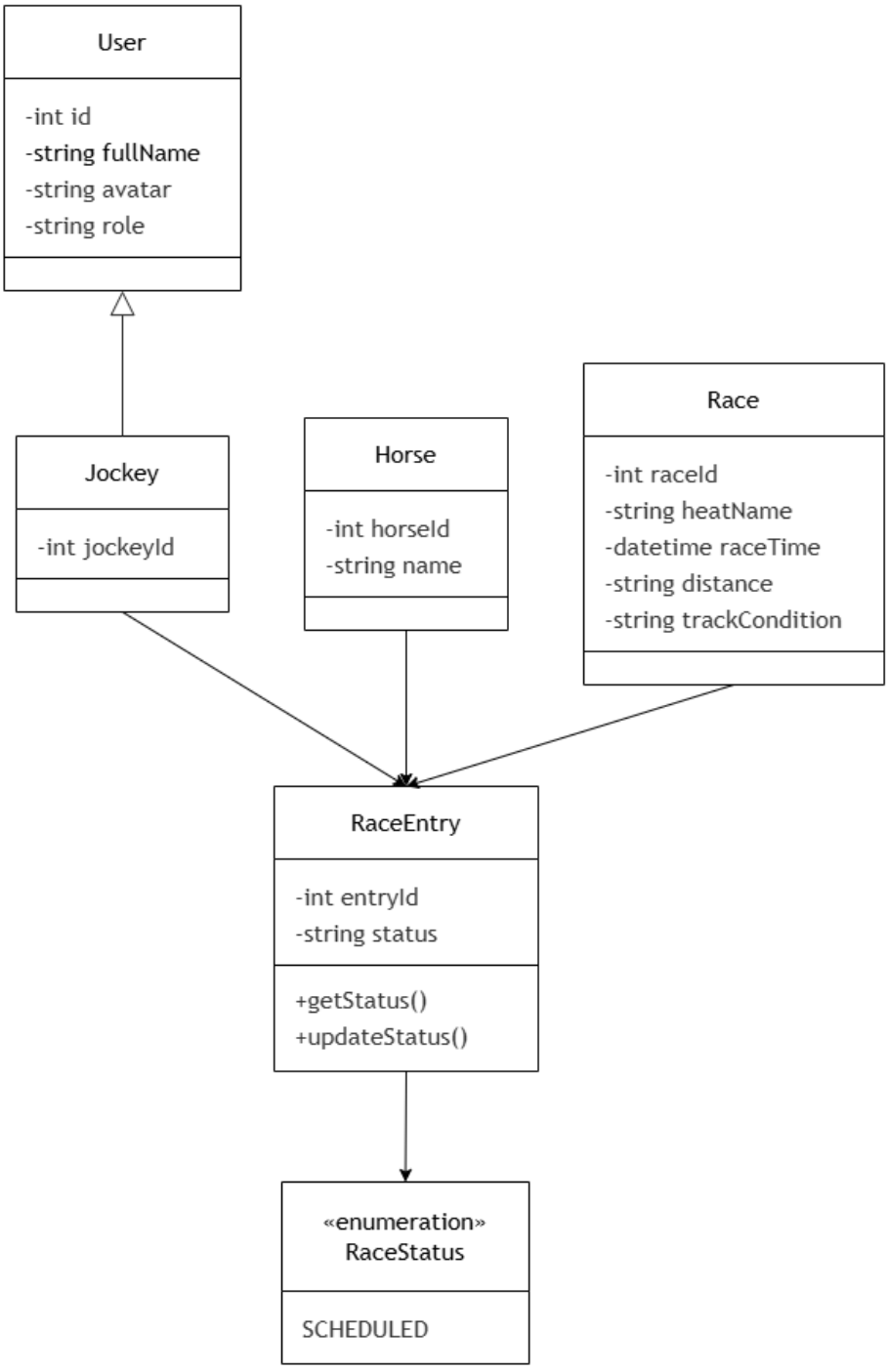


Figure 4.43: Race Schedule Class Diagram

Sequence Diagram

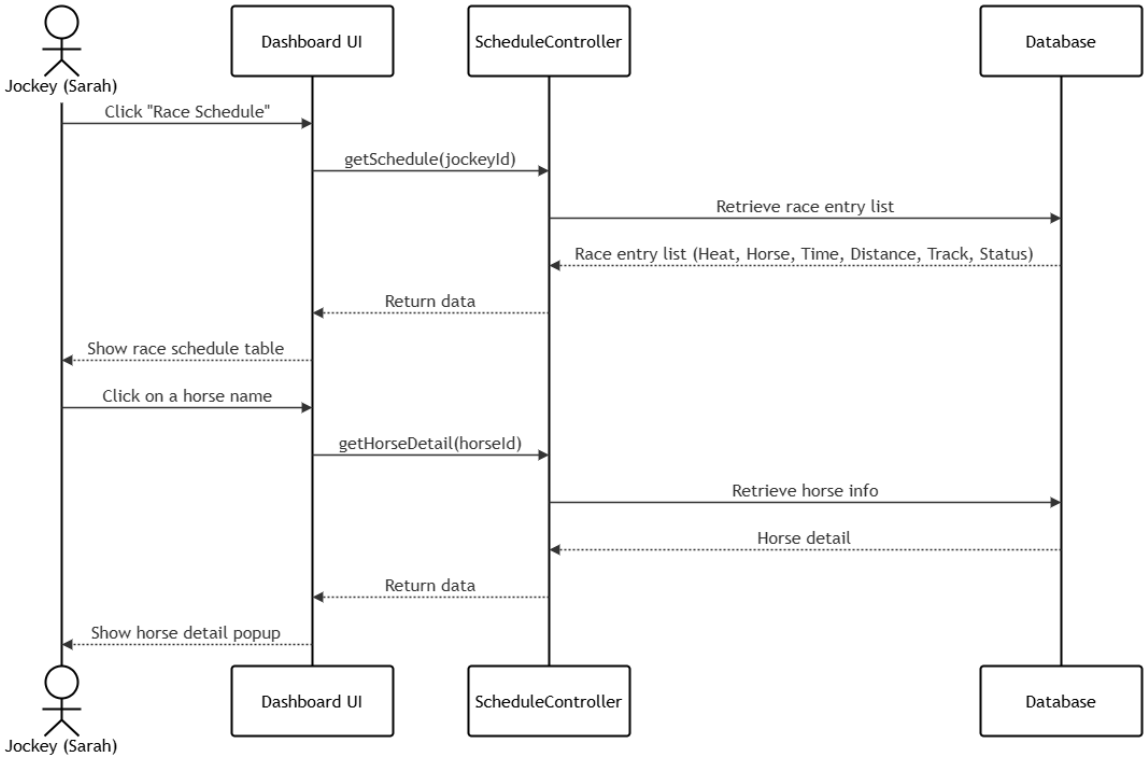


Figure 4.44: Race Schedule Sequence Diagram

Class Specification

Table 4.19: Race Schedule class specification

Class	Attributes / Methods	Type	Description
User	id, fullName, avatar, role	entity	Base account class; `Jockey` inherits from it to access the race schedule feature.
Jockey	jockeyId	entity	Extends `User`; the authenticated jockey whose assigned race entries are retrieved and displayed.
Horse	horseId, name	entity	Represents the horse assigned to the jockey for a specific race entry.

Class	Attributes / Methods	Type	Description
Race	raceId, heatName, raceTime, distance, trackCondition	entity	Stores the details of an individual race heat, including time, distance, and track conditions.
RaceEntry	entryId, status, getStatus(), updateStatus()	entity	Links a jockey and a horse to a specific race; tracks the entry status via `RaceStatus`.
RaceStatus	SCHEDULED	enumeration	Enumerates the status values of a race entry; currently supports the `SCHEDULED` state.
DashboardUI	getSchedule(jockeyId), getHorseDetail(horseId)	boundary / UI	Renders the race schedule table and displays a horse detail popup when the jockey clicks a horse name.
Schedule Controller	getSchedule(jockeyId), getHorseDetail(horseId)	controller	Retrieves the list of race entries for the jockey and fetches detailed horse information on demand.

4.2.19 Personal Profile

Class Diagram

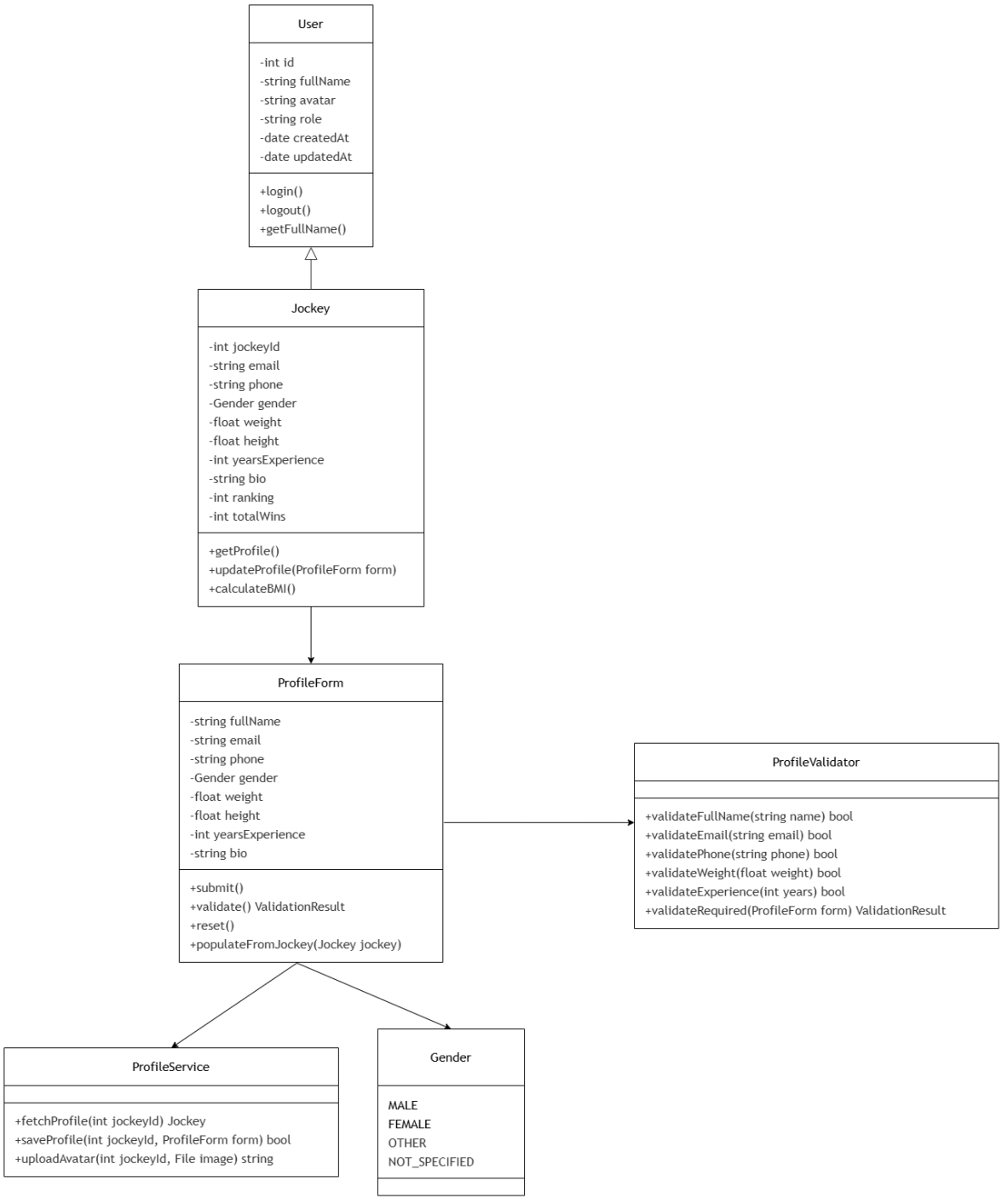


Figure 4.45: Personal Profile Class Diagram

Sequence Diagram

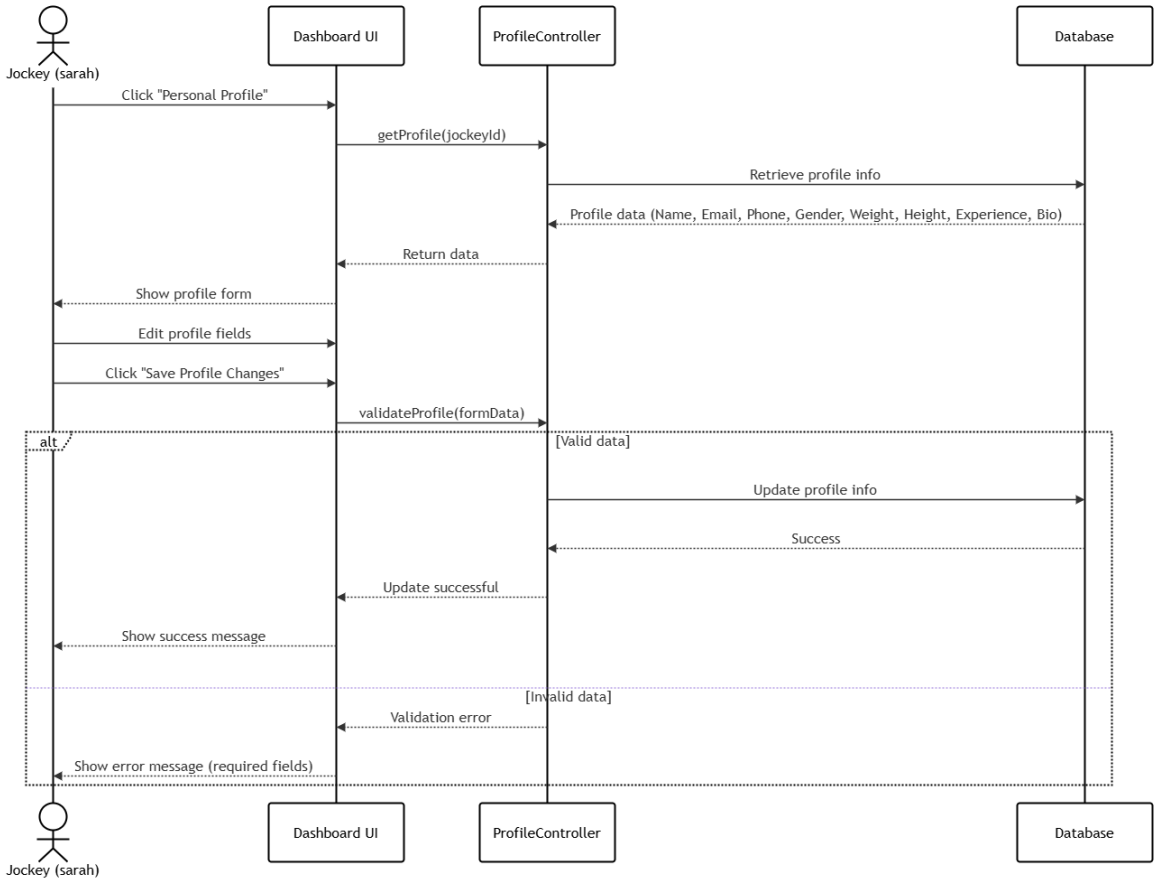


Figure 4.46: Personal Profile Sequence Diagram

Class Specification

Table 4.20: Personal Profile class specification

Class	Attributes / Methods	Type	Description
User	id, fullName, avatar, role, createdAt, updatedAt, login(), logout(), getFullName()	entity	Base account class providing core identity and authentication methods inherited by `Jockey`.
Jockey	jockeyId, email, phone, gender, weight, height, yearsExperience, bio, ranking, totalWins, getProfile(), updateProfile(ProfileForm), calculateBMI()	entity	Extends `User` with jockey-specific profile attributes; supports profile retrieval, update, and BMI calculation.

Class	Attributes / Methods	Type	Description
ProfileForm	fullName, email, phone, gender, weight, height, yearsExperience, bio, submit(), validate(), reset(), populate-FromJockey(Jockey)	DTO / form	Carries the form data submitted by the jockey when editing their personal profile, and validates required fields before saving.
Profile Validator	validateFullName(name), validateEmail(email), validatePhone(phone), validateWeight(weight), validateExperience(years), vali-dateRequired(ProfileForm)	service	Validates individual profile fields and the entire form, returning a `ValidationResult` for each check.
ProfileService	fetchProfile(jockeyId), saveProfile(jockeyId, ProfileForm), uploadAvatar(jockeyId, image)	service	Handles backend communication for loading the jockey profile, persisting updates, and uploading avatar images.
Gender	MALE, FEMALE, OTHER, NOT_SPECIFIED	enumeration	Defines the gender options available when the jockey edits their personal profile.
DashboardUI	getProfile(jockeyId), validateProfile(formData)	boundary / UI	Displays the profile form pre-filled with existing data, handles edit interactions, and shows success or validation error messages.
Profile Controller	getProfile(jockeyId), validateProfile(formData)	controller	Retrieves profile data from the database and processes profile update requests, delegating validation to `ProfileValidator`.

4.2.20 Achievements

Class Diagram

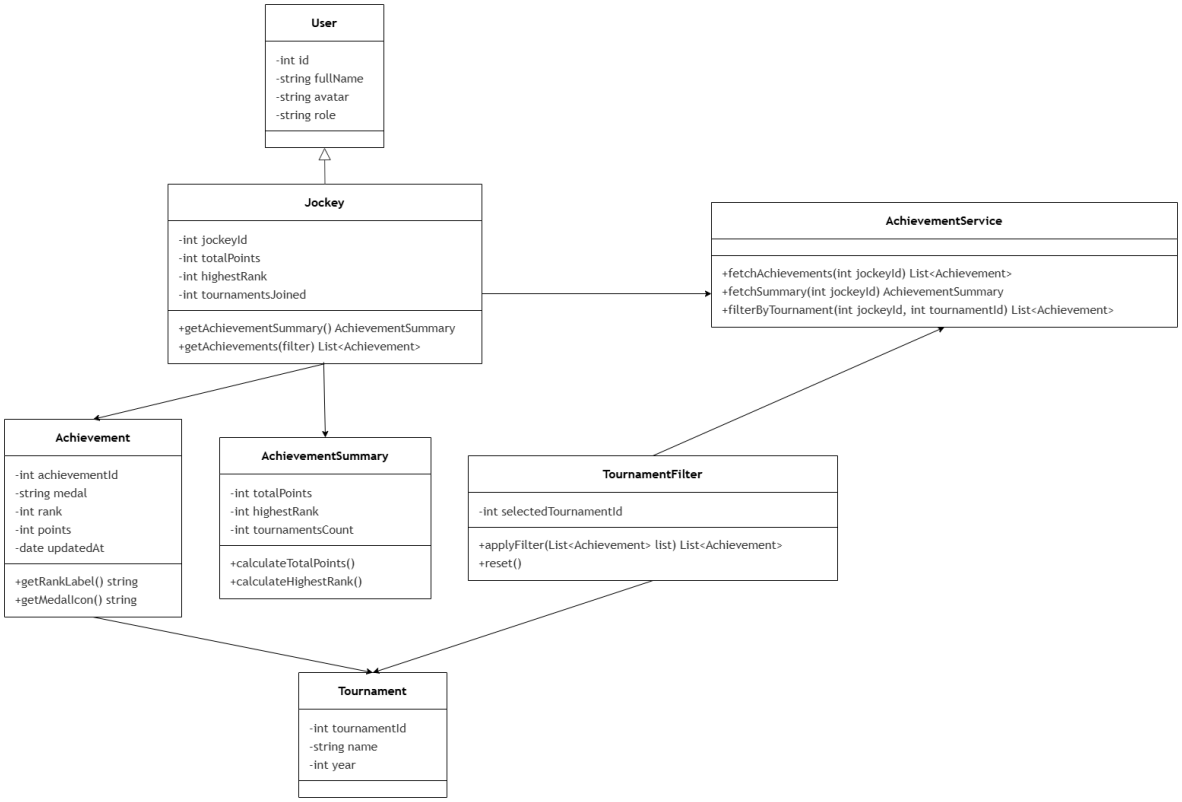


Figure 4.47: Achievements Class Diagram

Sequence Diagram

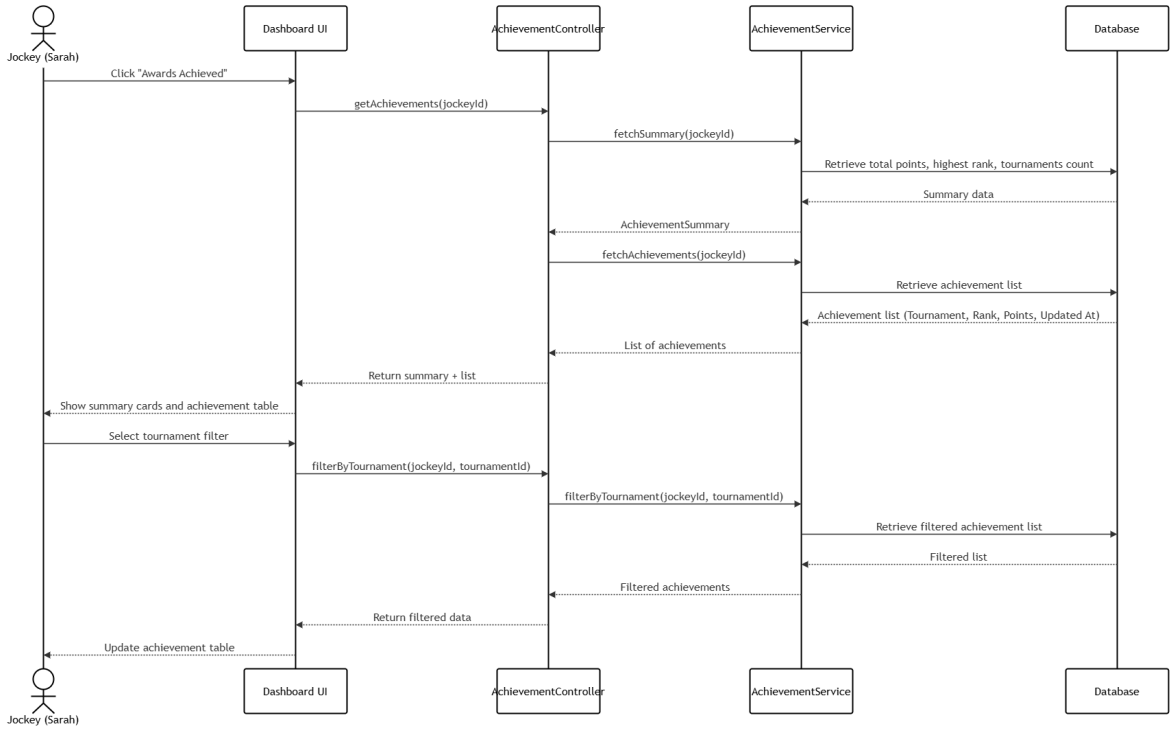


Figure 4.48: Achievements Sequence Diagram

Class Specification

Table 4.21: Achievements class specification

Class	Attributes / Methods	Type	Description
User	id, fullName, avatar, role	entity	Base account class inherited by `Jockey`, providing shared identity attributes for the achievements feature.
Jockey	jockeyId, totalPoints, highestRank, tournamentsJoined, getAchievementSummary(), getAchievements(filter)	entity	Extends `User`; stores cumulative performance statistics and delegates achievement retrieval to `AchievementService`.

Class	Attributes / Methods	Type	Description
Achievement	achievementId, medal, rank, points, updatedAt, getRankLabel(), getMedalIcon()	entity	Represents a single achievement record earned by the jockey in a tournament, with helper methods for display formatting.
Achievement Summary	totalPoints, highestRank, tournamentsCount, calculateTotalPoints(), calculateHighestRank()	DTO	Aggregates the jockey's overall performance statistics shown as summary cards on the achievements page.
Achievement Service	fetchAchievements(jockeyId), fetchSummary(jockeyId), filterByTournament(jockeyId, tournamentId)	service	Handles all data retrieval for achievements, including full lists, summary aggregation, and tournament-based filtering.
Tournament Filter	selectedTournamentId, applyFilter(list), reset()	service	Applies a tournament filter to the achievement list and supports resetting the filter to show all achievements.
Tournament	tournamentId, name, year	entity	Provides the tournament context for each achievement record and supports filter selection on the achievements page.
DashboardUI	getAchievements(jockeyId), filterByTournament(jockeyId, tournamentId)	boundary / UI	Renders the achievements summary cards and table, and updates the view when the jockey selects a tournament filter.
Achievement Controller	getAchievements(jockeyId), filterByTournament(jockeyId, tournamentId)	controller	Coordinates between the UI and `AchievementService` to fetch full or filtered achievement data for the jockey.

4.2.21 Spectator Prediction

Class Diagram

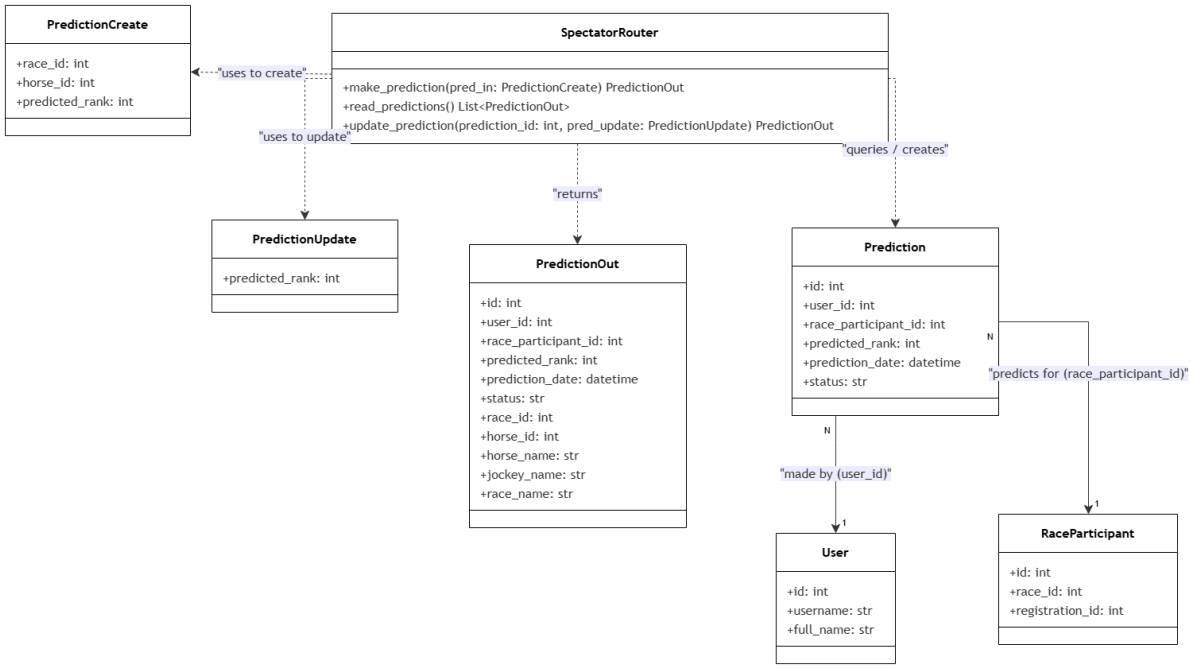


Figure 4.49: Spectator Prediction Class Diagram

Sequence Diagram

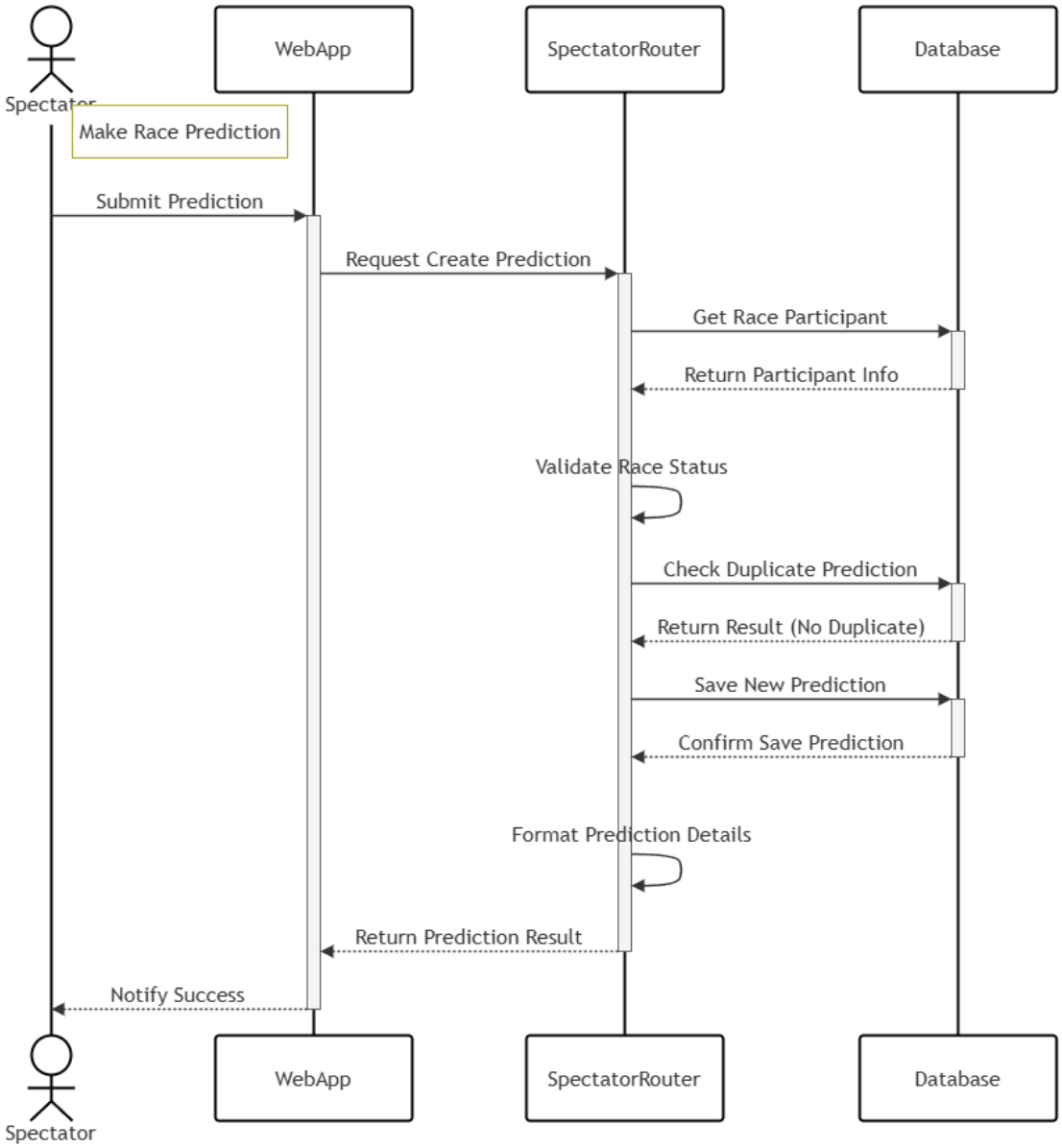


Figure 4.50: Spectator Prediction Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
SpectatorRouter	make_prediction(), read_predictions(), update_prediction()	controller / route	Handles HTTP requests related to spectator predictions (create, read, update).
PredictionCreate	race_id, horse_id, predicted_rank	DTO	Input data transfer object used to create a new prediction.
PredictionUpdate	predicted_rank	DTO	Input data transfer object used to update the predicted rank.
PredictionOut	id, user_id, race_participant_id, predicted_rank, prediction_date, status, race_id, horse_id, horse_name, jockey_name, race_name	DTO	Output data transfer object containing detailed prediction details and related information.
Prediction	id, user_id, race_participant_id, predicted_rank, prediction_date, status	entity	Database entity that stores the user's prediction information.
User	id, username, full_name	entity	Database entity representing an application user who makes predictions.
RaceParticipant	id, race_id, registration_id	entity	Database entity representing a participant in a race being predicted.

Table 4.22: Spectator Prediction class specification

4.2.22 Spectator Personal Profile

Class Diagram

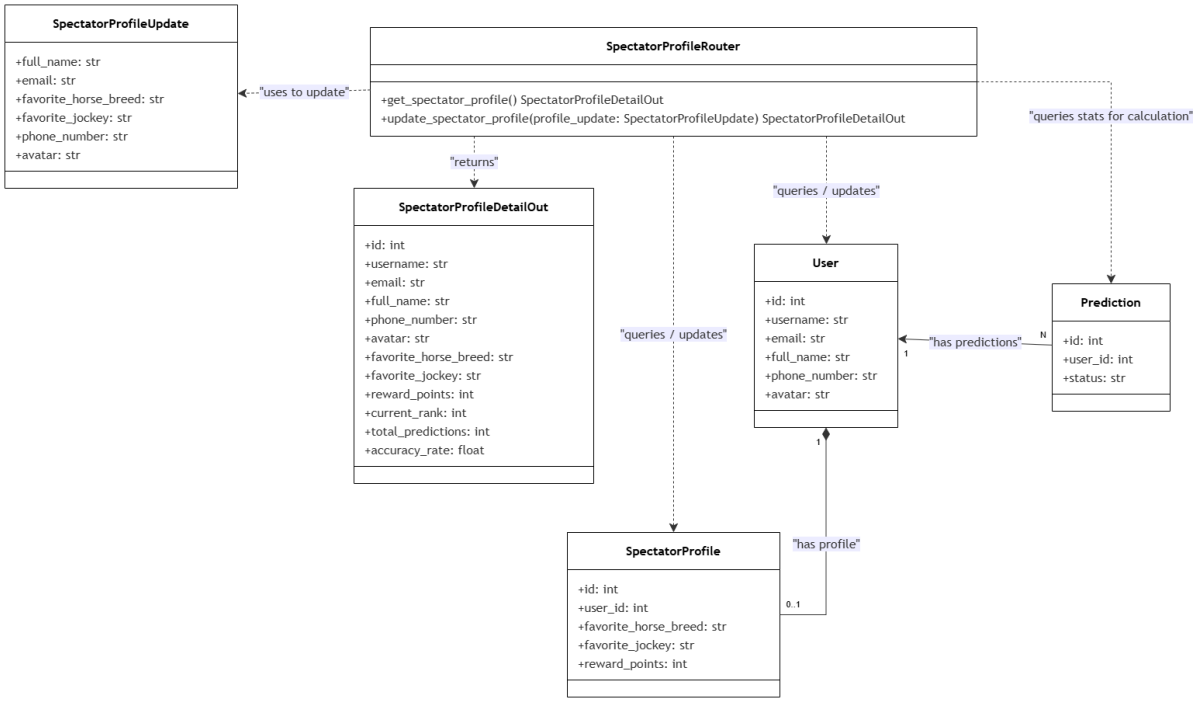


Figure 4.51: Spectator Personal Profile Class Diagram

Sequence Diagram

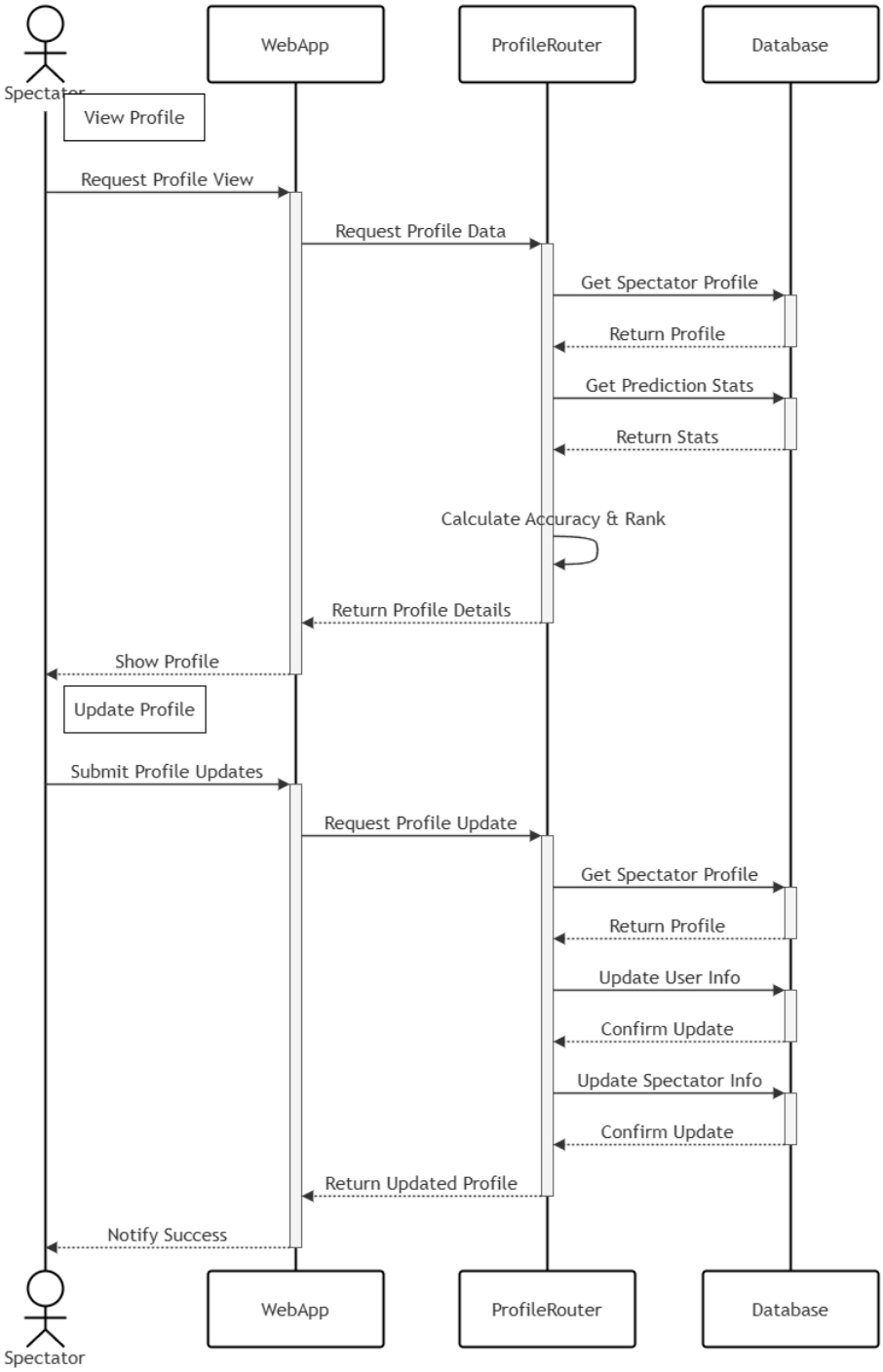


Figure 4.52: Spectator Personal Profile Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
SpectatorProfileRouter	get_spectator_profile(), update_spectator_profile()	controller / route	Handles HTTP requests related to spectator profiles, including retrieving profile details and updating information.
SpectatorProfileUpdate	full_name, email, favorite_horse_breed, favorite_jockey, phone_number, avatar	DTO	Input data transfer object containing the fields that a spectator can update in their profile.
SpectatorProfileDetailOut	id, username, email, full_name, phone_number, avatar, favorite_horse_breed, favorite_jockey, reward_points, current_rank, total_predictions, accuracy_rate	DTO	Output data transfer object returning the comprehensive profile information, including calculated statistical metrics.
User	id, username, email, full_name, phone_number, avatar	entity	Database entity representing the core authentication account information of the user.
SpectatorProfile	id, user_id, favorite_horse_breed, favorite_jockey, reward_points	entity	Database entity storing additional profile information specific to the spectator role.
Prediction	id, user_id, status	entity	Database entity referenced to aggregate user prediction counts and statistics for profile calculation.

Table 4.23: Spectator Profile class specification

4.2.23 Schedule and Result

Class Diagram

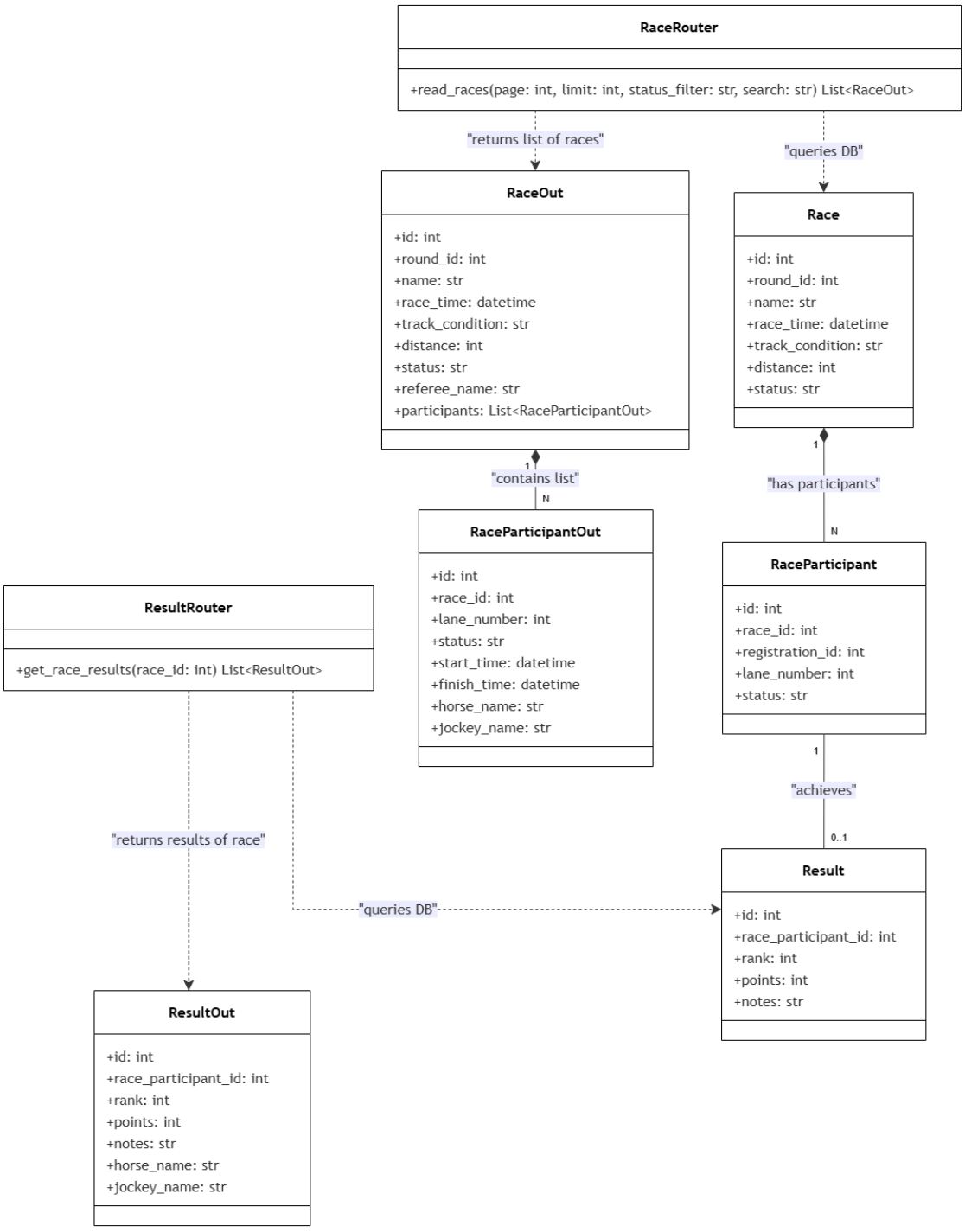


Figure 4.53: Schedule and Result Class Diagram

Sequence Diagram

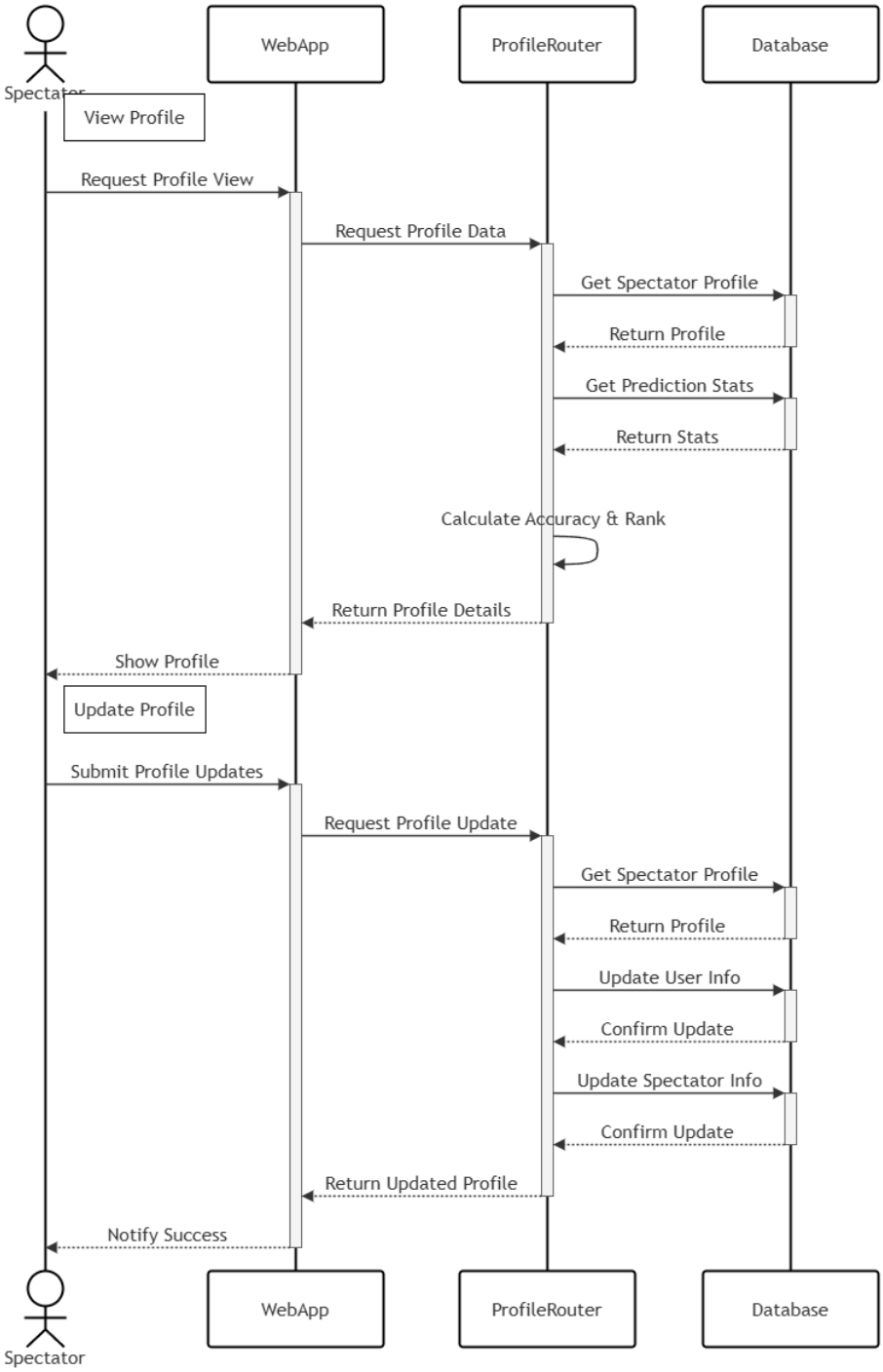


Figure 4.54: Spectator Schedule and Result Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
ScheduleRouter	read_schedules()	controller / route	Handles HTTP requests to retrieve and filter the list of scheduled horse races.
ResultRouter	get_race_results()	controller / route	Handles HTTP requests to retrieve the official results of a specific race.
RaceOut	id, round_id, name, race_time, track_condition, distance, status, referee_name, participants	DTO	Output data transfer object containing comprehensive race details and its participants.
RaceParticipantOut	id, race_id, lane_number, status, start_time, finish_time, horse_name, jockey_name	DTO	Output data transfer object representing detailed information of a participant within a race.
ResultOut	id, race_participant_id, rank, points, notes, horse_name, jockey_name	DTO	Output data transfer object returning the final standings and details of a race result.
Race	id, round_id, name, race_time, track_condition, distance, status	entity	Database entity storing core information of a horse racing event.
RaceParticipant	id, race_id, registration_id, lane_number, status	entity	Database entity representing a registered horse and jockey combination participating in a race.
Result	id, race_participant_id, rank, points, notes	entity	Database entity storing the official placement and points scored by a race participant.

Table 4.24: Schedule and Result class specification

4.2.24 Horse and Jockey LeaderBoard

Class Diagram

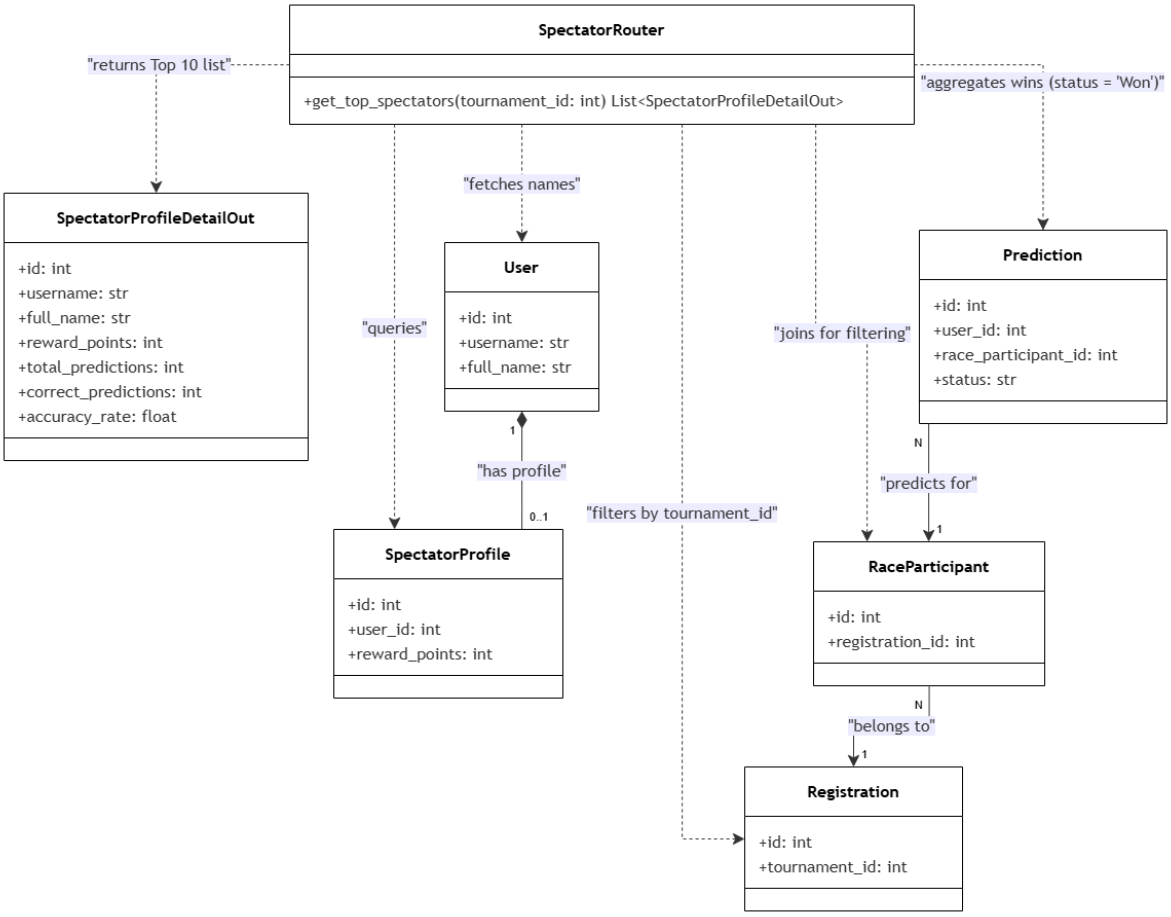


Figure 4.55: Horse and Jockey Rankings Class Diagram

Sequence Diagram

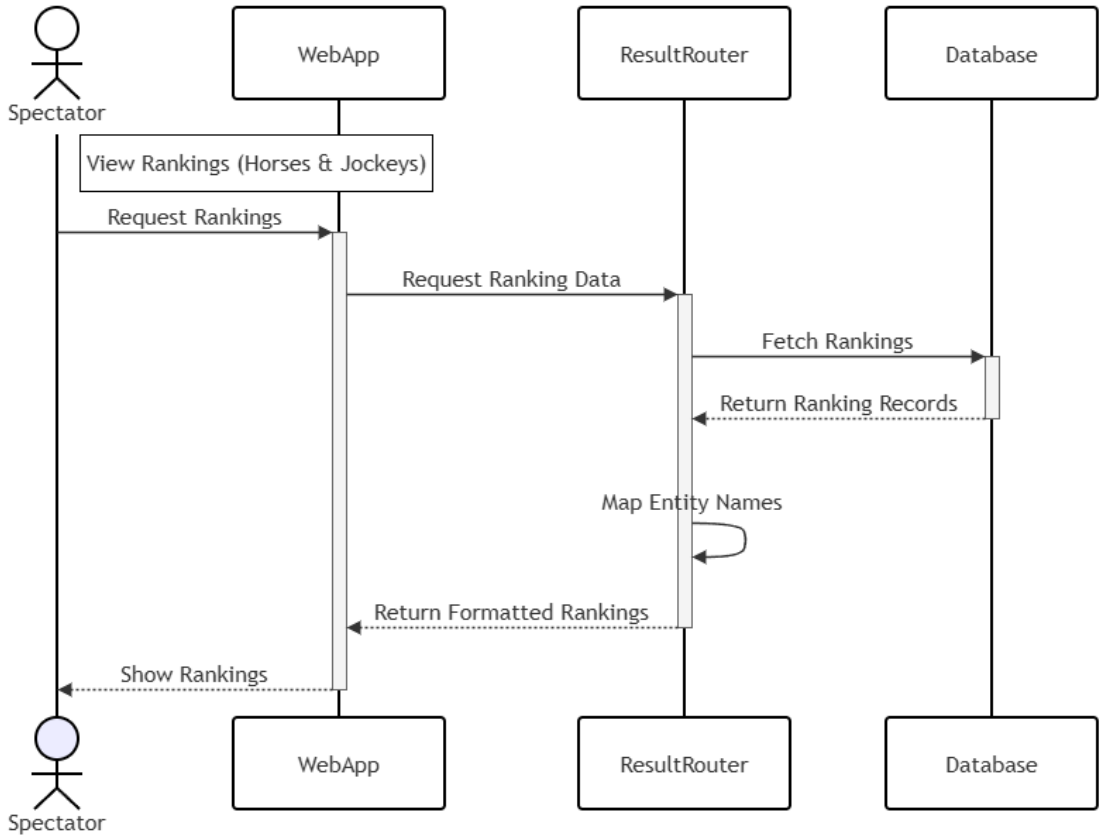


Figure 4.56: Horse and Jockey Rankings Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
ResultRouter	read_rankings()	controller / route	Handles HTTP requests to retrieve leaderboard rankings filtered by tournament.
RankingOut	id, entity_type, entity_id, entity_name, points, rank, updated_at	DTO	Output data transfer object returning the formatted ranking list of horses or jockeys.
Ranking	id, tournament_id, entity_type, entity_id, points, rank	entity	Database entity that stores the computed scores and standings within a tournament.
Horse	id, name	entity	Database entity representing horse information used when the ranking entity type is a horse.
JockeyProfile	id, user_id	entity	Database entity representing jockey profiles used when the ranking entity type is a jockey.
User	id, full_name	entity	Database entity representing core user accounts, used to fetch the full name of jockeys.

Table 4.25: Horse and Jockey Rankings class specification

4.2.25 Top Spectators Leaderboard

Class Diagram

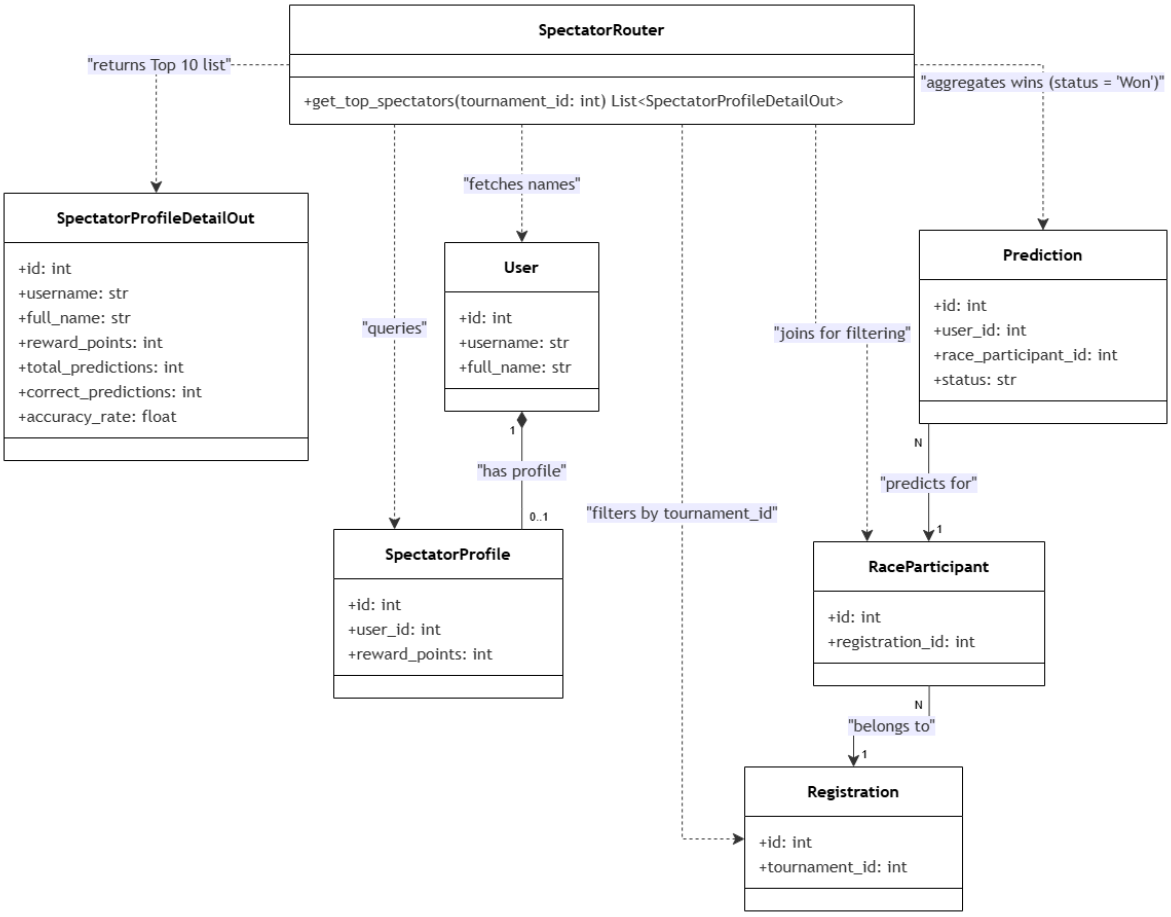


Figure 4.57: Top 10 Spectators Leaderboard Class Diagram

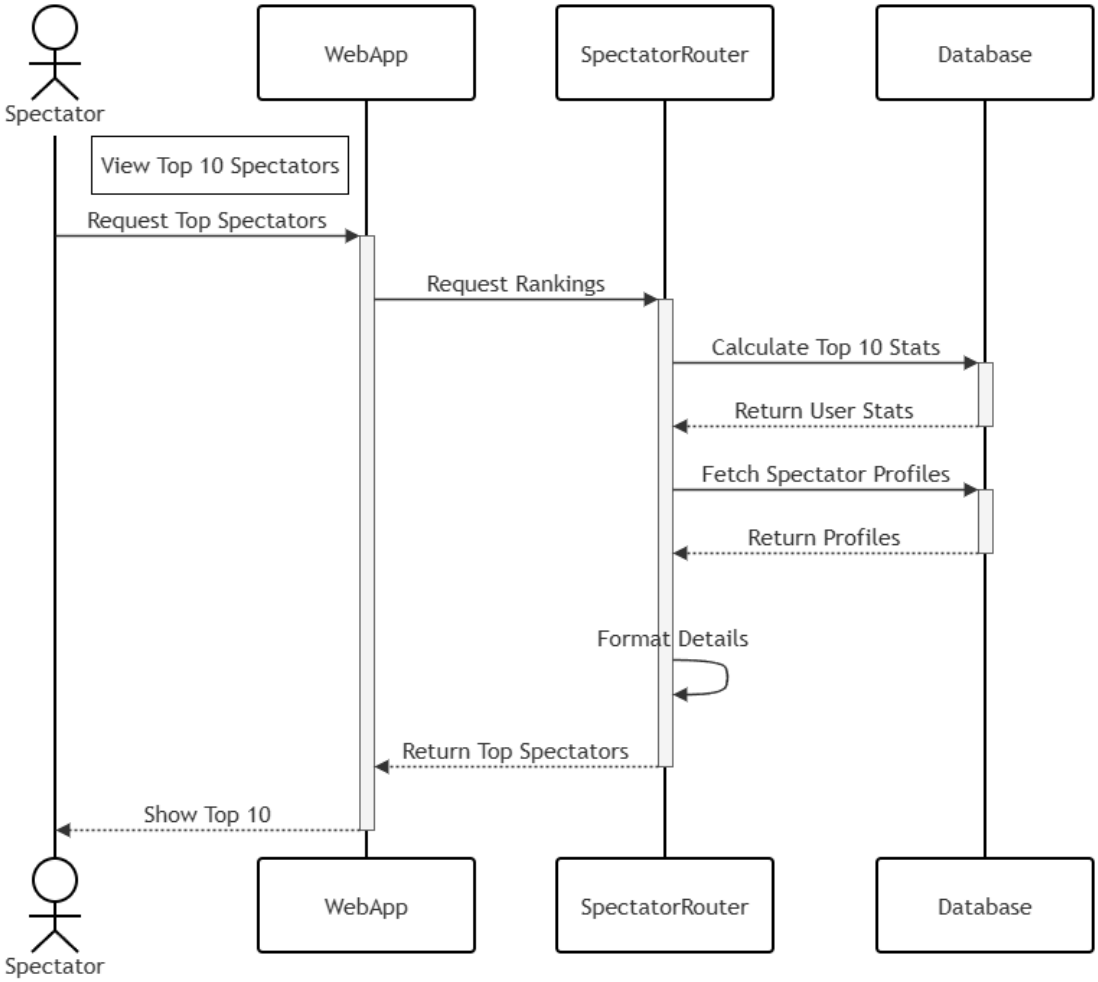


Figure 4.58: Top 10 Spectators Leaderboard Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
SpectatorRouter	get_top_spectators()	controller / route	Handles HTTP requests to aggregate and retrieve the leaderboard of top 10 spectators filtered by tournament.
SpectatorProfileDetailOut	id, username, full_name, reward_points, total_predictions, correct_predictions, accuracy_rate	DTO	Output data transfer object returning the top spectator profiles along with their performance metrics.
User	id, username, full_name	entity	Database entity representing the user accounts to fetch the profile identity names.
SpectatorProfile	id, user_id, reward_points	entity	Database entity storing spectator-specific statistics and total reward points.
Prediction	id, user_id, race_participant_id, status	entity	Database entity used to aggregate and calculate accurate prediction statistics (where status is 'Won').
RaceParticipant	id, registration_id	entity	Database entity representing race participants, utilized to join and filter predictions by race context.
Registration	id, tournament_id	entity	Database entity representing tournament registrations, used to filter leaderboard data by specific tournament.

Table 4.26: Top Spectators Leaderboard class specification

4.2.26 Assigned Race List

Class Diagram

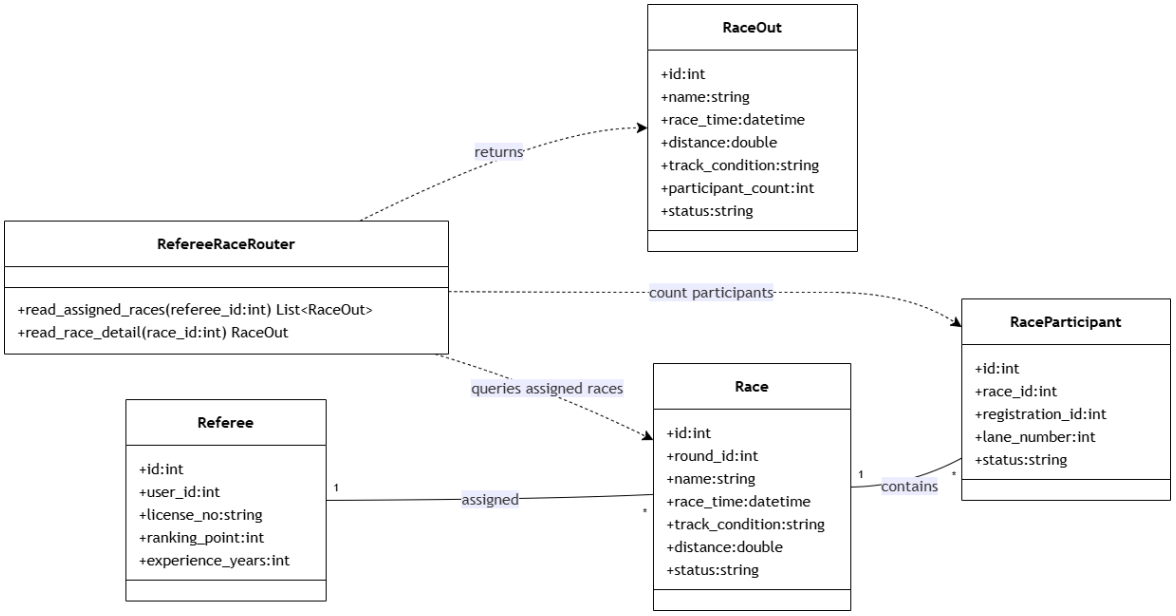


Figure 4.59: Assigned Race List Class Diagram

Sequence Diagram

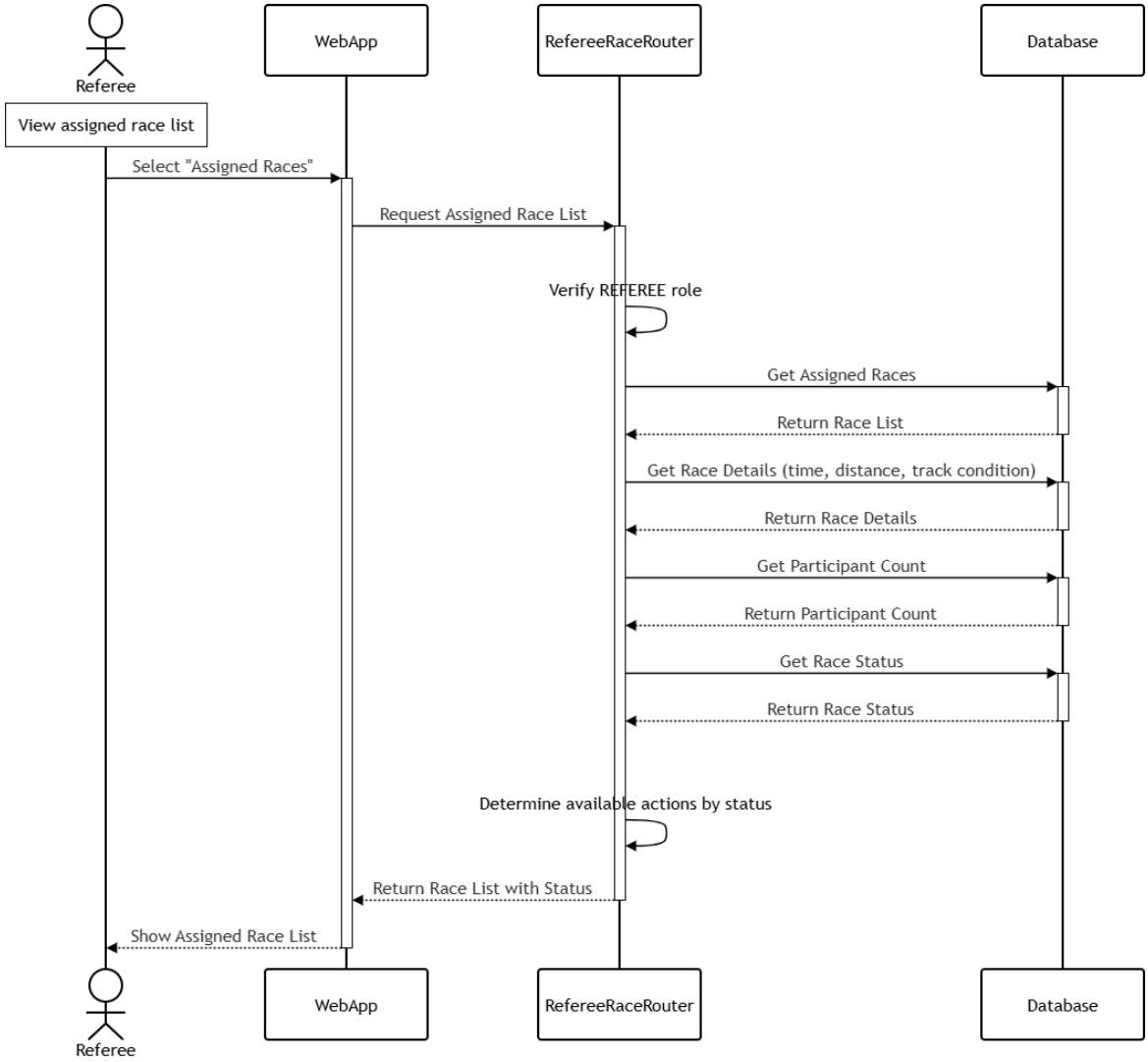


Figure 4.60: Assigned Race List Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
RefereeRaceRouter	read_assigned_races(), read_race_detail()	controller / route	Handles HTTP requests to retrieve the list of races assigned to the authenticated referee and their details.
RaceOut	id, name, race_time, distance, track_condition, participant_count, status	DTO	Output data transfer object returning summarized race information for the referee's assigned race list.
Race	id, round_id, name, race_time, track_condition, distance, status	entity	Database entity storing core information of a horse racing event.
RaceParticipant	id, race_id, registration_id, lane_number, status	entity	Database entity representing a registered horse and jockey combination participating in a race, used to compute participant count.
Referee	id, user_id, license_no, ranking_point, experience_years	entity	Database entity storing role-specific information of the race referee, including licensing and assignment eligibility.

Table 4.27: Assigned Race List class specification

4.2.27 Race Inspection

Class Diagram

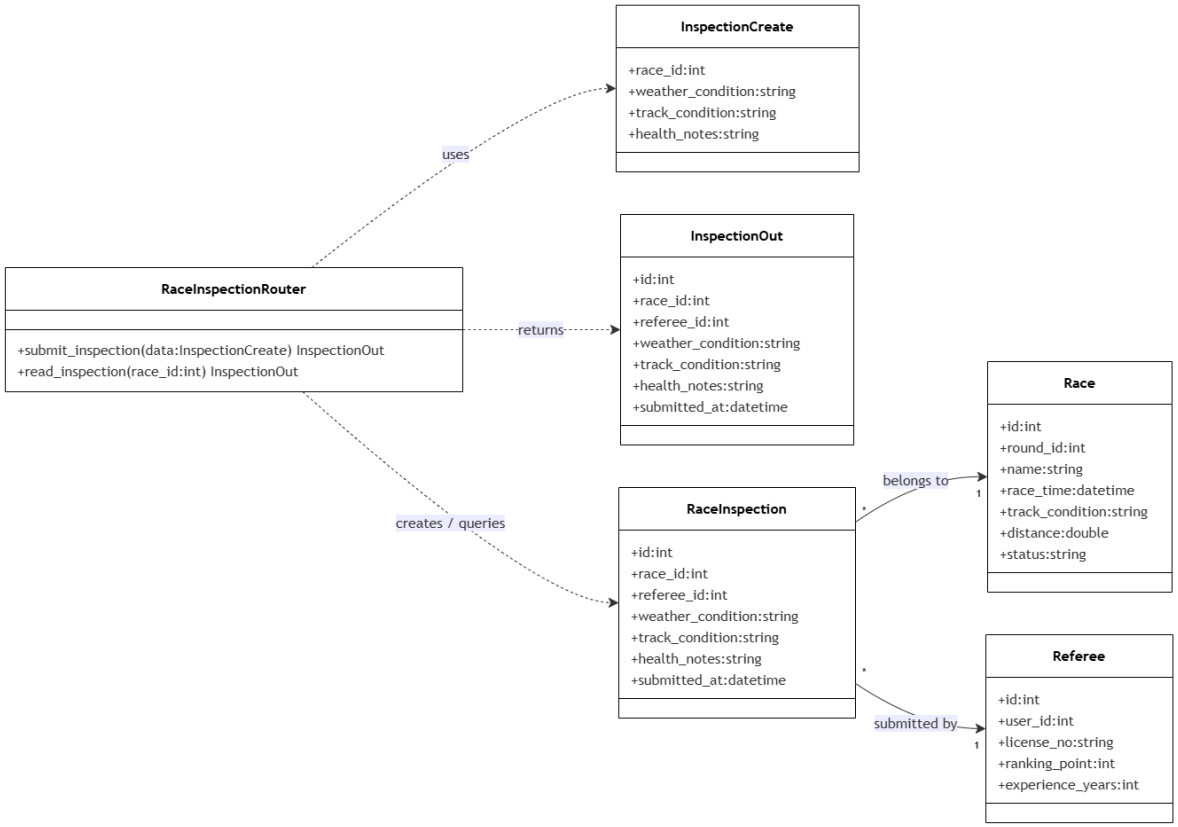


Figure 4.61: Race Inspection Class Diagram

Sequence Diagram

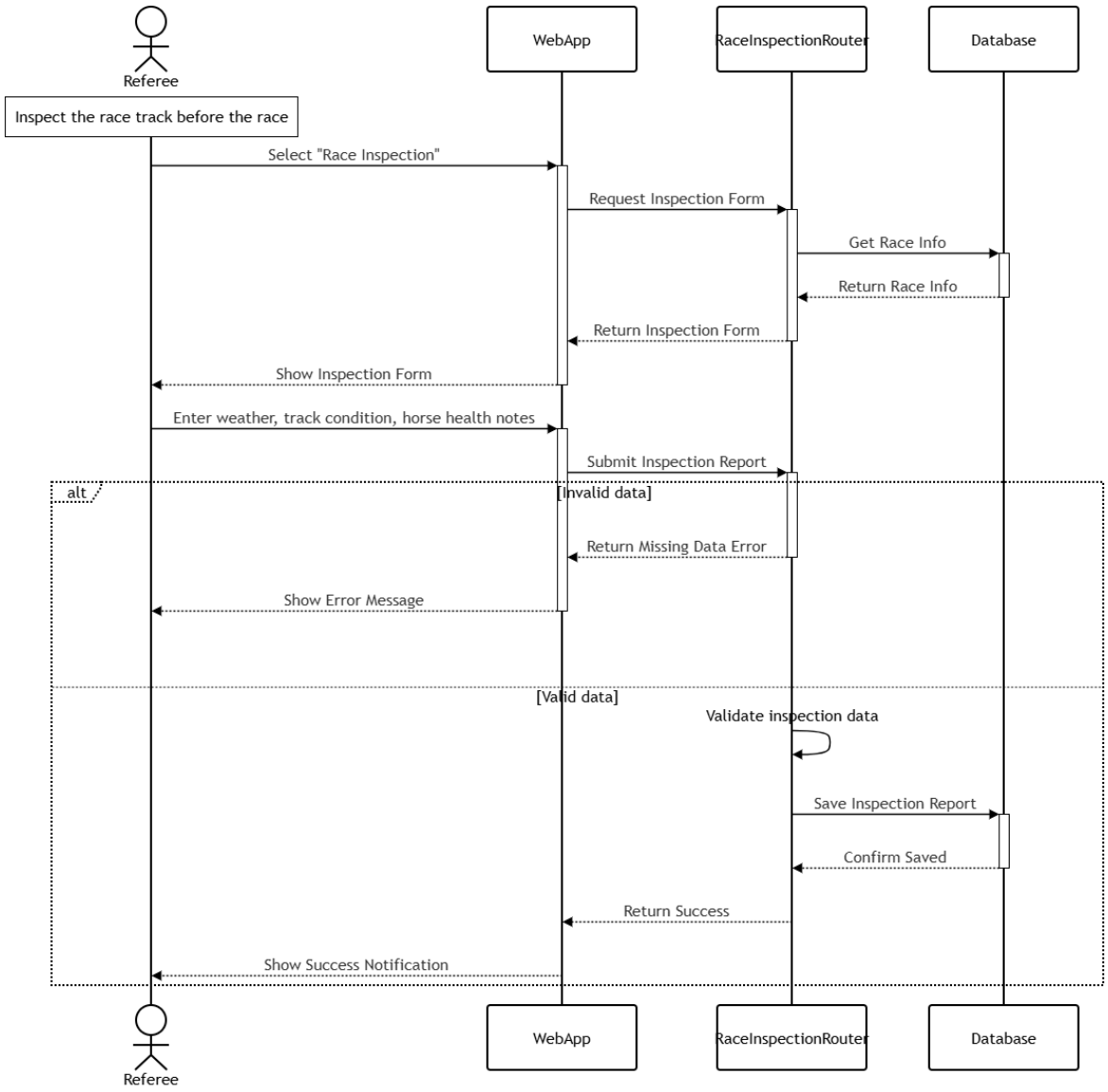


Figure 4.62: Race Inspection Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
RaceInspection Router	submit_inspection(), read_inspection()	controller / route	Handles HTTP requests related to submitting and retrieving pre-race inspection reports.
InspectionCreate	race_id, weather_condition, track_condition, health_notes	DTO	Input data transfer object used to submit a new race inspection report.
InspectionOut	id, race_id, referee_id, weather_condition, track_condition, health_notes, submitted_at	DTO	Output data transfer object returning the submitted inspection report details.
RaceInspection	id, race_id, referee_id, weather_condition, track_condition, health_notes, submitted_at	entity	Database entity storing the referee's pre-race inspection findings for a given race.
Race	id, round_id, name, race_time, track_condition, distance, status	entity	Database entity storing core information of the race being inspected.
Referee	id, user_id, license_no	entity	Database entity identifying the referee who submitted the inspection report.

Table 4.28: Race Inspection class specification

4.2.28 Input Result Form

Class Diagram

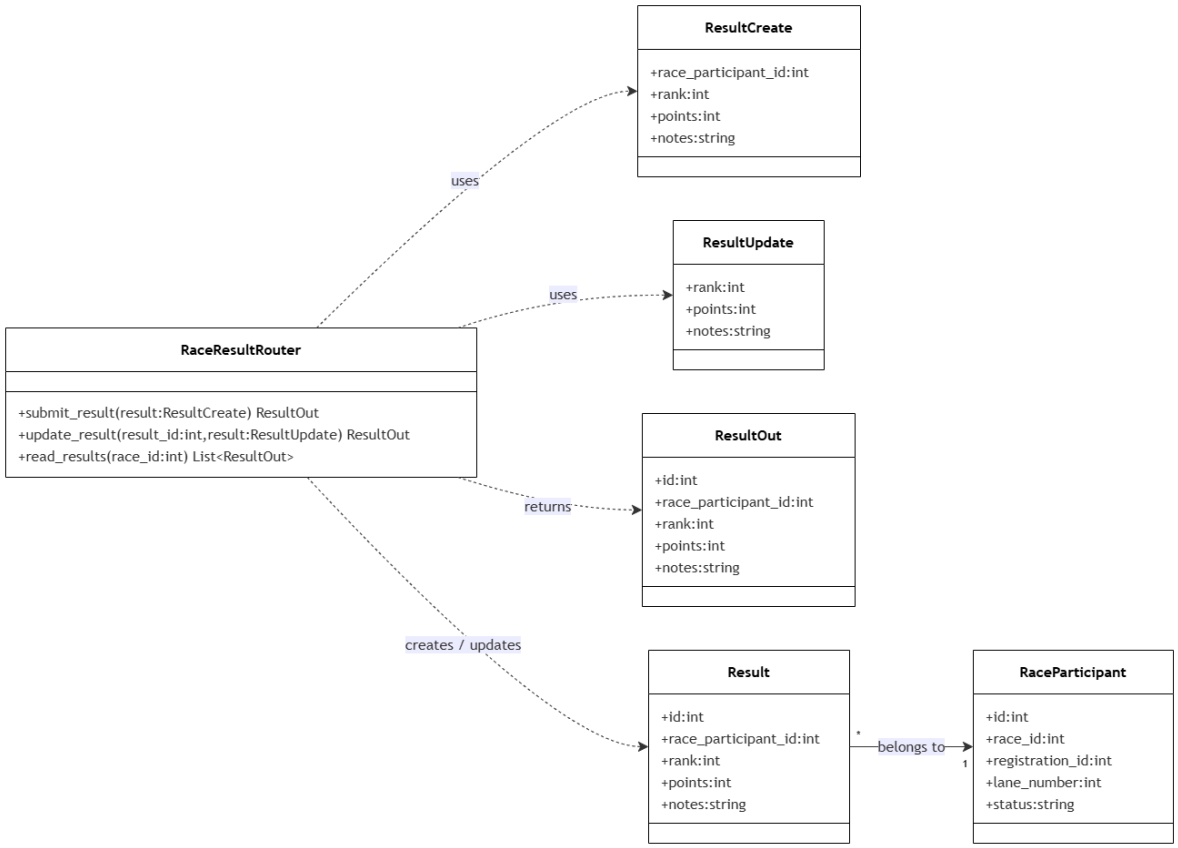


Figure 4.63: Input Result Form Class Diagram

Sequence Diagram

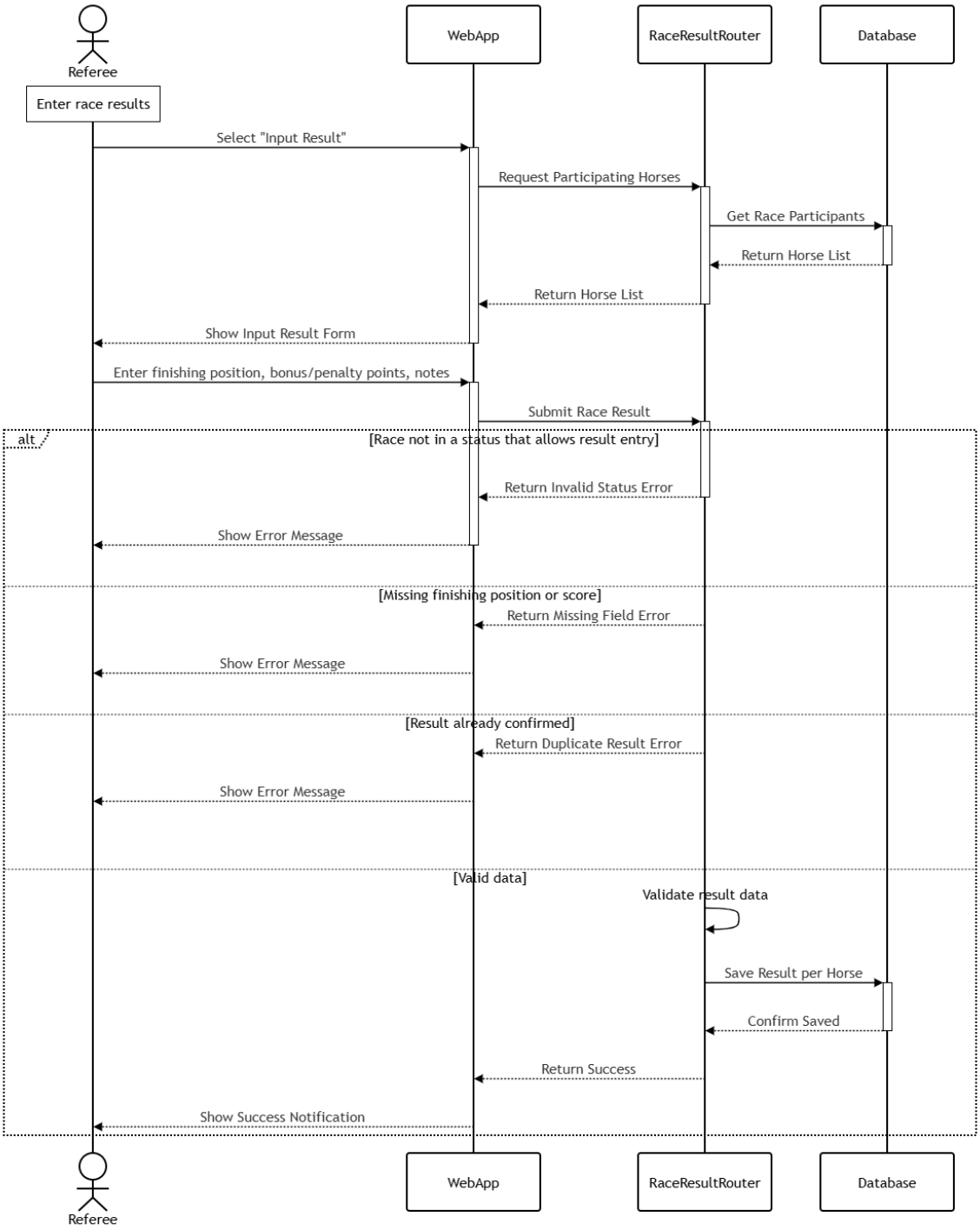


Figure 4.64: Input Result Form Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
RaceResultRouter	submit_result(), update_result(), read_results()	controller / route	Handles HTTP requests to create, update, and retrieve race results entered by the referee.
ResultCreate	race_participant_id, rank, points, notes	DTO	Input data transfer object used to submit a new result for a race participant.
ResultUpdate	rank, points, notes	DTO	Input data transfer object used to correct an existing race result.
ResultOut	id, race_participant_id, rank, points, notes, horse_name, jockey_name	DTO	Output data transfer object returning the saved result along with related participant information.
Result	id, race_participant_id, rank, points, notes	entity	Database entity storing the official placement and points scored by a race participant.
RaceParticipant	id, race_id, registration_id, lane_number, status	entity	Database entity representing the horse-jockey pair for which the result is being recorded.

Table 4.29: Input Result Form class specification

4.2.29 Violation Report

Class Diagram

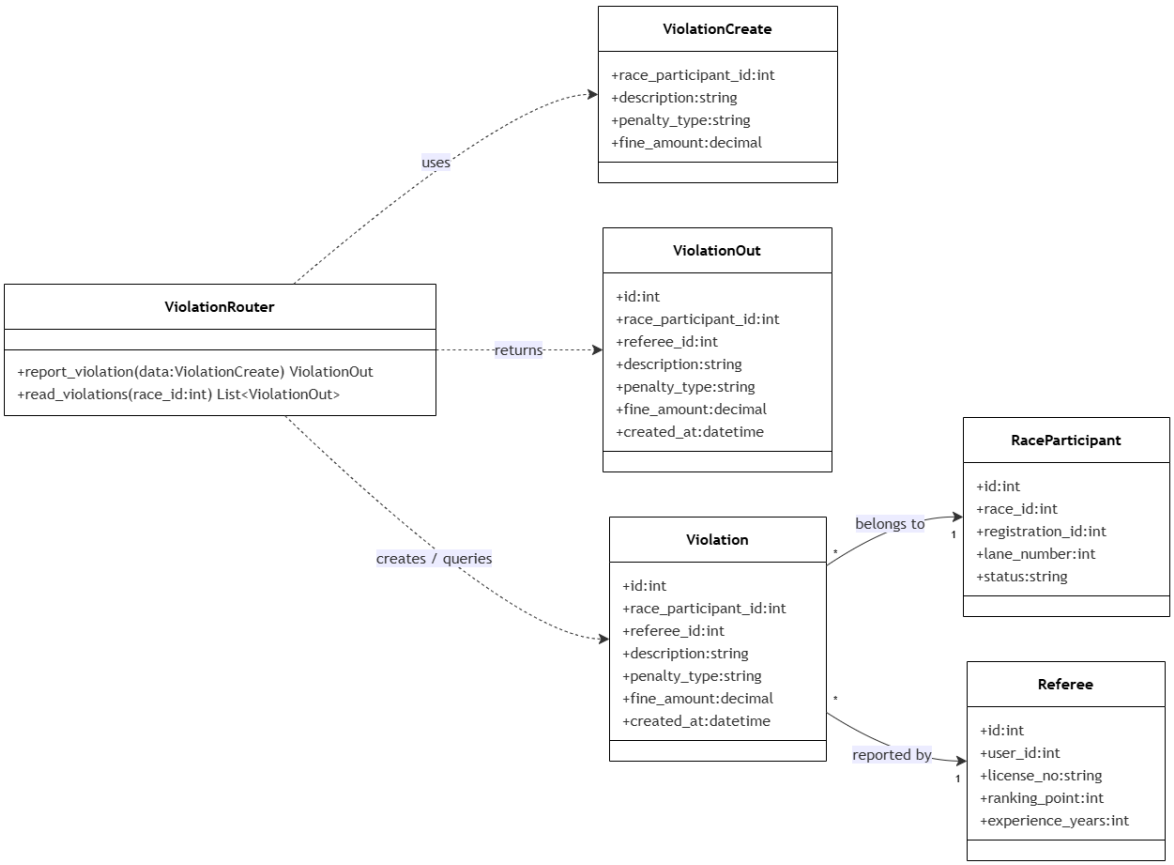


Figure 4.65: Violation Report Class Diagram

Sequence Diagram

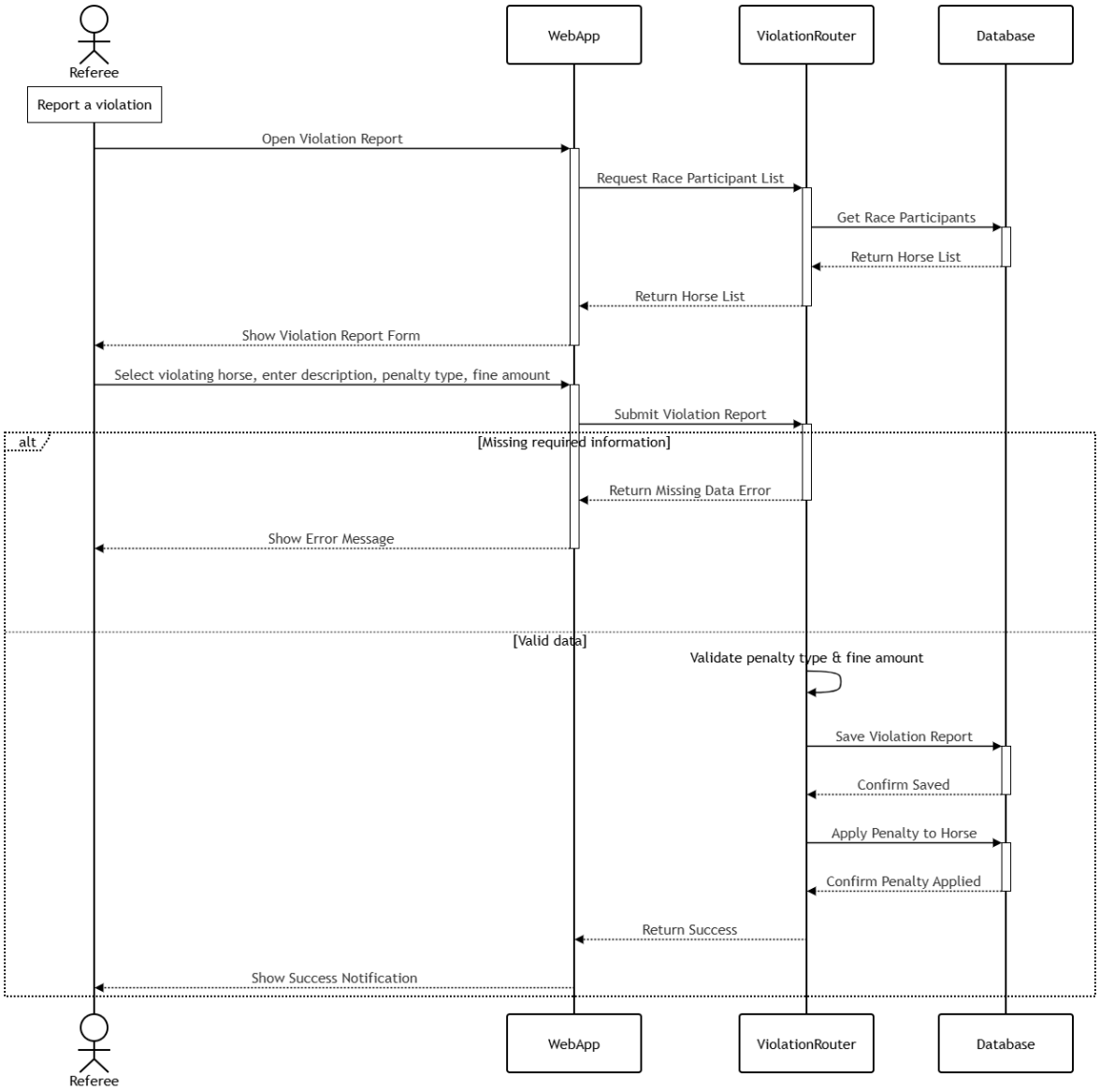


Figure 4.66: Violation Report Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
ViolationRouter	report_violation(), read_violations()	controller / route	Handles HTTP requests to submit and retrieve violation reports filed by the referee.
ViolationCreate	race_participant_id, description, penalty_type, fine_amount	DTO	Input data transfer object used to submit a new violation report for a race participant.
ViolationOut	id, race_participant_id, description, penalty_type, fine_amount, horse_name, created_at	DTO	Output data transfer object returning the submitted violation details along with related participant information.
Violation	id, race_participant_id, referee_id, description, penalty_type, fine_amount, created_at	entity	Database entity storing the details and penalty applied for a violation committed during a race.
RaceParticipant	id, race_id, registration_id, lane_number, status	entity	Database entity representing the horse-jockey pair being reported for the violation.
Referee	id, user_id, license_no	entity	Database entity identifying the referee who filed the violation report.

Table 4.30: Violation Report class specification

4.2.30 Result Confirmed

Class Diagram

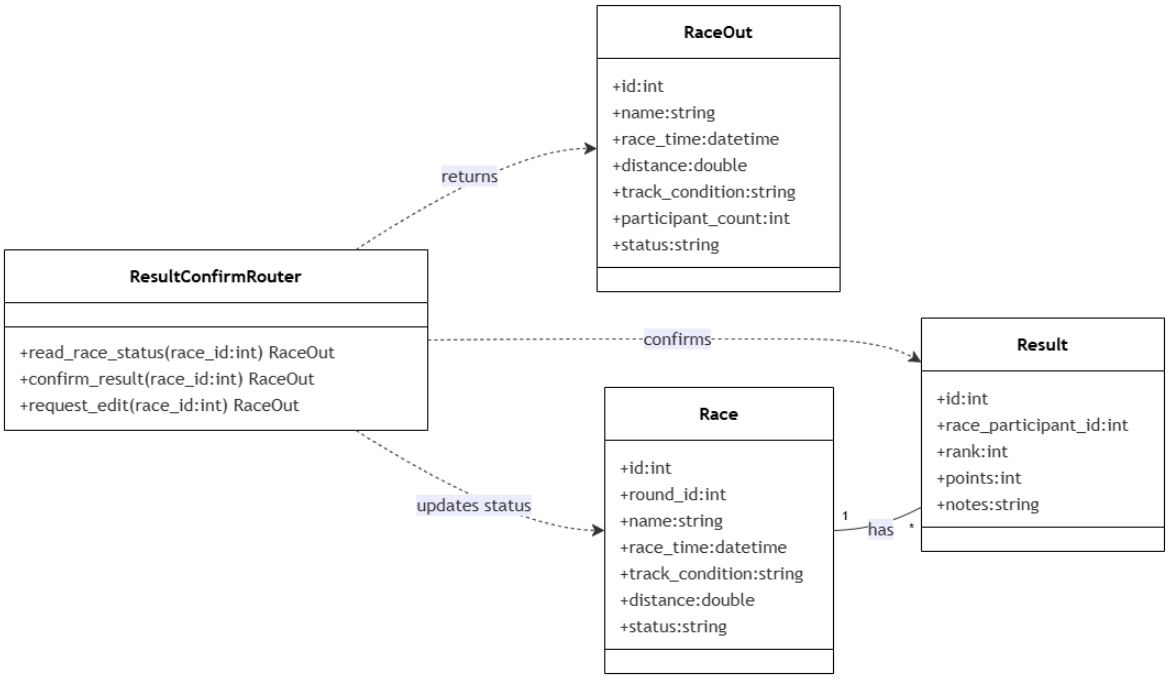


Figure 4.67: Result Confirmed Class Diagram

Sequence Diagram

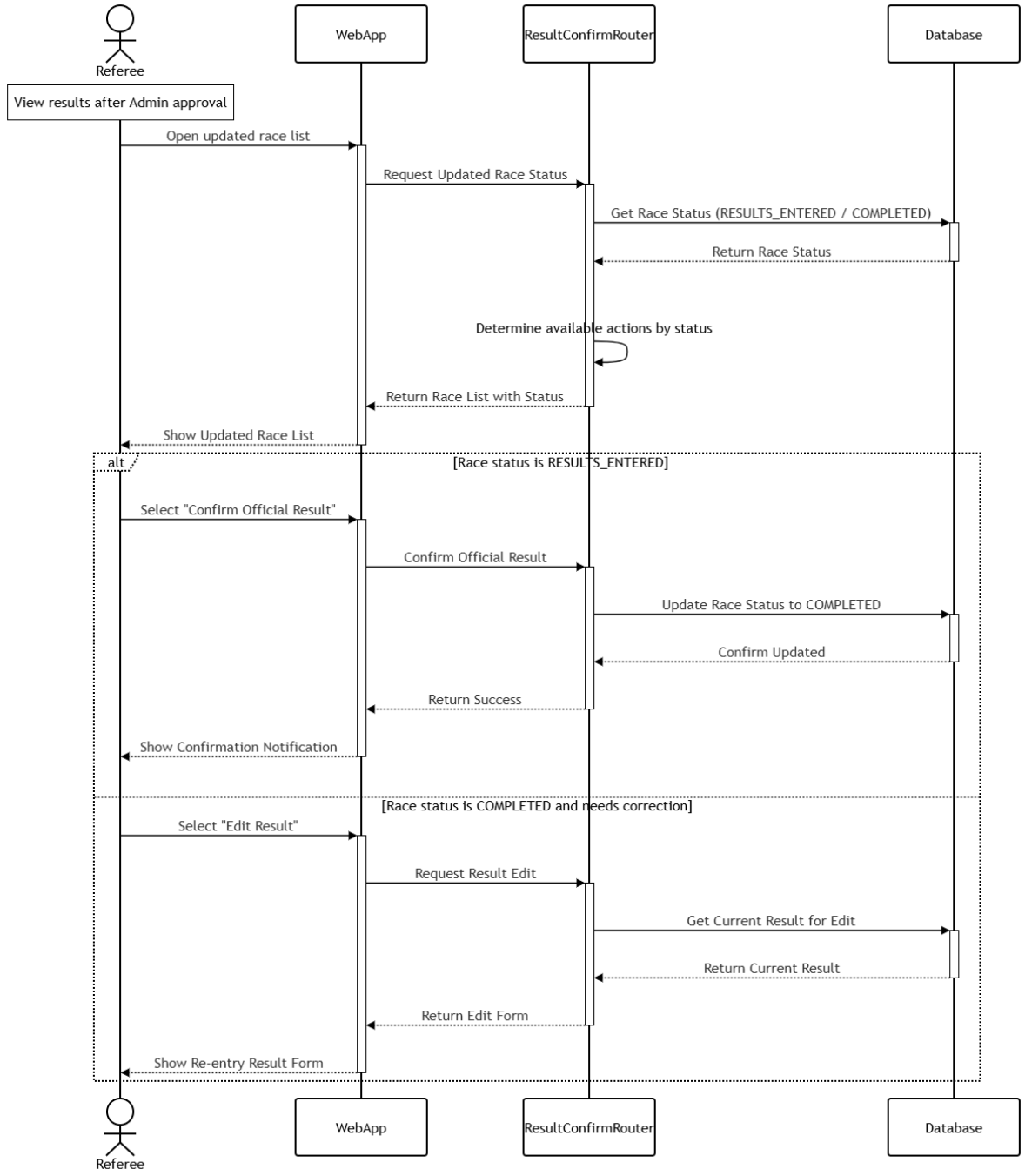


Figure 4.68: Result Confirmed Sequence Diagram

Class Specification

Class	Attributes / Methods	Type	Description
ResultConfirmRouter	read_race_status(), confirm_result(), request_edit()	controller / route	Handles HTTP requests to review the administrator-approved race status and confirm or request edits to the official result.
RaceOut	id, name, status	DTO	Output data transfer object returning the current status of the race after administrator review.
Race	id, round_id, name, race_time, track_condition, distance, status	entity	Database entity storing core information of the race, including its confirmation status.
Result	id, race_participant_id, rank, points, notes	entity	Database entity storing the official race result that is reviewed or corrected during confirmation.

Table 4.31: Result Confirmed class specification

V. Software Testing Documentation

5.1. Scope of Testing

5.1.1 In-Scope Testing

The test scope covers the main business functions of the Horse Racing Tournament Management System (HRTMS) web application. The purpose of testing is to verify that the core workflows work correctly for each role and that the backend data is stored and retrieved accurately.

- **Authentication and Registration**
  - Login with valid and invalid credentials.
  - Login handling for locked accounts.
  - Registration for Spectator, Horse Owner, Jockey, and Referee roles.
  - Logout and redirect back to the login page.
- **Admin and Tournament Management**
  - Load, create, update, and delete tournaments.
  - Create rounds and races.
  - Assign referees and validate scheduling conflicts.
  - Manage users, lock and unlock accounts, and change user roles.
  - Approve, reject, and reset registrations.
  - Manage prizes, awards, and tournament status.
- **Horse Owner Workflow**
  - Load, create, edit, and delete horse records.
  - Validate horse age and other required fields.
  - Select jockeys and send invitations.
  - Register horses for tournaments.
  - View registration status, profile details, upcoming races, and results.
- **Jockey Workflow**
  - View incoming invitations and respond with accept or reject.
  - View schedule, profile, and awards.
  - Update profile information.
  - Validate profile rules such as unique email and required fields.

  - Filter leaderboard data by tournament.

- **Referee Workflow**
  - View assigned races and race participant details.
  - Inspect races and record draft results.
  - Confirm official results.
  - Report violations and validate fine amounts.
  - Update result status from entered to completed.

- **Spectator Workflow**
  - Create, edit, and delete predictions.
  - Lock predictions after race time.
  - View upcoming schedules and completed race results.
  - Update personal profile and spectator statistics.
  - View leaderboard and filter by tournament.

- **API Verification**
  - Validate key backend endpoints with Postman.
  - Verify user data, rankings, race lists, invitations, prizes, and race result APIs.
  - Check response data, status codes, and persistence in the database.

5.1.2 Out-of-Scope Testing

The following items are excluded from the final test scope because they are not part of the current release or they require a separate non-functional test plan:

- External payment or betting integrations.
- Mobile application testing.
- Large-scale load or stress testing.
- Automated UI testing frameworks, because the team decided to use manual browser testing for UI verification.
- External services that are not implemented in the current HRTMS release.

5.1.3 Constraints & Assumptions

- Testing is performed in a production-like environment or a locally deployed environment with seeded test data.
- Test accounts for all roles are available before execution.
- Postman is used to verify backend endpoints and response payloads.
- UI verification is performed manually in the browser using the latest stable Chrome version.
- Test data is prepared in advance so that each role can follow its expected workflow without dependency issues.
- Database connectivity remains stable during the test window.

5.2. Test Strategy

5.2.1 Testing Types

This project uses three main testing types: Unit Testing, Integration Testing, and Functional Testing. The test strategy is built around manual UI verification and Postman-based API verification, which matches the final decision of the team.

- **Unit Testing**
  - **Objective:** Verify the correctness of individual functions, services, controllers, and validation rules in isolation. In HRTMS, unit-level checks focus on login logic, horse age validation, tournament registration rules, invitation validation, race result processing, and leaderboard calculation.
  - **Technique:** White-box testing is applied by developers during implementation. Individual functions are checked with controlled inputs and mock data when they depend on the database or other modules.
  - **Completion Criteria:**
    - Core business logic in important modules is covered by unit checks.
    - All unit test cases pass before code is merged.
    - No critical logic defect remains in authentication, validation, or result processing.
- **Integration Testing**
  - **Objective:** Validate that connected modules work correctly together, especially the interaction between the frontend, backend APIs, and database layer.
  - **Technique:** Black-box and gray-box testing are performed with Postman and browser-based verification. API requests are sent to the backend, and the results are checked against expected database changes and screen behavior.
  - **Completion Criteria:**
    - All main integration scenarios are executed and passed.
    - Data exchange between frontend, backend, and database is stable.
    - No critical defect remains in login flow, registration, scheduling, invitation processing, or race result storage.
- **Functional Testing**
  - **Objective:** Confirm that the implemented features satisfy the business requirements for each role in the system.
  - **Technique:** Black-box testing is carried out manually through the browser to simulate real user journeys for Admin, Horse Owner, Jockey, Referee, and Spectator.
  - **Completion Criteria:**
    - All major functional requirements are covered by manual test cases.
    - The tested features operate correctly for the intended role and screen.
    - No blocking issue remains that prevents normal use of the system.

5.2.2 Test Levels

The table below shows how the three testing types are mapped to the testing levels used in this project.

Type of Tests	Unit	Integration	System	Acceptance
Unit Testing	X			
Integration Testing		X	X	
Functional Testing			X	X

Table 5.1: Test Levels for Horse Racing Tournament Management System

5.2.3 Supporting Tools

Purpose	Tool	Provider / Vendor	Version
Integration Testing (API)	Postman	Postman, Inc.	Latest
Functional Testing (UI)	Manual Browser Testing	In-house	Latest Chrome
Database Verification	SQL Server Management Studio	Microsoft	Latest
Test Case Tracking	Microsoft Excel	Microsoft	Latest

Table 5.2: Supporting Tools for Testing

5.3. Test Plan

5.3.1 Human Resources

The table below assigns the main testing responsibilities to each project member.

Worker / Doer	Role	Specific Responsibilities / Comments
Ha Duy Hung	Project Leader	Execute and verify the login/logout testing flow, supervise overall testing progress, and approve the final report.
Dinh Thuy Anh	Test Leader	Execute and lead the Horse Owner testing flow, coordinate owner-related test cases, and report defects related to horse management and registration.
Ho Nguyen Thai Chau	Member	Execute Jockey and leaderboard test cases, and re-check failed cases after fixes.
Le Thi My Hue	Member	Execute Admin and tournament management test cases, including API verification with Postman.
Bui Le Quang Huy	Member	Execute Referee workflow test cases and verify race result processing.
Nguyen Gia Huy	Member	Execute integration tests, consolidate results, and update the final testing report.
Doan Thi Thu May	Member	Execute Spectator workflow test cases and verify prediction and profile functions.

Table 5.3: Testing Team Responsibilities

5.3.2 Test Environment

Purpose	Tool	Provider	Version
API validation	Postman	Postman, Inc.	Latest
UI validation	Manual browser testing	In-house	Latest Chrome
Database inspection	SQL Server Management Studio	Microsoft	Latest
Test case documentation	Microsoft Excel	Microsoft	Latest

Table 5.4: Test Environment

5.3.3 Test Milestones

The test execution window is based on the dates recorded in the test plan workbook.

Milestone Task	Start Date	End Date
Prepare test cases and test data	21/06/2026	21/06/2026
Execute authentication and registration tests	21/06/2026	22/06/2026
Execute admin and tournament management tests	22/06/2026	25/06/2026
Execute horse owner and jockey workflow tests	26/06/2026	29/06/2026
Execute referee and spectator workflow tests	30/06/2026	03/07/2026
Execute API checks and final retest	03/07/2026	05/07/2026

Table 5.5: Test Milestones

5.4. Test Cases

The detailed test cases are maintained in the Excel workbook Test_plan_nhom3.xlsx. The sheet contains the general function list to be tested, tester assignment, priority, deadline, notes, and execution status. It is a test function plan, not the final detailed test case specification.

- Main workbook: Test_plan_nhom3.xlsx
- Primary sheet: `Execution Plan`
- Tracking columns: No, Function Name, Priority, Deadline, Tester, Note, Pass, Fail, Untest

5.5. Test Reports

5.5.1 General Information

Field	Value
Project Name	Horse Racing Tournament Management System
Project Code	HRTMS
Document Code	Test Report
Creator	Testing Team
Reviewer / Approver	Team Leader and Supervisor
Issue Date	05/07/2026
Notes	Manual UI testing and Postman API verification were used for the final test execution.

Table 5.6: Test Report General Information

5.5.2 Result by Module

Module / Workflow	Passed	Failed	Untested	Total	Coverage	Pass Rate	Status
Authentication & Registration	10	0	0	10	100.00%	100.00%	Passed
Admin & Tournament Management	32	0	0	32	100.00%	100.00%	Passed
Horse Owner Workflow	17	0	0	17	100.00%	100.00%	Passed
Jockey Workflow	11	2	0	13	100.00%	84.62%	Open issues
Referee Workflow	14	0	0	14	100.00%	100.00%	Passed
Spectator Workflow	15	0	0	15	100.00%	100.00%	Passed
Backend API Checks	5	0	1	6	83.33%	100.00%	Follow-up needed
Total	**104**	**2**	**1**	**107**	**99.07%**	**98.11%**	**Overall passed**

Table 5.7: Result by Module

5.5.3 Open Issues

The following cases were not fully completed or still need follow-up:

- **validateJockeyProfileEmailDuplicate**: This case failed because some required data validation paths were missing during execution.
- **filterLeaderboardByTournament**: This case failed because the leaderboard filter did not return correct results in some scenarios.
- **getTournamentAwards**: This case was untested because the test data for awards was not sufficient at the time of execution.

5.5.4 Final Comment

The overall testing result shows that the system is stable for the main workflows. The report is summarized by module instead of a global test summary because the final reporting is based on the executed function groups. Most test cases passed successfully, and the remaining issues are concentrated in the Jockey workflow and one backend API case. These items should be retested after the related fixes and test data are prepared.